[CẦN BỔ SUNG: CƠ QUAN CHỦ QUẢN]

[CẦN BỔ SUNG: TÊN TRƯỜNG]

[CẦN BỔ SUNG: KHOA / BỘ MÔN]



BÁO CÁO ĐỒ ÁN

[CẦN BỔ SUNG: TÊN HỌC PHẦN]

XÂY DỰNG HỆ THỐNG BÁN VÀ QUẢN LÝ TRÀ SỮA UTE-TEA

GIẢNG VIÊN HƯỚNG DẪN: [CẦN BỔ SUNG]

LỚP: [CẦN BỔ SUNG]

NHÓM SINH VIÊN THỰC HIỆN:

HỌ VÀ TÊN	MSSV
[CẦN BỔ SUNG: HỌ TÊN]	[CẦN BỔ SUNG: MSSV]
[CẦN BỔ SUNG: HỌ TÊN]	[CẦN BỔ SUNG: MSSV]
[CẦN BỔ SUNG: HỌ TÊN]	[CẦN BỔ SUNG: MSSV]

[CẦN BỔ SUNG: ĐỊA ĐIỂM], tháng 10 năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

[CẦN BỔ SUNG: Nhận xét và đánh giá của giảng viên hướng dẫn.]

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

LỜI MỞ ĐẦU

Đồ án UTE-Tea nghiên cứu và xây dựng hệ thống bán trà sữa gắn với vận hành tại quầy và tại bàn. Bài toán không dừng ở việc hiển thị thực đơn: một đơn hàng cần có cấu hình món, topping, cơ sở phục vụ, hóa đơn, phương thức nhận tiền và trạng thái chế biến nhất quán. Đồng thời, nguyên liệu được quản lý bằng định mức, tồn ở kho tổng/bếp và các chứng từ nhập, sơ chế, hao hụt, kiểm kê.

Báo cáo này được biên soạn trên bản Word tham khảo, bảo toàn khung ba chương gồm kiến trúc ứng dụng, sơ đồ hoạt động các chức năng và giao diện hệ thống. Nội dung được đối chiếu trực tiếp với Controller, Service, Repository, Entity, DTO, template và JavaScript của UTE-Tea; tài liệu trong docs/skills chỉ đóng vai trò bổ trợ. Các phần chưa có bằng chứng mã nguồn không được trình bày như chức năng đã hoàn tất.

Phạm vi checkout khách hàng hiện tại chỉ phục vụ mang về hoặc tại quán theo ngữ cảnh bàn hợp lệ. Các ô chọn hình thức/cơ sở và nhập mã giảm giá đã được loại khỏi giao diện. Sau khi nhấn Gửi đơn hàng, khách chọn tiền mặt hoặc chuyển khoản trong hộp thoại rồi xác nhận. Việc chọn phương thức hoặc thông báo đã chuyển khoản chưa đủ để ghi nhận thanh toán; khoản thu cần được thu ngân xác nhận trong hệ thống.

Ngoài luồng POS chính, mã nguồn còn giữ module cơ sở, các vai trò VENDOR/ MANAGER/SHIPPER, lịch sử DeliveryOrder và tích hợp VNPAY của module cũ. Báo cáo phân biệt rõ những thành phần này với checkout hiện hành, tránh suy diễn khả năng giao tận nơi hoặc đối soát ngân hàng tự động. Ảnh giao diện được chụp từ ứng dụng demo H2 tách biệt, không đại diện cho số liệu kinh doanh thật.

Các thông tin hành chính chưa được chủ nhiệm đồ án xác nhận được đánh dấu [CẦN BỔ SUNG]. Đây là các trường cần hoàn thiện trước khi nộp; không tự sử dụng tên giảng viên, lớp hoặc danh sách sinh viên từ tài liệu tham khảo.

MỤC LỤC

1.KIẾN TRÚC ỨNG DỤNG 1

1.1.Sơ đồ kiến trúc ứng dụng 1

1.2.Mô tả kiến trúc ứng dụng 2

1.2.1.Công nghệ và tổ chức mã nguồn 2

1.2.2. Luồng xử lý yêu cầu và dữ liệu 3

1.2.3.Tác nhân, phạm vi và mức độ triển khai 4

1.2.4.Bảo mật, transaction và chống xử lý trùng 5

1.2.5.Mô hình dữ liệu và quan hệ chính 5

1.2.6.Từ điển Entity đối chiếu bảng và cột 7

1.2.7. Luồng trạng thái nghiệp vụ và giới hạn tích hợp 28

1.2.8. Phạm vi kiểm chứng và tài liệu nguồn 28

2.SƠ ĐỒ HOẠT ĐỘNG CÁC CHỨC NĂNG 30

2.1. Khách hàng 30

2.1.1.Đăng ký tài khoản chờ xác thực 30

2.1.2.Xác thực email bằng OTP 32

2.1.3.Gửi lại mã xác thực đăng ký 34

2.1.4.Đăng nhập theo trạng thái tài khoản 36

2.1.5.Đăng xuất và vô hiệu phiên 38

2.1.6.Yêu cầu khôi phục mật khẩu qua email 40

2.1.7.Xác minh OTP khôi phục và nhận vé tạm 42

2.1.8.Xác nhận mật khẩu mới 44

2.1.9. Xem menu, tìm kiếm và lọc sản phẩm 47

2.1.10. Xem chi tiết, chọn size và topping 49

2.1.11.Lưu và đồng bộ giỏ hàng tài khoản 52

2.1.12.Lấy lại giỏ đã lưu 54

2.1.13. Nhận diện bàn bằng QR hợp lệ 56

2.1.14. Kiểm tra tổng tiền trước gửi đơn 58

2.1.15.Gửi đơn mang về qua bước chọn thanh toán 61

2.1.16.Gửi đơn tại bàn theo phiên được xác thực 64

2.1.17. Thông báo chuyển khoản để thu ngân đối soát 67

2.1.18. Theo dõi chứng từ đơn vừa đặt 69

2.1.19. Xem lịch sử đơn hàng tài khoản 71

2.1.20.Yêu cầu hủy đơn trước chế biến 73

2.1.21. Xem và sửa thông tin cá nhân 76

2.1.22.Đổi mật khẩu khi đang đăng nhập 78

2.1.23.Đánh dấu yêu thích và ghi nhận món đã xem 80

2.1.24.Đánh giá món đã mua và hoàn tất82

2.1.25.Đính kèm ảnh hoặc video cho đánh giá 84

2.1.26. Quản lý địa chỉ đã lưu cho dữ liệu tương thích 87

2.2. Thu ngân 89

2.2.1. Xem hàng đợi và bộ lọc đơn 89

2.2.2. Tiếp nhận và xác nhận đơn khách 91

2.2.3.Tạo đơn tại quầy bằng POS 94

2.2.4. Ghi thu tiền mặt hoặc chuyển khoản 97

2.2.5.Xác nhận thông báo chuyển khoản của khách 99

2.2.6.Mở phiên phục vụ bàn 101

2.2.7.Gắn đơn quầy vào bàn cùng cơ sở 104

2.2.8.Đóng phiên bàn sau phục vụ 106

2.2.9. Duyệt hoặc từ chối yêu cầu hủy 108

2.2.10. Hoàn tiền mặt và lập yêu cầu hoàn chuyển khoản 111

2.3. Nhân viên bếp 114

2.3.1. Xem hàng đợi bếp theo cơ sở 114

2.3.2.Bắt đầu chế biến đơn116

2.3.3. Hoàn thành đơn và tiêu hao nguyên liệu 119

2.3.4. Theo dõi tồn kho và cảnh báo ngưỡng 121

2.3.5. Ghi mẻ sơ chế từ định mức 124

2.3.6. Nhập nguyên liệu thô vào kho tổng 126

2.3.7. Ghi hao hụt và nguyên liệu loại bỏ 129

2.3.8. Ghi nhận và xử lý sự cố chế biến 132

2.3.9. Ghi tiêu hao bổ sung khi pha bù 134

2.3.10. Kiểm kê và kết ca kho 137

2.3.11. Xem lịch sử nhập, sơ chế và biến động 139

2.4. Quản trị viên 141

2.4.1. Quản lý tài khoản nhân viên 141

2.4.2.Sửa, khóa và xóa tài khoản có kiểm soát 143

2.4.3. Quản lý danh mục menu 145

2.4.4. Quản lý món, size và giá bán 147

2.4.5. Quản lý ảnh sản phẩm trên Cloudinary 149

2.4.6. Thiết lập công thức định mức món 152

2.4.7. Quản lý bàn và mã QR 154

2.4.8. Quản lý nguyên liệu và đơn vị 156

2.4.9. Quản lý định mức sơ chế 158

2.4.10.Điều chỉnh kho tổng có lý do 161

2.4.11. Thiết lập ngưỡng cảnh báo tồn 163

2.4.12. Quản lý đơn và trạng thái thanh toán 165

2.4.13. Phê duyệt hoàn tiền chuyển khoản 167

2.4.14.Cấu hình cửa hàng, ngân hàng và giảm giá chung 170

2.4.15. Thống kê doanh thu theo khoảng thời gian 172

2.4.16. Dashboard đơn, sản phẩm và cảnh báo 175

2.4.17.Đổi logo và ảnh đại diện 177

2.5. Vai trò cơ sở và phân hệ mở rộng 179

2.5.1.Tạo và chỉnh sửa cơ sở trong phân hệ mở rộng 180

2.5.2. Phân công quản lý, thu ngân và bếp theo cơ sở 182

2.5.3. Quản lý menu và tồn bán theo cơ sở 184

2.5.4. Quản lý mã khuyến mãi qua API cơ sở 186

2.5.5.Hạn chế khách mua tại một cơ sở 188

2.5.6. Tra cứu đơn giao hàng lịch sử theo phạm vi 190

2.5.7. Phân công shipper cho đơn giao hàng cũ 192

2.5.8. Chuyển trạng thái đơn giao hàng và trả hàng lịch sử 194

2.5.9.Tích hợp VNPAY của module lịch sử 197

3.GIAO DIỆN HỆ THỐNG 200

3.1. Giao diện khách hàng 200

3.1.1.Đăng nhập 200

3.1.2.Đăng ký tài khoản 201

3.1.3.Yêu cầu khôi phục mật khẩu 202

3.1.4. Trang chủ và thực đơn 203

3.1.5. Danh sách món và bộ lọc 204

3.1.6. Chọn size, topping và ghi chú 205

3.1.7. Giỏ hàng và kiểm tra tổng tiền 206

3.1.8.Hộp chọn phương thức thanh toán 207

3.1.9. Checkout theo bàn đã quét QR 208

3.1.10.Lịch sử đơn của khách 209

3.1.11.Xác nhận đơn và thông tin chuyển khoản 210

3.1.12. Thông tin tài khoản 211

3.1.13.Yêu thích, món đã xem và địa chỉ lưu 212

3.2. Giao diện thu ngân 213

3.2.1.Bảng điều khiển thu ngân 213

3.3. Giao diện nhân viên bếp 214

3.3.1.Hàng đợi chế biến 214

3.4. Giao diện quản trị viên 215

3.4.1. Dashboard và báo cáo 215

3.4.2. Quản lý danh mục và món 216

3.4.3. Quản lý nhân viên 217

3.4.4. Quản lý bàn 218

3.4.5. Quản lý đơn và thu tiền 219

3.4.6. Nguyên liệu và tồn kho 220

3.4.7. Chứng từ vận hành kho 221

3.4.8.Cài đặt hệ thống 222

3.4.9. Quản lý cơ sở và phân công 223

3.5. Giao diện vai trò cơ sở và phân hệ mở rộng 224

3.5.1.Cổng nghiệp vụ VENDOR 224

3.5.2.Cổng nghiệp vụ MANAGER 225

3.5.3.Cổng nghiệp vụ SHIPPER 226

3.6.Đánh giá mức độ hoàn thiện và nội dung cần bổ sung 227

DANH MỤC HÌNH

[Cập nhật trường trong Word để hiển thị danh mục và số trang.]

DANH MỤC BẢNG

[Cập nhật trường trong Word để hiển thị danh mục và số trang.]

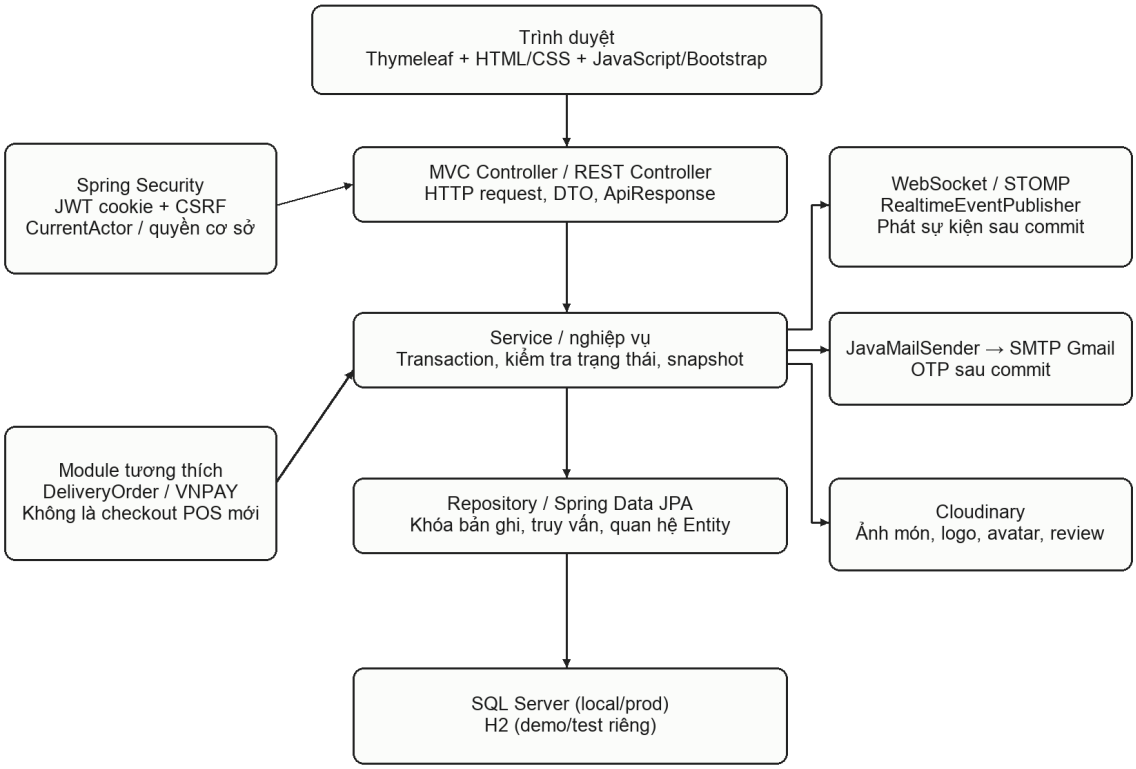
DANH MỤC ĐOẠN MÃ

[Cập nhật trường trong Word để hiển thị danh mục và số trang.]

1.KIẾN TRÚC ỨNG DỤNG

1.1.Sơ đồ kiến trúc ứng dụng

UTE-Tea là ứng dụng nguyên khối phân tầng trên Spring Boot. Tầng trình bày kết hợp các trang Thymeleaf render phía server với JavaScript gọi REST API. Spring Security xử lý JWT/CSRF và quyền truy cập trước khi yêu cầu vào Controller; Service giữ điều kiện nghiệp vụ và transaction; Repository/JPA truy vấn các Entity ánh xạ SQL Server. H2 được dùng cho demo/test với cấu hình tách biệt.



Hình 1.1. Kiến trúc UTE-Tea đối chiếu theo các package hiện có

Các tích hợp được nhận diện từ dependency và class thực tế gồm SMTP qua JavaMailSender, Cloudinary cho media và WebSocket/STOMP cho thông báo sau commit. VNPAY còn nằm ở phân hệ DeliveryOrder cũ; không phải lựa chọn trong hộp thanh toán POS hiện tại. Các đường nối biểu diễn trách nhiệm, không hàm ý hệ thống được triển khai thành nhiều microservice.

1.2.Mô tả kiến trúc ứng dụng

1.2.1.Công nghệ và tổ chức mã nguồn

Bảng 1.1. Công nghệ được xác nhận từ pom.xml và mã nguồn

Thành phần	Bằng chứng	Vai trò
Java 21	pom.xml: java.version và compiler release	Ngôn ngữ backend và môi trường biên dịch
Spring Boot 4.1.1	parent/dependency/plugin trong pom.xml	Khởi tạo ứng dụng, embedded web server và cấu hình ngoài
Spring MVC + Thymeleaf	controller/; templates/	Render trang HTML và cung cấp các API HTTP
Spring Security / JWT / BCrypt	SecurityConfig; JwtAuthenticationFilter; JwtService	Xác thực, phân quyền, chống CSRF và hash mật khẩu
Spring Data JPA / Hibernate	repository/; entity/	ORM, truy vấn và khóa bản ghi trong transaction
Microsoft SQL Server	mssql-jdbc; database/sqlserver/	CSDL quan hệ dùng cho local/ prod theo cấu hình
H2	application-demo.yml; application-test.yml	Demo/test; không là bằng chứng migration SQL Server đã chạy thật
Bootstrap + JavaScript	layouts/main.html; static/js/	Responsive UI, dialog, fetch API và cập nhật màn hình
WebSocket / STOMP	WebSocketConfig; RealtimeEventPublisher	Thông báo đơn, tiền, phiên bàn và thay đổi cấu hình
Spring Mail / Cloudinary	MailService; ProductImageService; ReviewService	Dịch vụ ngoài hệ thống; cần cấu hình hợp lệ để chạy thật

Phiên bản trong bảng phản ánh pom.xml của workspace, không phải tuyên bố đây là phiên bản mới nhất trên Internet. Spring Boot cung cấp cách đóng gói và chạy ứng dụng độc lập cùng cấu hình ngoài [TL1]. Việc đối chiếu phiên bản/tương thích của đồ án được thực hiện trên dependency của dự án, không lấy phiên bản từ báo cáo mẫu.

Bảng 1.2. Trách nhiệm các lớp và thư mục

Tầng	Thư mục	Trách nhiệm và ví dụ
Controller	src/main/java/vn/edu/ute/milktea/controller	Nhận request, @Valid, CurrentActor, headers; AuthController, OrderController, InventoryController.
Service	.../service	Áp quy tắc, transaction, trạng thái, tính tiền; OrderService, PaymentService, AuthService.
Repository	.../repository	JPA interface, truy vấn, pessimistic locking; OrderRepository, StockRepository.
Entity	.../entity	Bảng, cột và quan hệ; Order, Invoice, Payment, DiningTable, Stock.
DTO	.../dto	Hợp đồng đầu vào/đầu ra, validation; OrderDto, AuthDto, InventoryDto.
Security	.../security	JWT cookie/tokenVersion, capability guest, CSRF và kiểm tra role.
Frontend	src/main/resources/templates và static	Trang Thymeleaf, CSS/JS; giỏ, dialog thanh toán, polling và STOMP.
Database	database/sqlserver	Schema, seed và các migration tăng dần; không chạy script DROP cho dữ liệu đang vận hành.

1.2.2.Luồng xử lý yêu cầu và dữ liệu

Đối với trang HTML, browser gửi request đến CustomerPageController hoặc StaffPageController. Controller lấy dữ liệu từ repository/service, đưa vào Model và trả tên template. Thymeleaf kết hợp layouts/fragments với dữ liệu server để trả trang hoàn chỉnh. Các thao tác tiếp theo trên trang dùng MilkTeaApi trong static/js/common/api.js, thay vì submit form truyền thống ở mọi màn hình.

Đối với API thay đổi dữ liệu, JavaScript lấy/cập nhật CSRF token, gửi JSON hoặc multipart cùng cookie xác thực và header chống gửi lặp khi có. Filter JWT xây dựng CurrentActor từ tài khoản đang hoạt động và email đã xác thực; các token guest chỉ có phạm vi phiên hoặc một chứng từ. Controller xác thực DTO và chuyển thao tác vào Service. Service khóa bản ghi cần thiết, kiểm tra điều kiện và gọi Repository trong cùng transaction.

Khi tạo đơn, giá sản phẩm và topping được lấy lại từ database theo cơ sở phục vụ. Frontend không có quyền quyết định đơn giá và totalAmount. OrderItem lưu snapshot tên/size/giá, Invoice lưu snapshot giảm giá; vì vậy thay giá menu sau này không làm đổi hóa đơn đã phát sinh. expectedTotal chỉ dùng để phát hiện tổng hiển thị đã thay đổi, không được sử dụng làm công thức tính tiền.

Thông báo realtime được đăng ký sau commit để giao diện không nhìn thấy trạng thái chưa được lưu. Lệnh nghiệp vụ vẫn chạy qua HTTP và Service; STOMP không thay thế transaction. Khi socket mất kết nối, client có cơ chế tải lại/polling tùy trang. Email xác thực cũng được gửi sau khi tài khoản/OTP đã commit để SMTP thất bại không xóa dữ liệu chờ xác thực.

1.2.3.Tác nhân, phạm vi và mức độ triển khai

Bảng 1.3. Vai trò có trong Role.java và cách sử dụng thực tế

Vai trò	Phạm vi được xác nhận	Giới hạn cần ghi rõ
CUSTOMER / guest	Menu, giỏ, quote, tạo đơn, chứng từ, lịch sử tài khoản, hồ sơ, đánh giá có điều kiện.	Guest không được xem lịch sử chỉ nhờ ID bàn; cần chứng từ riêng cho một đơn.
CASHIER	POS, hàng đợi, xác nhận đơn, thu tiền, bàn/phiên, hủy/hoàn theo route.	Các lệnh đơn/thu tiền kiểm tra cơ sở; không suy ra mọi module hủy/kho đã đồng nhất phạm vi.
KITCHEN	Đơn cần pha, start/complete, tồn/ hao hụt/sơ chế/kiểm kê theo PreAuthorize.	Trạng thái toàn đơn; tiêu hao dùng snapshot và tồn nguyên liệu theo KHO/BEP.
ADMIN	Menu, nhân viên, bàn, kho, cài đặt, thống kê, phạm vi quản trị.	Không có entity Brand/luồng phản hồi kiểm duyệt review chỉ vì báo cáo mẫu có.
VENDOR / MANAGER	Shop owner hoặc được phân công; portal cơ sở, menu, voucher và dữ liệu tương thích.	Không áp quyền MANAGER của marketplace vào mọi route POS một cách suy đoán.
SHIPPER	Phân hệ DeliveryOrder lịch sử có API gán và chuyển trạng thái.	Không phải lựa chọn của checkout hiện tại; POST tạo DeliveryOrder từ khách đã bị chặn.

Bảng trên mô tả phạm vi đã đối chiếu ở SecurityConfig, @PreAuthorize và service. Các nội dung cần cải thiện được giữ như hạn chế kỹ thuật: một số module cũ chưa đồng nhất kiểm tra cơ sở; Stock hiện định danh theo nguyên liệu/vị trí, không có ShopId ở mọi dòng. Không mô tả toàn bộ kho như một hệ thống tồn nguyên liệu phân tán theo chi nhánh khi schema chưa thể hiện điều đó.

1.2.4.Bảo mật, transaction và chống xử lý trùng

Spring Security của ứng dụng servlet sử dụng filter chain để chặn hoặc chuyển tiếp yêu cầu trước DispatcherServlet; thứ tự filter ảnh hưởng đến việc xác thực và phân quyền [TL2]. Trong UTE-Tea, JwtAuthenticationFilter đặt trước UsernamePasswordAuthenticationFilter; SecurityConfig khai báo phạm vi route theo role và giữ CSRF cho các mutation HTTP. Các API auth public không đồng nghĩa được bỏ CSRF.

Mật khẩu tài khoản được hash bằng PasswordEncoder/BCrypt. JWT lưu trong cookie HttpOnly; tokenVersion và các cờ active/emailVerified được đối chiếu với DB ở mỗi request có xác thực. Token guest là chuỗi opaque lưu hash, có thời hạn và phạm vi cụ thể. ID bàn hoặc sessionId do client gửi không tự trở thành bằng chứng quyền truy cập.

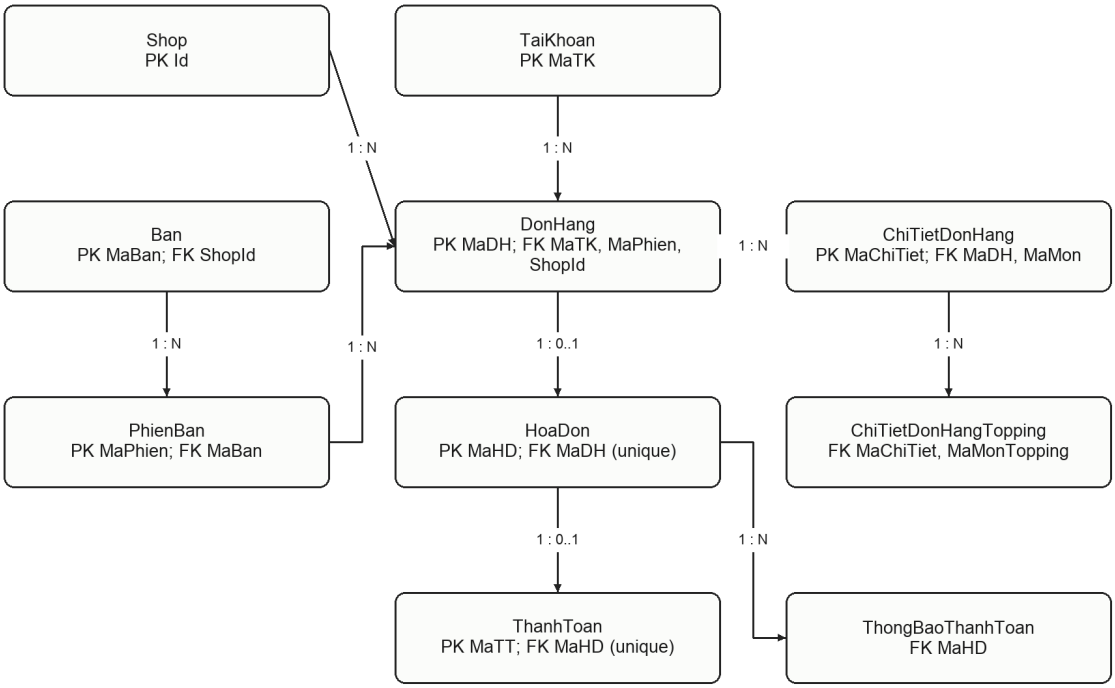
OTP sáu chữ số sinh bằng SecureRandom và chỉ lưu HMAC cùng salt, email, purpose, thời hạn và số lần thử. OTP_SECRET phải là khóa riêng hợp lệ; báo cáo không đọc hay đưa giá trị .env vào tài liệu. REGISTER_VERIFY và PASSWORD_RESET là hai mục đích khác nhau. Mã đúng được đánh dấu usedAt; mã sai/hết hạn có cập nhật trạng thái và giữ bộ đếm lỗi nghiệp vụ trong transaction.

Order dùng Idempotency-Key và fingerprint của nội dung để phát hiện gửi lặp; khóa cơ sở/bàn giúp tuần tự các thao tác có liên quan. Payment có khóa invoice/order, kiểm tra khoản thu đã tồn tại và không nhận số tiền do frontend quyết định. Một lần replay đúng không được chèn thêm hóa đơn hoặc khoản thu. Khách ẩn danh không được dùng key như quyền đọc lịch sử của người khác.

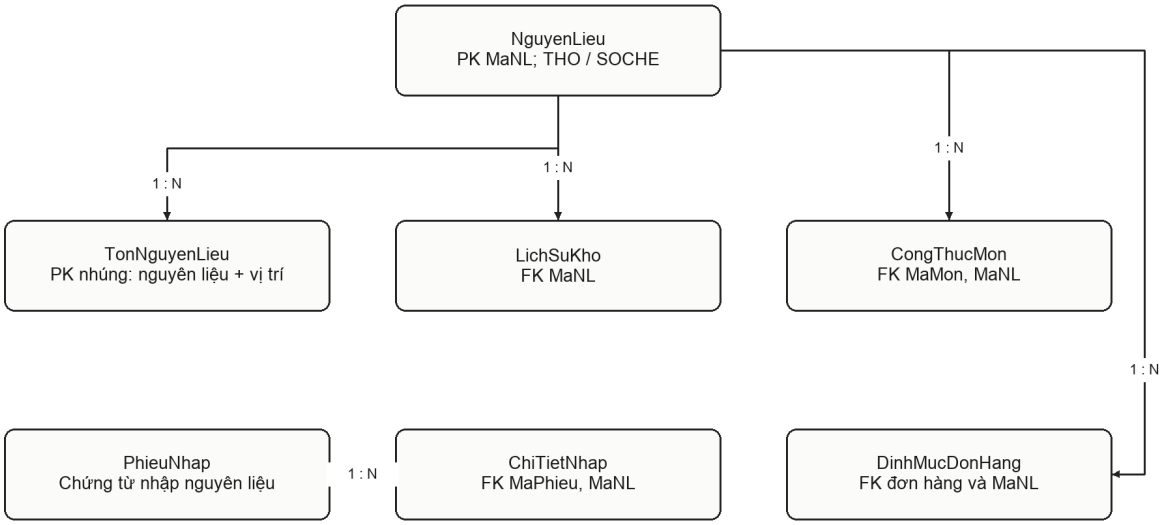
Transaction DB không đồng nhất với lời gọi SMTP/Cloudinary. MailService đăng ký gửi sau commit để tài khoản pending có thể gửi lại khi mail lỗi. ProductImageService có xử lý dọn ảnh mới khi rollback và xóa ảnh cũ sau commit theo nhánh source. Chưa có bằng chứng giao dịch thật của ngân hàng/Cloudinary trong phiên tạo báo cáo; chỉ xác nhận pipeline và các kiểm tra đã có trong code.

1.2.5.Mô hình dữ liệu và quan hệ chính

Tên bảng/cột sau đây lấy từ annotation JPA và SQL script của workspace. Khóa chính bảo vệ tính duy nhất của bản ghi; khóa ngoại ràng buộc quan hệ tham chiếu giữa các bảng [TL3]. Các ảnh là lược đồ quan hệ chọn lọc nhằm đọc hiểu; từ điển Entity phía sau giữ đầy đủ trường và nguồn để đối chiếu.



Hình 1.2. Các bảng và quan hệ cốt lõi của đơn, bàn và thanh toán



Hình 1.3. Các bảng kho và chứng từ; quan hệ chọn lọc từ JPA

Bảng 1.4. Nhóm dữ liệu và tính chất lưu trữ

Nhóm	Các Entity tiêu biểu	Ý nghĩa
Tài khoản/OTP	Account, EmailVerificationOtp, PasswordResetToken, AccountCart	Tách trạng thái active và emailVerified; vé/token lưu hash, giỏ có revision.
Menu	Product, Category, ProductRecipe	Biến thể theo tên/size/menuCode, giá theo Shop và công thức nguyên liệu.
Bàn	DiningTable, TableSession, TableSessionAccess, SessionCart	Lượt phục vụ OPEN/CLOSED và quyền truy cập scoped theo phiên.
Đơn/tiền	Order, OrderItem, OrderItemTopping, Invoice, Payment, PaymentNotice	Snapshot và trạng thái đơn độc lập khoản thu thực tế.
Kho	Material, Stock, StockMovement, ImportReceipt, PreparationBatch, ShiftSettlement	Nguyên liệu THO/SOCHE, vị trí KHO/BEP và lịch sử chứng từ.
Ngoại lệ	CancellationRequest, RefundRequest, BusinessAudit	Yêu cầu, quyết định, khoản hoàn và dấu vết xử lý.
Mở rộng/lịch sử	Shop, Voucher, DeliveryOrder, ProductReview, ReviewMedia	Phạm vi cơ sở; module giao hàng cũ; đánh giá liên kết bằng chứng mua.

1.2.6.Từ điển Entity đối chiếu bảng và cột

Bảng 1.5. Ánh xạ AccountCart → AccountCart

Nguồn: src/main/java/vn/edu/ute/milktea/entity/account/AccountCart.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
accountId	AccountId	Long	PK
account	AccountId	Account	Quan hệ JPA; N:1
itemsJson	ItemsJson	String	NOT NULL
revision	Revision	long	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.6. Ánh xạ DeliveryAddress → DeliveryAddress

Nguồn: src/main/java/vn/edu/ute/milktea/entity/account/DeliveryAddress.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Id	Long	PK
account	AccountId	Account	Quan hệ JPA; N:1
recipient	Recipient	String	NOT NULL; length=120
phone	Phone	String	NOT NULL; length=20
addressLine	AddressLine	String	NOT NULL; length=500
defaultAddress	IsDefault	boolean	NOT NULL

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
area	Area	String	length=120
latitude	Latitude	Double	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
longitude	Longitude	Double	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.7. Ánh xạ ProductInterest → ProductInterest

Nguồn: src/main/java/vn/edu/ute/milktea/entity/account/ProductInterest.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Id	Long	PK
account	AccountId	Account	Quan hệ JPA; N:1
product	ProductId	Product	Quan hệ JPA; N:1
favorite	Favorite	boolean	NOT NULL
lastViewedAt	LastViewedAt	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.8. Ánh xạ Account → TaiKhoan

Nguồn: src/main/java/vn/edu/ute/milktea/entity/account/Account.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaTK	Long	PK
email	Email	String	NOT NULL; UNIQUE; length=254
passwordHash	MatKhauBam	String	NOT NULL; length=255
fullName	HoTen	String	length=120
phone	DienThoai	String	length=20
role	VaiTro	Role	NOT NULL; length=16; Enum STRING
active	HoatDong	Boolean	NOT NULL
emailVerified	EmailXacThuc	Boolean	NOT NULL
tokenVersion	TokenVersion	Long	NOT NULL
createdAt	TaoLuc	Instant	NOT NULL
avatarUrl	AvatarUrl	String	length=1000
avatarPublicId	AvatarPublicId	String	length=255

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.9. Ánh xạ PasswordResetToken → TokenKhoiPhuc

Nguồn: src/main/java/vn/edu/ute/milktea/entity/account/PasswordResetToken.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaToken	Long	PK
account	MaTK	Account	Quan hệ JPA; NOT NULL; N:1
tokenHash	TokenHash	String	NOT NULL; UNIQUE; length=128
expiresAt	HetHanLuc	Instant	NOT NULL
usedAt	DaDungLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.10. Ánh xạ EmailVerificationOtp → XacThucEmail

Nguồn: src/main/java/vn/edu/ute/milktea/entity/account/EmailVerificationOtp.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaOTP	Long	PK
email	Email	String	NOT NULL; length=254
otpHash	OtpHash	String	NOT NULL; length=255
salt	Salt	String	NOT NULL; length=64
purpose	MucDich	String	NOT NULL; length=32
expiresAt	HetHanLuc	Instant	NOT NULL
attempts	SoLanThu	Integer	NOT NULL
maxAttempts	ToiDaThu	Integer	NOT NULL
usedAt	DaDungLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
revokedAt	VoHieuLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
lastSentAt	GuiCuoiLuc	Instant	NOT NULL
createdAt	TaoLuc	Instant	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.11. Ánh xạ BusinessAudit → NhatKyNghiepVu

Nguồn: src/main/java/vn/edu/ute/milktea/entity/audit/BusinessAudit.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaNK	Long	PK
parent	MaNKCha	BusinessAudit	Quan hệ JPA; N:1

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
sessionId	MaPhien	Long	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
orderId	MaDH	Long	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
accountId	MaTK	Long	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
action	HanhDong	String	NOT NULL; length=40
beforeState	Truoc	String	length=500
afterState	Sau	String	length=500
reason	LyDo	String	length=1000
dataJson	DuLieu	String	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
createdAt	TaoLuc	Instant	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.12. Ánh xạ RefundRequest → YeuCauHoan

Nguồn: src/main/java/vn/edu/ute/milktea/entity/cancellation/RefundRequest.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaHoan	Long	PK
order	MaDH	Order	Quan hệ JPA; NOT NULL; UNIQUE; 1:1
payment	MaTT	Payment	Quan hệ JPA; NOT NULL; UNIQUE; 1:1
amount	SoTien	BigDecimal	NOT NULL; precision=18; scale=2
reason	LyDo	String	NOT NULL; length=500
approvalReason	LyDoDuyet	String	length=500
status	TrangThai	RefundStatus	NOT NULL; length=16; Enum STRING
approvedBy	NguoiDuyet	Account	Quan hệ JPA; N:1
refundedBy	NguoiHoan	Account	Quan hệ JPA; N:1
approvedAt	DuyetLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
refundedAt	HoanLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
refundReference	ThamChieuHoan	String	length=120
idempotencyKey	MaYeuCau	String	NOT NULL; UNIQUE; length=64

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.13. Ánh xạ CancellationRequest → YeuCauHuy

Nguồn: src/main/java/vn/edu/ute/milktea/entity/cancellation/CancellationRequest.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaHuy	Long	PK
order	MaDH	Order	Quan hệ JPA; NOT NULL; N:1
reason	LyDo	String	NOT NULL; length=500
decisionReason	LyDoXuLy	String	length=500
status	TrangThai	CancellationStatus	NOT NULL; length=16; Enum STRING
createdAt	TaoLuc	Instant	NOT NULL
decidedAt	XuLyLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
decidedBy	NguoiXuLy	Account	Quan hệ JPA; N:1

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.14. Ánh xạ Category → DanhMuc

Nguồn: src/main/java/vn/edu/ute/milktea/entity/catalog/Category.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaDM	Long	PK
shop	ShopId	vn.edu.ute.milktea.entity.market.Shop	Quan hệ JPA; N:1
name	TenDM	String	NOT NULL; length=100
active	HoatDong	Boolean	NOT NULL
allowTopping	ChoPhepTopping	Boolean	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.15. Ánh xạ Product → MonUong

Nguồn: src/main/java/vn/edu/ute/milktea/entity/catalog/Product.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaMon	Long	PK
shop	ShopId	vn.edu.ute.milktea.entity.market.Shop	Quan hệ JPA; N:1
deliveryStock	DeliveryStock	Integer	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
catalogVersion	CatalogVersion	Long	NOT NULL; Optimistic version
category	MaDM	Category	Quan hệ JPA; NOT NULL; N:1
name	TenMon	String	NOT NULL; length=120

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
searchName	SearchName	String	length=240
menuCode	MenuCode	String	length=180
size	Size	String	NOT NULL; length=12
description	MoTa	String	length=1000
price	Gia	BigDecimal	NOT NULL; precision=18; scale=2
imageUrl	AnhUrl	String	length=1000
imagePublicId	AnhPublicId	String	length=255
active	HoatDong	Boolean	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.16. Ánh xạ RequestDedupRecord → BanGhiChongLap

Nguồn: src/main/java/vn/edu/ute/milktea/entity/common/RequestDedupRecord.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaBanGhi	Long	PK
scope	PhamVi	String	NOT NULL; length=128
idempotencyKey	KhoaYeuCau	String	NOT NULL; length=64
payloadFingerprint	DauVanPayload	String	length=128
resourceType	LoaiTaiNguyen	String	length=40
resourceId	MaTaiNguyen	Long	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
createdAt	TaoLuc	Instant	NOT NULL
expiresAt	HetHanLuc	Instant	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.17. Ánh xạ ShiftSettlementItem → ChiTietKetCaKho

Nguồn: src/main/java/vn/edu/ute/milktea/entity/inventory/ShiftSettlementItem.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaChiTiet	Long	PK
settlement	MaKetCa	ShiftSettlement	Quan hệ JPA; NOT NULL; N:1
material	MaNL	Material	Quan hệ JPA; NOT NULL; N:1
unit	DonVi	String	NOT NULL; length=16
bookQuantity	TonSo	BigDecimal	NOT NULL; precision=18; scale=3
actualQuantity	TonThucTe	BigDecimal	NOT NULL; precision=18; scale=3
discardedQuantity	LuongLoaiBo	BigDecimal	NOT NULL; precision=18; scale=3

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
retainedQuantity	LuongGiuLai	BigDecimal	NOT NULL; precision=18; scale=3
unexplainedDifference	ChenhLechKiemKe	BigDecimal	NOT NULL; precision=18; scale=3
unitCost	GiaVonDonVi	BigDecimal	NOT NULL; precision=18; scale=6
lossCost	ChiPhiHaoHut	BigDecimal	NOT NULL; precision=18; scale=2
reason	LyDo	String	length=500

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.18. Ánh xạ PreparationBatchItem → ChiTietMeSoChe

Nguồn: src/main/java/vn/edu/ute/milktea/entity/inventory/PreparationBatchItem.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Nhúng PreparationBatchItemId	PreparationBatchItemId	PK; Thành phần: MaMe, MaNL
batch	MaMe	PreparationBatch	Quan hệ JPA; NOT NULL; N:1
material	MaNL	Material	Quan hệ JPA; NOT NULL; N:1
recipeQuantity	LuongDinhMuc	BigDecimal	NOT NULL; precision=18; scale=3
actualQuantity	LuongThucDung	BigDecimal	NOT NULL; precision=18; scale=3
unitPrice	DonGiaVao	BigDecimal	precision=18; scale=4
totalCost	ThanhTien	BigDecimal	precision=18; scale=2

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.19. Ánh xạ ImportReceiptItem → ChiTietNhap

Nguồn: src/main/java/vn/edu/ute/milktea/entity/inventory/ImportReceiptItem.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Nhúng ImportReceiptItemId	ImportReceiptItemId	PK; Thành phần: MaPhieu, MaNL
receipt	MaPhieu	ImportReceipt	Quan hệ JPA; NOT NULL; N:1
material	MaNL	Material	Quan hệ JPA; NOT NULL; N:1
quantity	SoLuong	BigDecimal	NOT NULL; precision=18; scale=3
unitPrice	DonGiaNhap	BigDecimal	NOT NULL; precision=18; scale=2

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.20. Ánh xạ ShiftSettlement → KetCaKho

Nguồn: src/main/java/vn/edu/ute/milktea/entity/inventory/ShiftSettlement.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaKetCa	Long	PK
shiftCode	MaCa	String	length=64
location	ViTri	StockLocation	NOT NULL; length=8; Enum STRING
closedBy	NguoiChot	Account	Quan hệ JPA; NOT NULL; N:1
closedAt	ChotLuc	Instant	NOT NULL
fromTime	TuLuc	Instant	NOT NULL
toTime	DenLuc	Instant	NOT NULL
totalLossCost	TongChiPhiHaoHut	BigDecimal	NOT NULL; precision=18; scale=2
notes	GhiChu	String	length=500
idempotencyKey	MaYeuCau	String	UNIQUE; length=64
items	Quan hệ tập hợp	List<ShiftSettlementItem>	Quan hệ JPA; 1:N; không là cột scalar

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.21. Ánh xạ StockMovement → LichSuKho

Nguồn: src/main/java/vn/edu/ute/milktea/entity/inventory/StockMovement.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaBienDong	Long	PK
material	MaNL	Material	Quan hệ JPA; NOT NULL; N:1
location	ViTri	StockLocation	NOT NULL; length=8; Enum STRING
type	Loai	StockMovementType	NOT NULL; length=16; Enum STRING
deltaQuantity	LuongBienDong	BigDecimal	NOT NULL; precision=18; scale=3
beforeQuantity	TonTruoc	BigDecimal	NOT NULL; precision=18; scale=3
afterQuantity	TonSau	BigDecimal	NOT NULL; precision=18; scale=3
sourceType	NguonLoai	String	NOT NULL; length=16
sourceId	NguonMa	Long	NOT NULL
sourceLine	NguonDong	Long	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
createdBy	NguoiLap	Account	Quan hệ JPA; NOT NULL; N:1
createdAt	TaoLuc	Instant	NOT NULL

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
reason	LyDo	String	length=500
unitCost	GiaVonDonVi	BigDecimal	precision=18; scale=6
lossCost	ChiPhiHaoHut	BigDecimal	precision=18; scale=2

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.22. Ánh xạ PreparationBatch → MeSoChe

Nguồn: src/main/java/vn/edu/ute/milktea/entity/inventory/PreparationBatch.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaMe	Long	PK
recipe	MaCTSC	PreparationRecipe	Quan hệ JPA; NOT NULL; N:1
outputMaterial	MaNLDauRa	Material	Quan hệ JPA; NOT NULL; N:1
expectedQuantity	LuongDuKien	BigDecimal	NOT NULL; precision=18; scale=3
actualQuantity	LuongThucThu	BigDecimal	NOT NULL; precision=18; scale=3
status	TrangThai	DocumentStatus	NOT NULL; length=12; Enum STRING
discrepancyReason	LyDoChenhLech	String	length=500
totalCost	TongChiPhi	BigDecimal	precision=18; scale=2
unitCost	GiaVonBinhQuan	BigDecimal	precision=18; scale=6
ratioVariancePercent	TyLeChenhLech	BigDecimal	precision=7; scale=2
toleranceWarning	CanhBaoTyLe	Boolean	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
createdBy	NguoiLap	Account	Quan hệ JPA; NOT NULL; N:1
postedAt	GhiSoLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
idempotencyKey	MaYeuCau	String	UNIQUE; length=64

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.23. Ánh xạ Material → NguyenLieu

Nguồn: src/main/java/vn/edu/ute/milktea/entity/inventory/Material.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaNL	Long	PK
name	TenNL	String	NOT NULL; UNIQUE; length=120
type	Loai	MaterialType	NOT NULL; length=8; Enum STRING
unit	DonVi	String	NOT NULL; length=16

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
active	HoatDong	Boolean	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.24. Ánh xạ ImportReceipt → PhieuNhap

Nguồn: src/main/java/vn/edu/ute/milktea/entity/inventory/ImportReceipt.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaPhieu	Long	PK
createdBy	NguoiLap	Account	Quan hệ JPA; NOT NULL; N:1
reason	LyDo	String	length=500
supplier	NguonNhap	String	length=500
status	TrangThai	DocumentStatus	NOT NULL; length=12; Enum STRING
createdAt	LapLuc	Instant	NOT NULL
postedAt	GhiSoLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
idempotencyKey	MaYeuCau	String	UNIQUE; length=64

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.25. Ánh xạ Stock → TonNguyenLieu

Nguồn: src/main/java/vn/edu/ute/milktea/entity/inventory/Stock.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Nhúng StockId	StockId	PK; Thành phần: MaNL, ViTri
material	MaNL	Material	Quan hệ JPA; NOT NULL; N:1
quantity	SoLuong	BigDecimal	NOT NULL; precision=18; scale=3
threshold	NguongCanhBao	BigDecimal	precision=18; scale=3
costPrice	GiaVonDonVi	BigDecimal	precision=18; scale=6
version	Version	Long	NOT NULL; Optimistic version

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.26. Ánh xạ Carrier → Carrier

Nguồn: src/main/java/vn/edu/ute/milktea/entity/market/Carrier.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
shop	ShopId	Shop	Quan hệ JPA; N:1
id	Id	Long	PK

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
name	Name	String	NOT NULL; length=120
fee	Fee	java.math.BigDecimal	NOT NULL; precision=18; scale=2
active	Active	boolean	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.27. Ánh xạ DeliveryLine → DeliveryLine

Nguồn: src/main/java/vn/edu/ute/milktea/entity/market/DeliveryLine.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Id	Long	PK
order	OrderId	DeliveryOrder	Quan hệ JPA; N:1
product	ProductId	Product	Quan hệ JPA; N:1
productName	ProductName	String	NOT NULL; length=120
quantity	Quantity	int	NOT NULL
unitPrice	UnitPrice	java.math.BigDecimal	NOT NULL; precision=18; scale=2
snapshotSize	SnapshotSize	String	length=12
note	Note	String	length=500
topping	IsTopping	boolean	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.28. Ánh xạ DeliveryOrder → DeliveryOrder

Nguồn: src/main/java/vn/edu/ute/milktea/entity/market/DeliveryOrder.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Id	Long	PK
account	AccountId	Account	Quan hệ JPA; N:1
shop	ShopId	Shop	Quan hệ JPA; N:1
shipper	ShipperId	Account	Quan hệ JPA; N:1
carrier	CarrierId	Carrier	Quan hệ JPA; N:1
voucher	VoucherId	Voucher	Quan hệ JPA; N:1
recipient	Recipient	String	NOT NULL; length=120
phone	Phone	String	NOT NULL; length=20
addressLine	AddressLine	String	NOT NULL; length=500
subtotal	Subtotal	java.math.BigDecimal	NOT NULL; precision=18; scale=2
shippingFee	ShippingFee	java.math.BigDecimal	NOT NULL; precision=18; scale=2
discountAmount	DiscountAmount	java.math.BigDecimal	NOT NULL; precision=18; scale=2

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
totalAmount	TotalAmount	java.math.BigDecimal	NOT NULL; precision=18; scale=2
commissionPercent	CommissionPercent	java.math.BigDecimal	NOT NULL; precision=5; scale=2
status	Status	Status	NOT NULL; length=24; Enum STRING
paymentMethod	PaymentMethod	Method	NOT NULL; length=16; Enum STRING
paid	Paid	boolean	NOT NULL
gatewayReference	GatewayReference	String	length=120
refundReference	RefundReference	String	length=120
idempotencyKey	IdempotencyKey	String	NOT NULL; length=64
requestHash	RequestHash	String	NOT NULL; length=64
createdAt	CreatedAt	java.time.Instant	NOT NULL
deliveredAt	DeliveredAt	java.time.Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
deliveryFailed	DeliveryFailed	boolean	NOT NULL
version	Version	Long	NOT NULL; Optimistic version
lines	Quan hệ tập hợp	java.util.List<DeliveryLine>	Quan hệ JPA; 1:N; không là cột scalar

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.29. Ánh xạ ProductReview → ProductReview

Nguồn: src/main/java/vn/edu/ute/milktea/entity/market/ProductReview.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Id	Long	PK
account	AccountId	Account	Quan hệ JPA; N:1
product	ProductId	Product	Quan hệ JPA; N:1
deliveryOrder	DeliveryOrderId	DeliveryOrder	Quan hệ JPA; N:1
counterOrder	CounterOrderId	vn.edu.ute.milktea.entity.order.Order	Quan hệ JPA; N:1
stars	Stars	int	NOT NULL
comment	Comment	String	NOT NULL; length=3000
createdAt	CreatedAt	java.time.Instant	NOT NULL
media	Quan hệ tập hợp	java.util.List<ReviewMedia>	Quan hệ JPA; 1:N; không là cột scalar

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.30. Ánh xạ ReviewMedia → ReviewMedia

Nguồn: src/main/java/vn/edu/ute/milktea/entity/market/ReviewMedia.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Id	Long	PK
review	ReviewId	ProductReview	Quan hệ JPA; N:1
mediaType	MediaType	String	NOT NULL; length=12
url	Url	String	NOT NULL; length=1000
publicId	PublicId	String	NOT NULL; length=255

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.31. Ánh xạ Shop → Shop

Nguồn: src/main/java/vn/edu/ute/milktea/entity/market/Shop.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Id	Long	PK
name	Name	String	NOT NULL; length=120
owner	OwnerId	Account	Quan hệ JPA; N:1
managers	ShopId	java.util.Set<Account>	Quan hệ JPA; N:N qua bảng nối
active	Active	boolean	NOT NULL
managerCommissionEnabled	ManagerCommissionEnabled	boolean	NOT NULL
commissionPercent	CommissionPercent	java.math.BigDecimal	NOT NULL; precision=5; scale=2
address	Address	String	NOT NULL; length=500
area	Area	String	length=120
latitude	Latitude	Double	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
longitude	Longitude	Double	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
createdAt	CreatedAt	java.time.Instant	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.32. Ánh xạ ShopCustomerAccess → ShopCustomerAccess

Nguồn: src/main/java/vn/edu/ute/milktea/entity/market/ShopCustomerAccess.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Id	Long	PK
shop	ShopId	Shop	Quan hệ JPA; N:1
account	AccountId	Account	Quan hệ JPA; N:1
blocked	Blocked	boolean	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.33. Ánh xạ Voucher → Voucher

Nguồn: src/main/java/vn/edu/ute/milktea/entity/market/Voucher.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Id	Long	PK
code	Code	String	NOT NULL; UNIQUE; length=32
shop	ShopId	Shop	Quan hệ JPA; N:1
discountAmount	DiscountAmount	java.math.BigDecimal	NOT NULL; precision=18; scale=2
minimumAmount	MinimumAmount	java.math.BigDecimal	NOT NULL; precision=18; scale=2
startsAt	StartsAt	java.time.Instant	NOT NULL
endsAt	EndsAt	java.time.Instant	NOT NULL
usageLimit	UsageLimit	int	NOT NULL
usedCount	UsedCount	int	NOT NULL
perAccountLimit	PerAccountLimit	int	NOT NULL
active	Active	boolean	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.34. Ánh xạ OrderItem → ChiTietDonHang

Nguồn: src/main/java/vn/edu/ute/milktea/entity/order/OrderItem.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaChiTiet	Long	PK
order	MaDH	Order	Quan hệ JPA; NOT NULL; N:1
product	MaMon	Product	Quan hệ JPA; NOT NULL; N:1
snapshotName	TenMonChot	String	NOT NULL; length=120
snapshotSize	SizeChot	String	NOT NULL; length=12
quantity	SoLuong	Integer	NOT NULL
snapshotUnitPrice	DonGiaChot	BigDecimal	NOT NULL; precision=18; scale=2
note	GhiChu	String	length=500
toppings	Quan hệ tập hợp	java.util.List<OrderItemTopping>	Quan hệ JPA; 1:N; không là cột scalar

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.35. Ánh xạ OrderItemTopping → ChiTietDonHangTopping

Nguồn: src/main/java/vn/edu/ute/milktea/entity/order/OrderItemTopping.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaChiTietTopping	Long	PK

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
orderItem	MaChiTiet	OrderItem	Quan hệ JPA; NOT NULL; N:1
toppingProduct	MaMonTopping	Product	Quan hệ JPA; NOT NULL; N:1
snapshotName	TenToppingChot	String	NOT NULL; length=120
snapshotUnitPrice	DonGiaToppingChot	BigDecimal	NOT NULL; precision=18; scale=2

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.36. Ánh xạ OrderIngredientSnapshot → DinhMucDonHang

Nguồn: src/main/java/vn/edu/ute/milktea/entity/order/OrderIngredientSnapshot.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Nhúng OrderIngredientSnapshotId	OrderIngredientSnapshotId	PK; Thành phần: MaDH, MaNL
order	MaDH	Order	Quan hệ JPA; NOT NULL; N:1
material	MaNL	Material	Quan hệ JPA; NOT NULL; N:1
totalQuantity	TongLuong	BigDecimal	NOT NULL; precision=18; scale=3
snapshotType	LoaiChot	MaterialType	NOT NULL; length=8; Enum STRING
snapshotUnit	DonViChot	String	NOT NULL; length=16

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.37. Ánh xạ Order → DonHang

Nguồn: src/main/java/vn/edu/ute/milktea/entity/order/Order.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaDH	Long	PK
session	MaPhien	TableSession	Quan hệ JPA; N:1
account	MaTK	Account	Quan hệ JPA; N:1
source	Nguon	OrderSource	NOT NULL; length=10; Enum STRING
status	TrangThai	OrderStatus	NOT NULL; length=24; Enum STRING
replacementOfOrder	ThayTheMaDH	Order	Quan hệ JPA; N:1
idempotencyKey	MaYeuCau	String	length=64
shop	ShopId	vn.edu.ute.milktea.entity.market.Shop	Quan hệ JPA; N:1
requestedPaymentMethod	RequestedPaymentMethod	vn.edu.ute.milktea.entity.payment.PaymentMethod	length=20; Enum STRING
voucher	VoucherId	vn.edu.ute.milktea.entity.market.Voucher	Quan hệ JPA; N:1

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
requestFingerprint	RequestFingerprint	String	length=64
note	GhiChu	String	length=500
createdAt	TaoLuc	Instant	NOT NULL
startedAt	BatDauLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
completedAt	HoanThanhLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
version	Version	Long	NOT NULL; Optimistic version

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.38. Ánh xạ Invoice → HoaDon

Nguồn: src/main/java/vn/edu/ute/milktea/entity/order/Invoice.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaHD	Long	PK
order	MaDH	Order	Quan hệ JPA; NOT NULL; UNIQUE; 1:1
subtotal	TongTien	BigDecimal	NOT NULL; precision=18; scale=2
discountAmount	TienGiam	BigDecimal	NOT NULL; precision=18; scale=2
totalAmount	PhaiTra	BigDecimal	NOT NULL; precision=18; scale=2
snapshotDiscountPercent	TyLeGiamChot	BigDecimal	NOT NULL; precision=5; scale=2
snapshotDiscountType	LoaiGiamChot	String	NOT NULL; length=20
snapshotDiscountValue	GiaTriGiamChot	BigDecimal	NOT NULL; precision=18; scale=2
status	TrangThai	InvoiceStatus	NOT NULL; length=12; Enum STRING
createdAt	LapLuc	Instant	NOT NULL
cancelledAt	HuyLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.39. Ánh xạ CounterOrderAccess → TruyCapDonQuay

Nguồn: src/main/java/vn/edu/ute/milktea/entity/order/CounterOrderAccess.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaTruyCap	Long	PK
order	MaDH	Order	Quan hệ JPA; NOT NULL; N:1

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
tokenHash	TokenHash	String	NOT NULL; UNIQUE; length=128
expiresAt	HetHanLuc	Instant	NOT NULL
revokedAt	ThuHoiLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.40. Ánh xạ Payment → ThanhToan

Nguồn: src/main/java/vn/edu/ute/milktea/entity/payment/Payment.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaTT	Long	PK
invoice	MaHD	Invoice	Quan hệ JPA; NOT NULL; UNIQUE; 1:1
method	PhuongThuc	PaymentMethod	NOT NULL; length=16; Enum STRING
amount	SoTien	BigDecimal	NOT NULL; precision=18; scale=2
reference	ThamChieu	String	length=120
recordedBy	NguoiThu	Account	Quan hệ JPA; NOT NULL; N:1
paidAt	ThuLuc	Instant	NOT NULL
idempotencyKey	MaYeuCau	String	length=64

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.41. Ánh xạ PaymentNotice → ThongBaoThanhToan

Nguồn: src/main/java/vn/edu/ute/milktea/entity/payment/PaymentNotice.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaTB	Long	PK
invoice	MaHD	Invoice	Quan hệ JPA; NOT NULL; N:1
method	PhuongThuc	PaymentMethod	NOT NULL; length=16; Enum STRING
reference	ThamChieu	String	length=120
createdAt	TaoLuc	Instant	NOT NULL
idempotencyKey	MaYeuCau	String	length=64

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.42. Ánh xạ PreparationRecipeItem → ChiTietCTSoChe

Nguồn: src/main/java/vn/edu/ute/milktea/entity/recipe/PreparationRecipeItem.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Nhúng PreparationRecipeItemId	PreparationRecipeItemId	PK; Thành phần: MaCTSC, MaNL
recipe	MaCTSC	PreparationRecipe	Quan hệ JPA; NOT NULL; N:1
inputMaterial	MaNL	Material	Quan hệ JPA; NOT NULL; N:1
standardInputQuantity	LuongDauVaoChuan	BigDecimal	NOT NULL; precision=18; scale=3

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.43. Ánh xạ ProductRecipe → CongThucMon

Nguồn: src/main/java/vn/edu/ute/milktea/entity/recipe/ProductRecipe.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	Nhúng ProductRecipeId	ProductRecipeId	PK; Thành phần: MaMon, MaNL
product	MaMon	Product	Quan hệ JPA; NOT NULL; N:1
material	MaNL	Material	Quan hệ JPA; NOT NULL; N:1
quantity	DinhLuong	BigDecimal	NOT NULL; precision=18; scale=3

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.44. Ánh xạ PreparationRecipe → CongThucSoChe

Nguồn: src/main/java/vn/edu/ute/milktea/entity/recipe/PreparationRecipe.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaCTSC	Long	PK
name	TenCT	String	NOT NULL; length=120
outputMaterial	MaNLDauRa	Material	Quan hệ JPA; NOT NULL; N:1
standardOutputQuantity	LuongDauRaChuan	BigDecimal	NOT NULL; precision=18; scale=3
tolerancePercent	NguongCanhBaoTyLe	BigDecimal	precision=5; scale=2
active	HoatDong	Boolean	NOT NULL
note	GhiChu	String	length=500
version	Version	Long	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.45. Ánh xạ GlobalSettings → CauHinhChung

Nguồn: src/main/java/vn/edu/ute/milktea/entity/settings/GlobalSettings.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaCauHinh	Integer	PK
shopName	TenQuan	String	NOT NULL; length=120
address	DiaChi	String	NOT NULL; length=255
logoUrl	LogoUrl	String	length=1000
logoPublicId	LogoPublicId	String	length=255
bankName	NganHang	String	length=120
bankAccountNumber	SoTaiKhoan	String	length=120
bankAccountHolder	ChuTK	String	length=120
discountPercent	TyLeGiam	BigDecimal	NOT NULL; precision=5; scale=2
discountType	LoaiGiam	String	NOT NULL; length=20
discountValue	GiaTriGiam	BigDecimal	NOT NULL; precision=18; scale=2
modifiedBy	NguoiSua	Account	Quan hệ JPA; N:1
modifiedAt	SuaLuc	Instant	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.46. Ánh xạ DiningTable → Ban

Nguồn: src/main/java/vn/edu/ute/milktea/entity/table/DiningTable.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaBan	Long	PK
name	TenBan	String	NOT NULL; UNIQUE; length=40
shop	ShopId	vn.edu.ute.milktea.entity.market.Shop	Quan hệ JPA; N:1
qrCode	MaQR	String	NOT NULL; UNIQUE; length=64
status	TrangThai	TableStatus	NOT NULL; length=16; Enum STRING
activeSessionId	MaPhienDangMo	Long	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
active	HoatDong	Boolean	NOT NULL
version	Version	Long	NOT NULL; Optimistic version

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.47. Ánh xạ SessionCartItem → ChiTietGioPhien

Nguồn: src/main/java/vn/edu/ute/milktea/entity/table/SessionCartItem.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaDong	Long	PK

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
cart	MaGio	SessionCart	Quan hệ JPA; NOT NULL; N:1
product	MaMon	Product	Quan hệ JPA; NOT NULL; N:1
quantity	SoLuong	Integer	NOT NULL
note	GhiChu	String	length=500
toppingIds	ToppingIds	String	length=255

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.48. Ánh xạ SessionCart → GioPhien

Nguồn: src/main/java/vn/edu/ute/milktea/entity/table/SessionCart.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaGio	Long	PK
session	MaPhien	TableSession	Quan hệ JPA; NOT NULL; UNIQUE; 1:1
version	Version	Long	NOT NULL; Optimistic version
modifiedAt	SuaLuc	Instant	NOT NULL

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.49. Ánh xạ TableSession → PhienBan

Nguồn: src/main/java/vn/edu/ute/milktea/entity/table/TableSession.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaPhien	Long	PK
table	MaBan	DiningTable	Quan hệ JPA; NOT NULL; N:1
status	TrangThai	TableSessionStatus	NOT NULL; length=8; Enum STRING
openedAt	MoLuc	Instant	NOT NULL
closedAt	DongLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung
openedBy	NguoiMo	Account	Quan hệ JPA; N:1
closedBy	NguoiDong	Account	Quan hệ JPA; N:1
version	Version	Long	NOT NULL; Optimistic version

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Bảng 1.50. Ánh xạ TableSessionAccess → TruyCapPhienBan

Nguồn: src/main/java/vn/edu/ute/milktea/entity/table/TableSessionAccess.java.

Thuộc tính Java	Cột/ánh xạ	Kiểu Java	Ràng buộc/quan hệ
id	MaTruyCap	Long	PK
session	MaPhien	TableSession	Quan hệ JPA; NOT NULL; N:1
tokenHash	TokenHash	String	NOT NULL; UNIQUE; length=128
expiresAt	HetHanLuc	Instant	NOT NULL
revokedAt	ThuHoiLuc	Instant	Theo mặc định JPA/ annotation; không suy diễn ràng buộc bổ sung

Các cột FK thuộc @JoinColumn; quan hệ dạng tập hợp được trình bày là ánh xạ, không coi tên collection là cột vật lý. Khóa nhúng được đối chiếu thêm với lớp Id tương ứng.

Shop.managers dùng bảng nối ShopManager với ShopId/AccountId, không có một Entity độc lập tên ShopManager. Các lớp Id/enum không phải bảng riêng. SQL script cũng có các ràng buộc/index và dữ liệu lịch sử; vì vậy số class Entity không được đồng nhất một cách máy móc với toàn bộ đối tượng trong database triển khai.

1.2.7.Luồng trạng thái nghiệp vụ và giới hạn tích hợp

Bảng 1.51. Trạng thái và bằng chứng trong mã hiện tại

Nội dung	Luồng/điều kiện	Bằng chứng
Đơn khách	CHO_XAC_NHAN → CHO_CHE_BIEN → DANG_CHE_BIEN → HOAN_THANH; nhánh DA_HUY theo điều kiện.	OrderStatus; OrderService; CancellationService
Đơn POS	Nhân viên tạo CHO_THANH_TOAN; ghi thu/ điều phối cập nhật theo logic hiện có.	OrderService.createCounterOrder ; PaymentService.recordReceipt
Thanh toán	CASH/BANK_TRANSFER; chưa có Payment tương ứng thì response PENDING_PAYMENT. Notice không làm PAID.	PaymentMethod; PaymentRepository; mapToOrderResponse
Phiên bàn	Xem QR không mở phiên; đơn đầu hoặc thu ngân mở; chỉ đóng thủ công khi điều kiện thỏa.	TableContextService; TableSessionService
Ngân hàng	Thông tin cấu hình quản trị; VietQR theo số tiền/nội dung; thu ngân xác minh khoản thu.	GlobalSettings; OrderService.vietQr; PaymentService
VNPAY	Có chữ ký/IPN ở module DeliveryOrder cũ; không nằm trong dialog checkout mới.	VnpayService; VnpayController; MarketController.checkout
Cloudinary/SMTP	Có code tích hợp và xử lý lỗi; cấu hình bên ngoài quyết định khả năng dùng thực.	MailService; ProductImageService; ReviewService

1.2.8.Phạm vi kiểm chứng và tài liệu nguồn

Phiên biên soạn đã chạy một demo H2 riêng ở cổng 18081 để chụp 27 giao diện và thực hiện các lệnh mẫu tạo đơn, ghi thu, xác nhận và chế biến. SMTP bị vô hiệu trong demo chụp ảnh; .env và dữ liệu đang vận hành không được đọc để đưa vào báo cáo. Các ảnh là ảnh thật của ứng dụng với dữ liệu mẫu, không phải ảnh do AI vẽ.

Các báo cáo Surefire hiện có trong target/surefire-reports được dùng như bằng chứng kiểm thử đã lưu; không tự khẳng định mọi chức năng đã được E2E. Chưa xác minh SQL Server thật, giao dịch ngân hàng, email tới hộp thư thật hay upload Cloudinary trong phiên biên soạn. Những điều kiện này được nêu tại từng nhóm chức năng và bảng kiểm tra kèm theo.

[TL1] Spring Boot, tài liệu chính thức: https://docs.spring.io/spring-boot/index.html. Truy cập ngày 27/10/2026; dùng cho khái niệm cấu hình/chạy ứng dụng, phiên bản đồ án vẫn lấy từ pom.xml.

[TL2] Spring Security, Servlet Architecture: https://docs.spring.io/spring-security/reference/servlet/architecture.html. Truy cập ngày 27/10/2026; tham khảo mô hình filter chain.

[TL3] Microsoft Learn, Primary and foreign key constraints: https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints. Truy cập ngày 27/10/2026.

[TL4] Mã nguồn UTE-Tea: src/main/java, src/main/resources, database/sqlserver, tests và src/test/java. Các trích đoạn trong Chương 2 có đường dẫn và số dòng cụ thể.

[TL5] Tài liệu bổ trợ của dự án: skills/domain_system.md, skills/workflow_system.md, skills/system_architecture.md, docs/CHECKOUT_PAYMENT_FIX.md và docs/STARTUP_FIX.md; các quyết định mới được đối chiếu lại với source hiện tại.

2.SƠ ĐỒ HOẠT ĐỘNG CÁC CHỨC NĂNG

Chương trình bày theo thứ tự tác nhân → chức năng → mô tả → sơ đồ hoạt động → mã xử lý → giải thích, giữ cấu trúc của tài liệu tham khảo. Có 73 nhóm chức năng được đối chiếu. Chỉ những nhánh có mã nguồn mới được mô tả; các module tương thích được đánh dấu riêng. Sơ đồ rút gọn các kiểm tra liên quan thành nút quyết định đọc hiểu, không bổ sung một bước nghiệp vụ chưa tồn tại.

2.1.Khách hàng

2.1.1.Đăng ký tài khoản chờ xác thực

2.1.1.1.Mô tả chức năng

Bảng 2.1. Đặc tả: Đăng ký tài khoản chờ xác thực

Tiêu chí	Nội dung
Mục đích	Tạo hồ sơ khách hàng có email chưa xác thực, chuẩn bị OTP để kích hoạt tài khoản.
Tác nhân	Khách hàng
Điều kiện trước	Chưa đăng nhập; thông tin phải thỏa AuthDto.RegisterRequest; cấu hình OTP/SMTP hợp lệ.
Đầu vào	Email, mật khẩu, họ tên, số điện thoại 10 chữ số.
Kết quả / điều kiện sau	Không cấp JWT trước xác thực. SMTP thất bại vẫn giữ tài khoản pending sau commit để có thể gửi lại.
Trường hợp lỗi	Email đã dùng, tài khoản bị khóa, DTO sai, thiếu khóa OTP hoặc SMTP lỗi.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Chuẩn hóa email và khóa tài khoản trùng email.

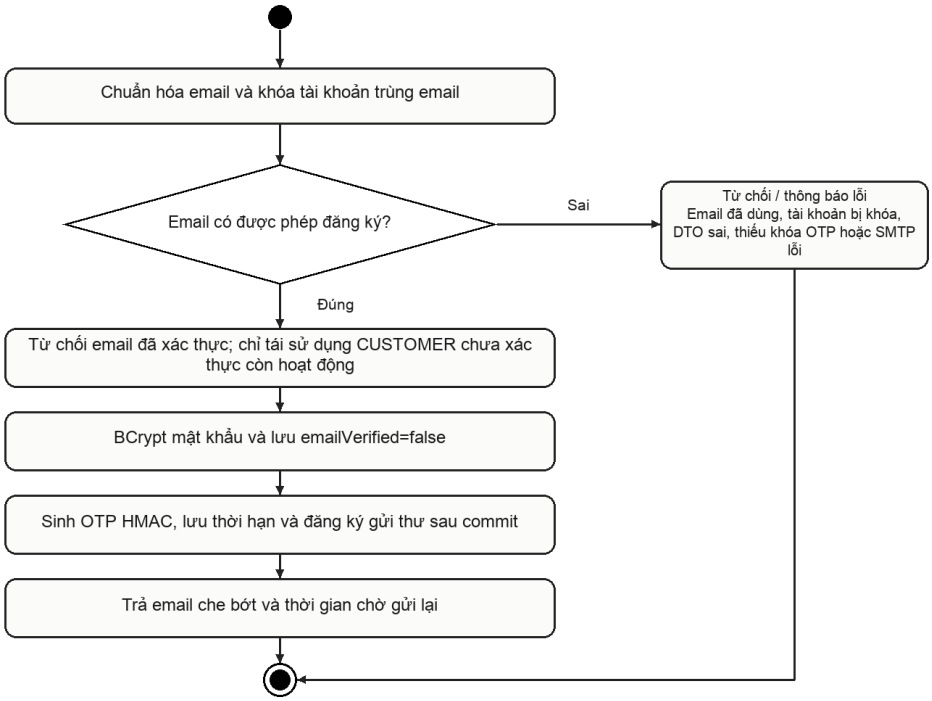
Bước 2. Từ chối email đã xác thực; chỉ tái sử dụng CUSTOMER chưa xác thực còn hoạt động.

Bước 3. BCrypt mật khẩu và lưu emailVerified=false.

Bước 4. Sinh OTP HMAC, lưu thời hạn và đăng ký gửi thư sau commit.

Bước 5. Trả email che bớt và thời gian chờ gửi lại.

2.1.1.2.Sơ đồ hoạt động



Hình 2.1. Hoạt động: Đăng ký tài khoản chờ xác thực

2.1.1.3.Code xử lý chức năng

Đoạn mã 2.1. AuthService.register — Đăng ký tài khoản chờ xác thực

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.register; dòng 86–137.

```
    public AuthDto.RegisterResponse register(AuthDto.RegisterRequest request) {
        String email = request.getEmail().trim().toLowerCase();
                                        Optional<Account>     existingOpt     =
accountRepository.findByEmailForUpdate(email);

        Account account;
        if (existingOpt.isPresent()) {
            Account existing = existingOpt.get();
            if (Boolean.TRUE.equals(existing.getEmailVerified())) {
                    throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED,
"Địa chỉ email này đã được sử dụng");
            }
                                if   (existing.getRole()   !=   Role.CUSTOMER   ||   !
Boolean.TRUE.equals(existing.getActive())) {
                    throw BusinessException.forbidden(ErrorCode.ACCESS_DENIED, "Tài
khoản không thể đăng ký lại qua biểu mẫu công khai");
            }
```

[Đã lược bỏ các dòng 99–100 của phương thức gốc.]

```
            if (request.getFullName() != null && !request.getFullName().isBlank())
{
                existing.setFullName(request.getFullName().trim());
            }
            if (request.getPhone() != null && !request.getPhone().isBlank()) {
                existing.setPhone(request.getPhone().trim());
            }
```

```
            account = accountRepository.save(existing);
        } else {
            account = Account.builder()
                    .email(email)
                    .passwordHash(passwordEncoder.encode(request.getPassword()))
```

[Đã lược bỏ các dòng 112–125 của phương thức gốc.]

```
        // Gửi email thật qua SMTP
                mailService.sendRegistrationOtpEmail(email,  account.getFullName(),
generated.rawOtp(), 10);

        return AuthDto.RegisterResponse.builder()
                .email(email)
                .maskedEmail(MailService.maskEmail(email))
                  .message("Tài khoản đang chờ kích hoạt. Mã xác thực 6 chữ số đã
được gửi tới email của bạn. Vui lòng nhập mã để kích hoạt tài khoản.")
                .emailVerified(false)
                .cooldownSeconds(60)
                .build();
    }
```

2.1.1.4.Giải thích code

Role CUSTOMER được gán ở server; client không được tự nâng quyền. Mật khẩu và OTP thô không trả trong response.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findByEmailForUpdate(); passwordEncoder.encode(); accountRepository.save(); otpService.generateAndSaveOtp(). Đường dẫn giao diện liên quan: src/main/resources/templates/auth/login.html.

2.1.2.Xác thực email bằng OTP

2.1.2.1.Mô tả chức năng

Bảng 2.2. Đặc tả: Xác thực email bằng OTP

Tiêu chí	Nội dung
Mục đích	Kích hoạt email sau khi người dùng chứng minh quyền sở hữu hộp thư.
Tác nhân	Khách hàng
Điều kiện trước	Tài khoản đã được tạo; mã chưa dùng, chưa thu hồi và còn thời hạn.
Đầu vào	Email và OTP gồm đúng 6 chữ số.
Kết quả / điều kiện sau	OTP đúng được đánh dấu usedAt; trạng thái active do quản trị khóa không tự mở lại.
Trường hợp lỗi	Mã sai, hết hạn, bị khóa sau giới hạn thử hoặc không có tài khoản.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm tài khoản theo email chuẩn hóa.

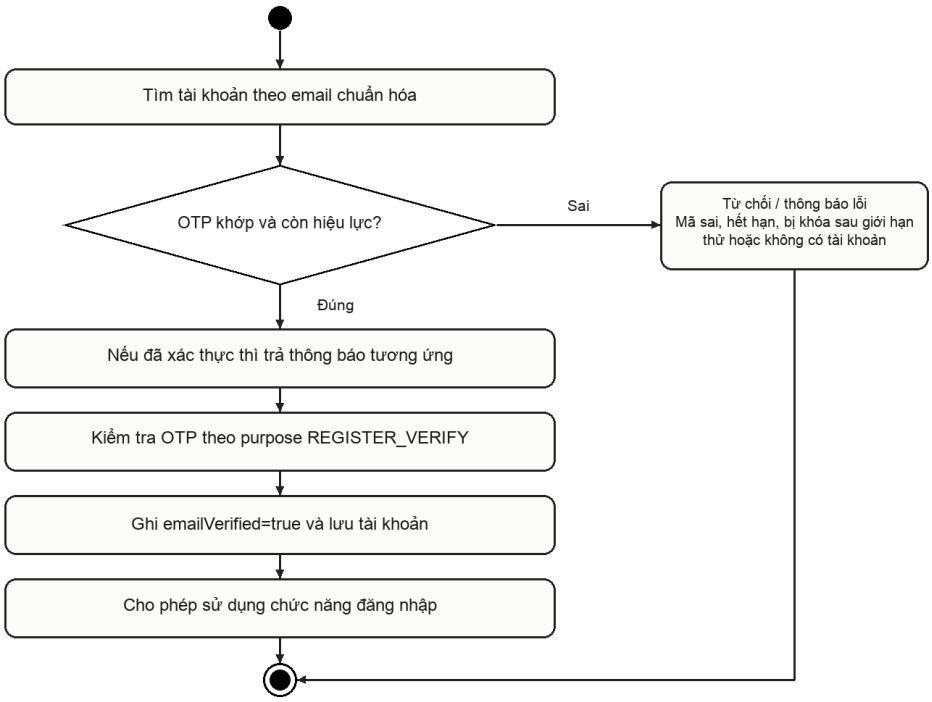
Bước 2. Nếu đã xác thực thì trả thông báo tương ứng.

Bước 3. Kiểm tra OTP theo purpose REGISTER_VERIFY.

Bước 4. Ghi emailVerified=true và lưu tài khoản.

Bước 5. Cho phép sử dụng chức năng đăng nhập.

2.1.2.2.Sơ đồ hoạt động



Hình 2.2. Hoạt động: Xác thực email bằng OTP

2.1.2.3.Code xử lý chức năng

Đoạn mã 2.2. AuthService.verifyEmail — Xác thực email bằng OTP

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.verifyEmail; dòng 143–161.

```
    public String verifyEmail(AuthDto.VerifyEmailRequest request) {
        String email = request.getEmail().trim().toLowerCase();
        Account account = accountRepository.findByEmail(email)
                                                    .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED, "Không tìm thấy tài khoản
với email này"));

        if (Boolean.TRUE.equals(account.getEmailVerified())) {
             return "Tài khoản của bạn đã được xác thực trước đó. Bạn có thể đăng
nhập ngay.";
        }

        // Xác thực mã OTP (kiểm tra hạn dùng, băm HMAC, số lần thử)
                otpService.verifyOtp(email,  EmailVerificationOtp.PURPOSE_REGISTER,
request.getOtp());

        // Cập nhật trạng thái xác thực email
```

```
          // LƯU Ý: Không tự mở khóa nếu tài khoản đang bị admin khóa (active =
false)
        account.setEmailVerified(true);
        accountRepository.save(account);

         return "Xác thực email thành công! Bạn có thể đăng nhập vào hệ thống ngay
bây giờ.";
    }
```

2.1.2.4.Giải thích code

noRollbackFor với lỗi nghiệp vụ giữ số lần nhập sai/thu hồi mã trong DB; không suy diễn mã hóa HMAC thành mã hóa có thể giải ngược.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findByEmail(); otpService.verifyOtp(); accountRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/auth/login.html.

2.1.3.Gửi lại mã xác thực đăng ký

2.1.3.1.Mô tả chức năng

Bảng 2.3. Đặc tả: Gửi lại mã xác thực đăng ký

Tiêu chí	Nội dung
Mục đích	Cho phép khôi phục quy trình kích hoạt khi chưa nhận được email hoặc OTP đã hết hạn.
Tác nhân	Khách hàng
Điều kiện trước	Tài khoản chưa emailVerified; đã qua cooldown mặc định 60 giây.
Đầu vào	Email đã đăng ký.
Kết quả / điều kiện sau	Chỉ OTP mới nhất còn khả năng xác thực, mã trước không được tiếp tục sử dụng.
Trường hợp lỗi	Gửi dồn dập, email không tồn tại, email đã xác thực, SMTP chưa hoạt động.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa tài khoản theo email.

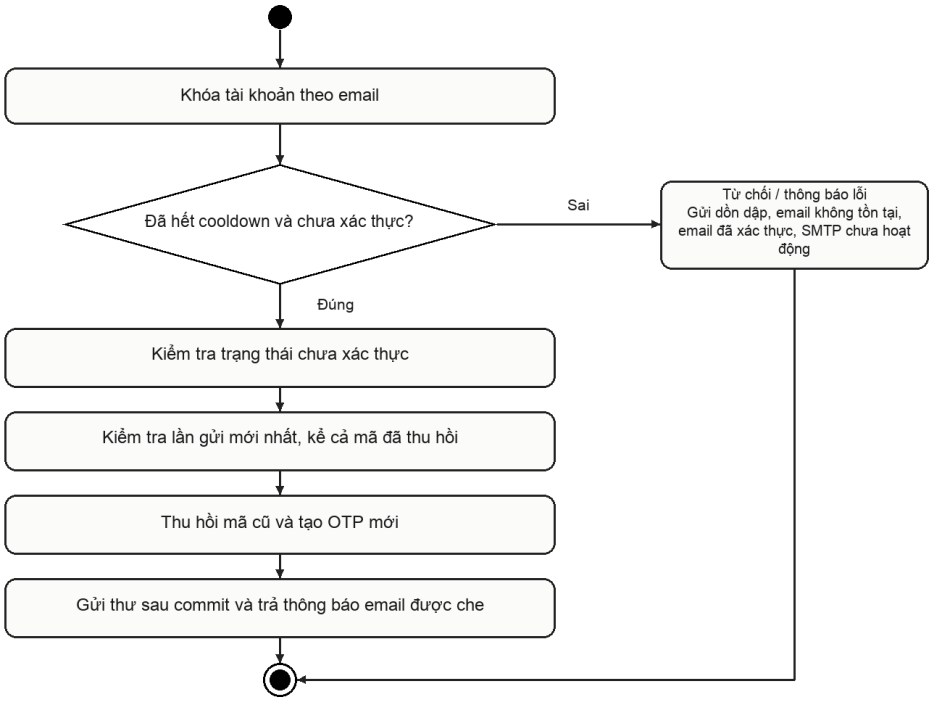
Bước 2. Kiểm tra trạng thái chưa xác thực.

Bước 3. Kiểm tra lần gửi mới nhất, kể cả mã đã thu hồi.

Bước 4. Thu hồi mã cũ và tạo OTP mới.

Bước 5. Gửi thư sau commit và trả thông báo email được che.

2.1.3.2.Sơ đồ hoạt động



Hình 2.3. Hoạt động: Gửi lại mã xác thực đăng ký

2.1.3.3.Code xử lý chức năng

Đoạn mã 2.3. AuthService.resendRegisterOtp — Gửi lại mã xác thực đăng ký

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.resendRegisterOtp; dòng 167–182.

```
    public String resendRegisterOtp(String email) {
        String cleanEmail = email.trim().toLowerCase();
        Account account = accountRepository.findByEmailForUpdate(cleanEmail)
                                                        .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED, "Địa chỉ email chưa được
đăng ký trong hệ thống"));

        if (Boolean.TRUE.equals(account.getEmailVerified())) {
              throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED, "Tài
khoản đã được xác thực. Bạn có thể đăng nhập ngay.");
        }

        OtpService.GeneratedOtp generated = otpService.generateAndSaveOtp(
                cleanEmail, EmailVerificationOtp.PURPOSE_REGISTER, 10);

           mailService.sendRegistrationOtpEmail(cleanEmail, account.getFullName(),
generated.rawOtp(), 10);

                  return   "Mã   xác   thực   mới   đã   được   gửi   tới   email   "   +
MailService.maskEmail(cleanEmail) + ". Vui lòng kiểm tra hộp thư.";
    }
```

2.1.3.4.Giải thích code

Giao diện có đồng hồ chờ; giới hạn thật được kiểm tra lại ở server, không dựa vào nút disabled của trình duyệt.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findByEmailForUpdate(); otpService.generateAndSaveOtp(); mailService.sendRegistrationOtpEmail(); MailService.maskEmail(). Đường dẫn giao diện liên quan: src/main/resources/templates/auth/login.html.

2.1.4.Đăng nhập theo trạng thái tài khoản

2.1.4.1.Mô tả chức năng

Bảng 2.4. Đặc tả: Đăng nhập theo trạng thái tài khoản

Tiêu chí	Nội dung
Mục đích	Xác thực thông tin đăng nhập và tạo phiên JWT theo vai trò.
Tác nhân	Khách hàng
Điều kiện trước	Tài khoản tồn tại, active=true, emailVerified=true.
Đầu vào	Email và mật khẩu.
Kết quả / điều kiện sau	Controller lưu JWT vào cookie HttpOnly; CSRF được cấp lại sau đăng nhập.
Trường hợp lỗi	Sai thông tin, tài khoản tạm khóa hoặc chưa xác thực email.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Chuẩn hóa email và tìm Account.

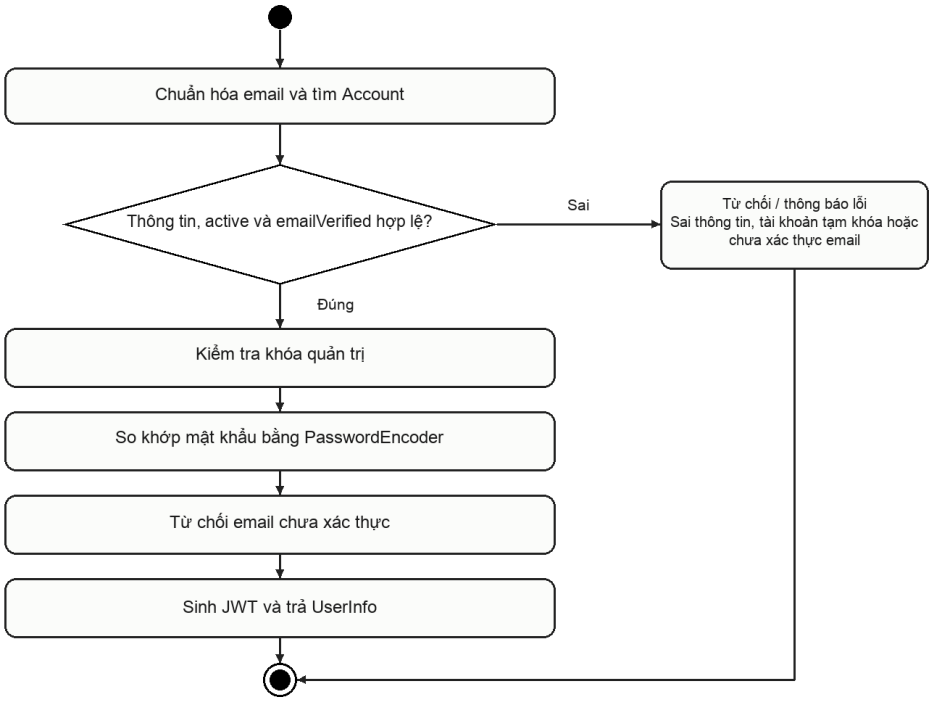
Bước 2. Kiểm tra khóa quản trị.

Bước 3. So khớp mật khẩu bằng PasswordEncoder.

Bước 4. Từ chối email chưa xác thực.

Bước 5. Sinh JWT và trả UserInfo.

2.1.4.2.Sơ đồ hoạt động



Hình 2.4. Hoạt động: Đăng nhập theo trạng thái tài khoản

2.1.4.3.Code xử lý chức năng

Đoạn mã 2.4. AuthService.login — Đăng nhập theo trạng thái tài khoản

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.login; dòng 40–76.

```
    public AuthDto.AuthResponse login(AuthDto.LoginRequest request) {
        String email = request.getEmail().trim().toLowerCase();
        Account account = accountRepository.findByEmail(email)
                .orElseThrow(() -> BusinessException.unauthorized(
                        ErrorCode.AUTH_INVALID_CREDENTIALS, "Tài khoản hoặc mật
khẩu không đúng"));

        // 1. Kiểm tra tài khoản bị admin khóa
        if (!Boolean.TRUE.equals(account.getActive())) {
            throw BusinessException.forbidden(
                    ErrorCode.ACCESS_DENIED, "Tài khoản đã bị tạm khóa. Vui lòng
liên hệ ban quản trị.");
        }

        // 2. Kiểm tra mật khẩu
                            if    (!passwordEncoder.matches(request.getPassword(),
account.getPasswordHash())) {
            throw BusinessException.unauthorized(
                    ErrorCode.AUTH_INVALID_CREDENTIALS, "Tài khoản hoặc mật khẩu
không đúng");
        }

        // 3. Kiểm tra xác thực email (Tách biệt khỏi trạng thái khóa)
        if (!Boolean.TRUE.equals(account.getEmailVerified())) {
            throw BusinessException.unauthorized(
                    ErrorCode.AUTH_EMAIL_NOT_VERIFIED,
```

```
                    "Địa chỉ email chưa được xác thực. Vui lòng xác thực email để
kích hoạt tài khoản trước khi đăng nhập.");
        }

        String token = jwtService.generateToken(account);
        return AuthDto.AuthResponse.builder()
                .token(token)
                .user(AuthDto.UserInfo.builder()
                        .id(account.getId())
                        .email(account.getEmail())
                        .fullName(account.getFullName())
                        .phone(account.getPhone())
                        .role(account.getRole())
                        .build())
                .build();
    }
```

2.1.4.4.Giải thích code

JWT mang tokenVersion; filter đối chiếu lại trạng thái tài khoản và phiên bản để vô hiệu token cũ.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findByEmail(); passwordEncoder.matches(); jwtService.generateToken(). Đường dẫn giao diện liên quan: src/main/resources/templates/auth/login.html.

2.1.5.Đăng xuất và vô hiệu phiên

2.1.5.1.Mô tả chức năng

Bảng 2.5. Đặc tả: Đăng xuất và vô hiệu phiên

Tiêu chí	Nội dung
Mục đích	Kết thúc sử dụng tài khoản và giảm khả năng tái sử dụng token cũ.
Tác nhân	Khách hàng
Điều kiện trước	Có cookie hoặc ngữ cảnh tài khoản của phiên hiện tại.
Đầu vào	Tác nhân hiện tại từ JWT cookie; CSRF cho POST logout.
Kết quả / điều kiện sau	Trình duyệt không tiếp tục dùng cookie cũ; tokenVersion cũ bị filter từ chối.
Trường hợp lỗi	Thiếu CSRF hoặc token không còn hợp lệ sẽ đi qua chính sách bảo mật chung.
Trạng thái xác minh	Đã xác minh mã nguồn

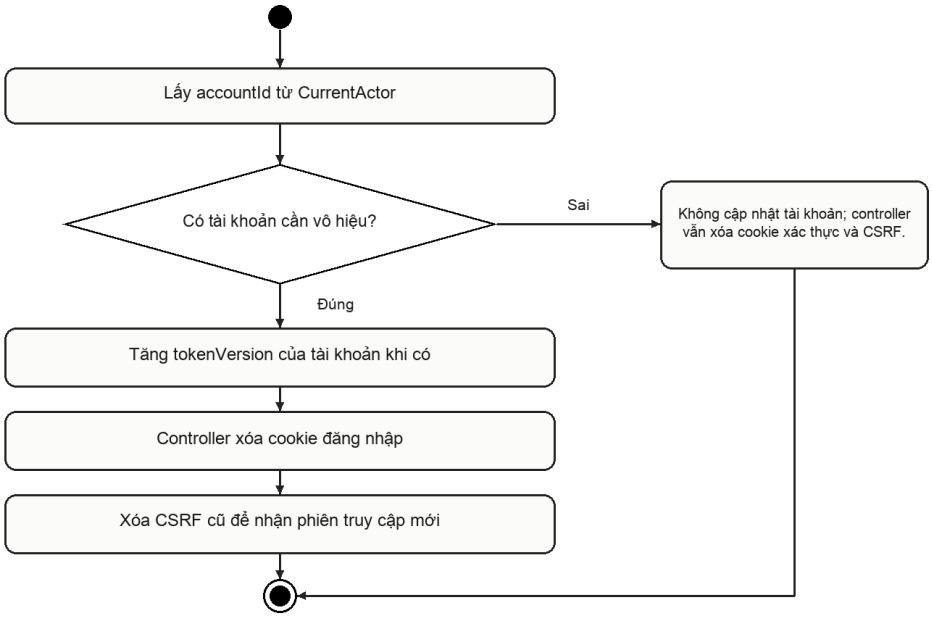
Bước 1. Lấy accountId từ CurrentActor.

Bước 2. Tăng tokenVersion của tài khoản khi có.

Bước 3. Controller xóa cookie đăng nhập.

Bước 4. Xóa CSRF cũ để nhận phiên truy cập mới.

2.1.5.2.Sơ đồ hoạt động



Hình 2.5. Hoạt động: Đăng xuất và vô hiệu phiên

2.1.5.3.Code xử lý chức năng

Đoạn mã 2.5. AuthService.logout — Đăng xuất và vô hiệu phiên

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.logout; dòng 185–193.

```
    public void logout(Long accountId) {
        if (accountId != null) {
            accountRepository.findById(accountId).ifPresent(account -> {
                // Tăng tokenVersion để vô hiệu hóa mọi token cũ
                account.setTokenVersion(account.getTokenVersion() + 1);
                accountRepository.save(account);
            });
        }
    }
```

2.1.5.4.Giải thích code

Đăng xuất không tự đóng phiên bàn và không xóa chứng từ đơn hàng.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findById(); accountRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/auth/login.html.

2.1.6.Yêu cầu khôi phục mật khẩu qua email

2.1.6.1.Mô tả chức năng

Bảng 2.6. Đặc tả: Yêu cầu khôi phục mật khẩu qua email

Tiêu chí	Nội dung
Mục đích	Khởi tạo quy trình lấy lại quyền truy cập khi quên mật khẩu.
Tác nhân	Khách hàng
Điều kiện trước	Cấu hình mail/OTP hợp lệ; địa chỉ phải đúng định dạng.
Đầu vào	Email cần khôi phục.
Kết quả / điều kiện sau	Chưa thay đổi mật khẩu ở bước yêu cầu.
Trường hợp lỗi	Cooldown hoặc SMTP không gửi được; phản hồi không công khai OTP.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Chuẩn hóa email.

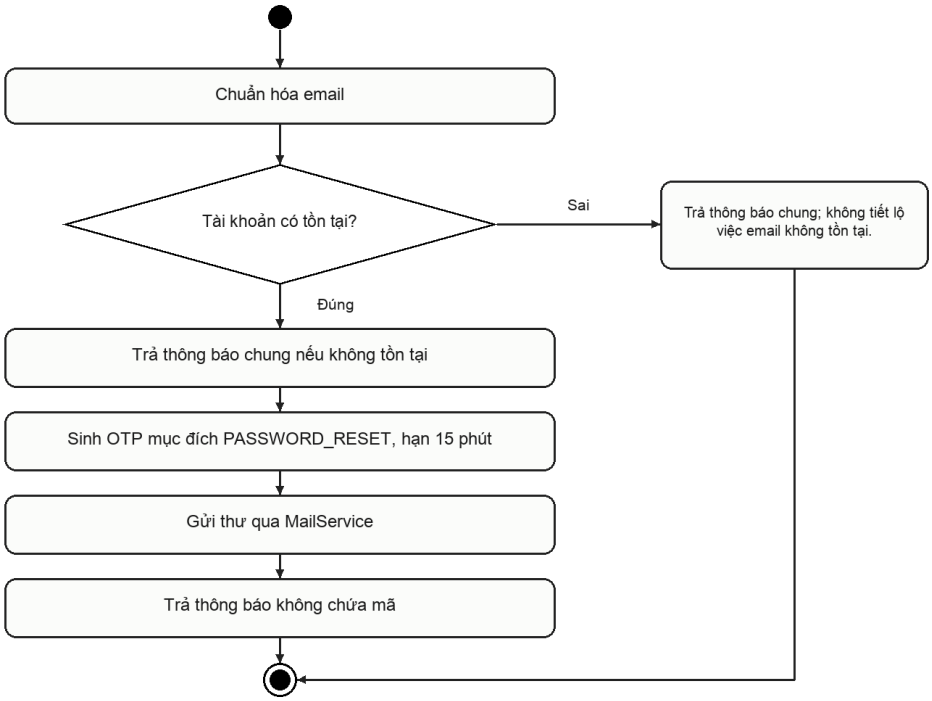
Bước 2. Trả thông báo chung nếu không tồn tại.

Bước 3. Sinh OTP mục đích PASSWORD_RESET, hạn 15 phút.

Bước 4. Gửi thư qua MailService.

Bước 5. Trả thông báo không chứa mã.

2.1.6.2.Sơ đồ hoạt động



Hình 2.6. Hoạt động: Yêu cầu khôi phục mật khẩu qua email

2.1.6.3.Code xử lý chức năng

Đoạn mã 2.6. AuthService.requestPasswordReset — Yêu cầu khôi phục mật khẩu qua email

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.requestPasswordReset; dòng 202–226.

```
    public String requestPasswordReset(AuthDto.PasswordResetRequest request) {
        String email = request.getEmail().trim().toLowerCase();
         String genericMessage = "Nếu địa chỉ email tồn tại trên hệ thống, mã xác
thực khôi phục mật khẩu đã được gửi đến hộp thư của bạn.";

        Optional<Account> accountOpt = accountRepository.findByEmail(email);
        if (accountOpt.isEmpty()) {
                log.info("Yêu cầu reset mật khẩu cho email không tồn tại: {}",
MailService.maskEmail(email));
            return genericMessage;
        }

        Account account = accountOpt.get();
        if (!Boolean.TRUE.equals(account.getActive())) {
               log.warn("Yêu cầu reset mật khẩu cho tài khoản đang bị khóa: {}",
MailService.maskEmail(email));
            return genericMessage;
        }

        // Sinh mã OTP 6 số mục đích PASSWORD_RESET, thời hạn 15 phút
        OtpService.GeneratedOtp generated = otpService.generateAndSaveOtp(
                email, EmailVerificationOtp.PURPOSE_PASSWORD_RESET, 15);

        // Gửi email thật
```

```
                mailService.sendPasswordResetOtpEmail(email,  account.getFullName(),
generated.rawOtp(), 15);

        return genericMessage;
    }
```

2.1.6.4.Giải thích code

Thông báo chung hạn chế tiết lộ tài khoản. Không mô tả bước này là đặt lại mật khẩu thành công.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findByEmail(); MailService.maskEmail(); otpService.generateAndSaveOtp(); mailService.sendPasswordResetOtpEmail(). Đường dẫn giao diện liên quan: src/main/resources/templates/auth/login.html.

2.1.7.Xác minh OTP khôi phục và nhận vé tạm

2.1.7.1.Mô tả chức năng

Bảng 2.7. Đặc tả: Xác minh OTP khôi phục và nhận vé tạm

Tiêu chí	Nội dung
Mục đích	Tách bước kiểm tra OTP khỏi bước nhập mật khẩu mới.
Tác nhân	Khách hàng
Điều kiện trước	Tài khoản còn hoạt động; OTP đúng purpose và chưa được dùng.
Đầu vào	Email và OTP khôi phục.
Kết quả / điều kiện sau	Mã OTP bị tiêu thụ; vé tạm có hạn dùng để xác nhận bước tiếp theo.
Trường hợp lỗi	Sai OTP, hết hạn, vượt số lần thử hoặc tài khoản khóa.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra email và trạng thái tài khoản.

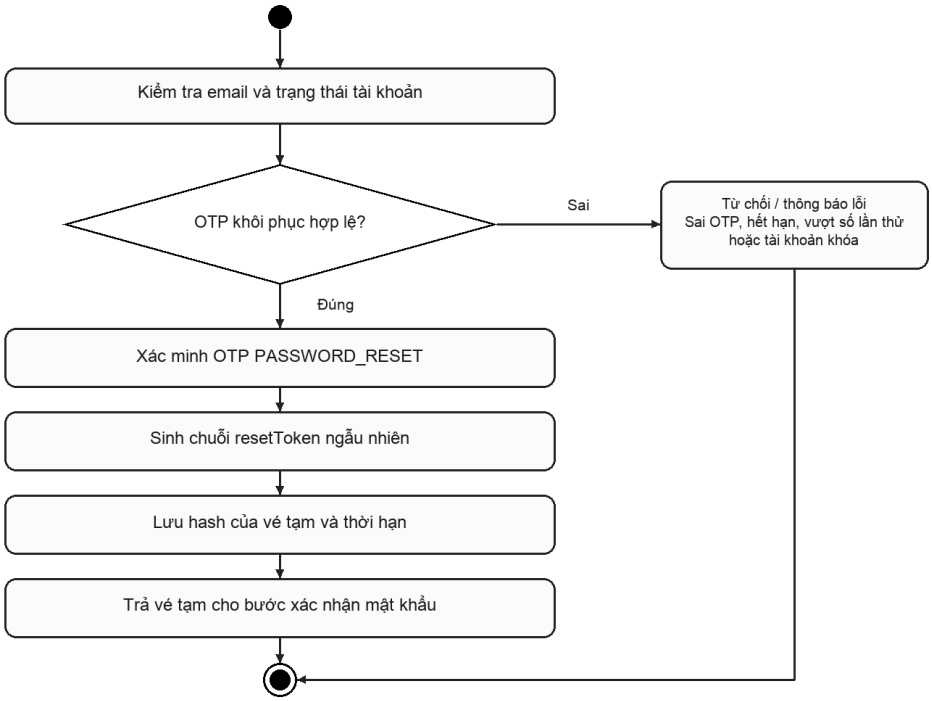
Bước 2. Xác minh OTP PASSWORD_RESET.

Bước 3. Sinh chuỗi resetToken ngẫu nhiên.

Bước 4. Lưu hash của vé tạm và thời hạn.

Bước 5. Trả vé tạm cho bước xác nhận mật khẩu.

2.1.7.2.Sơ đồ hoạt động



Hình 2.7. Hoạt động: Xác minh OTP khôi phục và nhận vé tạm

2.1.7.3.Code xử lý chức năng

Đoạn mã 2.7. AuthService.verifyPasswordResetOtp — Xác minh OTP khôi phục và nhận vé tạm

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.verifyPasswordResetOtp; dòng 232–262.

```
    public String verifyPasswordResetOtp(String rawEmail, String otp) {
        if (rawEmail == null || rawEmail.isBlank()) {
            throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED, "Email
không được để trống");
        }
        String cleanEmail = rawEmail.trim().toLowerCase();
        Account account = accountRepository.findByEmail(cleanEmail)
                .orElseThrow(() -> BusinessException.badRequest(
                        ErrorCode.AUTH_INVALID_CREDENTIALS, "Tài khoản hoặc mã xác
thực không hợp lệ"));

        if (!Boolean.TRUE.equals(account.getActive())) {
            throw BusinessException.badRequest(ErrorCode.ACCESS_DENIED, "Tài khoản
đang bị tạm khóa.");
        }

        // 1. Xác thực mã OTP 6 số với purpose PASSWORD_RESET
                                                otpService.verifyOtp(cleanEmail,
EmailVerificationOtp.PURPOSE_PASSWORD_RESET, otp);

        // 2. Sinh resetToken ngẫu nhiên an toàn, lưu hash vào TokenKhoiPhuc
        String rawToken = UUID.randomUUID().toString();
        String tokenHash = guestAccessService.hashToken(rawToken);
```

```
        PasswordResetToken resetTokenEntity = PasswordResetToken.builder()
                .account(account)
                .tokenHash(tokenHash)
                .expiresAt(Instant.now().plus(15, ChronoUnit.MINUTES))
                .build();

        passwordResetTokenRepository.save(resetTokenEntity);
          log.info("Xác thực OTP đặt lại mật khẩu thành công cho tài khoản: {}",
MailService.maskEmail(cleanEmail));

        return rawToken;
    }
```

2.1.7.4.Giải thích code

Vé reset chỉ dành cho luồng quên mật khẩu, không phải JWT đăng nhập.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findByEmail(); otpService.verifyOtp(); guestAccessService.hashToken(); passwordResetTokenRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/auth/login.html.

2.1.8.Xác nhận mật khẩu mới

2.1.8.1.Mô tả chức năng

Bảng 2.8. Đặc tả: Xác nhận mật khẩu mới

Tiêu chí	Nội dung
Mục đích	Cập nhật mật khẩu sau khi người dùng hoàn thành bằng chứng khôi phục.
Tác nhân	Khách hàng
Điều kiện trước	Vé tạm/chứng cứ khôi phục đúng tài khoản, chưa dùng và còn hạn.
Đầu vào	Email, resetToken hoặc mã theo nhánh tương thích và newPassword.
Kết quả / điều kiện sau	Mật khẩu mới thay mật khẩu cũ; các JWT mang phiên bản cũ bị vô hiệu.
Trường hợp lỗi	Vé sai/hết hạn/đã dùng, mật khẩu không đủ điều kiện hoặc email không khớp.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra thông tin yêu cầu.

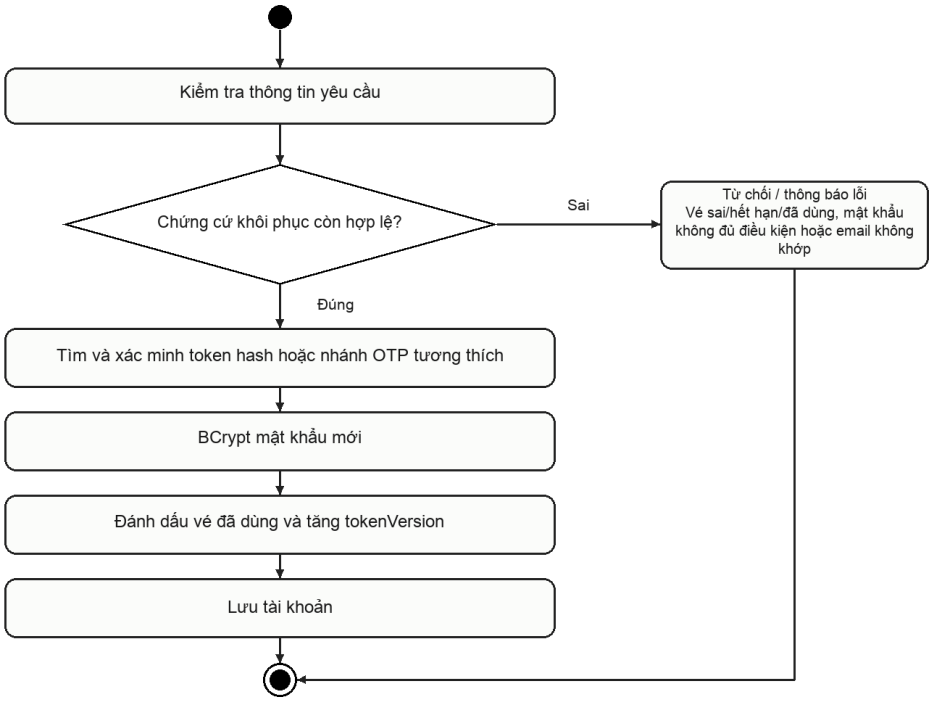
Bước 2. Tìm và xác minh token hash hoặc nhánh OTP tương thích.

Bước 3. BCrypt mật khẩu mới.

Bước 4. Đánh dấu vé đã dùng và tăng tokenVersion.

Bước 5. Lưu tài khoản.

2.1.8.2.Sơ đồ hoạt động



Hình 2.8. Hoạt động: Xác nhận mật khẩu mới

2.1.8.3.Code xử lý chức năng

Đoạn mã 2.8. AuthService.confirmPasswordReset — Xác nhận mật khẩu mới

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.confirmPasswordReset; dòng 268–323.

```
    public void confirmPasswordReset(AuthDto.PasswordResetConfirmRequest request)
{
        String tokenOrCode = request.getEffectiveToken();
        if (tokenOrCode.isBlank()) {
               throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED, "Mã
xác thực không được để trống");
        }

        Account account = null;

        // Trường hợp 1: Sử dụng mã OTP 6 chữ số kèm email
             if (request.getEmail() != null && !request.getEmail().isBlank() &&
tokenOrCode.matches("^\\d{6}$")) {
            String email = request.getEmail().trim().toLowerCase();
            account = accountRepository.findByEmail(email)
                    .orElseThrow(() -> BusinessException.badRequest(
```

[Đã lược bỏ các dòng 281–289 của phương thức gốc.]

```
            if (resetTokenOpt.isPresent()) {
                PasswordResetToken resetToken = resetTokenOpt.get();
                                        if (resetToken.getUsedAt() != null ||
resetToken.getExpiresAt().isBefore(Instant.now())) {
                                                                    throw
BusinessException.badRequest(ErrorCode.AUTH_TOKEN_EXPIRED, "Mã khôi phục đã hết
hạn hoặc đã sử dụng");
```

```
                }
                account = resetToken.getAccount();
                resetToken.setUsedAt(Instant.now());
                passwordResetTokenRepository.save(resetToken);
                            } else if (request.getEmail() != null && !
request.getEmail().isBlank()) {
                // Thử xác thực qua OTP service nếu mã gửi dạng OTP
                String email = request.getEmail().trim().toLowerCase();
```

[Đã lược bỏ các dòng 301–311 của phương thức gốc.]

```
        }

        // Cập nhật mật khẩu băm mới
        account.setPasswordHash(passwordEncoder.encode(request.getNewPassword()));

        // Tăng tokenVersion để vô hiệu hóa toàn bộ session/JWT cũ
        account.setTokenVersion(account.getTokenVersion() + 1);

        // Lưu tài khoản (Lưu ý: Không tự ý mở khóa active nếu tài khoản bị khóa)
        accountRepository.save(account);
                log.info("Đặt lại mật khẩu thành công cho tài khoản: {}",
MailService.maskEmail(account.getEmail()));
    }
```

2.1.8.4.Giải thích code

Báo cáo phân biệt luồng giao diện ba bước với nhánh API tương thích code/token trong backend.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findByEmail(); otpService.verifyOtp(); guestAccessService.hashToken(); passwordResetTokenRepository.findByTokenHash(). Đường dẫn giao diện liên quan: src/main/resources/templates/auth/login.html.

2.1.9.Xem menu, tìm kiếm và lọc sản phẩm

2.1.9.1.Mô tả chức năng

Bảng 2.9. Đặc tả: Xem menu, tìm kiếm và lọc sản phẩm

Tiêu chí	Nội dung
Mục đích	Tra cứu menu có phân trang và các tiêu chí sắp xếp được triển khai.
Tác nhân	Khách hàng
Điều kiện trước	Endpoint công khai; chỉ sản phẩm/danh mục và cơ sở đang hoạt động được đưa vào tập kết quả.
Đầu vào	Từ khóa, category/shop tùy chọn, min/max, sort, page, size.
Kết quả / điều kiện sau	Danh sách phản ánh dữ liệu hiện có; không tự gán cơ sở phục vụ của đơn.
Trường hợp lỗi	Giá âm, min lớn hơn max, page/size vượt giới hạn hoặc truy vấn quá dài.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra bộ lọc và phân trang.

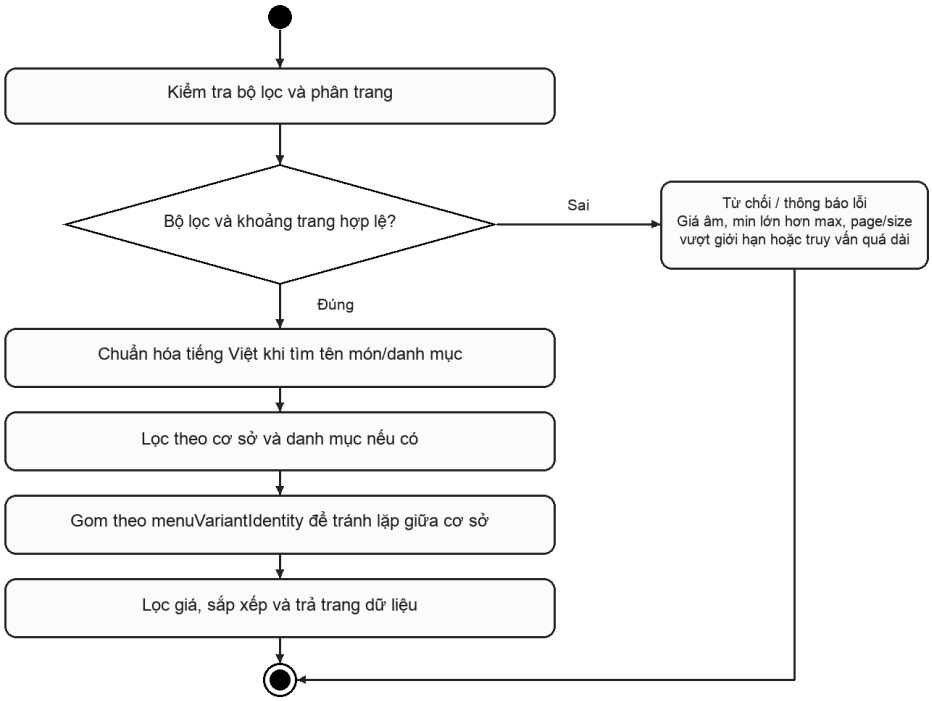
Bước 2. Chuẩn hóa tiếng Việt khi tìm tên món/danh mục.

Bước 3. Lọc theo cơ sở và danh mục nếu có.

Bước 4. Gom theo menuVariantIdentity để tránh lặp giữa cơ sở.

Bước 5. Lọc giá, sắp xếp và trả trang dữ liệu.

2.1.9.2.Sơ đồ hoạt động



Hình 2.9. Hoạt động: Xem menu, tìm kiếm và lọc sản phẩm

2.1.9.3.Code xử lý chức năng

Đoạn mã 2.9. CatalogSearchService.search — Xem menu, tìm kiếm và lọc sản phẩm

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/CatalogSearchService.java; CatalogSearchService.search; dòng 68–111.

```
    public CatalogPage search(String term, Long category, Long shop, BigDecimal
min, BigDecimal max, String sort, int page, int size) {
        if (page < 0 || page > 100000 || size < 1 || size > 100
                || (min != null && min.signum() < 0)
                || (max != null && max.signum() < 0)
                || (min != null && max != null && min.compareTo(max) > 0)
                || term != null && term.length() > 120) {
               throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED, "Bộ
lọc hoặc phân trang không hợp lệ");
        }

        String normalizedTerm = Product.normalizeSearch(term == null ? "" : term);
        List<Product> matched = products.findAllByActiveTrue().stream()
                                        .filter(p  ->  p.getCategory()  !=  null  &&
Boolean.TRUE.equals(p.getCategory().getActive()))
                .filter(p -> p.getShop() == null || p.getShop().isActive())
```

[Đã lược bỏ các dòng 81–87 của phương thức gốc.]

```
          // Checkout resolves a menu variant to a branch later. Do not show the
same
         // variant once per branch/category, or collapse distinct sizes into one
SKU.
        Map<String, Product> unique = new LinkedHashMap<>();
        matched.stream().sorted(Comparator
                .comparingInt((Product p) -> p.getShop() == null ? 0 : 1)
```

```
                .thenComparing(Product::getPrice)
                .thenComparing(Product::getId))
                .forEach(p -> unique.putIfAbsent(p.menuVariantIdentity(), p));
        List<Product> displayProducts = new ArrayList<>(unique.values());
        if (min != null) {
                        displayProducts.removeIf(p  ->  p.getPrice()  ==  null  ||
p.getPrice().compareTo(min) < 0);
```

[Đã lược bỏ các dòng 99–99 của phương thức gốc.]

```
        if (max != null) {
                        displayProducts.removeIf(p  ->  p.getPrice()  ==  null  ||
p.getPrice().compareTo(max) > 0);
        }

        displayProducts.sort(comparator(sort, matched));

        long total = displayProducts.size();
        int fromIndex = Math.min(page * size, displayProducts.size());
        int toIndex = Math.min(fromIndex + size, displayProducts.size());
                            var    items    =    displayProducts.subList(fromIndex,
toIndex).stream().map(market::productView).toList();
        return new CatalogPage(items, total, page, size);
    }
```

2.1.9.4.Giải thích code

Sort có newest, giá, đánh giá, yêu thích và bán chạy theo các truy vấn trong service.

Method liên quan: CatalogSearchService.search. Controller: MarketController. Giao diện: src/main/resources/templates/customer/marketplace.html; src/main/resources/static/js/customer/marketplace.js.

2.1.10.Xem chi tiết, chọn size và topping

2.1.10.1.Mô tả chức năng

Bảng 2.10. Đặc tả: Xem chi tiết, chọn size và topping

Tiêu chí	Nội dung
Mục đích	Cung cấp hộp chọn món trên menu, không tạo một trang chi tiết không tồn tại.
Tác nhân	Khách hàng
Điều kiện trước	Món tồn tại và còn một biến thể đang bán trong menu.
Đầu vào	ID món được chọn, size, số lượng, ghi chú, topping khi cho phép.
Kết quả / điều kiện sau	Khách lựa chọn cấu hình trước khi đưa vào giỏ; giá cuối vẫn kiểm tra theo cơ sở ở backend.
Trường hợp lỗi	Món không tồn tại hoặc tất cả biến thể đã ngừng bán.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm món nguồn.

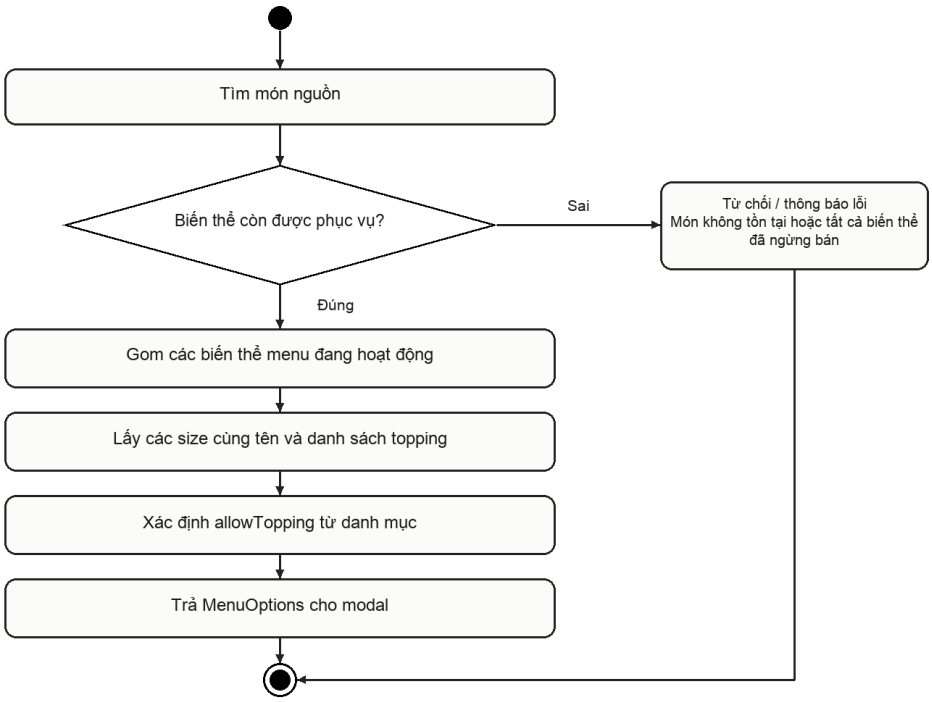
Bước 2. Gom các biến thể menu đang hoạt động.

Bước 3. Lấy các size cùng tên và danh sách topping.

Bước 4. Xác định allowTopping từ danh mục.

Bước 5. Trả MenuOptions cho modal.

2.1.10.2.Sơ đồ hoạt động



Hình 2.10. Hoạt động: Xem chi tiết, chọn size và topping

2.1.10.3.Code xử lý chức năng

Đoạn mã 2.10. CatalogSearchService.options — Xem chi tiết, chọn size và topping

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/CatalogSearchService.java; CatalogSearchService.options; dòng 32–65.

```
    public MenuOptions options(Long id) {
        var source = products.findById(id)
                                                        .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.PRODUCT_UNAVAILABLE, "Món không tồn tại"));

        Map<String, Product> unique = new LinkedHashMap<>();
        products.findAllByActiveTrue().stream()
                .sorted(Comparator.comparing(Product::getId))
                .forEach(p -> unique.putIfAbsent(p.menuVariantIdentity(), p));

        var selected = unique.get(source.menuVariantIdentity());
        if (selected == null) {
            throw BusinessException.badRequest(ErrorCode.PRODUCT_UNAVAILABLE, "Món
đã ngừng bán");
        }
```

```
        var variants = unique.values().stream()
                                                                        .filter(p    ->
Product.normalizeSearch(p.getName()).equals(Product.normalizeSearch(selected.getNa
me()))
                .sorted(Comparator.comparing(Product::getPrice))
                .map(market::productView)
                .toList();
        var toppings = unique.values().stream()
                                                                        .filter(p    ->
Product.normalizeSearch(p.getCategory().getName()).contains("topping"))
                .map(market::productView)
                .toList();
        boolean allowTopping = products.findAllByActiveTrue().stream().anyMatch(
                                                                             p     ->
p.menuVariantIdentity().equals(selected.menuVariantIdentity())
                        && Boolean.TRUE.equals(p.getCategory().getAllowTopping())
        );
        return new MenuOptions(
                market.productView(selected),
                variants,
                toppings,
                                                                allowTopping    &&    !
Product.normalizeSearch(selected.getCategory().getName()).contains("topping")
        );
    }
```

2.1.10.4.Giải thích code

Chi tiết sản phẩm là modal có thật, không mô tả route /product-detail tự tạo.

Method liên quan: CatalogSearchService.options. Controller: MarketController. Giao diện: src/main/resources/templates/fragments/product-options.html; src/main/resources/static/js/customer/menu-selection.js.

2.1.11.Lưu và đồng bộ giỏ hàng tài khoản

2.1.11.1.Mô tả chức năng

Bảng 2.11. Đặc tả: Lưu và đồng bộ giỏ hàng tài khoản

Tiêu chí	Nội dung
Mục đích	Lưu giỏ của khách đăng nhập và chống ghi đè thay đổi từ thiết bị khác.
Tác nhân	Khách hàng
Điều kiện trước	CurrentActor có accountId; phiên bản gửi lên khớp bản hiện tại.
Đầu vào	Danh sách món, số lượng, ghi chú, toppingIds và If-Match version.
Kết quả / điều kiện sau	Giỏ server có phiên bản mới; client tải lại khi xung đột.
Trường hợp lỗi	Giỏ đã thay đổi, topping không được phép hoặc dữ liệu không hợp lệ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa AccountCart theo tài khoản.

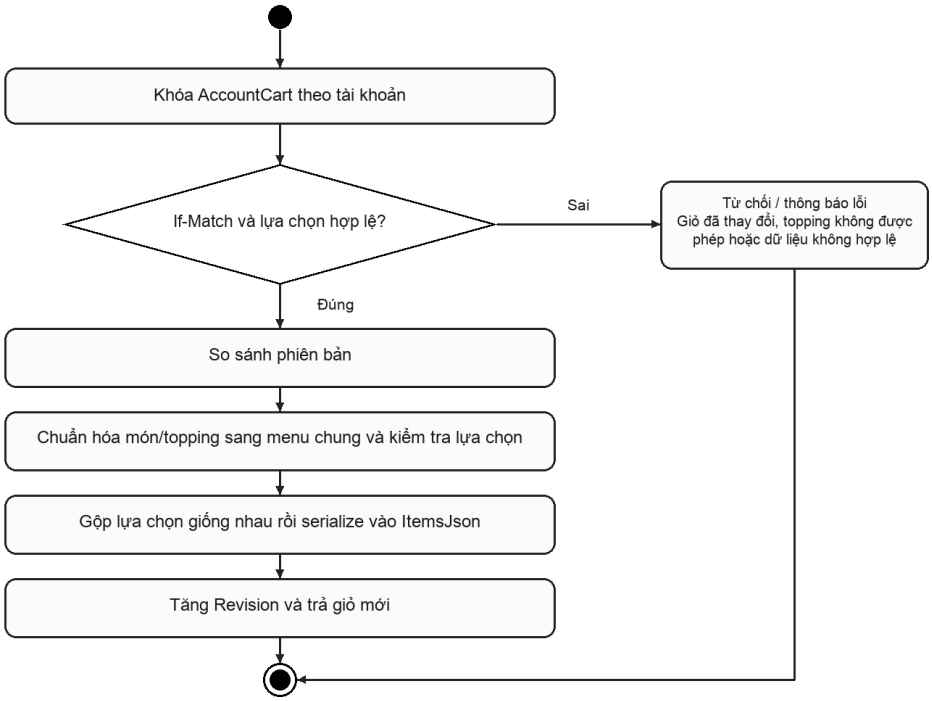
Bước 2. So sánh phiên bản.

Bước 3. Chuẩn hóa món/topping sang menu chung và kiểm tra lựa chọn.

Bước 4. Gộp lựa chọn giống nhau rồi serialize vào ItemsJson.

Bước 5. Tăng Revision và trả giỏ mới.

2.1.11.2.Sơ đồ hoạt động



Hình 2.11. Hoạt động: Lưu và đồng bộ giỏ hàng tài khoản

2.1.11.3.Code xử lý chức năng

Đoạn mã 2.11. CustomerDataService.replaceCart — Lưu và đồng bộ giỏ hàng tài khoản

Nguồn: src/main/java/vn/edu/ute/milktea/service/account/CustomerDataService.java; CustomerDataService.replaceCart; dòng 44–70.

```
                    public      CartDto.CartResponse      replaceCart(CurrentActor
actor,CartDto.ReplaceCartRequest input,String version) {
        Account account=lock(actor);
                var  cart=carts.findById(account.getId()).orElseGet(()->{var  c=new
AccountCart();c.setAccount(account);c.setAccountId(account.getId());return c;});
        if(version==null||!version.equals(Long.toString(cart.getRevision())))
             throw BusinessException.conflict(ErrorCode.CART_VERSION_CONFLICT,"Giỏ
hàng đã thay đổi. Vui lòng tải lại trước khi sửa.");
         if(input==null||input.getItems()==null||input.getItems().size()>100) throw
invalid("Giỏ tối đa 100 dòng");
        // Validate and reprice before storing only IDs, quantity and note.
        var candidates=products.findAllByActiveTrue();
        Map<String,CartDto.CartItemRequest> normalized=new LinkedHashMap<>();
        for(var line:input.getItems()) {
            lineView(line,false);
            Long productId=canonical(line.getProductId(),candidates);
                                    var   toppingIds=(line.getToppingIds()==null?
List.<Long>of():line.getToppingIds()).stream().map(t-
>canonical(t,candidates)).sorted().toList();
            String note=line.getNote()==null?"":line.getNote();
            String identity=available(productId,false).menuVariantIdentity();
                        if(!toppingIds.isEmpty()&&candidates.stream().noneMatch(p-
>p.menuVariantIdentity().equals(identity)&&Boolean.TRUE.equals(p.getCategory().get
AllowTopping())))throw invalid("Món không cho phép topping");
```

```
            String key=productId+"|"+toppingIds+"|"+note;
            var previous=normalized.get(key);

if(previous==null)normalized.put(key,CartDto.CartItemRequest.builder().productId(p
roductId).quantity(line.getQuantity()).note(note).toppingIds(toppingIds).build());
                                else  {previous.setQuantity(previous.getQuantity()
+line.getQuantity());if(previous.getQuantity()>99)throw invalid("Mỗi lựa chọn món
tối đa 99 ly");}
        }
        try {cart.setItemsJson(mapper.writeValueAsString(normalized.values()));}
        catch(Exception e){throw invalid("Dữ liệu giỏ không hợp lệ");}
        cart.setRevision(cart.getRevision()+1);
        carts.save(cart);
        return cartView(cart);
    }
```

2.1.11.4.Giải thích code

Khách chưa đăng nhập dùng localStorage trong CustomerApp; không tuyên bố giỏ khách tạm đã được lưu vào DB.

Method liên quan: CustomerDataService.replaceCart. Controller: CustomerDataController. Giao diện: src/main/resources/templates/customer/cart.html; src/main/resources/static/js/customer/customer.js.

2.1.12.Lấy lại giỏ đã lưu

2.1.12.1.Mô tả chức năng

Bảng 2.12. Đặc tả: Lấy lại giỏ đã lưu

Tiêu chí	Nội dung
Mục đích	Phục hồi giỏ tài khoản khi tải trang hoặc đăng nhập.
Tác nhân	Khách hàng
Điều kiện trước	Đã đăng nhập và phiên bản JWT còn hợp lệ.
Đầu vào	Tác nhân từ tài khoản hiện tại.
Kết quả / điều kiện sau	Client khôi phục giỏ cùng phiên bản; giỏ tạm có thể được gộp theo mã JavaScript hiện có.
Trường hợp lỗi	Khách ẩn danh bị chặn đối với endpoint giỏ tài khoản.
Trạng thái xác minh	Đã xác minh mã nguồn

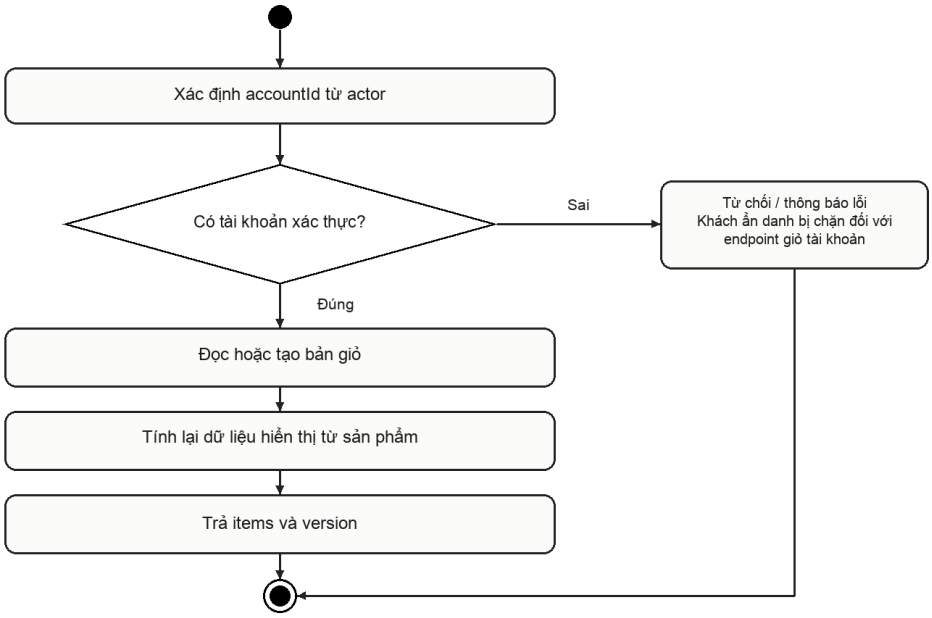
Bước 1. Xác định accountId từ actor.

Bước 2. Đọc hoặc tạo bản giỏ.

Bước 3. Tính lại dữ liệu hiển thị từ sản phẩm.

Bước 4. Trả items và version.

2.1.12.2.Sơ đồ hoạt động



Hình 2.12. Hoạt động: Lấy lại giỏ đã lưu

2.1.12.3.Code xử lý chức năng

Đoạn mã 2.12. CustomerDataService.cart — Lấy lại giỏ đã lưu

Nguồn: src/main/java/vn/edu/ute/milktea/service/account/CustomerDataService.java; CustomerDataService.cart; dòng 39–42.

```
    public CartDto.CartResponse cart(CurrentActor actor) {
        var cart=carts.findById(accountId(actor)).orElseGet(AccountCart::new);
        return cartView(cart);
    }
```

2.1.12.4.Giải thích code

Không lấy accountId tùy ý do frontend gửi làm quyền truy cập.

Method liên quan: CustomerDataService.cart. Controller: CustomerDataController. Giao diện: src/main/resources/templates/customer/cart.html; src/main/resources/static/js/customer/customer.js.

2.1.13.Nhận diện bàn bằng QR hợp lệ

2.1.13.1.Mô tả chức năng

Bảng 2.13. Đặc tả: Nhận diện bàn bằng QR hợp lệ

Tiêu chí	Nội dung
Mục đích	Lấy ngữ cảnh bàn/cơ sở từ mã QR thực tế, tái dùng cơ chế phiên bàn.
Tác nhân	Khách hàng
Điều kiện trước	Bàn tồn tại, active=true; cơ sở phục vụ được xác định hợp lệ.
Đầu vào	Mã qrCode từ đường dẫn hoặc ngữ cảnh đã quét trong tab.
Kết quả / điều kiện sau	Xem QR chưa tự mở phiên; số bàn không thay thế mã QR bí mật.
Trường hợp lỗi	QR rỗng/sai, bàn ngừng hoạt động hoặc cơ sở không được cấu hình.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm bàn theo mã QR.

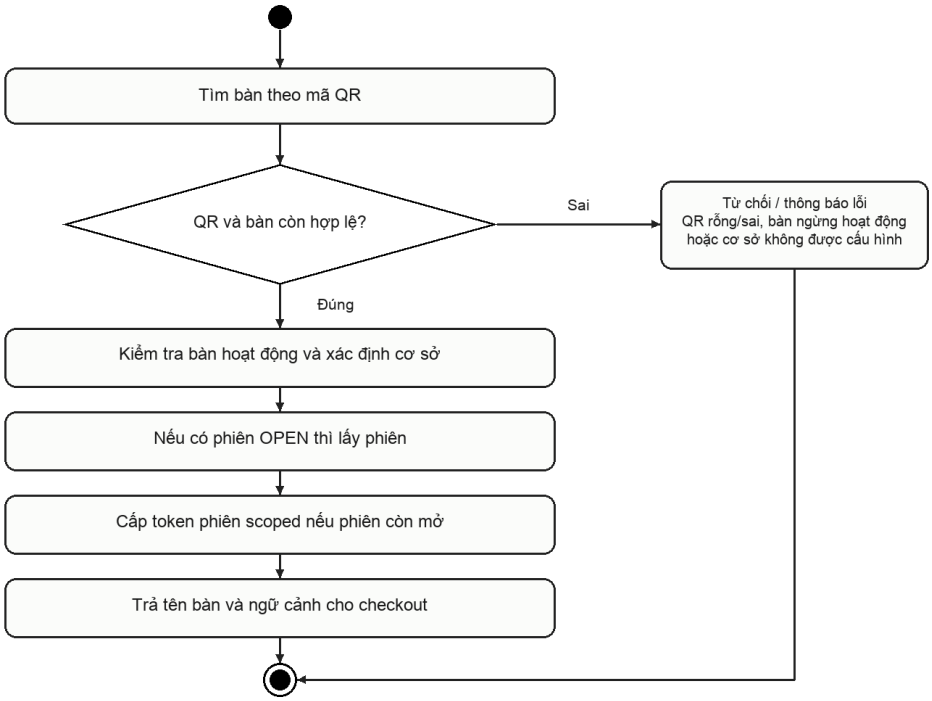
Bước 2. Kiểm tra bàn hoạt động và xác định cơ sở.

Bước 3. Nếu có phiên OPEN thì lấy phiên.

Bước 4. Cấp token phiên scoped nếu phiên còn mở.

Bước 5. Trả tên bàn và ngữ cảnh cho checkout.

2.1.13.2.Sơ đồ hoạt động



Hình 2.13. Hoạt động: Nhận diện bàn bằng QR hợp lệ

2.1.13.3.Code xử lý chức năng

Đoạn mã 2.13. TableContextService.readByQr — Nhận diện bàn bằng QR hợp lệ

Nguồn: src/main/java/vn/edu/ute/milktea/service/table/TableContextService.java; TableContextService.readByQr; dòng 28–63.

```
    public TableDto.TableContextResponse readByQr(String qrCode) {
        if (qrCode == null || qrCode.isBlank()) {
             throw BusinessException.badRequest(ErrorCode.QR_INVALID, "Mã QR không
hợp lệ");
        }

        DiningTable table = tableRepository.findByQrCode(qrCode)
                                                              .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.TABLE_NOT_FOUND, "Không tìm thấy bàn"));

        if (!Boolean.TRUE.equals(table.getActive())) {
               throw BusinessException.badRequest(ErrorCode.TABLE_NOT_FOUND, "Bàn
đang ngừng hoạt động");
        }

        var shop = branchService.tableBranch(table);
             TableDto.TableContextResponse.TableContextResponseBuilder  builder  =
TableDto.TableContextResponse.builder()
                .shopId(shop.getId())
                .shopName(shop.getName())
                .tableId(table.getId())
                .tableName(table.getName())
                .qrCode(table.getQrCode())
                .status(table.getStatus());
```

```
        if (table.getActiveSessionId() != null) {
                                            Optional<TableSession>    sessionOpt    =
sessionRepository.findById(table.getActiveSessionId());
                    if (sessionOpt.isPresent() && sessionOpt.get().getStatus() ==
TableSessionStatus.OPEN) {
                TableSession session = sessionOpt.get();
                builder.sessionId(session.getId())
                        .sessionStatus(session.getStatus());

                   // Cấp token truy cập phiên scoped riêng cho tab hiện tại (lưu
hash, trả raw token)
                    var issued = guestAccessService.issueTableSessionToken(session,
12);
                builder.tableToken(issued.rawToken());
            }
        }

        return builder.build();
    }
```

2.1.13.4.Giải thích code

Checkout không hiện ô tự chọn hình thức/cơ sở; ngữ cảnh bàn hợp lệ tự quyết định nhánh DINE_IN.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: tableRepository.findByQrCode(); branchService.tableBranch(); sessionRepository.findById(); guestAccessService.issueTableSessionToken(). Đường dẫn giao diện liên quan: src/main/resources/static/js/customer/customer.js; src/main/resources/templates/customer/cart.html.

2.1.14.Kiểm tra tổng tiền trước gửi đơn

2.1.14.1.Mô tả chức năng

Bảng 2.14. Đặc tả: Kiểm tra tổng tiền trước gửi đơn

Tiêu chí	Nội dung
Mục đích	Hiển thị giá trị phải thanh toán từ dữ liệu server trước khi xác nhận tạo đơn.
Tác nhân	Khách hàng
Điều kiện trước	Cơ sở phục vụ đúng ngữ cảnh hoặc mặc định POS; món còn hoạt động.
Đầu vào	Các dòng giỏ và shopId chỉ khi có ngữ cảnh bàn; không lấy giá do client cung cấp.
Kết quả / điều kiện sau	Không tạo Order hay Payment ở bước quote.
Trường hợp lỗi	Món không tồn tại/ngừng bán, cơ sở chưa sẵn sàng hoặc cấu hình không hợp lệ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Xác định cơ sở và quyền mua.

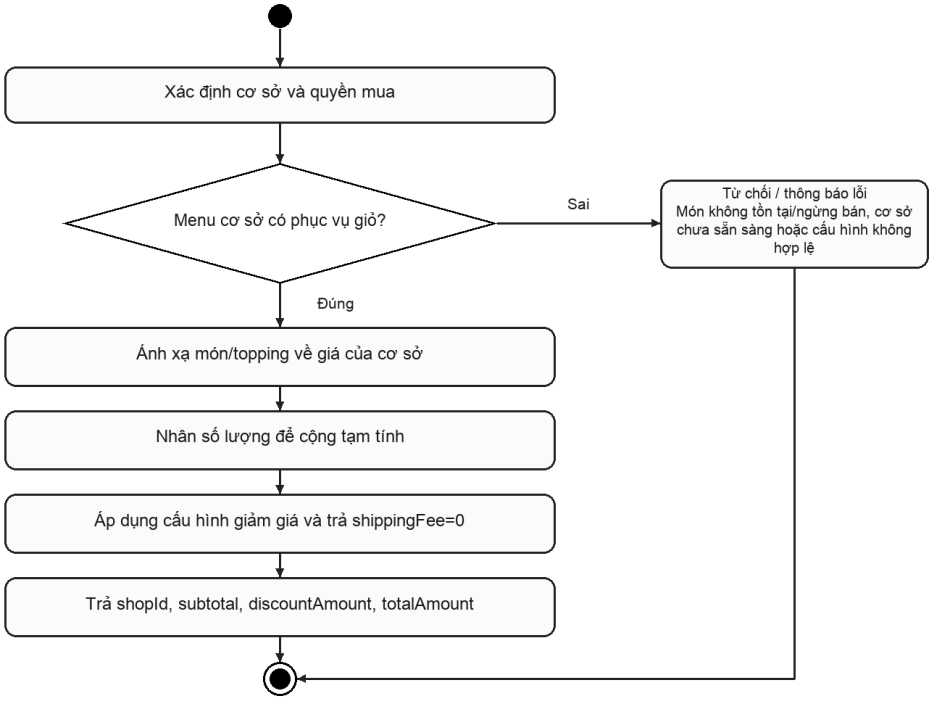
Bước 2. Ánh xạ món/topping về giá của cơ sở.

Bước 3. Nhân số lượng để cộng tạm tính.

Bước 4. Áp dụng cấu hình giảm giá và trả shippingFee=0.

Bước 5. Trả shopId, subtotal, discountAmount, totalAmount.

2.1.14.2.Sơ đồ hoạt động



Hình 2.14. Hoạt động: Kiểm tra tổng tiền trước gửi đơn

2.1.14.3.Code xử lý chức năng

Đoạn mã 2.14. OrderService.quote — Kiểm tra tổng tiền trước gửi đơn

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.quote; dòng 775–798.

```
      public Map<String,Object> quote(OrderDto.CreateCounterOrderRequest request,
CurrentActor actor) {
        var shop = branchService.resolve(request.getShopId());
        branchService.assertCustomer(shop, actor);
        BigDecimal subtotal = BigDecimal.ZERO;
        for (var item : request.getItems()) {
                                                          Product     source     =
productRepository.findById(item.getProductId()).orElseThrow(() ->

BusinessException.badRequest(ErrorCode.PRODUCT_UNAVAILABLE,"Món không tồn tại"));
            Product product = branchService.product(source, shop);
                            if  (!Boolean.TRUE.equals(product.getActive()))  throw
BusinessException.badRequest(ErrorCode.PRODUCT_UNAVAILABLE,"Món ngừng bán");
            BigDecimal price = product.getPrice();
             if (item.getToppingIds() != null) for (Long id : item.getToppingIds())
{
                                                                  var    topping    =
branchService.product(productRepository.findById(id).orElseThrow(() ->
```

```
BusinessException.badRequest(ErrorCode.PRODUCT_UNAVAILABLE,"Topping    không    tồn
tại")), shop);
                              if  (!Boolean.TRUE.equals(topping.getActive()))  throw
BusinessException.badRequest(ErrorCode.PRODUCT_UNAVAILABLE,"Topping ngừng bán");
                price = price.add(topping.getPrice());
            }
                                                                          subtotal      =
subtotal.add(price.multiply(BigDecimal.valueOf(item.getQuantity())));
        }
               var  draft  =  Order.builder().shop(shop).account(actor  !=  null  &&
actor.isAuthenticated()
                  ? accountRepository.findById(actor.getAccountId()).orElse(null) :
null).build();
        var discount = applyVoucher(draft, request.getVoucherCode(), subtotal,
                                                          computeDiscount(subtotal,
settingsRepository.findById(1).orElse(null)), false);
                                                                                 return
Map.of("shopId",shop.getId(),"subtotal",subtotal,"discountAmount",discount.discoun
tAmount,"totalAmount",discount.totalAmount,"shippingFee",BigDecimal.ZERO);
    }
```

2.1.14.4.Giải thích code

Ô mã giảm giá đã bỏ khỏi giao diện khách; backend còn nhánh voucher cho tính tương thích và API quản trị.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: branchService.resolve(); branchService.assertCustomer(); productRepository.findById(); branchService.product(). Đường dẫn giao diện liên quan: src/main/resources/static/js/customer/delivery-checkout.js; src/main/resources/templates/fragments/delivery-checkout.html.

2.1.15.Gửi đơn mang về qua bước chọn thanh toán

2.1.15.1.Mô tả chức năng

Bảng 2.15. Đặc tả: Gửi đơn mang về qua bước chọn thanh toán

Tiêu chí	Nội dung
Mục đích	Tạo đơn mang về sau khi khách bấm Gửi đơn hàng, chọn phương thức và xác nhận trong dialog.
Tác nhân	Khách hàng
Điều kiện trước	Giỏ không rỗng; quote hợp lệ; cơ sở POS đã được xác định; công thức món hợp lệ.
Đầu vào	Danh sách món/topping, paymentMethod, expectedTotal, Idempotency-Key.
Kết quả / điều kiện sau	Đơn CUSTOMER ở CHO_XAC_NHAN, chưa có Payment. Chỉ xóa giỏ khi API tạo đơn thành công.
Trường hợp lỗi	Replay khác chủ/nội dung, giá đổi, món thiếu công thức, cơ sở ngừng phục vụ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa cơ sở và kiểm tra quyền mua.

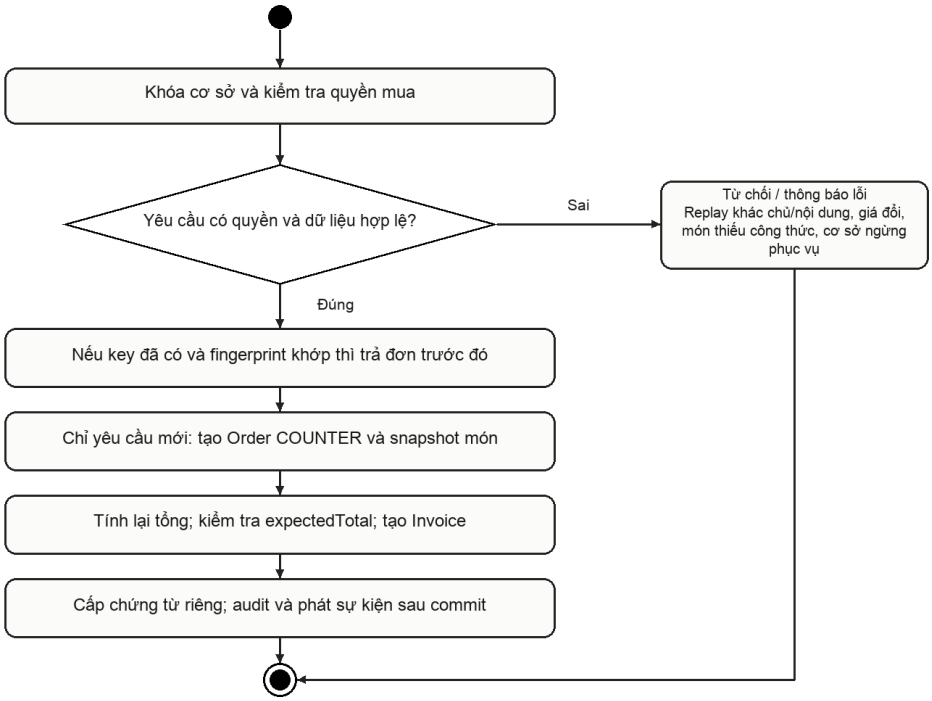
Bước 2. Nếu key đã có và fingerprint khớp thì trả đơn trước đó.

Bước 3. Chỉ yêu cầu mới: tạo Order COUNTER và snapshot món.

Bước 4. Tính lại tổng; kiểm tra expectedTotal; tạo Invoice.

Bước 5. Cấp chứng từ riêng; audit và phát sự kiện sau commit.

2.1.15.2.Sơ đồ hoạt động



Hình 2.15. Hoạt động: Gửi đơn mang về qua bước chọn thanh toán

2.1.15.3.Code xử lý chức năng

Đoạn mã 2.15. OrderService.createCounterOrder — Gửi đơn mang về qua bước chọn thanh toán

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.createCounterOrder; dòng 219–285.

```
                                        public            OrderDto.OrderResponse
createCounterOrder(OrderDto.CreateCounterOrderRequest request, CurrentActor actor,
String idempotencyKey) {
                if  (idempotencyKey  !=  null  &&  (idempotencyKey.isBlank()  ||
idempotencyKey.length() > 64))
            throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED,"Mã yêu
cầu không hợp lệ");
        if (idempotencyKey == null) idempotencyKey = UUID.randomUUID().toString();
        var shop = branchService.resolve(request.getShopId());
        branchService.assertCustomer(shop, actor);
                    if   (actor   !=   null   &&   (actor.hasRole(Role.ADMIN)   ||
actor.hasRole(Role.CASHIER)))
                    branchService.assertStaff(Order.builder().shop(shop).build(),
actor.getAccountId());
        String fingerprint = fingerprint(request);
        if (idempotencyKey != null && !idempotencyKey.isBlank()) {
            var existing = orderRepository.findByIdempotencyKey(idempotencyKey);
            if (existing.isPresent()) {
                Order order = existing.get();
```

[Đã lược bỏ các dòng 232–260 của phương thức gốc.]

```
                            if    (request.getExpectedTotal()    !=    null    &&
request.getExpectedTotal().compareTo(discount.totalAmount) != 0)
```

```
                throw BusinessException.conflict(ErrorCode.VALIDATION_FAILED, "Giá
hoặc giảm giá đã thay đổi. Vui lòng kiểm tra tổng tiền mới.");
        Invoice invoice = Invoice.builder()
                .order(order)
                .subtotal(subtotal)
                .discountAmount(discount.discountAmount)
                .totalAmount(discount.totalAmount)
                .snapshotDiscountPercent(discount.snapshotPercent)
                .snapshotDiscountType(discount.discountType)
                .snapshotDiscountValue(discount.configValue)
                .status(InvoiceStatus.HIEU_LUC)
```

[Đã lược bỏ các dòng 272–273 của phương thức gốc.]

```
        invoice = invoiceRepository.save(invoice);
        var issued = guestAccessService.issueCounterOrderToken(order, 24);
        auditRepository.save(BusinessAudit.builder()
                .orderId(order.getId())
                .accountId(creator != null ? creator.getId() : null)
                .action("COUNTER_ORDER_CREATED")
                .afterState(order.getStatus().name())
                .createdAt(Instant.now())
                .build());
                        realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"ORDER_CREATED",    order.getId().toString(),    null,    "1",    Map.of("orderId",
order.getId(), "status", order.getStatus().name()));
                  return  mapToOrderResponse(order,  invoice,  savedItems,  null,
issued.rawToken());
    }
```

2.1.15.4.Giải thích code

Đóng hộp thanh toán chưa tạo đơn; lựa chọn tiền mặt/chuyển khoản không tự làm PAID.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: branchService.resolve(); branchService.assertCustomer(); branchService.assertStaff(); orderRepository.findByIdempotencyKey(). Đường dẫn giao diện liên quan: src/main/resources/templates/customer/cart.html; src/main/resources/static/js/customer/customer.js; src/main/resources/static/js/customer/delivery-checkout.js.

2.1.16.Gửi đơn tại bàn theo phiên được xác thực

2.1.16.1.Mô tả chức năng

Bảng 2.16. Đặc tả: Gửi đơn tại bàn theo phiên được xác thực

Tiêu chí	Nội dung
Mục đích	Gắn đơn mới vào đúng bàn, đúng phiên và cơ sở phục vụ.
Tác nhân	Khách hàng
Điều kiện trước	Có QR đúng bàn hoặc token phiên còn hạn; nhân viên POS có quyền tương ứng.
Đầu vào	tableId, expectedSessionId, qrCode hoặc tableToken, items và phương thức.
Kết quả / điều kiện sau	Đơn TABLE tham chiếu phiên xác thực; người khác không được truy cập lịch sử từ ID bàn.
Trường hợp lỗi	Giả mạo QR/token, đổi cơ sở, phiên cũ/đóng hoặc món không thể phục vụ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa bàn và kiểm tra bằng chứng truy cập.

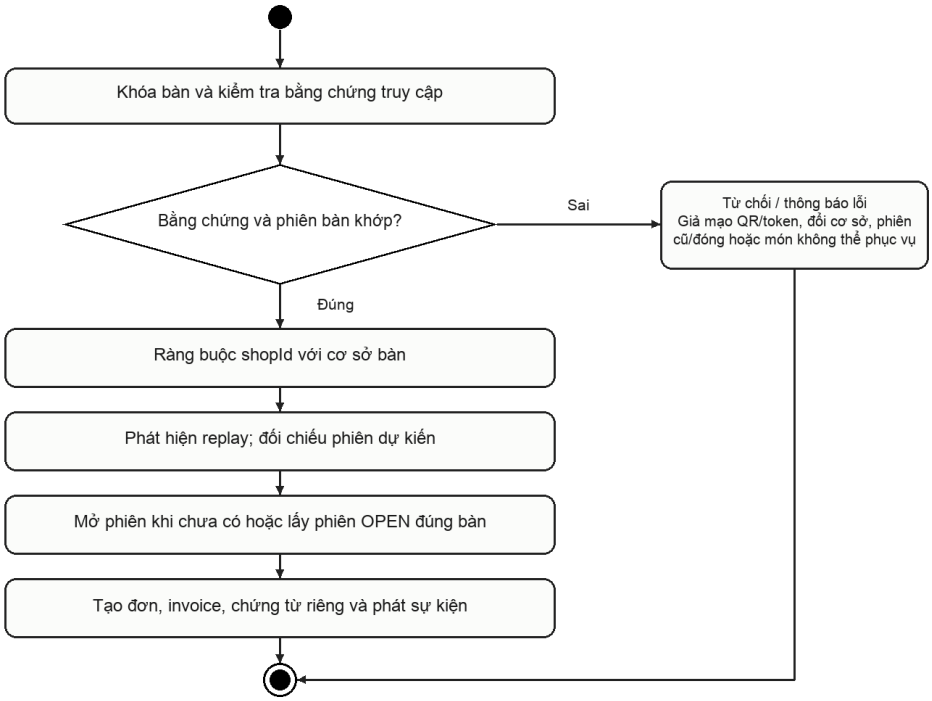
Bước 2. Ràng buộc shopId với cơ sở bàn.

Bước 3. Phát hiện replay; đối chiếu phiên dự kiến.

Bước 4. Mở phiên khi chưa có hoặc lấy phiên OPEN đúng bàn.

Bước 5. Tạo đơn, invoice, chứng từ riêng và phát sự kiện.

2.1.16.2.Sơ đồ hoạt động



Hình 2.16. Hoạt động: Gửi đơn tại bàn theo phiên được xác thực

2.1.16.3.Code xử lý chức năng

Đoạn mã 2.16. OrderService.createTableOrder — Gửi đơn tại bàn theo phiên được xác thực

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.createTableOrder; dòng 82–216.

```
                                                public            OrderDto.OrderResponse
createTableOrder(OrderDto.CreateTableOrderRequest   request,   CurrentActor   actor,
String idempotencyKey) {
                  if   (idempotencyKey   !=   null   &&   (idempotencyKey.isBlank()   ||
idempotencyKey.length() > 64))
             throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED,"Mã yêu
cầu không hợp lệ");
        if (idempotencyKey == null) idempotencyKey = UUID.randomUUID().toString();
        if (request == null || request.getTableId() == null)
             throw BusinessException.forbidden(ErrorCode.ACCESS_DENIED,"Hãy quét mã
QR bàn hợp lệ");
        DiningTable table = tableRepository.findByIdWithLock(request.getTableId())
                                                                  .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.TABLE_NOT_FOUND, "Không tìm thấy bàn"));
             boolean   staff   =   actor   !=   null   &&   (actor.hasRole(Role.CASHIER)   ||
actor.hasRole(Role.ADMIN));
        if (!staff) {
                         boolean   qrValid   =   request.getQrCode()   !=   null   &&   !
request.getQrCode().isBlank()
                                                                                  &&
java.security.MessageDigest.isEqual(request.getQrCode().getBytes(java.nio.charset.
StandardCharsets.UTF_8),

 table.getQrCode().getBytes(java.nio.charset.StandardCharsets.UTF_8));
```

[Đã lược bỏ các dòng 95–97 của phương thức gốc.]

```
                                    if    (!qrValid    &&    !sessionValid)    throw
BusinessException.forbidden(ErrorCode.ACCESS_DENIED, "Hãy quét mã QR bàn hợp lệ");
        }
        var shop = branchService.tableBranch(table);
        branchService.assertCustomer(shop, actor);
                                if    (request.getShopId()    !=    null    &&    !
request.getShopId().equals(shop.getId()))
            throw BusinessException.forbidden(ErrorCode.ACCESS_DENIED, "Không được
đổi cơ sở của bàn");
          if (staff) branchService.assertStaff(Order.builder().shop(shop).build(),
actor.getAccountId());
        String fingerprint = fingerprint(request);
        if (idempotencyKey != null && !idempotencyKey.isBlank()) {
            var existing = orderRepository.findByIdempotencyKey(idempotencyKey);
            if (existing.isPresent()) {
```

[Đã lược bỏ các dòng 109–204 của phương thức gốc.]

```
                .orderId(order.getId())
                   .accountId(customerAccount != null ? customerAccount.getId() :
null)
                .action("ORDER_CREATED")
                .afterState(order.getStatus().name())
                .createdAt(Instant.now())
                .build());
                       realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"ORDER_CREATED",    order.getId().toString(),    session.getId().toString(),    "1",
Map.of("orderId", order.getId(), "status", order.getStatus().name()));
             realtimeEventPublisher.publishAfterCommit("/topic/table-sessions/"  +
session.getId(),             "ORDER_CREATED",             order.getId().toString(),
session.getId().toString(),   "1",   Map.of("orderId",   order.getId(),   "status",
order.getStatus().name()));
          // Per-order guest proof: order tracking must never reuse a table-wide
access token.
        var receiptToken = guestAccessService.issueCounterOrderToken(order, 24);
         return mapToOrderResponse(order, invoice, savedItems, rawGuestTableToken,
receiptToken.rawToken());
    }
```

2.1.16.4.Giải thích code

Giỏ riêng của khách không xóa giỏ phiên POS dùng chung. Không cho khách tự gán bàn bằng ID có thể đoán.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: tableRepository.findByIdWithLock(); guestAccessService.validateTableToken(); branchService.tableBranch(); branchService.assertCustomer(). Đường dẫn giao diện liên quan: src/main/resources/templates/customer/cart.html; src/main/resources/static/js/customer/customer.js.

2.1.17.Thông báo chuyển khoản để thu ngân đối soát

2.1.17.1.Mô tả chức năng

Bảng 2.17. Đặc tả: Thông báo chuyển khoản để thu ngân đối soát

Tiêu chí	Nội dung
Mục đích	Ghi nhận khách đã chuyển khoản để thu ngân kiểm tra giao dịch thực tế.
Tác nhân	Khách hàng
Điều kiện trước	Controller xác minh chủ đơn hoặc chứng từ riêng; hóa đơn chưa thu, đơn chưa hủy.
Đầu vào	Mã đơn, method=BANK_TRANSFER, reference và khóa yêu cầu.
Kết quả / điều kiện sau	Đơn giữ trạng thái chưa thanh toán; chưa tạo Payment và không tự xác nhận tiền ngân hàng.
Trường hợp lỗi	Không có quyền, hóa đơn hủy, đã được ghi thu hoặc dữ liệu phương thức sai.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra quyền đọc đơn ở controller.

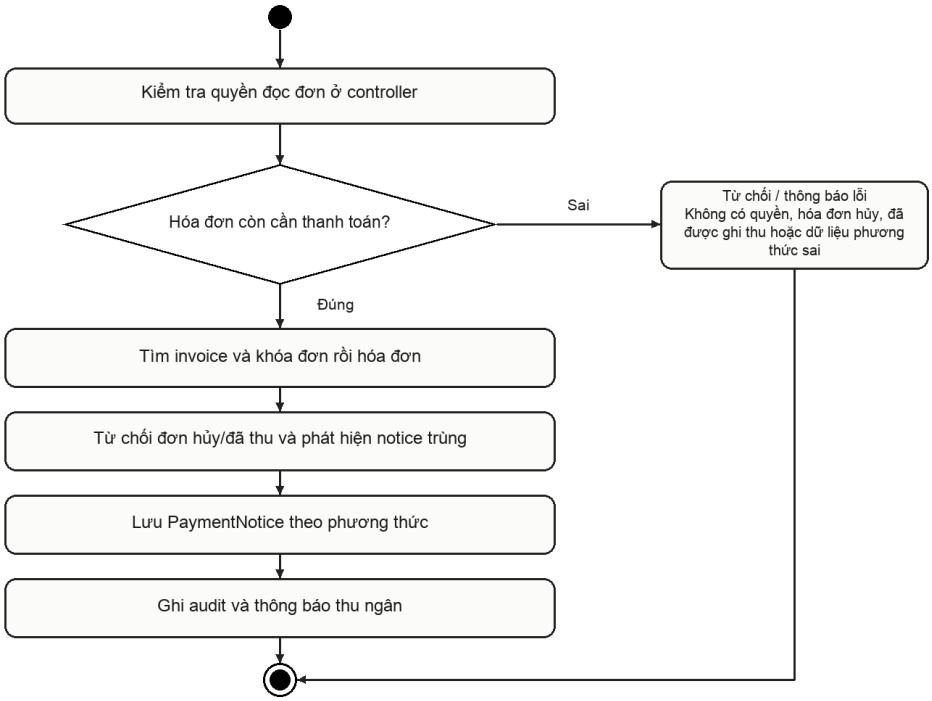
Bước 2. Tìm invoice và khóa đơn rồi hóa đơn.

Bước 3. Từ chối đơn hủy/đã thu và phát hiện notice trùng.

Bước 4. Lưu PaymentNotice theo phương thức.

Bước 5. Ghi audit và thông báo thu ngân.

2.1.17.2.Sơ đồ hoạt động



Hình 2.17. Hoạt động: Thông báo chuyển khoản để thu ngân đối soát

2.1.17.3.Code xử lý chức năng

Đoạn mã 2.17. PaymentService.createNoticeForOrder — Thông báo chuyển khoản để thu ngân đối soát

Nguồn: src/main/java/vn/edu/ute/milktea/service/payment/PaymentService.java; PaymentService.createNoticeForOrder; dòng 307–311.

```
    public void createNoticeForOrder(Long orderId, PaymentDto.PaymentNoticeRequest
request, String idempotencyKey) {
        Invoice invoice = invoiceRepository.findByOrderId(orderId)
                                                        .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED, "Không tìm thấy hóa đơn
cho đơn hàng #" + orderId));
        createNotice(invoice.getId(), request, idempotencyKey);
    }
```

2.1.17.4.Giải thích code

Tài khoản nhận tiền lấy từ GlobalSettings; URL VietQR chỉ có khi cấu hình BIN/ tài khoản phù hợp. Chưa có xác nhận giao dịch ngân hàng tự động cho luồng POS này.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: invoiceRepository.findByOrderId(). Đường dẫn giao diện liên quan: src/main/resources/templates/ customer/order-confirmation.html; src/main/resources/static/js/customer/order-confirmation.js.

2.1.18.Theo dõi chứng từ đơn vừa đặt

2.1.18.1.Mô tả chức năng

Bảng 2.18. Đặc tả: Theo dõi chứng từ đơn vừa đặt

Tiêu chí	Nội dung
Mục đích	Cho phép xem một đơn được sở hữu hoặc được cấp capability riêng.
Tác nhân	Khách hàng
Điều kiện trước	Có quyền với chính đơn cần xem.
Đầu vào	orderId và JWT tài khoản hoặc token riêng của đúng đơn.
Kết quả / điều kiện sau	Trả dữ liệu một chứng từ; token bàn không cấp quyền xem mọi đơn trong phiên.
Trường hợp lỗi	Không tìm thấy đơn hoặc token/tài khoản không có quyền.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm Order theo ID.

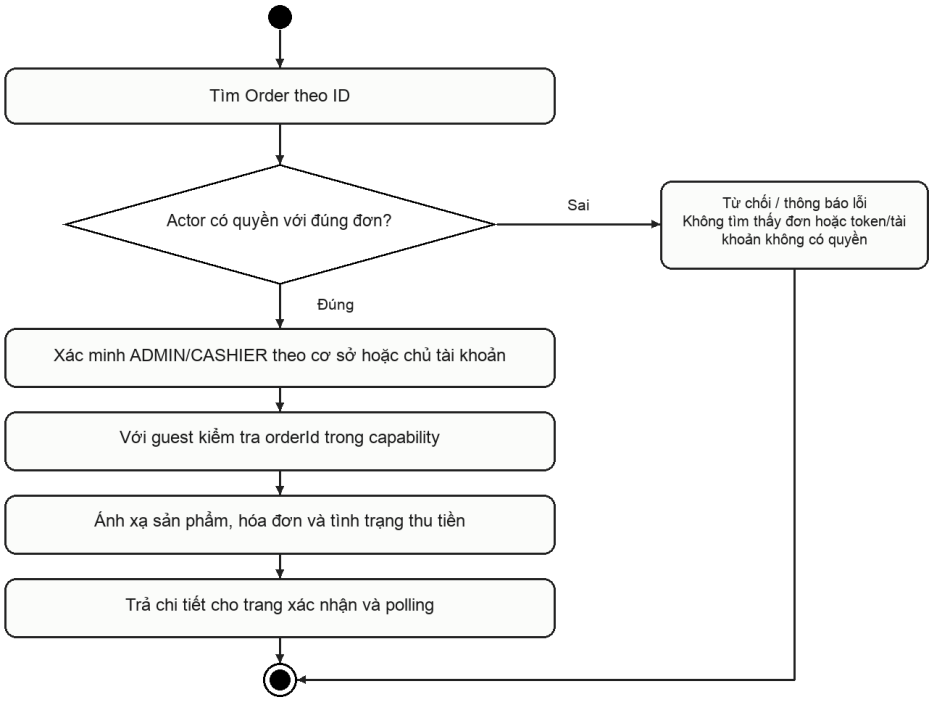
Bước 2. Xác minh ADMIN/CASHIER theo cơ sở hoặc chủ tài khoản.

Bước 3. Với guest kiểm tra orderId trong capability.

Bước 4. Ánh xạ sản phẩm, hóa đơn và tình trạng thu tiền.

Bước 5. Trả chi tiết cho trang xác nhận và polling.

2.1.18.2.Sơ đồ hoạt động



Hình 2.18. Hoạt động: Theo dõi chứng từ đơn vừa đặt

2.1.18.3.Code xử lý chức năng

Đoạn mã 2.18. OrderService.getOrderForCustomer — Theo dõi chứng từ đơn vừa đặt

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.getOrderForCustomer; dòng 737–756.

```
     public OrderDto.OrderResponse getOrderForCustomer(Long orderId, CurrentActor
actor) {
        Order order = orderRepository.findById(orderId)
                                                          .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT,  "Không  tìm  thấy  đơn
hàng"));
        if (actor == null) {
             throw BusinessException.forbidden(ErrorCode.ACCESS_DENIED, "Bạn không
có quyền xem đơn hàng này");
        }
                                                    boolean       allowed       =
actor.hasRole(vn.edu.ute.milktea.entity.account.Role.CASHIER)                    ||
actor.hasRole(vn.edu.ute.milktea.entity.account.Role.ADMIN);
           if (allowed) { branchService.assertStaff(order, actor.getAccountId());
return mapOrderToResponse(order); }
        if (actor.getAccountId() != null && order.getAccount() != null) {
            allowed = actor.getAccountId().equals(order.getAccount().getId());
        } else if (actor.getOrderId() != null) {
            // An opaque token authorizes only this exact order, not every order
            // made during an open dining-table session.
            allowed = actor.getOrderId().equals(order.getId());
        }
        if (!allowed) {
```

```
            throw BusinessException.forbidden(ErrorCode.ACCESS_DENIED, "Bạn không
có quyền xem đơn hàng này");
        }
        return mapOrderToResponse(order);
    }
```

2.1.18.4.Giải thích code

Tiền mặt hiện yêu cầu trả tại quầy; chuyển khoản hiện thông tin cấu hình hoặc thông báo chưa có tài khoản ngân hàng.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findById(); branchService.assertStaff(). Đường dẫn giao diện liên quan: src/main/resources/templates/customer/order-confirmation.html; src/main/resources/static/js/customer/order-confirmation.js.

2.1.19.Xem lịch sử đơn hàng tài khoản

2.1.19.1.Mô tả chức năng

Bảng 2.19. Đặc tả: Xem lịch sử đơn hàng tài khoản

Tiêu chí	Nội dung
Mục đích	Tra cứu những đơn thuộc tài khoản đăng nhập.
Tác nhân	Khách hàng
Điều kiện trước	Customer history controller yêu cầu tài khoản xác thực.
Đầu vào	JWT tài khoản; không nhận accountId từ trình duyệt làm phạm vi.
Kết quả / điều kiện sau	Danh sách mới nhất trước, dữ liệu chỉ thuộc phạm vi cho phép.
Trường hợp lỗi	Guest không được liệt kê lịch sử chỉ bằng QR/ token phiên bàn.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Xác định actor.

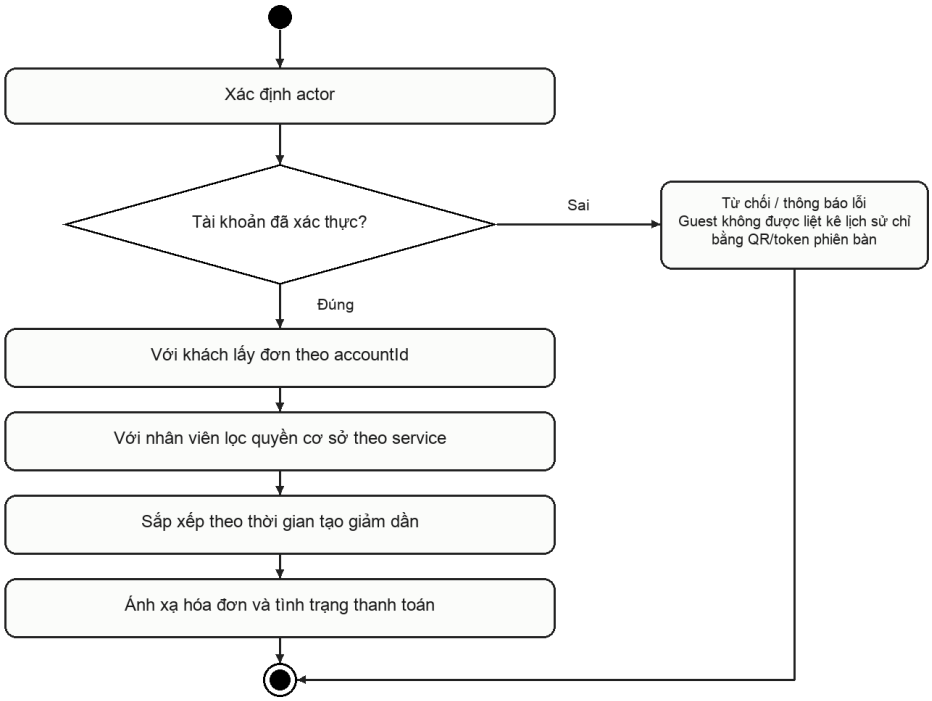
Bước 2. Với khách lấy đơn theo accountId.

Bước 3. Với nhân viên lọc quyền cơ sở theo service.

Bước 4. Sắp xếp theo thời gian tạo giảm dần.

Bước 5. Ánh xạ hóa đơn và tình trạng thanh toán.

2.1.19.2.Sơ đồ hoạt động



Hình 2.19. Hoạt động: Xem lịch sử đơn hàng tài khoản

2.1.19.3.Code xử lý chức năng

Đoạn mã 2.19. OrderService.getOrdersForCustomer — Xem lịch sử đơn hàng tài khoản

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.getOrdersForCustomer; dòng 723–733.

```
    public List<OrderDto.OrderResponse> getOrdersForCustomer(CurrentActor actor) {
        List<Order> orders = new ArrayList<>();
                                            if      (actor      !=      null      &&
(actor.hasRole(vn.edu.ute.milktea.entity.account.Role.CASHIER)                    ||
actor.hasRole(vn.edu.ute.milktea.entity.account.Role.ADMIN))) {
            orders = orderRepository.findAll();
        } else if (actor != null && actor.getAccountId() != null) {
                                                                    orders      =
orderRepository.findByAccountIdOrderByCreatedAtDesc(actor.getAccountId());
        }
        // Neither a QR nor a guest capability can list history. The order receipt
        // is fetched separately with getOrderForCustomer() and a per-order token.
                                                                          return
orders.stream().filter(this::staffCanRead).sorted(java.util.Comparator.comparing(O
rder::getCreatedAt).reversed()).map(this::mapOrderToResponse).toList();
    }
```

2.1.19.4.Giải thích code

Lịch sử giao hàng cũ được lưu riêng; không gộp DeliveryOrder thành Order một cách giả định.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findAll(); orderRepository.findByAccountIdOrderByCreatedAtDesc(). Đường dẫn giao diện liên quan: src/main/resources/templates/customer/orders.html; src/main/resources/static/js/order/orders.js.

2.1.20.Yêu cầu hủy đơn trước chế biến

2.1.20.1.Mô tả chức năng

Bảng 2.20. Đặc tả: Yêu cầu hủy đơn trước chế biến

Tiêu chí	Nội dung
Mục đích	Hủy trực tiếp hoặc xin duyệt theo trạng thái chế biến và việc thu tiền.
Tác nhân	Khách hàng
Điều kiện trước	Là chủ đơn/actor có quyền; đơn chưa DANG_CHE_BIEN hoặc HOAN_THANH.
Đầu vào	orderId và lý do hủy.
Kết quả / điều kiện sau	Hủy không đồng nghĩa tự hoàn tiền; refund là nghiệp vụ riêng có chứng từ.
Trường hợp lỗi	Đơn đã pha/hoàn tất, actor khác hoặc đã có yêu cầu CHO.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa đơn và xác minh quyền.

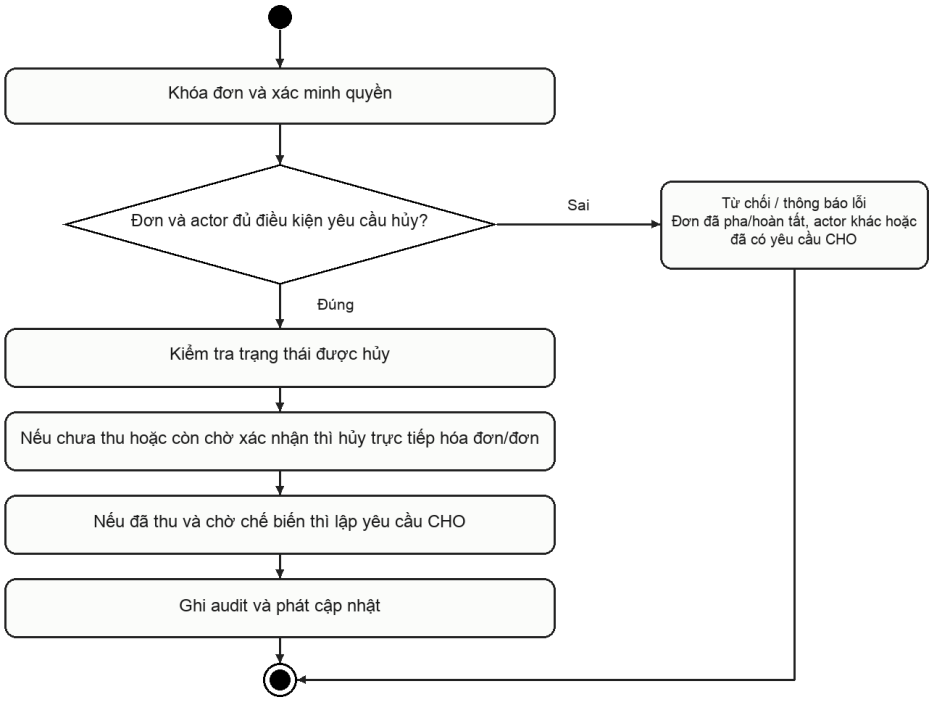
Bước 2. Kiểm tra trạng thái được hủy.

Bước 3. Nếu chưa thu hoặc còn chờ xác nhận thì hủy trực tiếp hóa đơn/đơn.

Bước 4. Nếu đã thu và chờ chế biến thì lập yêu cầu CHO.

Bước 5. Ghi audit và phát cập nhật.

2.1.20.2.Sơ đồ hoạt động



Hình 2.20. Hoạt động: Yêu cầu hủy đơn trước chế biến

2.1.20.3.Code xử lý chức năng

Đoạn mã 2.20. CancellationService.requestCancellation — Yêu cầu hủy đơn trước chế biến

Nguồn: src/main/java/vn/edu/ute/milktea/service/cancellation/CancellationService.java; CancellationService.requestCancellation; dòng 46–134.

```
    public CancellationDto.CancellationResponse requestCancellation(
                Long orderId, CancellationDto.CreateCancellationRequest request,
CurrentActor actor) {

        Order order = orderRepository.findByIdWithLock(orderId)
                                                        .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT,  "Không  tìm  thấy  đơn
hàng"));

        if (!canActorAccessOrder(order, actor)) {
             throw BusinessException.forbidden(ErrorCode.ACCESS_DENIED, "Bạn không
có quyền hủy đơn hàng này");
        }

        // Chỉ được hủy trước khi bếp bắt đầu pha chế
        if (order.getStatus() != OrderStatus.CHO_THANH_TOAN
                && order.getStatus() != OrderStatus.CHO_XAC_NHAN
```

[Đã lược bỏ các dòng 59–59 của phương thức gốc.]

```
            throw BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT,
                            "Không thể hủy đơn đã hoặc đang được pha chế: " +
order.getStatus());
        }
```

```
                                                    Invoice       invoice       =
invoiceRepository.findByOrderId(order.getId()).orElse(null);
                              boolean    isPaid    =    invoice    !=    null    &&
paymentRepository.existsByInvoiceId(invoice.getId());

          // Đơn Chờ xác nhận hoặc Đơn Chờ chế biến chưa thu tiền: Hủy trực tiếp
ngay lập tức
         if (order.getStatus() == OrderStatus.CHO_XAC_NHAN || (order.getStatus() ==
OrderStatus.CHO_CHE_BIEN && !isPaid)) {
            order.setStatus(OrderStatus.DA_HUY);
            orderRepository.save(order);
```

[Đã lược bỏ các dòng 71–122 của phương thức gốc.]

```
                .createdAt(Instant.now())
                .build());

        publish(order);
        return CancellationDto.CancellationResponse.builder()
                .id(cancelReq.getId())
                .orderId(order.getId())
                .reason(cancelReq.getReason())
                .status(cancelReq.getStatus())
                .createdAt(cancelReq.getCreatedAt())
                .build();
    }
```

2.1.20.4.Giải thích code

Không ghi trả nguyên liệu đã tiêu hao bằng một thao tác hủy đơn; không mô tả chức năng trả hàng POS sau khi uống nếu source không có.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findByIdWithLock(); invoiceRepository.findByOrderId(); paymentRepository.existsByInvoiceId(); orderRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/customer/orders.html; src/main/resources/static/js/order/orders.js.

2.1.21.Xem và sửa thông tin cá nhân

2.1.21.1.Mô tả chức năng

Bảng 2.21. Đặc tả: Xem và sửa thông tin cá nhân

Tiêu chí	Nội dung
Mục đích	Cập nhật thông tin liên hệ thuộc tài khoản đang đăng nhập.
Tác nhân	Khách hàng
Điều kiện trước	Actor có accountId hợp lệ.
Đầu vào	Họ tên, số điện thoại qua UpdateProfileRequest.
Kết quả / điều kiện sau	Không thay đổi vai trò hay quyền quản trị qua chức năng sửa hồ sơ.
Trường hợp lỗi	Không tìm thấy tài khoản hoặc dữ liệu không thỏa validation phía giao diện/API.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm Account hiện tại.

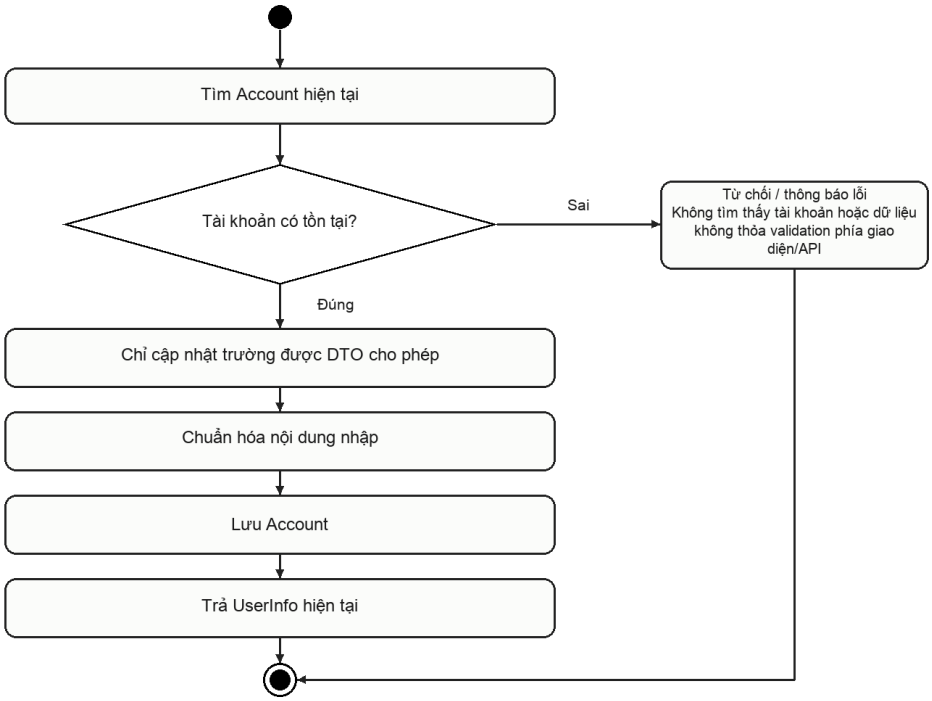
Bước 2. Chỉ cập nhật trường được DTO cho phép.

Bước 3. Chuẩn hóa nội dung nhập.

Bước 4. Lưu Account.

Bước 5. Trả UserInfo hiện tại.

2.1.21.2.Sơ đồ hoạt động



Hình 2.21. Hoạt động: Xem và sửa thông tin cá nhân

2.1.21.3.Code xử lý chức năng

Đoạn mã 2.21. AuthService.updateProfile — Xem và sửa thông tin cá nhân

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.updateProfile; dòng 339–358.

```
                public AuthDto.UserInfo updateProfile(Long accountId,
AuthDto.UpdateProfileRequest request) {
        Account account = accountRepository.findById(accountId)
                                                        .orElseThrow(() ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED, "Không tìm thấy tài
khoản"));

        if (request.getFullName() != null) {
            account.setFullName(request.getFullName().trim());
        }
        if (request.getPhone() != null) {
            account.setPhone(request.getPhone().trim());
        }
        accountRepository.save(account);

        return AuthDto.UserInfo.builder()
                .id(account.getId())
                .email(account.getEmail())
                .fullName(account.getFullName())
                .phone(account.getPhone())
                .role(account.getRole())
                .build();
    }
```

2.1.21.4.Giải thích code

Ảnh đại diện có endpoint upload riêng, không phải trường tùy ý sửa avatar nội bộ.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findById(); accountRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/profile.html.

2.1.22.Đổi mật khẩu khi đang đăng nhập

2.1.22.1.Mô tả chức năng

Bảng 2.22. Đặc tả: Đổi mật khẩu khi đang đăng nhập

Tiêu chí	Nội dung
Mục đích	Cho phép người dùng chủ động đổi mật khẩu bằng mật khẩu hiện tại.
Tác nhân	Khách hàng
Điều kiện trước	Đang đăng nhập; mật khẩu hiện tại phải khớp hash trong DB.
Đầu vào	currentPassword và newPassword.
Kết quả / điều kiện sau	Thông tin đăng nhập cũ bị thay thế và JWT phiên bản cũ không còn hiệu lực.
Trường hợp lỗi	Mật khẩu cũ sai, tài khoản không tồn tại hoặc DTO mật khẩu mới không hợp lệ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm tài khoản hiện tại.

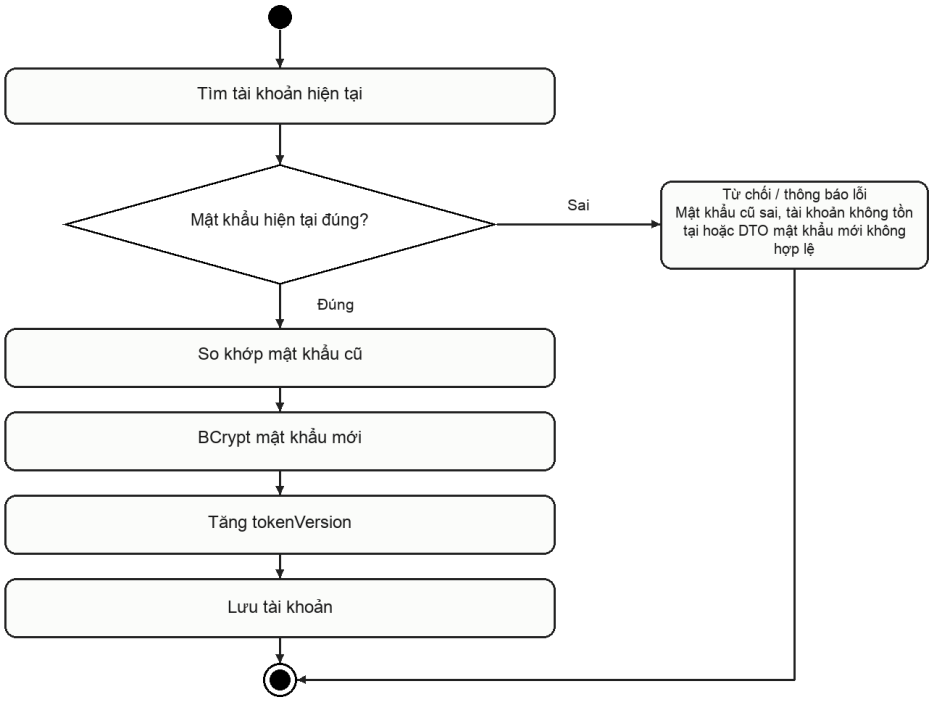
Bước 2. So khớp mật khẩu cũ.

Bước 3. BCrypt mật khẩu mới.

Bước 4. Tăng tokenVersion.

Bước 5. Lưu tài khoản.

2.1.22.2.Sơ đồ hoạt động



Hình 2.22. Hoạt động: Đổi mật khẩu khi đang đăng nhập

2.1.22.3.Code xử lý chức năng

Đoạn mã 2.22. AuthService.changePassword — Đổi mật khẩu khi đang đăng nhập

Nguồn: src/main/java/vn/edu/ute/milktea/service/auth/AuthService.java; AuthService.changePassword; dòng 361–372.

```
       public void changePassword(Long accountId, AuthDto.ChangePasswordRequest
request) {
        Account account = accountRepository.findById(accountId)
                                                              .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED,   "Không   tìm   thấy   tài
khoản"));

                           if    (!passwordEncoder.matches(request.getOldPassword(),
account.getPasswordHash())) {
            throw BusinessException.badRequest(ErrorCode.AUTH_INVALID_CREDENTIALS,
"Mật khẩu hiện tại không chính xác");
        }

        account.setPasswordHash(passwordEncoder.encode(request.getNewPassword()));
        account.setTokenVersion(account.getTokenVersion() + 1);
        accountRepository.save(account);
    }
```

2.1.22.4.Giải thích code

Đây là luồng khác với khôi phục mật khẩu bằng OTP; không yêu cầu email OTP trong method này.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findById(); passwordEncoder.matches(); passwordEncoder.encode(); accountRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/profile.html.

2.1.23.Đánh dấu yêu thích và ghi nhận món đã xem

2.1.23.1.Mô tả chức năng

Bảng 2.23. Đặc tả: Đánh dấu yêu thích và ghi nhận món đã xem

Tiêu chí	Nội dung
Mục đích	Lưu lựa chọn và lịch sử xem phục vụ trải nghiệm tài khoản.
Tác nhân	Khách hàng
Điều kiện trước	Có accountId và sản phẩm hợp lệ.
Đầu vào	productId; enabled khi yêu thích hoặc cờ viewed qua endpoint riêng.
Kết quả / điều kiện sau	Dữ liệu quan tâm thuộc từng tài khoản, không phát sinh đơn hàng.
Trường hợp lỗi	Không có tài khoản hoặc mã món không hợp lệ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Xác định chủ tài khoản.

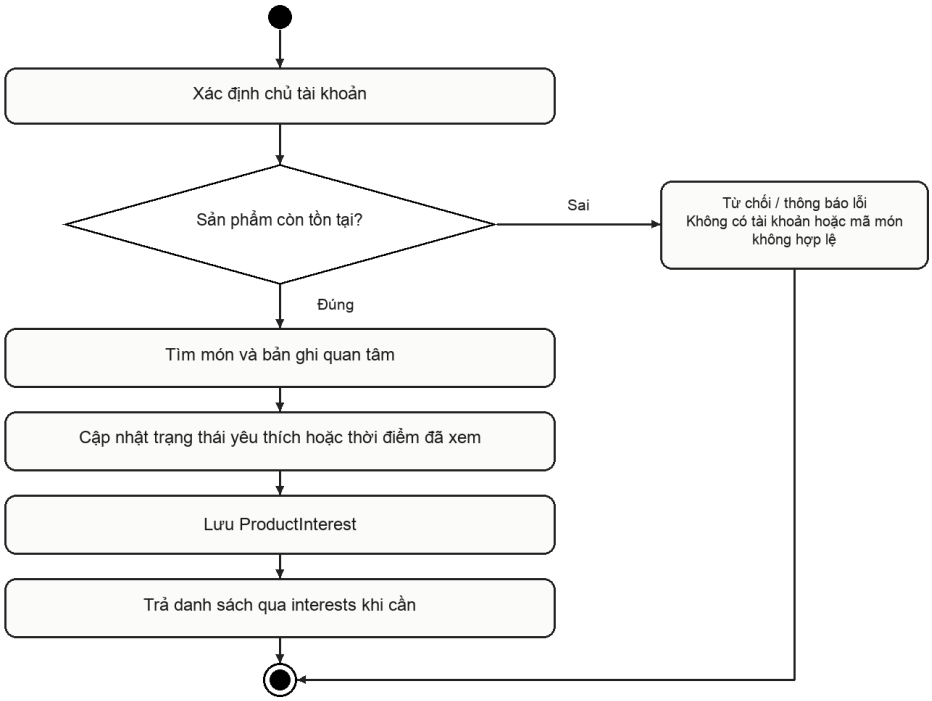
Bước 2. Tìm món và bản ghi quan tâm.

Bước 3. Cập nhật trạng thái yêu thích hoặc thời điểm đã xem.

Bước 4. Lưu ProductInterest.

Bước 5. Trả danh sách qua interests khi cần.

2.1.23.2.Sơ đồ hoạt động



Hình 2.23. Hoạt động: Đánh dấu yêu thích và ghi nhận món đã xem

2.1.23.3.Code xử lý chức năng

Đoạn mã 2.23. CustomerDataService.interest — Đánh dấu yêu thích và ghi nhận món đã xem

Nguồn: src/main/java/vn/edu/ute/milktea/service/account/CustomerDataService.java; CustomerDataService.interest; dòng 139–145.

```
            public    void    interest(CurrentActor    actor,Long    productId,Boolean
favorite,boolean viewed) {
        Account account=lock(actor);Product p=available(productId);
                                                                                var
interest=interests.findByAccountIdAndProductId(account.getId(),p.getId()).orElseGe
t(()->{var    i=new    ProductInterest();i.setAccount(account);i.setProduct(p);return
i;});
        if(favorite!=null)interest.setFavorite(favorite);
        if(viewed)interest.setLastViewedAt(Instant.now());
        interests.save(interest);
    }
```

2.1.23.4.Giải thích code

Phân hệ preferences có đường dẫn thực /api/v1/account/preferences; lịch sử quan tâm không thay thế lịch sử mua hàng.

Method liên quan: CustomerDataService.interest. Controller: CustomerDataController. Giao diện: src/main/resources/templates/customer/account-data.html; src/main/resources/static/js/customer/account-data.js; src/main/resources/static/js/customer/marketplace.js.

2.1.24.Đánh giá món đã mua và hoàn tất

2.1.24.1.Mô tả chức năng

Bảng 2.24. Đặc tả: Đánh giá món đã mua và hoàn tất

Tiêu chí	Nội dung
Mục đích	Ghi nhận đánh giá có điều kiện mua hàng và ngăn đánh giá lặp.
Tác nhân	Khách hàng
Điều kiện trước	Tài khoản có đơn chứa món; POS phải HOAN_THANH và đã thu đủ; comment ít nhất 50 ký tự.
Đầu vào	productId, orderId, stars, comment, delivery.
Kết quả / điều kiện sau	Đánh giá được gắn với bằng chứng đơn tương ứng.
Trường hợp lỗi	Chưa mua/chưa trả tiền, bình luận ngắn, đơn không khớp hoặc đánh giá trùng.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa Account và kiểm tra chưa đánh giá món.

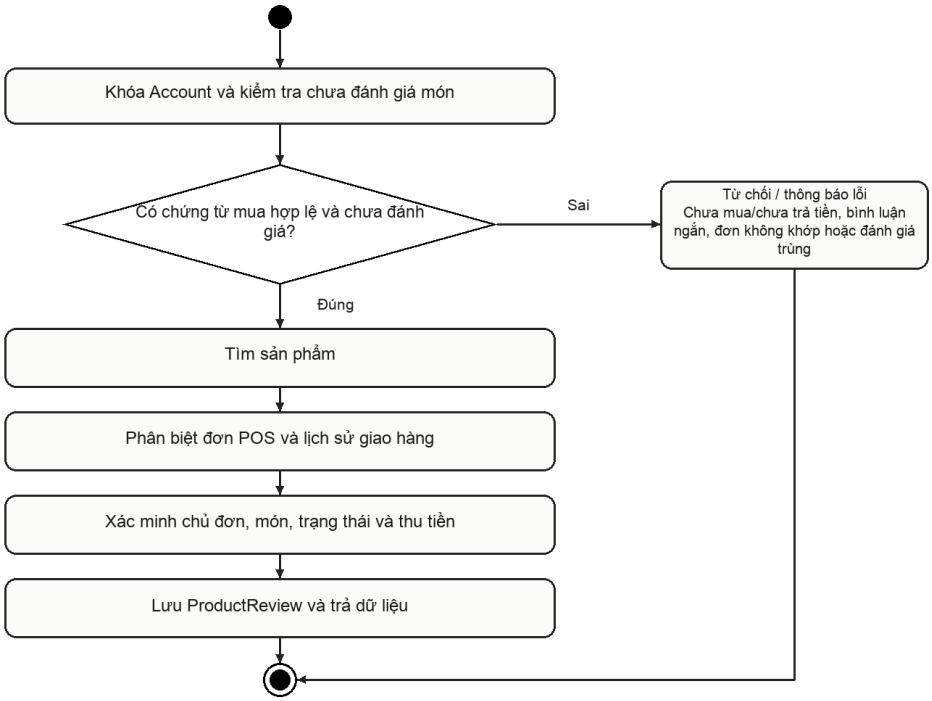
Bước 2. Tìm sản phẩm.

Bước 3. Phân biệt đơn POS và lịch sử giao hàng.

Bước 4. Xác minh chủ đơn, món, trạng thái và thu tiền.

Bước 5. Lưu ProductReview và trả dữ liệu.

2.1.24.2.Sơ đồ hoạt động



Hình 2.24. Hoạt động: Đánh giá món đã mua và hoàn tất

2.1.24.3.Code xử lý chức năng

Đoạn mã 2.24. ReviewService.create — Đánh giá món đã mua và hoàn tất

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/ReviewService.java; ReviewService.create; dòng 32–50.

```
 public ReviewView create(CurrentActor actor,Long productId,ReviewInput input){
  Long accountId=CustomerDataService.accountId(actor);
   var  account=accounts.findByIdWithLock(accountId).orElseThrow(()->invalid("Không
có tài khoản"));
  if(input.comment().trim().length()<50)throw invalid("Bình luận tối thiểu 50 ký
tự");
               if(reviews.existsByAccountIdAndProductId(accountId,productId))throw
BusinessException.conflict(ErrorCode.VALIDATION_FAILED,"Bạn đã đánh giá món này");
  var product=products.findById(productId).orElseThrow(()->invalid("Món không tồn
tại"));
                                      ProductReview                          r=new
ProductReview();r.setAccount(account);r.setProduct(product);r.setStars(input.stars
());r.setComment(input.comment().trim());
  if(input.delivery()){
     var  order=deliveries.findById(input.orderId()).orElseThrow(()->invalid("Không
có đơn hàng"));
              if(!order.getAccount().getId().equals(accountId)||order.getStatus()!
=DeliveryOrder.Status.DELIVERED||!order.isPaid()||
order.getLines().stream().noneMatch(l-
>l.getProduct().getId().equals(productId)))throw
BusinessException.forbidden(ErrorCode.ACCESS_DENIED,"Chỉ đánh giá món đã mua và
nhận thành công");
   r.setDeliveryOrder(order);
  }else{
```

```
    long eligible=em.createQuery("select count(i) from OrderItem i, Payment pay
where       i.order.id=:orderId       and       i.product.id=:productId       and
i.order.account.id=:accountId                                                   and
i.order.status=vn.edu.ute.milktea.entity.order.OrderStatus.HOAN_THANH           and
pay.invoice.order=i.order                                                       and
pay.invoice.status=vn.edu.ute.milktea.entity.order.InvoiceStatus.HIEU_LUC       and
pay.amount>=pay.invoice.totalAmount",Long.class)
     .setParameter("orderId",input.orderId()).setParameter("productId",productId).s
etParameter("accountId",accountId).getSingleResult();
     if(eligible==0)throw  BusinessException.forbidden(ErrorCode.ACCESS_DENIED,"Chỉ
đánh giá món đã mua, hoàn thành và thanh toán");
          r.setCounterOrder(counterOrders.findById(input.orderId()).orElseThrow(()-
>invalid("Không có đơn hàng")));
  }
  return view(reviews.save(r));
 }
```

2.1.24.4.Giải thích code

Chưa có API kiểm duyệt/phản hồi đánh giá trong ReviewController hiện tại; không đưa chức năng đó thành đã triển khai.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: CustomerDataService.accountId(). Đường dẫn giao diện liên quan: src/main/resources/templates/customer/orders.html; src/main/resources/static/js/order/orders.js; src/main/resources/static/js/customer/marketplace.js.

2.1.25.Đính kèm ảnh hoặc video cho đánh giá

2.1.25.1.Mô tả chức năng

Bảng 2.25. Đặc tả: Đính kèm ảnh hoặc video cho đánh giá

Tiêu chí	Nội dung
Mục đích	Lưu minh họa đánh giá qua dịch vụ ảnh đã tích hợp.
Tác nhân	Khách hàng
Điều kiện trước	Là chủ đánh giá; Cloudinary được cấu hình hợp lệ.
Đầu vào	reviewId và MultipartFile JPEG/PNG/WebP/MP4 trong giới hạn 20MB.
Kết quả / điều kiện sau	Chỉ URL/publicId cần quản lý được lưu vào chứng từ media; không lưu file lớn trong bảng.
Trường hợp lỗi	Tệp sai nội dung/kích thước, không có quyền hoặc dịch vụ ảnh lỗi.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm đánh giá và xác minh chủ sở hữu.

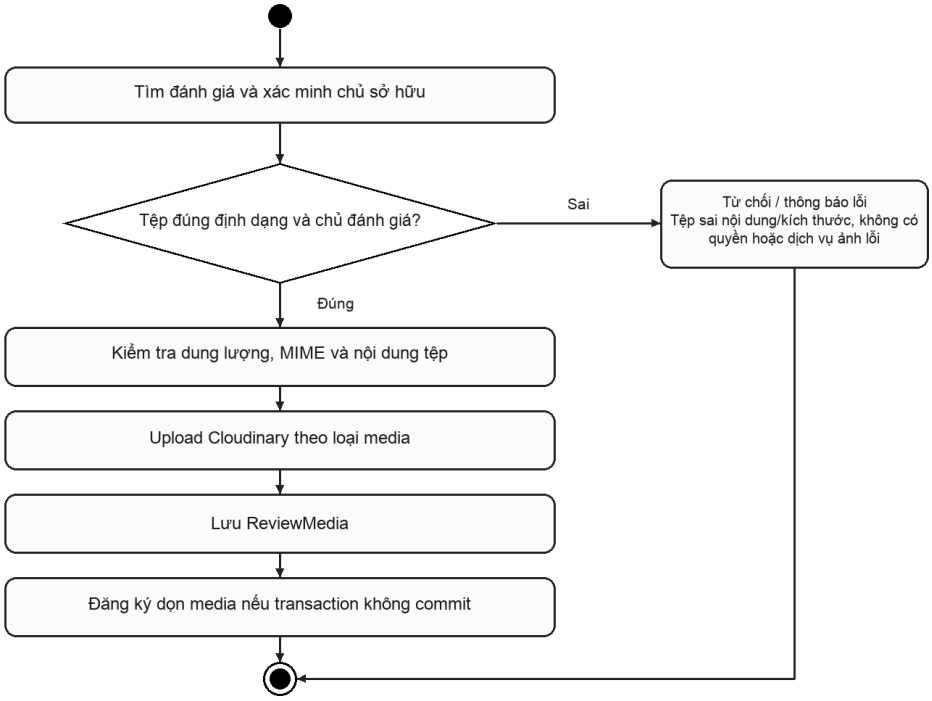
Bước 2. Kiểm tra dung lượng, MIME và nội dung tệp.

Bước 3. Upload Cloudinary theo loại media.

Bước 4. Lưu ReviewMedia.

Bước 5. Đăng ký dọn media nếu transaction không commit.

2.1.25.2.Sơ đồ hoạt động



Hình 2.25. Hoạt động: Đính kèm ảnh hoặc video cho đánh giá

2.1.25.3.Code xử lý chức năng

Đoạn mã 2.25. ReviewService.media — Đính kèm ảnh hoặc video cho đánh giá

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/ReviewService.java; ReviewService.media; dòng 55–75.

```
 public ReviewView media(CurrentActor actor,Long reviewId,MultipartFile file){
                                                                                Long
accountId=CustomerDataService.accountId(actor);accounts.findByIdWithLock(accountId
).orElseThrow(()->invalid("Không có tài khoản"));
    var  review=reviews.findById(reviewId).orElseThrow(()->invalid("Không  có  đánh
giá"));
                            if(!review.getAccount().getId().equals(accountId))throw
BusinessException.forbidden(ErrorCode.ACCESS_DENIED,"Không  có  quyền  sửa  đánh
giá");
  if(review.getMedia().size()>=5)throw invalid("Mỗi đánh giá tối đa 5 ảnh/video");
    if(file==null||file.isEmpty()||file.getSize()>20*1024*1024)throw  invalid("Ảnh/
video không hợp lệ hoặc vượt 20MB");
   if(!images.isCloudinaryConfigured())throw invalid("Chưa cấu hình Cloudinary để
tải ảnh/video");
  try{
                                    byte[]              bytes=file.getBytes();String
mime=Objects.toString(file.getContentType(),"");String type;
                                                                              boolean
jpeg=bytes.length>=3&&(bytes[0]&255)==255&&(bytes[1]&255)==216&&(bytes[2]&255)==25
5;
```

```
     boolean  png=bytes.length>=8&&Arrays.equals(Arrays.copyOf(bytes,8),new  byte[]
{(byte)137,80,78,71,13,10,26,10});
                                    boolean              webp=bytes.length>=12&&new
String(bytes,0,4,java.nio.charset.StandardCharsets.US_ASCII).equals("RIFF")&&new
String(bytes,8,4,java.nio.charset.StandardCharsets.US_ASCII).equals("WEBP");
                                     boolean              mp4=bytes.length>=12&&new
String(bytes,4,4,java.nio.charset.StandardCharsets.US_ASCII).equals("ftyp");
           if((jpeg&&mime.equals("image/jpeg"))||(png&&mime.equals("image/png"))||
(webp&&mime.equals("image/webp"))){type="image";if(bytes.length>5*1024*1024)throw
invalid("Ảnh tối đa 5MB");}
   else if(mp4&&mime.equals("video/mp4"))type="video";else throw invalid("Chỉ nhận
JPEG, PNG, WebP hoặc MP4 có nội dung hợp lệ");
   String publicId="ute-tea/reviews/"+UUID.randomUUID();
                                                                                  var
uploaded=cloudinary.uploader().upload(bytes,Map.of("resource_type",type,"public_id
",publicId,"overwrite",false));
                      TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronization(){@Override  public  void  afterCompletion(int  status)
{if(status!
=STATUS_COMMITTED)try{cloudinary.uploader().destroy(publicId,Map.of("resource_type
",type));}catch(Exception ignored){}}});
                                                    var                    media=new
ReviewMedia();media.setReview(review);media.setMediaType(type);media.setPublicId(p
ublicId);media.setUrl((String)uploaded.get("secure_url"));review.getMedia().add(me
dia);reviews.saveAndFlush(review);return view(review);
         }catch(BusinessException     e){throw     e;}catch(Exception     e){throw
BusinessException.badRequest(ErrorCode.IMAGE_UPLOAD_FAILED,"Không  thể  tải  ảnh/
video. Vui lòng thử lại.");}
 }
```

2.1.25.4.Giải thích code

Đã xác minh source validation và cleanup; phiên báo cáo không upload ảnh/video thật lên tài khoản bên ngoài.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: CustomerDataService.accountId(). Đường dẫn giao diện liên quan: src/main/resources/static/js/customer/marketplace.js.

2.1.26.Quản lý địa chỉ đã lưu cho dữ liệu tương thích

2.1.26.1.Mô tả chức năng

Bảng 2.26. Đặc tả: Quản lý địa chỉ đã lưu cho dữ liệu tương thích

Tiêu chí	Nội dung
Mục đích	Bảo toàn hồ sơ địa chỉ đang tồn tại trong module tài khoản cũ.
Tác nhân	Khách hàng
Điều kiện trước	Địa chỉ thuộc tài khoản; tối đa 20 bản ghi.
Đầu vào	Người nhận, điện thoại, địa chỉ, mặc định và tọa độ tùy chọn.
Kết quả / điều kiện sau	Địa chỉ được lưu nhưng không là điều kiện bắt buộc của checkout mang về/tại quán.
Trường hợp lỗi	Không tồn tại, không có quyền hoặc vượt 20 địa chỉ.
Trạng thái xác minh	Mã nguồn tương thích; tách khỏi checkout mới

Bước 1. Xác định accountId.

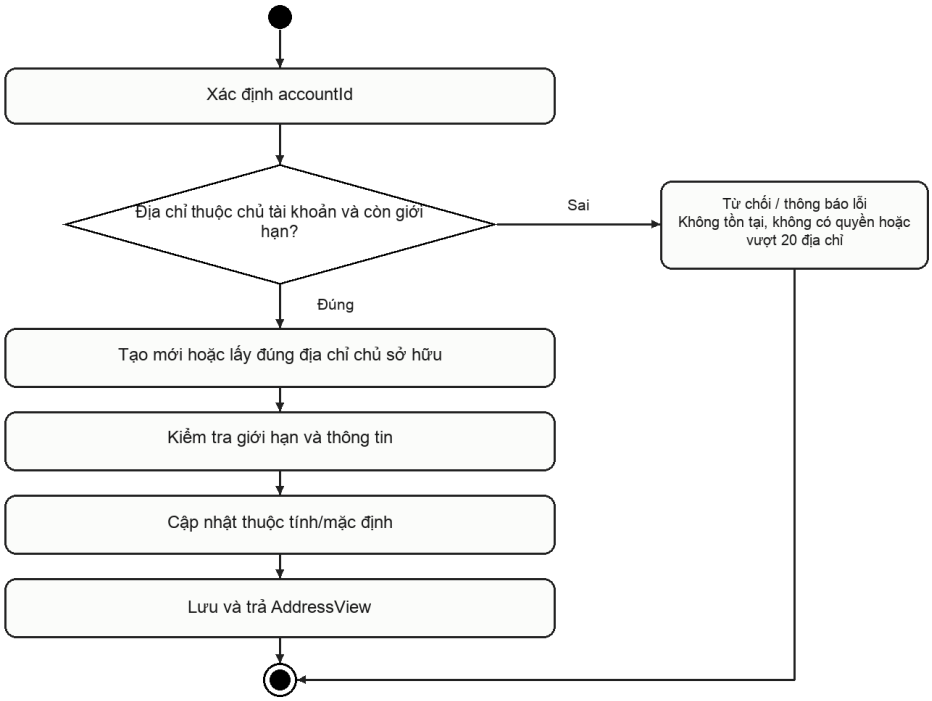
Bước 2. Tạo mới hoặc lấy đúng địa chỉ chủ sở hữu.

Bước 3. Kiểm tra giới hạn và thông tin.

Bước 4. Cập nhật thuộc tính/mặc định.

Bước 5. Lưu và trả AddressView.

2.1.26.2.Sơ đồ hoạt động



Hình 2.26. Hoạt động: Quản lý địa chỉ đã lưu cho dữ liệu tương thích

2.1.26.3.Code xử lý chức năng

Đoạn mã 2.26. CustomerDataService.saveAddress — Quản lý địa chỉ đã lưu cho dữ liệu tương thích

Nguồn: src/main/java/vn/edu/ute/milktea/service/account/CustomerDataService.java; CustomerDataService.saveAddress; dòng 118–129.

```
        public CustomerDataDto.AddressView saveAddress(CurrentActor actor,Long
id,CustomerDataDto.AddressInput input) {
        Account account=lock(actor);
        var existing=addresses.findByAccountIdOrderByIdAsc(account.getId());
        if(id==null&&existing.size()>=20)throw invalid("Tối đa 20 địa chỉ");
                                                    var       address=id==null?new
DeliveryAddress():addresses.findByIdAndAccountId(id,account.getId()).orElseThrow((
)->BusinessException.notFound(ErrorCode.ACCESS_DENIED,"Không tìm thấy địa chỉ"));
                boolean makeDefault=input.defaultAddress()||existing.isEmpty()||
address.isDefaultAddress();
                                                  if(makeDefault){existing.forEach(a-
>a.setDefaultAddress(false));addresses.flush();}

address.setAccount(account);address.setRecipient(input.recipient().trim());address
.setPhone(input.phone().trim());

address.setAddressLine(input.addressLine().trim());address.setDefaultAddress(makeD
efault);
                                                  address.setArea(input.area()==null?
null:input.area().trim());address.setLatitude(input.latitude());address.setLongitu
de(input.longitude());
        return addressView(addresses.save(address));
    }
```

2.1.26.4.Giải thích code

Module dữ liệu còn tồn tại; không được mô tả như UTE-Tea đang cung cấp giao tận nơi.

Method liên quan: CustomerDataService.saveAddress. Controller: CustomerDataController. Giao diện: src/main/resources/templates/customer/account-data.html; src/main/resources/static/js/customer/account-data.js.

2.2.Thu ngân

2.2.1.Xem hàng đợi và bộ lọc đơn

2.2.1.1.Mô tả chức năng

Bảng 2.27. Đặc tả: Xem hàng đợi và bộ lọc đơn

Tiêu chí	Nội dung
Mục đích	Tập trung các đơn cần xác nhận, chế biến, thu tiền, hủy và hoàn tiền.
Tác nhân	Thu ngân
Điều kiện trước	Vai trò CASHIER/ADMIN; nhân viên thuộc đúng cơ sở với đơn mới.
Đầu vào	status, paymentStatus, tab, query.
Kết quả / điều kiện sau	Không cấp quyền chi nhánh khác chỉ do nhập ID/mã đơn.
Trường hợp lỗi	Bị Security từ chối hoặc không có bản ghi trong phạm vi; danh sách rỗng là kết quả hợp lệ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Lấy tập đơn phù hợp.

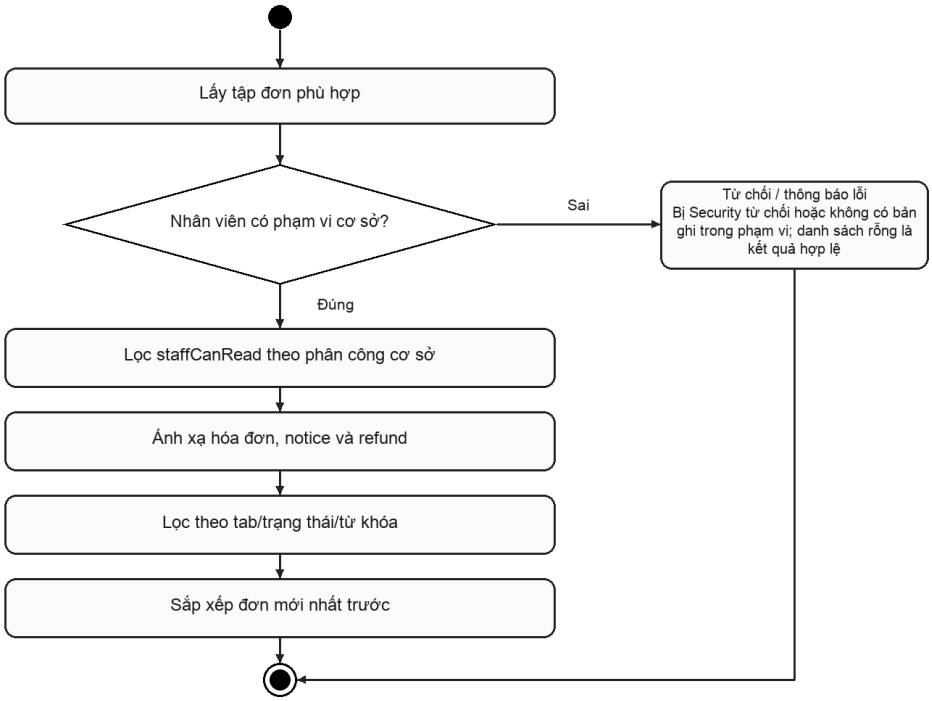
Bước 2. Lọc staffCanRead theo phân công cơ sở.

Bước 3. Ánh xạ hóa đơn, notice và refund.

Bước 4. Lọc theo tab/trạng thái/từ khóa.

Bước 5. Sắp xếp đơn mới nhất trước.

2.2.1.2.Sơ đồ hoạt động



Hình 2.27. Hoạt động: Xem hàng đợi và bộ lọc đơn

2.2.1.3.Code xử lý chức năng

Đoạn mã 2.27. OrderService.getOrdersForCashier — Xem hàng đợi và bộ lọc đơn

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.getOrdersForCashier; dòng 888–930.

```
      public List<OrderDto.OrderResponse> getOrdersForCashier(OrderStatus status,
String paymentStatus, String tab, String query) {
        List<Order> orders;
                    boolean   isAllTab   =   (tab   ==   null   ||   tab.isBlank()   ||
"all".equalsIgnoreCase(tab));
        if (isAllTab && status != null) {
                                                                          orders      =
orderRepository.findByStatusInOrderByCreatedAtAsc(List.of(status));
        } else {
            orders = orderRepository.findAll();
        }
        var stream = orders.stream().filter(this::staffCanRead)
                .sorted(java.util.Comparator.comparing(Order::getCreatedAt).revers
ed())
                .map(this::mapOrderToResponse);
        if (query != null && !query.isBlank()) {
            String q = query.trim().toUpperCase().replaceFirst("^#", "");
```

[Đã lược bỏ các dòng 901–906 của phương thức gốc.]

```
        }
        if (!isAllTab) {
            stream = stream.filter(o -> matchesTab(o, tab));
        } else {
            if (status != null) {
                stream = stream.filter(o -> o.getStatus() == status);
```

```
            }
            if (paymentStatus != null && !paymentStatus.isBlank()) {
                String ps = paymentStatus.trim().toUpperCase();
                if ("PAID".equals(ps)) {
                    stream = stream.filter(OrderDto.OrderResponse::isPaid);
```

[Đã lược bỏ các dòng 918–918 của phương thức gốc.]

```
                                    stream = stream.filter(o -> o.isPaid() &&
(!"BANK_TRANSFER".equalsIgnoreCase(o.getPaymentMethod())           &&           !
o.isTransferNotified()));
                } else if ("PAID_CK".equals(ps) || "PAID_TRANSFER".equals(ps) ||
"PAID_BANK_TRANSFER".equals(ps)) {
                                    stream = stream.filter(o -> o.isPaid() &&
("BANK_TRANSFER".equalsIgnoreCase(o.getPaymentMethod())                          ||
o.isTransferNotified()));
                } else if ("UNPAID".equals(ps)) {
                        stream = stream.filter(o -> !o.isPaid() && o.getStatus() !=
OrderStatus.DA_HUY);
                } else if ("TRANSFER_PENDING".equals(ps)) {
                           stream = stream.filter(o -> o.isTransferNotified() && !
o.isPaid());
                }
            }
        }
        return stream.toList();
    }
```

2.2.1.4.Giải thích code

Hàng đợi hiển thị trạng thái nghiệp vụ và thu tiền riêng; chưa PAID vẫn có thể được điều phối pha chế.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findByStatusInOrderByCreatedAtAsc(); orderRepository.findAll(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.2.2.Tiếp nhận và xác nhận đơn khách

2.2.2.1.Mô tả chức năng

Bảng 2.28. Đặc tả: Tiếp nhận và xác nhận đơn khách

Tiêu chí	Nội dung
Mục đích	Chuyển đơn khách đang chờ xác nhận sang hàng đợi bếp.
Tác nhân	Thu ngân
Điều kiện trước	Đúng cơ sở và trạng thái CHO_XAC_NHAN.
Đầu vào	orderId, actor thu ngân.
Kết quả / điều kiện sau	Bếp nhìn thấy đơn trong hàng đợi chế biến.
Trường hợp lỗi	Đơn không đúng trạng thái hoặc không thuộc cơ sở.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa Order.

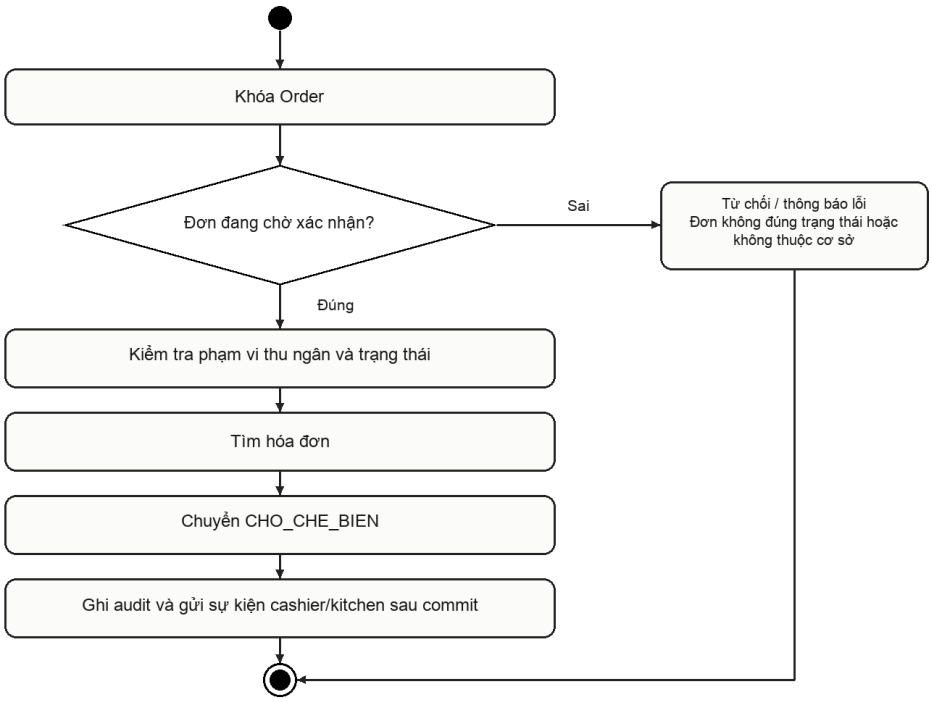
Bước 2. Kiểm tra phạm vi thu ngân và trạng thái.

Bước 3. Tìm hóa đơn.

Bước 4. Chuyển CHO_CHE_BIEN.

Bước 5. Ghi audit và gửi sự kiện cashier/kitchen sau commit.

2.2.2.2.Sơ đồ hoạt động



Hình 2.28. Hoạt động: Tiếp nhận và xác nhận đơn khách

2.2.2.3.Code xử lý chức năng

Đoạn mã 2.28. OrderService.confirm — Tiếp nhận và xác nhận đơn khách

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.confirm; dòng 320–352.

```
    public void confirm(Long orderId, Long cashierId) {
        Order order = orderRepository.findByIdWithLock(orderId)
                                                    .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT,  "Không  tìm  thấy  đơn
hàng"));
        branchService.assertStaff(order, cashierId);
        if (order.getStatus() == OrderStatus.CHO_CHE_BIEN) {
            return;
        }
        if (order.getStatus() != OrderStatus.CHO_XAC_NHAN) {
            throw BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT,
                    "Không thể xác nhận đơn ở trạng thái: " + order.getStatus());
        }
        order.setStatus(OrderStatus.CHO_CHE_BIEN);
        Invoice invoice = invoiceRepository.findByOrderId(orderId)
                .orElseThrow(() -> BusinessException.notFound(
                        ErrorCode.ORDER_STATE_CONFLICT,
```

```
                        "Không tìm thấy hóa đơn của đơn hàng"));
        // Không áp dụng paid gate: Hỗ trợ tiền mặt hoặc chuyển khoản trước/trong/
sau pha chế theo GEMINI.md
        orderRepository.save(order);
        auditRepository.save(BusinessAudit.builder()
                .orderId(order.getId())
                .accountId(cashierId)
                .action("ORDER_CONFIRMED")
                .beforeState("CHO_XAC_NHAN")
                .afterState("CHO_CHE_BIEN")
                .createdAt(Instant.now())
                .build());
                        String sId = order.getSession() != null ?
order.getSession().getId().toString() : null;
                        realtimeEventPublisher.publishAfterCommit("/topic/kitchen",
"ORDER_STATUS_CHANGED", order.getId().toString(), sId, "1", Map.of("orderId",
order.getId(), "status", order.getStatus().name()));
                        realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"ORDER_STATUS_CHANGED", order.getId().toString(), sId, "1", Map.of("orderId",
order.getId(), "status", order.getStatus().name()));
        if (order.getSession() != null) {
              realtimeEventPublisher.publishAfterCommit("/topic/table-sessions/" +
order.getSession().getId(), "ORDER_STATUS_CHANGED", order.getId().toString(), sId,
"1", Map.of("orderId", order.getId(), "status", order.getStatus().name()));
        }
    }
```

2.2.2.4.Giải thích code

Không áp đặt paid gate mới: quy trình cho phép tiền mặt/chuyển khoản trước, trong hoặc sau pha chế.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findByIdWithLock(); branchService.assertStaff(); invoiceRepository.findByOrderId(); orderRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.2.3.Tạo đơn tại quầy bằng POS

2.2.3.1.Mô tả chức năng

Bảng 2.29. Đặc tả: Tạo đơn tại quầy bằng POS

Tiêu chí	Nội dung
Mục đích	Cho thu ngân lập đơn trực tiếp từ menu và nhận tiền theo thao tác POS.
Tác nhân	Thu ngân
Điều kiện trước	Nhân viên đúng quyền cơ sở; món và công thức còn hợp lệ.
Đầu vào	items, note, shopId mặc định, paymentMethod, Idempotency-Key.
Kết quả / điều kiện sau	Đơn do nhân viên tạo ở CHO_THANH_TOAN; JavaScript ghi thu bằng API riêng.
Trường hợp lỗi	Món ngừng bán, cơ sở khác hoặc key đã dùng cho nội dung khác.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa cơ sở và kiểm tra phạm vi nhân viên.

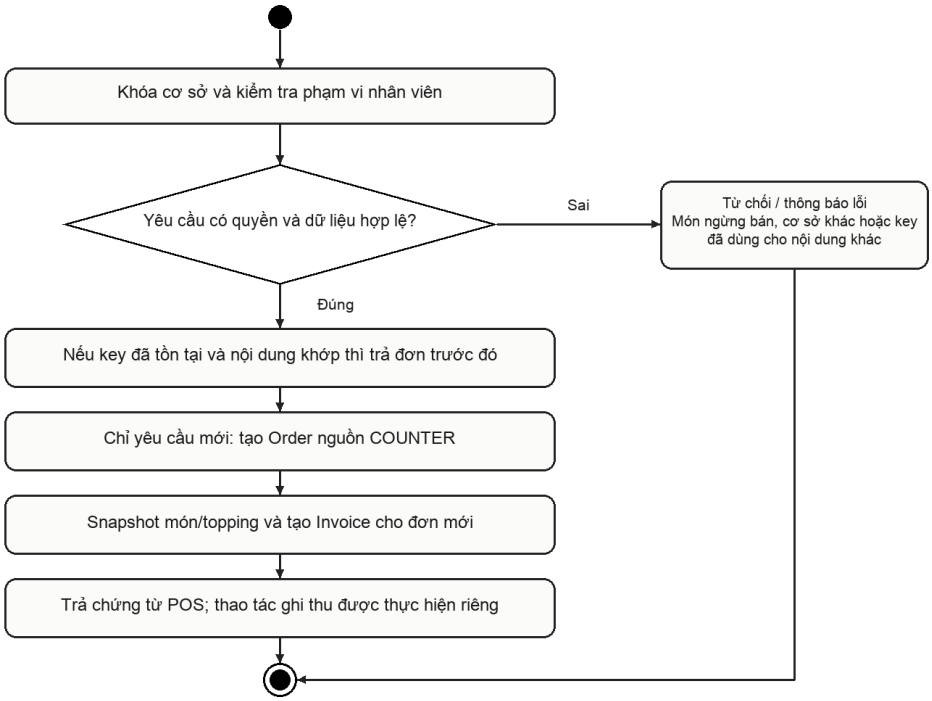
Bước 2. Nếu key đã tồn tại và nội dung khớp thì trả đơn trước đó.

Bước 3. Chỉ yêu cầu mới: tạo Order nguồn COUNTER.

Bước 4. Snapshot món/topping và tạo Invoice cho đơn mới.

Bước 5. Trả chứng từ POS; thao tác ghi thu được thực hiện riêng.

2.2.3.2.Sơ đồ hoạt động



Hình 2.29. Hoạt động: Tạo đơn tại quầy bằng POS

2.2.3.3.Code xử lý chức năng

Đoạn mã 2.29. OrderService.createCounterOrder — Tạo đơn tại quầy bằng POS

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.createCounterOrder; dòng 219–285.

```
                                                public            OrderDto.OrderResponse
createCounterOrder(OrderDto.CreateCounterOrderRequest request, CurrentActor actor,
String idempotencyKey) {
                 if  (idempotencyKey  !=  null  &&  (idempotencyKey.isBlank()  ||
idempotencyKey.length() > 64))
             throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED,"Mã yêu
cầu không hợp lệ");
        if (idempotencyKey == null) idempotencyKey = UUID.randomUUID().toString();
        var shop = branchService.resolve(request.getShopId());
        branchService.assertCustomer(shop, actor);
                     if   (actor   !=   null   &&   (actor.hasRole(Role.ADMIN)   ||
actor.hasRole(Role.CASHIER)))
                     branchService.assertStaff(Order.builder().shop(shop).build(),
actor.getAccountId());
        String fingerprint = fingerprint(request);
        if (idempotencyKey != null && !idempotencyKey.isBlank()) {
            var existing = orderRepository.findByIdempotencyKey(idempotencyKey);
            if (existing.isPresent()) {
                Order order = existing.get();
```

[Đã lược bỏ các dòng 232–260 của phương thức gốc.]

```
                              if    (request.getExpectedTotal()    !=    null    &&
request.getExpectedTotal().compareTo(discount.totalAmount) != 0)
                throw BusinessException.conflict(ErrorCode.VALIDATION_FAILED, "Giá
hoặc giảm giá đã thay đổi. Vui lòng kiểm tra tổng tiền mới.");
```

```
        Invoice invoice = Invoice.builder()
                .order(order)
                .subtotal(subtotal)
                .discountAmount(discount.discountAmount)
                .totalAmount(discount.totalAmount)
                .snapshotDiscountPercent(discount.snapshotPercent)
                .snapshotDiscountType(discount.discountType)
                .snapshotDiscountValue(discount.configValue)
                .status(InvoiceStatus.HIEU_LUC)
```

[Đã lược bỏ các dòng 272–273 của phương thức gốc.]

```
        invoice = invoiceRepository.save(invoice);
        var issued = guestAccessService.issueCounterOrderToken(order, 24);
        auditRepository.save(BusinessAudit.builder()
                .orderId(order.getId())
                .accountId(creator != null ? creator.getId() : null)
                .action("COUNTER_ORDER_CREATED")
                .afterState(order.getStatus().name())
                .createdAt(Instant.now())
                .build());
                        realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"ORDER_CREATED",    order.getId().toString(),    null,    "1",    Map.of("orderId",
order.getId(), "status", order.getStatus().name()));
                  return  mapToOrderResponse(order,  invoice,  savedItems,  null,
issued.rawToken());
    }
```

2.2.3.4.Giải thích code

Không coi việc tạo đơn POS là bằng chứng đã thu tiền; cần PaymentService.recordReceipt thật sự hoàn tất.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: branchService.resolve(); branchService.assertCustomer(); branchService.assertStaff(); orderRepository.findByIdempotencyKey(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.2.4.Ghi thu tiền mặt hoặc chuyển khoản

2.2.4.1.Mô tả chức năng

Bảng 2.30. Đặc tả: Ghi thu tiền mặt hoặc chuyển khoản

Tiêu chí	Nội dung
Mục đích	Tạo khoản thu duy nhất sau khi thu ngân thực tế nhận tiền.
Tác nhân	Thu ngân
Điều kiện trước	Hóa đơn hiệu lực, đơn chưa hủy, nhân viên đúng cơ sở.
Đầu vào	invoiceId, method, reference, Idempotency-Key và cashierId từ actor.
Kết quả / điều kiện sau	Thanh toán được chứng minh bằng Payment; không tạo khoản thu thứ hai cho hóa đơn.
Trường hợp lỗi	Sai quyền, key khác invoice/phương thức, số tiền không hợp lệ hoặc đã thanh toán.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa bàn nếu có rồi khóa đơn và hóa đơn.

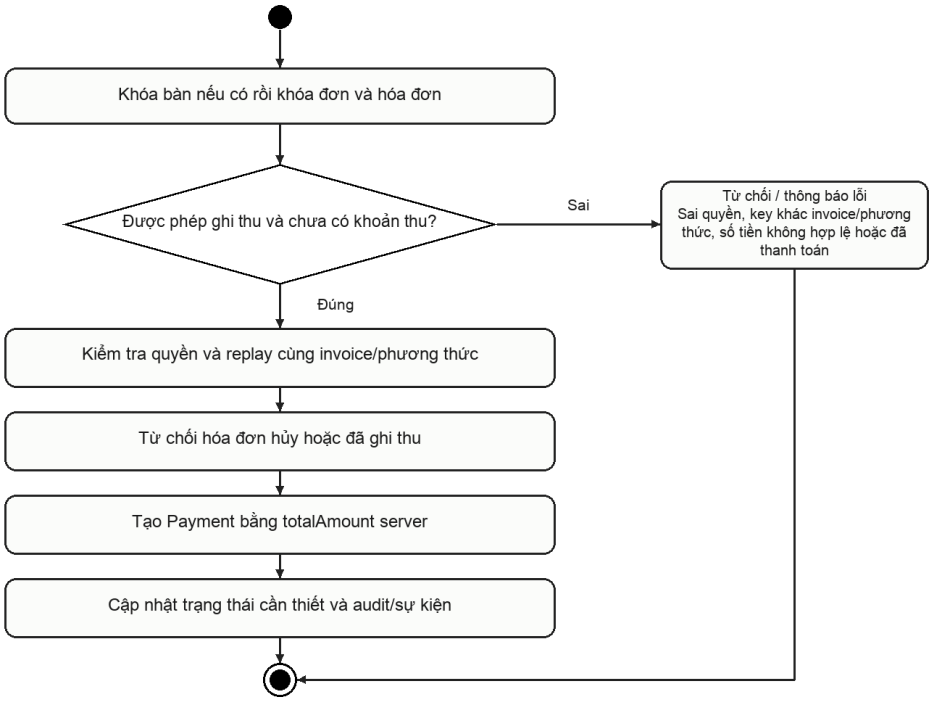
Bước 2. Kiểm tra quyền và replay cùng invoice/phương thức.

Bước 3. Từ chối hóa đơn hủy hoặc đã ghi thu.

Bước 4. Tạo Payment bằng totalAmount server.

Bước 5. Cập nhật trạng thái cần thiết và audit/sự kiện.

2.2.4.2.Sơ đồ hoạt động



Hình 2.30. Hoạt động: Ghi thu tiền mặt hoặc chuyển khoản

2.2.4.3.Code xử lý chức năng

Đoạn mã 2.30. PaymentService.recordReceipt — Ghi thu tiền mặt hoặc chuyển khoản

Nguồn: src/main/java/vn/edu/ute/milktea/service/payment/PaymentService.java; PaymentService.recordReceipt; dòng 44–225.

```
    public PaymentDto.PaymentResponse recordReceipt(
            Long invoiceId,
            PaymentDto.RecordPaymentRequest request,
            Long cashierId,
            String idempotencyKey) {

        // 1. Kiểm tra dữ liệu đầu vào
        if (request == null || request.getMethod() == null) {
            throw BusinessException.conflict(
                    ErrorCode.VALIDATION_FAILED,
                    "Phương thức thanh toán không hợp lệ");
        }
```

[Đã lược bỏ các dòng 57–57 của phương thức gốc.]

```
                && idempotencyKey.length() > 64) {
            throw BusinessException.conflict(
                    ErrorCode.VALIDATION_FAILED,
                    "Mã yêu cầu thanh toán quá dài");
        }

        // 2. Đọc hóa đơn để xác định đơn hàng
        Invoice invoiceLookup = invoiceRepository.findById(invoiceId)
                .orElseThrow(() -> BusinessException.notFound(
                        ErrorCode.VALIDATION_FAILED,
```

```
                        "Không tìm thấy hóa đơn"));
```

[Đã lược bỏ các dòng 69–213 của phương thức gốc.]

```
            realtimeEventPublisher.publishAfterCommit(
                    "/topic/table-sessions/"
                            + order.getSession().getId(),
                    "PAYMENT_RECORDED",
                    payment.getId().toString(),
                    sessionId,
                    "1",
                    eventData);
        }

        return toPaymentResponse(payment);
    }
```

2.2.4.4.Giải thích code

Thu ngân phải đối soát chuyển khoản thủ công; code không liên hệ ngân hàng để chứng minh số dư.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: invoiceRepository.findById(); tableRepository.findByIdWithLock(); orderRepository.findByIdWithLock(); invoiceRepository.findByIdWithLock(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.2.5.Xác nhận thông báo chuyển khoản của khách

2.2.5.1.Mô tả chức năng

Bảng 2.31. Đặc tả: Xác nhận thông báo chuyển khoản của khách

Tiêu chí	Nội dung
Mục đích	Đối soát thông báo của khách trước khi ghi thu BANK_TRANSFER.
Tác nhân	Thu ngân
Điều kiện trước	Thu ngân được phân công và đã kiểm tra giao dịch ngoài hệ thống.
Đầu vào	orderId; Idempotency-Key tùy chọn.
Kết quả / điều kiện sau	Tái sử dụng kiểm tra và khóa ghi thu thay vì tạo cổng thanh toán giả.
Trường hợp lỗi	Sai cơ sở, đơn không có hóa đơn hoặc trạng thái không cho ghi thu.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm invoice theo Order.

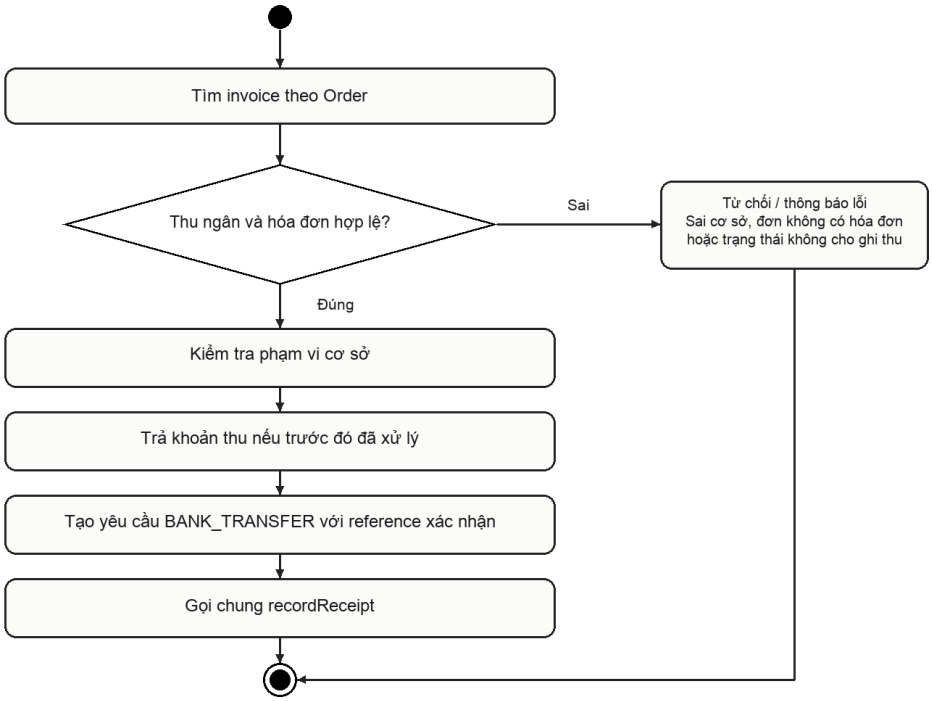
Bước 2. Kiểm tra phạm vi cơ sở.

Bước 3. Trả khoản thu nếu trước đó đã xử lý.

Bước 4. Tạo yêu cầu BANK_TRANSFER với reference xác nhận.

Bước 5. Gọi chung recordReceipt.

2.2.5.2.Sơ đồ hoạt động



Hình 2.31. Hoạt động: Xác nhận thông báo chuyển khoản của khách

2.2.5.3.Code xử lý chức năng

Đoạn mã 2.31. PaymentService.confirmTransferReceiptForOrder — Xác nhận thông báo chuyển khoản của khách

Nguồn: src/main/java/vn/edu/ute/milktea/service/payment/PaymentService.java; PaymentService.confirmTransferReceiptForOrder; dòng 279–297.

```
    public PaymentDto.PaymentResponse confirmTransferReceiptForOrder(Long orderId,
Long cashierId, String idempotencyKey) {
        Invoice invoice = invoiceRepository.findByOrderId(orderId)
                                                              .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED, "Không tìm thấy hóa đơn
cho đơn hàng #" + orderId));
        branchService.assertStaff(invoice.getOrder(), cashierId);
        if (paymentRepository.existsByInvoiceId(invoice.getId())) {
                                                          Payment     existing     =
paymentRepository.findByInvoiceId(invoice.getId()).orElse(null);
            if (existing != null) {
                return toPaymentResponse(existing);
            }
        }
                                    PaymentDto.RecordPaymentRequest     request     =
PaymentDto.RecordPaymentRequest.builder()
                .method(PaymentMethod.BANK_TRANSFER)
                .reference("TRANSFER_CONFIRMED_" + orderId)
                .build();
                    String   effectiveKey   =   (idempotencyKey   !=   null   &&   !
idempotencyKey.isBlank())
```

```
                ? idempotencyKey
                : "TRANSFER_CONFIRM_" + orderId;
        return recordReceipt(invoice.getId(), request, cashierId, effectiveKey);
    }
```

2.2.5.4.Giải thích code

Hành động xác nhận là trách nhiệm thu ngân; không kích hoạt từ nút Tôi đã chuyển khoản của khách.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: invoiceRepository.findByOrderId(); branchService.assertStaff(); paymentRepository.existsByInvoiceId(); paymentRepository.findByInvoiceId(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.2.6.Mở phiên phục vụ bàn

2.2.6.1.Mô tả chức năng

Bảng 2.32. Đặc tả: Mở phiên phục vụ bàn

Tiêu chí	Nội dung
Mục đích	Chủ động mở lượt phục vụ trước khi khách hoặc thu ngân đặt món.
Tác nhân	Thu ngân
Điều kiện trước	Bàn hoạt động; nếu đã ánh xạ cơ sở thì staff có quyền tương ứng.
Đầu vào	tableId và cashierId.
Kết quả / điều kiện sau	Một activeSessionId được gắn trên bàn và giỏ phiên được khởi tạo.
Trường hợp lỗi	Không tìm thấy bàn hoặc nhân viên không có quyền cơ sở.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa bàn và kiểm tra quyền thu ngân.

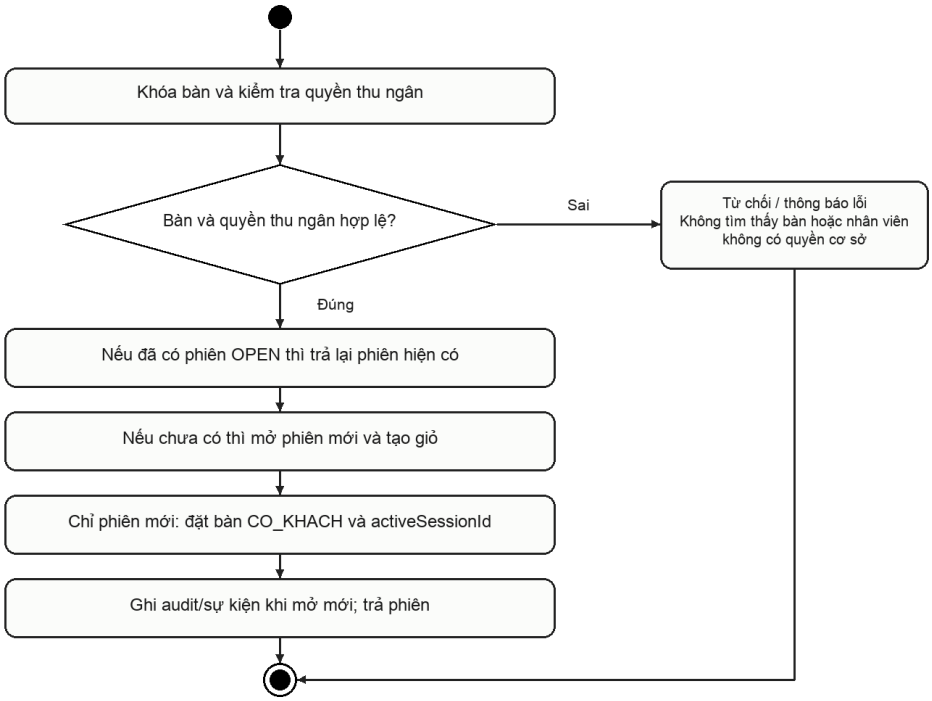
Bước 2. Nếu đã có phiên OPEN thì trả lại phiên hiện có.

Bước 3. Nếu chưa có thì mở phiên mới và tạo giỏ.

Bước 4. Chỉ phiên mới: đặt bàn CO_KHACH và activeSessionId.

Bước 5. Ghi audit/sự kiện khi mở mới; trả phiên.

2.2.6.2.Sơ đồ hoạt động



Hình 2.32. Hoạt động: Mở phiên phục vụ bàn

2.2.6.3.Code xử lý chức năng

Đoạn mã 2.32. TableSessionService.openByCashier — Mở phiên phục vụ bàn

Nguồn: src/main/java/vn/edu/ute/milktea/service/table/TableSessionService.java; TableSessionService.openByCashier; dòng 43–89.

```
    public TableSession openByCashier(Long tableId, Long cashierId) {
        DiningTable table = tableRepository.findByIdWithLock(tableId)
                                                              .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.TABLE_NOT_FOUND, "Không tìm thấy bàn"));
                                           if     (table.getShop()     !=     null)
branchService.assertStaff(Order.builder().shop(table.getShop()).build(),
cashierId);

        if (table.getActiveSessionId() != null) {
                                                                  var     existingOpt     =
sessionRepository.findById(table.getActiveSessionId());
                 if  (existingOpt.isPresent()  &&  existingOpt.get().getStatus()  ==
TableSessionStatus.OPEN) {
                return existingOpt.get();
            }
        }

                              Account    cashier    =    cashierId    !=    null    ?
accountRepository.findById(cashierId).orElse(null) : null;
```

[Đã lược bỏ các dòng 56–65 của phương thức gốc.]

```
        table.setStatus(TableStatus.CO_KHACH);
        table.setActiveSessionId(session.getId());
        tableRepository.save(table);
```

```
        SessionCart cart = SessionCart.builder()
                .session(session)
                .version(1L)
                .modifiedAt(Instant.now())
                .build();
        cartRepository.save(cart);
```

[Đã lược bỏ các dòng 77–77 của phương thức gốc.]

```
                .sessionId(session.getId())
                .accountId(cashierId)
                .action("TABLE_SESSION_OPENED_BY_CASHIER")
                .afterState("OPEN")
                .createdAt(Instant.now())
                .build());

                        realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"TABLE_SESSION_OPENED",   session.getId().toString(),   session.getId().toString(),
"1", java.util.Map.of("tableId", table.getId(), "tableName", table.getName()));
             realtimeEventPublisher.publishAfterCommit("/topic/table-sessions/"  +
session.getId(),         "TABLE_SESSION_OPENED",         session.getId().toString(),
session.getId().toString(),    "1",    java.util.Map.of("tableId",    table.getId(),
"tableName", table.getName()));

        return session;
    }
```

2.2.6.4.Giải thích code

Không tự mở thêm phiên chỉ vì khách xem QR; thao tác mở phiên này thuộc thu ngân.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: tableRepository.findByIdWithLock(); branchService.assertStaff(); sessionRepository.findById(); accountRepository.findById(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.2.7.Gắn đơn quầy vào bàn cùng cơ sở

2.2.7.1.Mô tả chức năng

Bảng 2.33. Đặc tả: Gắn đơn quầy vào bàn cùng cơ sở

Tiêu chí	Nội dung
Mục đích	Chuyển vị trí phục vụ của cùng đơn quầy, không tạo đơn hoặc thu lại tiền.
Tác nhân	Thu ngân
Điều kiện trước	Đơn nguồn COUNTER chưa có session; thu ngân đúng cơ sở; bàn cùng cơ sở.
Đầu vào	orderId và tableId.
Kết quả / điều kiện sau	Mã đơn/hóa đơn giữ nguyên; vị trí phục vụ có session.
Trường hợp lỗi	Đã có session, nguồn không phải COUNTER hoặc bàn ở cơ sở khác.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa đơn và kiểm tra nguồn.

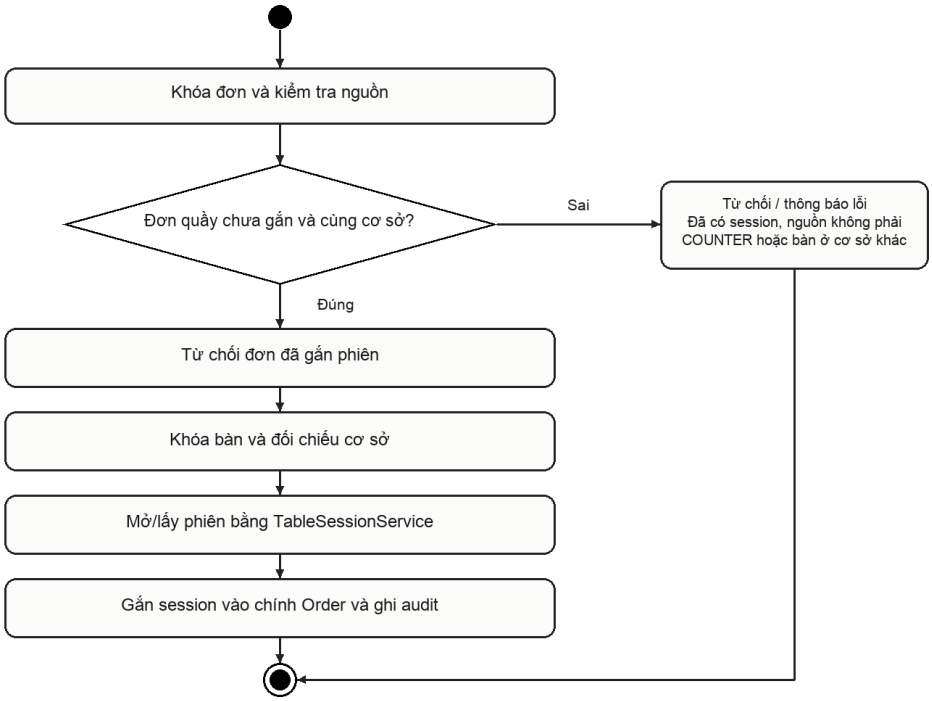
Bước 2. Từ chối đơn đã gắn phiên.

Bước 3. Khóa bàn và đối chiếu cơ sở.

Bước 4. Mở/lấy phiên bằng TableSessionService.

Bước 5. Gắn session vào chính Order và ghi audit.

2.2.7.2.Sơ đồ hoạt động



Hình 2.33. Hoạt động: Gắn đơn quầy vào bàn cùng cơ sở

2.2.7.3.Code xử lý chức năng

Đoạn mã 2.33. OrderService.assignCounterOrderToTable — Gắn đơn quầy vào bàn cùng cơ sở

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.assignCounterOrderToTable; dòng 288–317.

```
       public void assignCounterOrderToTable(Long orderId, Long tableId, Long
cashierId) {
        Order order = orderRepository.findByIdWithLock(orderId)
                                                          .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT, "Không tìm thấy đơn
hàng"));
        branchService.assertStaff(order, cashierId);
        if (order.getSource() != OrderSource.COUNTER) {
                throw BusinessException.badRequest(ErrorCode.ORDER_STATE_CONFLICT,
"Chỉ được gắn đơn hàng tại quầy vào bàn");
        }
        if (order.getSession() != null) {
                throw BusinessException.badRequest(ErrorCode.ORDER_STATE_CONFLICT,
"Đơn hàng đã được gắn vào phiên bàn khác");
        }
        // Mở phiên nếu bàn chưa có phiên, hoặc lấy phiên hiện tại
        var targetTable = tableRepository.findByIdWithLock(tableId).orElseThrow(()
->
                BusinessException.notFound(ErrorCode.TABLE_NOT_FOUND,"Không tìm
thấy bàn"));
        var targetShop = branchService.tableBranch(targetTable);
                                if    (order.getShop()    !=    null    &&    !
order.getShop().getId().equals(targetShop.getId()))
```

```
            throw BusinessException.forbidden(ErrorCode.ACCESS_DENIED,"Không được
gắn đơn sang bàn khác cơ sở");
             TableSession session = tableSessionService.openByCashier(tableId,
cashierId);
        order.setSession(session);
        orderRepository.save(order);
        auditRepository.save(BusinessAudit.builder()
                .sessionId(session.getId())
                .orderId(order.getId())
                .accountId(cashierId)
                .action("ORDER_ASSIGNED_TO_TABLE")
                .reason("Gắn đơn quầy vào bàn #" + tableId)
                .createdAt(Instant.now())
                .build());
                       realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"ORDER_ASSIGNED_TO_TABLE",  order.getId().toString(),  session.getId().toString(),
"1", Map.of("orderId", order.getId()));
              realtimeEventPublisher.publishAfterCommit("/topic/table-sessions/"  +
session.getId(),        "ORDER_ASSIGNED_TO_TABLE",        order.getId().toString(),
session.getId().toString(), "1", Map.of("orderId", order.getId()));
    }
```

2.2.7.4.Giải thích code

Không thể dùng frontend tableId để chuyển một đơn sang chi nhánh khác.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findByIdWithLock(); branchService.assertStaff(); tableRepository.findByIdWithLock(); branchService.tableBranch(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.2.8.Đóng phiên bàn sau phục vụ

2.2.8.1.Mô tả chức năng

Bảng 2.34. Đặc tả: Đóng phiên bàn sau phục vụ

Tiêu chí	Nội dung
Mục đích	Kết thúc lượt dùng bàn khi các đơn/chứng từ đã đủ điều kiện.
Tác nhân	Thu ngân
Điều kiện trước	Phiên hiện tại của bàn; đơn đã xong/hủy và hóa đơn hiệu lực đã được thu đủ theo closeSession.
Đầu vào	sessionId và nhân viên thu ngân.
Kết quả / điều kiện sau	Bàn sẵn sàng cho lượt mới; chứng từ lịch sử không bị xóa.
Trường hợp lỗi	Phiên cũ, còn đơn xử lý hoặc hóa đơn chưa thu đủ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm phiên và đối chiếu bàn.

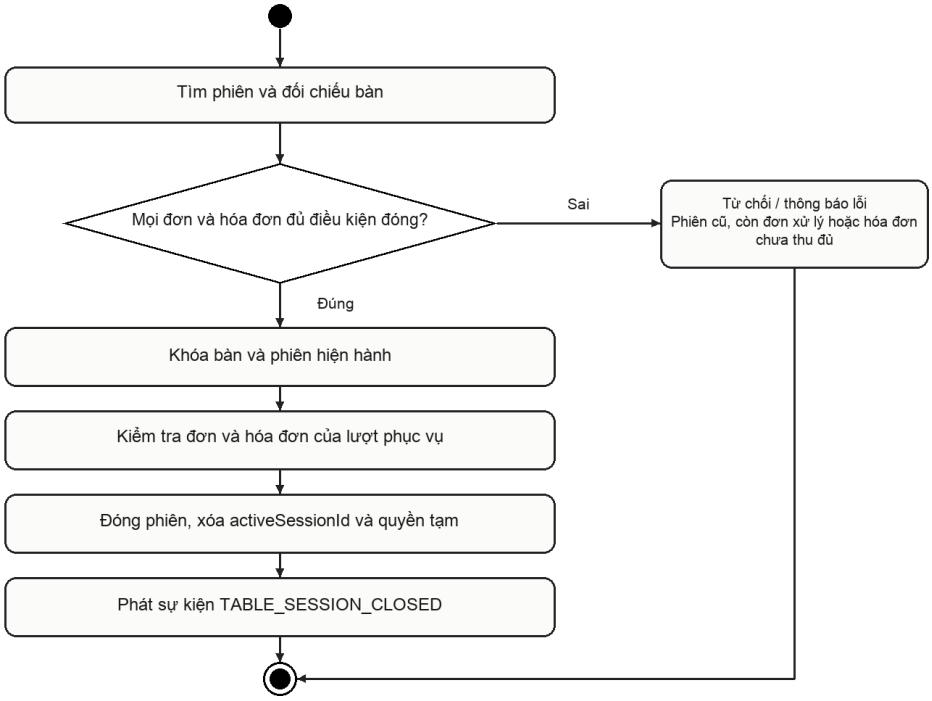
Bước 2. Khóa bàn và phiên hiện hành.

Bước 3. Kiểm tra đơn và hóa đơn của lượt phục vụ.

Bước 4. Đóng phiên, xóa activeSessionId và quyền tạm.

Bước 5. Phát sự kiện TABLE_SESSION_CLOSED.

2.2.8.2.Sơ đồ hoạt động



Hình 2.34. Hoạt động: Đóng phiên bàn sau phục vụ

2.2.8.3.Code xử lý chức năng

Đoạn mã 2.34. TableSessionService.closeBySessionId — Đóng phiên bàn sau phục vụ

Nguồn: src/main/java/vn/edu/ute/milktea/service/table/TableSessionService.java; TableSessionService.closeBySessionId; dòng 183–188.

```
                    public     vn.edu.ute.milktea.dto.TableDto.TableSessionResponse
closeBySessionId(Long sessionId, Long cashierId) {
        TableSession session = sessionRepository.findByIdWithLock(sessionId)
                                                              .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.TABLE_SESSION_CLOSED,    "Không    tìm    thấy
phiên"));
        closeByCashier(session.getTable().getId(), cashierId);
        return getSessionResponse(sessionId);
    }
```

2.2.8.4.Giải thích code

Thanh toán, logout hoặc mất WebSocket không tự đóng phiên; thao tác phải từ thu ngân.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: sessionRepository.findByIdWithLock(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.2.9.Duyệt hoặc từ chối yêu cầu hủy

2.2.9.1.Mô tả chức năng

Bảng 2.35. Đặc tả: Duyệt hoặc từ chối yêu cầu hủy

Tiêu chí	Nội dung
Mục đích	Giải quyết yêu cầu CHO của khách khi đơn đã thu tiền nhưng chưa pha.
Tác nhân	Thu ngân
Điều kiện trước	Vai trò được endpoint cho phép; yêu cầu còn CHO và đơn chưa bắt đầu pha.
Đầu vào	requestId, approved và lý do quyết định.
Kết quả / điều kiện sau	Yêu cầu kết thúc có chứng từ quyết định; hoàn tiền được xử lý riêng.
Trường hợp lỗi	Đã quyết định hoặc đơn chuyển sang DANG_CHE_BIEN/HOAN_THANH.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa yêu cầu và Order.

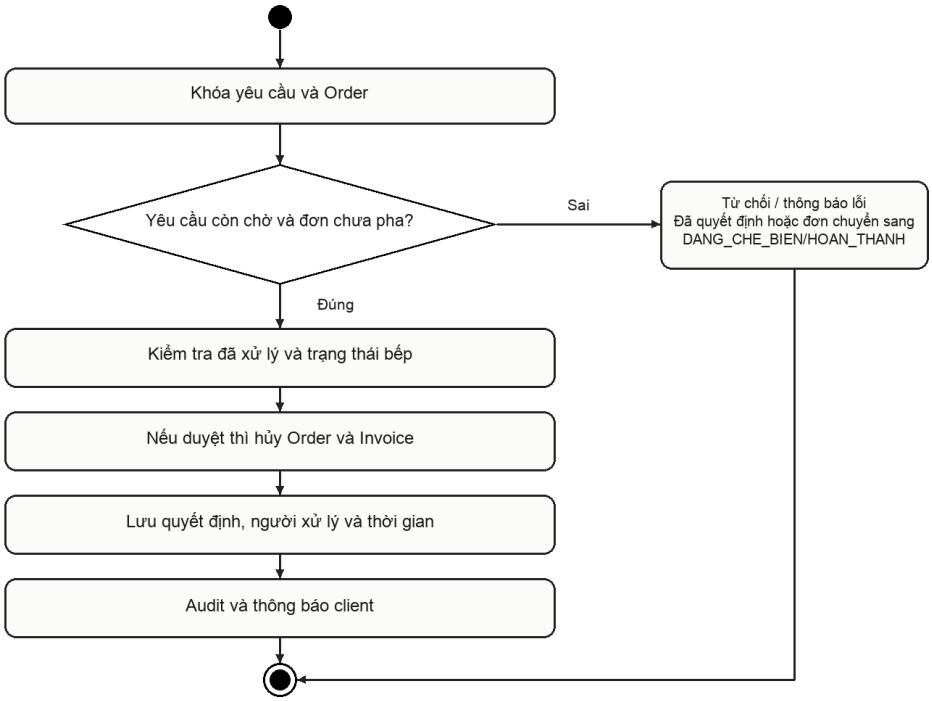
Bước 2. Kiểm tra đã xử lý và trạng thái bếp.

Bước 3. Nếu duyệt thì hủy Order và Invoice.

Bước 4. Lưu quyết định, người xử lý và thời gian.

Bước 5. Audit và thông báo client.

2.2.9.2.Sơ đồ hoạt động



Hình 2.35. Hoạt động: Duyệt hoặc từ chối yêu cầu hủy

2.2.9.3.Code xử lý chức năng

Đoạn mã 2.35. CancellationService.decideCancellation — Duyệt hoặc từ chối yêu cầu hủy

Nguồn: src/main/java/vn/edu/ute/milktea/service/cancellation/CancellationService.java; CancellationService.decideCancellation; dòng 151–211.

```
                    public      void      decideCancellation(Long      requestId,
CancellationDto.DecideCancellationRequest request, Long cashierId) {
        CancellationRequest cancelReq = cancellationRepository.findById(requestId)
                                                                .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED,  "Không  tìm  thấy  yêu  cầu
hủy"));

        if (cancelReq.getStatus() != CancellationStatus.CHO) {
             throw BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT, "Yêu
cầu hủy đã được xử lý trước đó");
        }

                                                            Order       order       =
orderRepository.findByIdWithLock(cancelReq.getOrder().getId())
                                                                .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT,  "Không  tìm  thấy  đơn
hàng"));

        entityManager.refresh(cancelReq);
        if (cancelReq.getStatus() != CancellationStatus.CHO) {
```

[Đã lược bỏ các dòng 164–164 của phương thức gốc.]

```
        }
                            Account    cashier    =    cashierId    !=    null    ?
accountRepository.findById(cashierId).orElse(null) : null;
```

```
        cancelReq.setDecidedBy(cashier);
        cancelReq.setDecidedAt(Instant.now());
        cancelReq.setDecisionReason(request.getReason());

        if (Boolean.TRUE.equals(request.getApproved())) {
            // Chấp thuận hủy
                        if (order.getStatus() != OrderStatus.CHO_THANH_TOAN &&
order.getStatus() != OrderStatus.CHO_XAC_NHAN && order.getStatus() !=
OrderStatus.CHO_CHE_BIEN) {
                throw BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT,
                            "Không thể chấp thuận hủy vì đơn đã bắt đầu hoặc hoàn
thành pha chế: " + order.getStatus());
```

[Đã lược bỏ các dòng 176–199 của phương thức gốc.]

```
            auditRepository.save(BusinessAudit.builder()
                    .orderId(order.getId())
                    .accountId(cashierId)
                    .action("CANCELLATION_REJECTED")
                    .reason(request.getReason())
                    .createdAt(Instant.now())
                    .build());
        }

        cancellationRepository.save(cancelReq);
        publish(order);
    }
```

2.2.9.4.Giải thích code

Cần phân biệt quyền role endpoint với kiểm tra cơ sở: các method hủy/hoàn chưa đồng nhất assertStaff như OrderService; báo cáo không khẳng định đã bao phủ tất cả.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: cancellationRepository.findById(); orderRepository.findByIdWithLock(); accountRepository.findById(); orderRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.2.10.Hoàn tiền mặt và lập yêu cầu hoàn chuyển khoản

2.2.10.1.Mô tả chức năng

Bảng 2.36. Đặc tả: Hoàn tiền mặt và lập yêu cầu hoàn chuyển khoản

Tiêu chí	Nội dung
Mục đích	Theo dõi việc trả lại khoản đã thu cho một đơn bị hủy.
Tác nhân	Thu ngân
Điều kiện trước	Order đã DA_HUY và có Payment; chưa hoàn trước đó.
Đầu vào	orderId, cashierId; nhánh ngân hàng dùng requestBankRefund.
Kết quả / điều kiện sau	Tiền mặt ghi nhận hoàn trực tiếp; chuyển khoản có quy trình xin duyệt quản trị riêng.
Trường hợp lỗi	Đơn chưa hủy, chưa thu hoặc đã hoàn.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa đơn và kiểm tra trạng thái hủy.

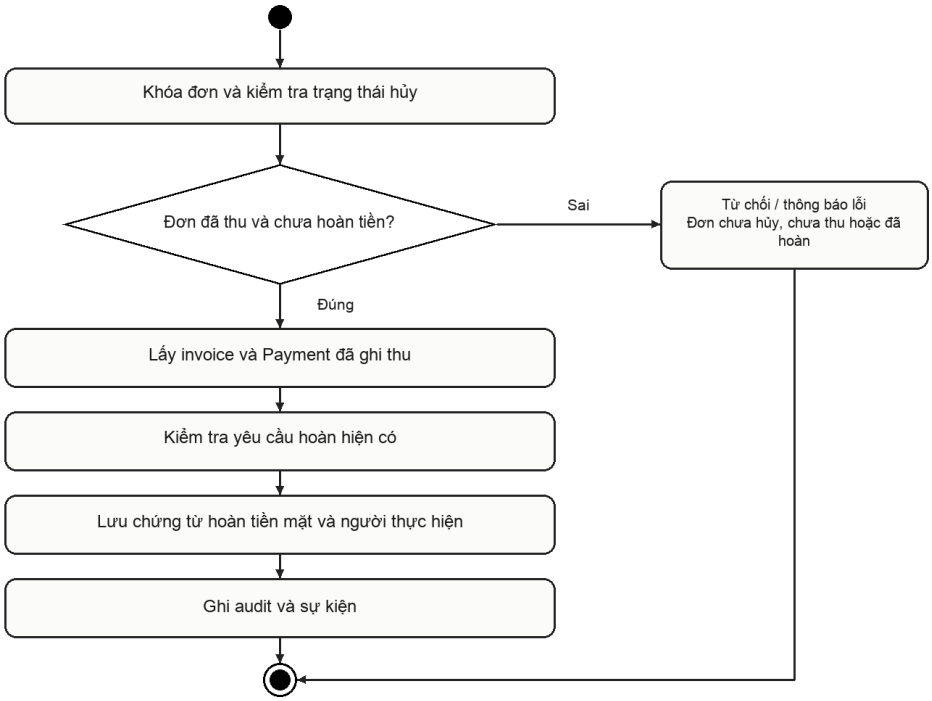
Bước 2. Lấy invoice và Payment đã ghi thu.

Bước 3. Kiểm tra yêu cầu hoàn hiện có.

Bước 4. Lưu chứng từ hoàn tiền mặt và người thực hiện.

Bước 5. Ghi audit và sự kiện.

2.2.10.2.Sơ đồ hoạt động



Hình 2.36. Hoạt động: Hoàn tiền mặt và lập yêu cầu hoàn chuyển khoản

2.2.10.3.Code xử lý chức năng

Đoạn mã 2.36. RefundService.refundCash — Hoàn tiền mặt và lập yêu cầu hoàn chuyển khoản

Nguồn: src/main/java/vn/edu/ute/milktea/service/cancellation/RefundService.java; RefundService.refundCash; dòng 72–120.

```
     public vn.edu.ute.milktea.dto.CancellationDto.RefundResponse refundCash(Long
orderId, Long cashierId) {
        var order = orderRepository.findByIdWithLock(orderId)
                                                              .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT,  "Không  tìm  thấy  đơn
hàng"));
                                                    if      (order.getStatus()      !=
vn.edu.ute.milktea.entity.order.OrderStatus.DA_HUY) {
            throw BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT, "Chỉ
có thể hoàn tiền cho đơn đã hủy");
        }
        var existing = refundRepository.findByOrderId(order.getId());
                    if  (existing.isPresent()  &&  existing.get().getStatus()  ==
RefundStatus.DA_HOAN) {
            throw BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT, "Đơn
hàng đã được hoàn tiền trước đó");
        }
        var invoice = invoiceRepository.findByOrderId(order.getId())
                                                              .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT,  "Không  tìm  thấy  hóa
đơn"));
        var payment = paymentRepository.findByInvoiceId(invoice.getId())
```

[Đã lược bỏ các dòng 85–85 của phương thức gốc.]

```
                                                    if      (payment.getMethod()      !=
vn.edu.ute.milktea.entity.payment.PaymentMethod.CASH) {
                throw BusinessException.conflict(ErrorCode.VALIDATION_FAILED, "Đơn
thanh toán chuyển khoản phải gửi yêu cầu hoàn tiền đến Admin");
        }
                                Account    cashier    =    cashierId    !=    null    ?
accountRepository.findById(cashierId).orElse(null) : null;
        RefundRequest refund;
        if (existing.isPresent()) {
            refund = existing.get();
            refund.setStatus(RefundStatus.DA_HOAN);
            refund.setRefundedBy(cashier);
            refund.setRefundedAt(Instant.now());
            refund.setRefundReference("CASH_REFUND_" + orderId);
```

[Đã lược bỏ các dòng 97–108 của phương thức gốc.]

```
        }
        refund = refundRepository.save(refund);
        auditRepository.save(BusinessAudit.builder()
                .orderId(order.getId())
                .accountId(cashierId)
                .action("CASH_REFUNDED")
                .reason("Hoàn tiền mặt thực tế: " + refund.getAmount() + " VND")
                .createdAt(Instant.now())
                .build());
        publish(refund);
        return toResponse(refund);
    }
```

2.2.10.4.Giải thích code

Không xóa Payment gốc để giả lập hoàn tiền. Chứng từ hoàn giữ liên kết với đơn bị hủy.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findByIdWithLock(); refundRepository.findByOrderId(); invoiceRepository.findByOrderId(); paymentRepository.findByInvoiceId(). Đường dẫn giao diện liên quan: src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js.

2.3.Nhân viên bếp

2.3.1.Xem hàng đợi bếp theo cơ sở

2.3.1.1.Mô tả chức năng

Bảng 2.37. Đặc tả: Xem hàng đợi bếp theo cơ sở

Tiêu chí	Nội dung
Mục đích	Cung cấp những đơn được thu ngân điều phối và đang thực hiện.
Tác nhân	Nhân viên bếp
Điều kiện trước	JWT vai trò bếp; có phân công cơ sở khi đơn mới mang ShopId.
Đầu vào	Actor KITCHEN/ADMIN, trạng thái hàng đợi server.
Kết quả / điều kiện sau	Không đưa đơn CHO_XAC_NHAN chưa điều phối vào hàng đợi chế biến.
Trường hợp lỗi	Không có quyền; hàng đợi rỗng là trạng thái hợp lệ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Truy vấn CHO_CHE_BIEN và DANG_CHE_BIEN.

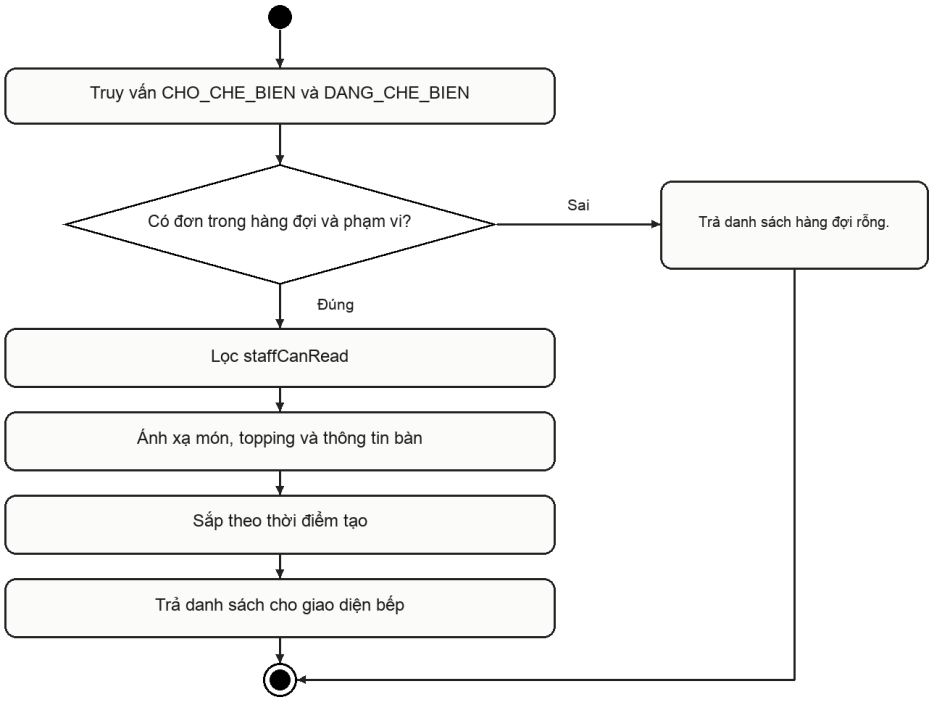
Bước 2. Lọc staffCanRead.

Bước 3. Ánh xạ món, topping và thông tin bàn.

Bước 4. Sắp theo thời điểm tạo.

Bước 5. Trả danh sách cho giao diện bếp.

2.3.1.2.Sơ đồ hoạt động



Hình 2.37. Hoạt động: Xem hàng đợi bếp theo cơ sở

2.3.1.3.Code xử lý chức năng

Đoạn mã 2.37. OrderService.getOrdersForKitchen — Xem hàng đợi bếp theo cơ sở

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.getOrdersForKitchen; dòng 958–962.

```
    public List<OrderDto.OrderResponse> getOrdersForKitchen() {
        List<Order> orders = orderRepository.findByStatusInOrderByCreatedAtAsc(
                List.of(OrderStatus.CHO_CHE_BIEN, OrderStatus.DANG_CHE_BIEN));
                                                                        return
orders.stream().filter(this::staffCanRead).map(this::mapOrderToResponse).toList();
    }
```

2.3.1.4.Giải thích code

Payment không phải điều kiện bắt buộc để bếp thấy đơn đã được xác nhận.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findByStatusInOrderByCreatedAtAsc(). Đường dẫn giao diện liên quan: src/main/resources/templates/kitchen/orders.html; src/main/resources/static/js/kitchen/kitchen.js.

2.3.2.Bắt đầu chế biến đơn

2.3.2.1.Mô tả chức năng

Bảng 2.38. Đặc tả: Bắt đầu chế biến đơn

Tiêu chí	Nội dung
Mục đích	Xác nhận bếp đã bắt đầu xử lý toàn đơn.
Tác nhân	Nhân viên bếp
Điều kiện trước	Đơn CHO_CHE_BIEN, đúng cơ sở, không có yêu cầu hủy CHO.
Đầu vào	orderId và kitchenId.
Kết quả / điều kiện sau	Hủy trước pha không còn được xử lý như đơn chưa bắt đầu.
Trường hợp lỗi	Đơn sai trạng thái, khác cơ sở hoặc CANCELLATION_PENDING.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa Order và kiểm tra nhân viên.

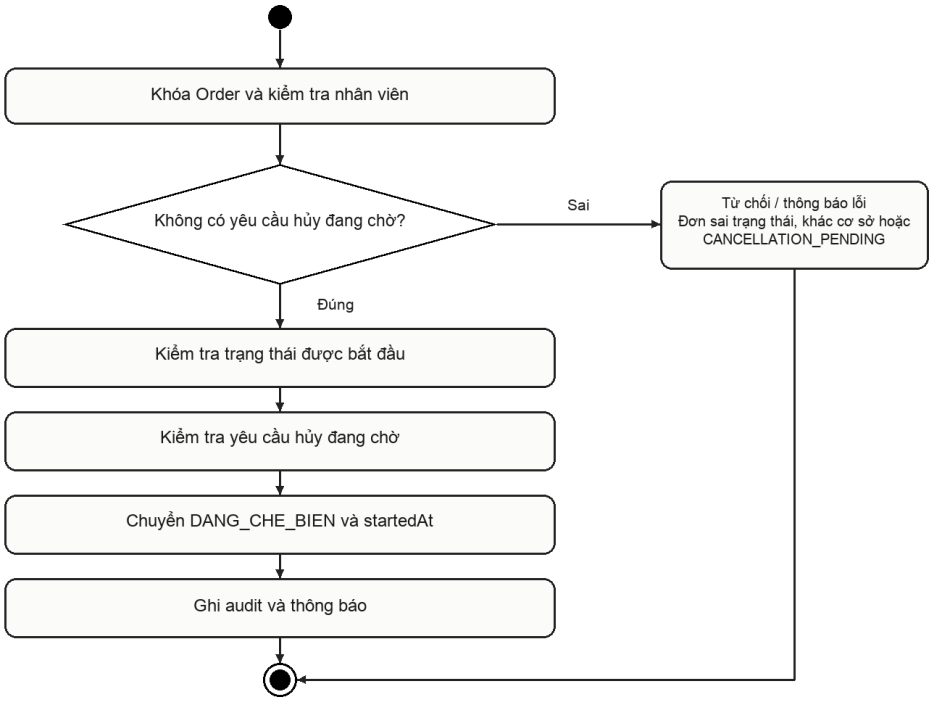
Bước 2. Kiểm tra trạng thái được bắt đầu.

Bước 3. Kiểm tra yêu cầu hủy đang chờ.

Bước 4. Chuyển DANG_CHE_BIEN và startedAt.

Bước 5. Ghi audit và thông báo.

2.3.2.2.Sơ đồ hoạt động



Hình 2.38. Hoạt động: Bắt đầu chế biến đơn

2.3.2.3.Code xử lý chức năng

Đoạn mã 2.38. OrderService.start — Bắt đầu chế biến đơn

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.start; dòng 355–395.

```
    public void start(Long orderId, Long kitchenId) {
        Order order = orderRepository.findByIdWithLock(orderId)
                                                        .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT,  "Không  tìm  thấy  đơn
hàng"));
        branchService.assertStaff(order, kitchenId);
        if (order.getStatus() == OrderStatus.DANG_CHE_BIEN) {
            return;
        }
        if (order.getStatus() != OrderStatus.CHO_CHE_BIEN) {
            throw BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT,
                        "Không  thể  bắt  đầu  chế  biến  đơn  ở  trạng  thái:  " +
order.getStatus());
        }
        Invoice invoice = invoiceRepository.findByOrderId(orderId)
                .orElseThrow(() -> BusinessException.notFound(
```

[Đã lược bỏ các dòng 368–371 của phương thức gốc.]

```
                                                    var       pendingCancel       =
cancellationRepository.findFirstByOrderIdAndStatus(orderId,
                vn.edu.ute.milktea.entity.cancellation.CancellationStatus.CHO);
        if (pendingCancel.isPresent()) {
            throw BusinessException.conflict(ErrorCode.CANCELLATION_PENDING,
                    "Đơn hàng đang có yêu cầu hủy chờ xử lý từ khách hàng. Không
thể bắt đầu.");
        }
```

```
        order.setStatus(OrderStatus.DANG_CHE_BIEN);
        order.setStartedAt(Instant.now());
        orderRepository.save(order);
        auditRepository.save(BusinessAudit.builder()
                .orderId(order.getId())
```

[Đã lược bỏ các dòng 383–383 của phương thức gốc.]

```
                .action("ORDER_STARTED")
                .beforeState("CHO_CHE_BIEN")
                .afterState("DANG_CHE_BIEN")
                .createdAt(Instant.now())
                .build());
                          String    sId    =    order.getSession()    !=    null    ?
order.getSession().getId().toString() : null;
                        realtimeEventPublisher.publishAfterCommit("/topic/kitchen",
"ORDER_STATUS_CHANGED",  order.getId().toString(),  sId,  "1",  Map.of("orderId",
order.getId(), "status", order.getStatus().name()));
                        realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"ORDER_STATUS_CHANGED",  order.getId().toString(),  sId,  "1",  Map.of("orderId",
order.getId(), "status", order.getStatus().name()));
        if (order.getSession() != null) {
              realtimeEventPublisher.publishAfterCommit("/topic/table-sessions/" +
order.getSession().getId(), "ORDER_STATUS_CHANGED", order.getId().toString(), sId,
"1", Map.of("orderId", order.getId(), "status", order.getStatus().name()));
        }
    }
```

2.3.2.4.Giải thích code

Hệ thống cập nhật trạng thái toàn đơn, không dựng trạng thái chế biến cho từng món.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findByIdWithLock(); branchService.assertStaff(); invoiceRepository.findByOrderId(); cancellationRepository.findFirstByOrderIdAndStatus(). Đường dẫn giao diện liên quan: src/main/resources/templates/kitchen/orders.html; src/main/resources/static/js/kitchen/kitchen.js.

2.3.3.Hoàn thành đơn và tiêu hao nguyên liệu

2.3.3.1.Mô tả chức năng

Bảng 2.39. Đặc tả: Hoàn thành đơn và tiêu hao nguyên liệu

Tiêu chí	Nội dung
Mục đích	Hoàn tất chế biến cùng việc ghi trừ nguyên liệu theo snapshot.
Tác nhân	Nhân viên bếp
Điều kiện trước	Đúng trạng thái được complete, đúng cơ sở và tồn kho đủ theo snapshot.
Đầu vào	orderId và nhân viên bếp.
Kết quả / điều kiện sau	Đơn hoàn thành và biến động kho cùng được commit; không tự đóng bàn.
Trường hợp lỗi	Thiếu tồn kho, snapshot không hợp lệ hoặc trạng thái không phù hợp.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa Order và kiểm tra trạng thái.

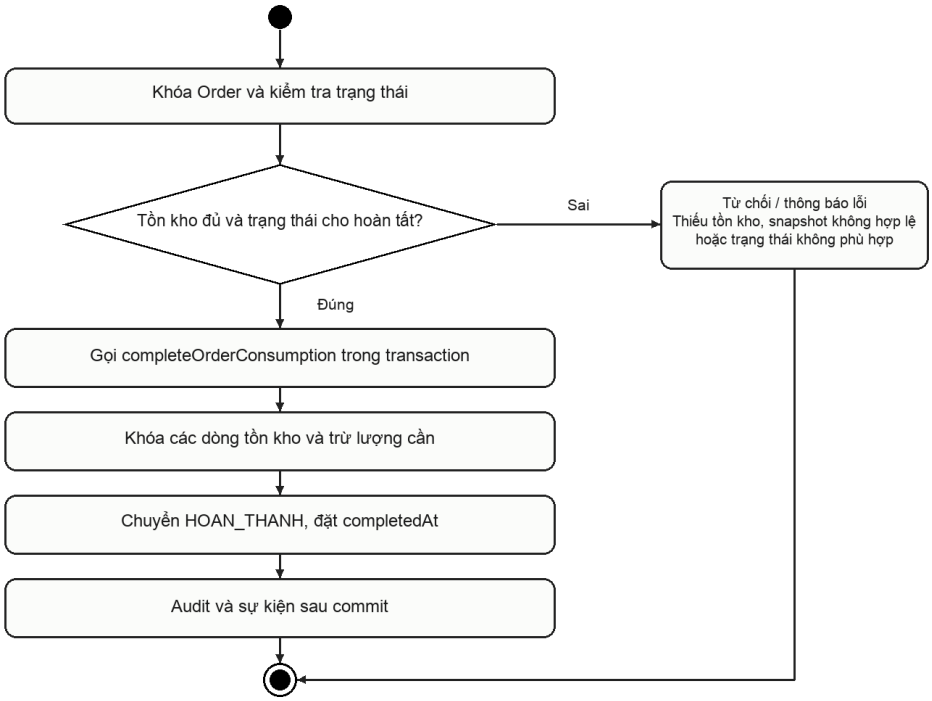
Bước 2. Gọi completeOrderConsumption trong transaction.

Bước 3. Khóa các dòng tồn kho và trừ lượng cần.

Bước 4. Chuyển HOAN_THANH, đặt completedAt.

Bước 5. Audit và sự kiện sau commit.

2.3.3.2.Sơ đồ hoạt động



Hình 2.39. Hoạt động: Hoàn thành đơn và tiêu hao nguyên liệu

2.3.3.3.Code xử lý chức năng

Đoạn mã 2.39. OrderService.complete — Hoàn thành đơn và tiêu hao nguyên liệu

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.complete; dòng 398–432.

```
    public void complete(Long orderId, Long kitchenId) {
        Order order = orderRepository.findByIdWithLock(orderId)
                                                        .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT,  "Không  tìm  thấy  đơn
hàng"));
        branchService.assertStaff(order, kitchenId);
        if (order.getStatus() == OrderStatus.HOAN_THANH) {
            return;
        }
        if (order.getStatus() != OrderStatus.DANG_CHE_BIEN) {
            throw BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT,
                    "Chỉ có thể hoàn thành đơn đang ở trạng thái DANG_CHE_BIEN.
Trạng thái hiện tại: " + order.getStatus());
        }
        Account kitchen = accountRepository.findById(kitchenId).orElse(null);
         // Tiêu hao kho BEP theo công thức đang lưu, nhân số ly và lưu snapshot
thực tế.
        // Nếu không đủ tồn kho, ném BusinessException.conflict và không đổi trạng
thái đơn
        inventoryService.completeOrderConsumption(order, kitchen);
        // Chuyển trạng thái đơn sang HOAN_THANH khi tiêu hao kho thành công
        order.setStatus(OrderStatus.HOAN_THANH);
        order.setCompletedAt(Instant.now());
        orderRepository.save(order);
        auditRepository.save(BusinessAudit.builder()
                .orderId(order.getId())
```

```
                .accountId(kitchenId)
                .action("ORDER_COMPLETED")
                .beforeState("DANG_CHE_BIEN")
                .afterState("HOAN_THANH")
                .createdAt(Instant.now())
                .build());
                        String sId = order.getSession() != null ?
order.getSession().getId().toString() : null;
                    realtimeEventPublisher.publishAfterCommit("/topic/kitchen",
"ORDER_STATUS_CHANGED", order.getId().toString(), sId, "1", Map.of("orderId",
order.getId(), "status", order.getStatus().name()));
                    realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"ORDER_STATUS_CHANGED", order.getId().toString(), sId, "1", Map.of("orderId",
order.getId(), "status", order.getStatus().name()));
        if (order.getSession() != null) {
              realtimeEventPublisher.publishAfterCommit("/topic/table-sessions/" +
order.getSession().getId(), "ORDER_STATUS_CHANGED", order.getId().toString(), sId,
"1", Map.of("orderId", order.getId(), "status", order.getStatus().name()));
        }
         realtimeEventPublisher.publishAfterCommit("/topic/stock", "STOCK_CHANGED",
order.getId().toString(), sId, "1", Map.of("orderId", order.getId()));
    }
```

2.3.3.4.Giải thích code

Nếu transaction thất bại không được báo hoàn thành thành công chỉ vì UI đổi nút.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findByIdWithLock(); branchService.assertStaff(); accountRepository.findById(); inventoryService.completeOrderConsumption(). Đường dẫn giao diện liên quan: src/main/resources/templates/kitchen/orders.html; src/main/resources/static/js/kitchen/kitchen.js.

2.3.4.Theo dõi tồn kho và cảnh báo ngưỡng

2.3.4.1.Mô tả chức năng

Bảng 2.40. Đặc tả: Theo dõi tồn kho và cảnh báo ngưỡng

Tiêu chí	Nội dung
Mục đích	Tra cứu lượng tồn ở KHO/BEP và các mức cảnh báo.
Tác nhân	Nhân viên bếp
Điều kiện trước	Vai trò được nhóm API inventory cho phép; bộ lọc theo dữ liệu hiện có.
Đầu vào	location, type, alertFilter, query.
Kết quả / điều kiện sau	Chỉ đọc tồn; không làm thay đổi số lượng khi đổi tab/lọc.
Trường hợp lỗi	Không có quyền hoặc tập kết quả rỗng.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Chọn tập tồn theo vị trí và loại nguyên liệu.

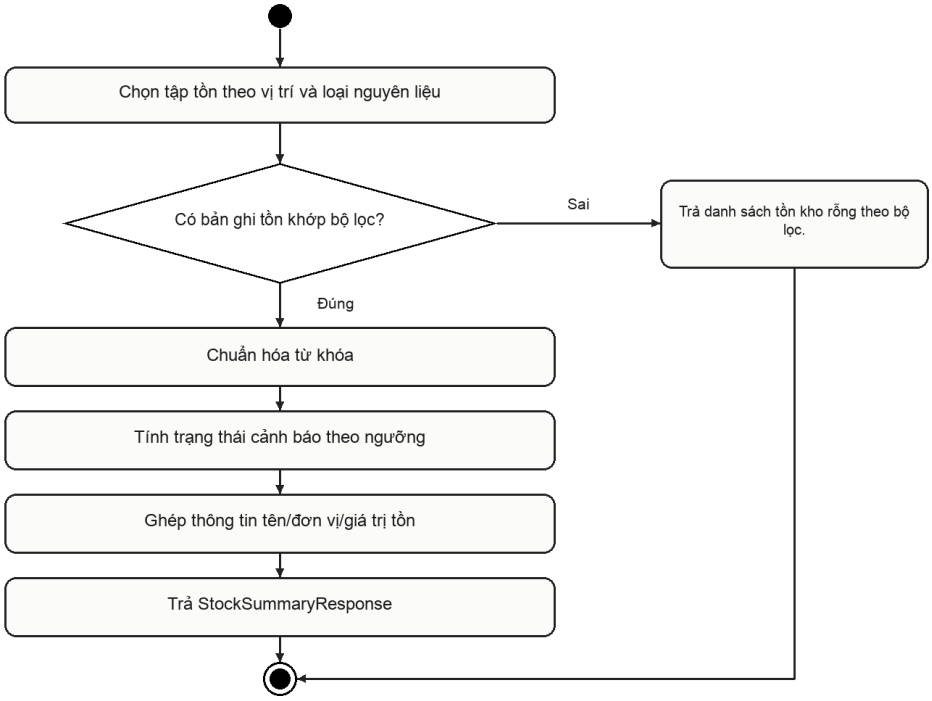
Bước 2. Chuẩn hóa từ khóa.

Bước 3. Tính trạng thái cảnh báo theo ngưỡng.

Bước 4. Ghép thông tin tên/đơn vị/giá trị tồn.

Bước 5. Trả StockSummaryResponse.

2.3.4.2.Sơ đồ hoạt động



Hình 2.40. Hoạt động: Theo dõi tồn kho và cảnh báo ngưỡng

2.3.4.3.Code xử lý chức năng

Đoạn mã 2.40. InventoryService.getStocksWithFilter — Theo dõi tồn kho và cảnh báo ngưỡng

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.getStocksWithFilter; dòng 506–556.

```
       public InventoryDto.StockSummaryResponse getStocksWithFilter(StockLocation
location, MaterialType type, String alertFilter, String query) {
        StockLocation loc = location != null ? location : StockLocation.KHO;
        List<Stock> stocks = stockRepository.findByIdLocation(loc);

        var streamInit = stocks.stream()
                                    .filter(s -> s.getMaterial() != null &&
Boolean.TRUE.equals(s.getMaterial().getActive()));
        if (type != null) {
                streamInit = streamInit.filter(s -> s.getMaterial().getType() ==
type);
        } else if (loc == StockLocation.KHO) {
                streamInit = streamInit.filter(s -> s.getMaterial().getType() ==
MaterialType.THO);
        }

        List<InventoryDto.StockItemResponse> allItems = streamInit
```

[Đã lược bỏ các dòng 519–529 của phương thức gốc.]

```
        if (query != null && !query.isBlank()) {
            String q = query.trim().toLowerCase();
                                                stream    =    stream.filter(it    ->
it.getMaterialName().toLowerCase().contains(q));
        }

                        if   (alertFilter   !=   null   &&   !alertFilter.isBlank()
&& !"ALL".equalsIgnoreCase(alertFilter)) {
            String filter = alertFilter.trim().toUpperCase();
            if ("HET_TON".equals(filter)) {
                                                stream   =   stream.filter(it   ->
"HET_TON".equals(it.getAlertStatus()));
            } else if ("TON_THAP".equals(filter)) {
                                                stream   =   stream.filter(it   ->
"TON_THAP".equals(it.getAlertStatus()));
```

[Đã lược bỏ các dòng 541–544 của phương thức gốc.]

```
            }
        }

        return InventoryDto.StockSummaryResponse.builder()
                .totalCount(totalCount)
                .hetTonCount(hetTonCount)
                .tonThapCount(tonThapCount)
                .binhThuongCount(binhThuongCount)
                .chuaCauHinhCount(chuaCauHinhCount)
                .items(stream.toList())
                .build();
    }
```

2.3.4.4.Giải thích code

Giao diện tồn được dùng chung cho nhóm quyền phù hợp; không khẳng định mọi báo cáo kho đã tách ShopId vì schema Stock khóa theo material/location.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: stockRepository.findByIdLocation(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/inventory.html; src/main/resources/static/js/admin/admin.js.

2.3.5.Ghi mẻ sơ chế từ định mức

2.3.5.1.Mô tả chức năng

Bảng 2.41. Đặc tả: Ghi mẻ sơ chế từ định mức

Tiêu chí	Nội dung
Mục đích	Chuyển nguyên liệu đầu vào thành bán thành phẩm với lượng thực tế, giá vốn và dấu vết mẻ.
Tác nhân	Nhân viên bếp
Điều kiện trước	Định mức sơ chế hoạt động; loại đầu ra phù hợp; tồn đầu vào đủ; người dùng đúng quyền.
Đầu vào	Nguyên liệu đầu ra, sản lượng thực tế, định mức, số lượng đầu vào và Idempotency-Key.
Kết quả / điều kiện sau	Có chứng từ mẻ và nhật ký các biến động liên quan trong một giao dịch.
Trường hợp lỗi	Thiếu tồn, sản lượng không hợp lệ, công thức thiếu hoặc yêu cầu trùng khác nội dung.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra chống lặp và cấu hình định mức.

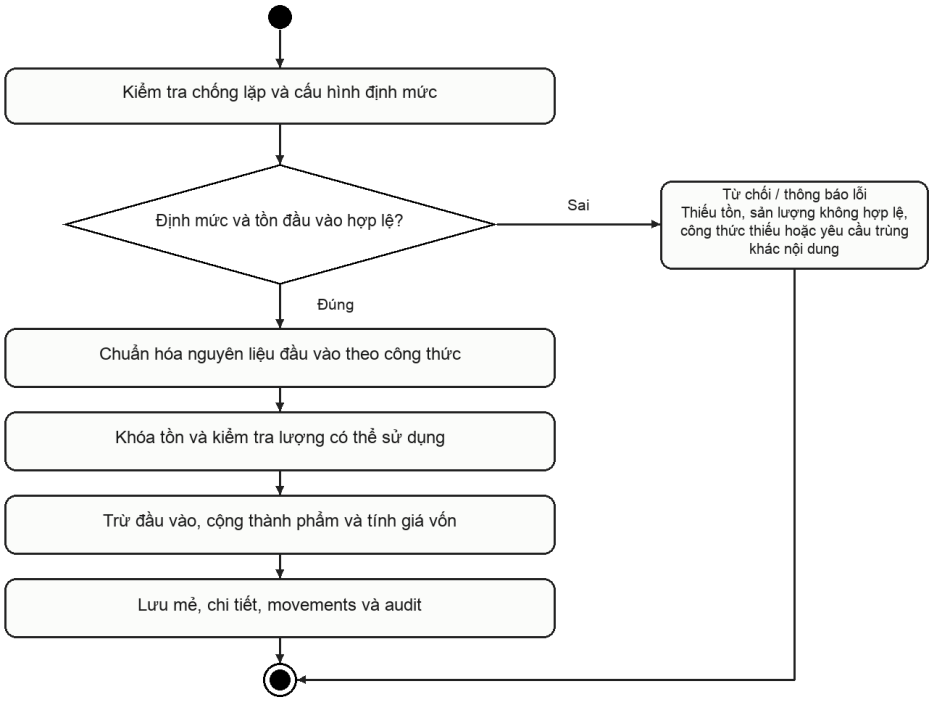
Bước 2. Chuẩn hóa nguyên liệu đầu vào theo công thức.

Bước 3. Khóa tồn và kiểm tra lượng có thể sử dụng.

Bước 4. Trừ đầu vào, cộng thành phẩm và tính giá vốn.

Bước 5. Lưu mẻ, chi tiết, movements và audit.

2.3.5.2.Sơ đồ hoạt động



Hình 2.41. Hoạt động: Ghi mẻ sơ chế từ định mức

2.3.5.3.Code xử lý chức năng

Đoạn mã 2.41. InventoryService.recordPreparation — Ghi mẻ sơ chế từ định mức

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.recordPreparation; dòng 141–414.

```
    public void recordPreparation(
            InventoryDto.CreatePreparationBatchRequest request,
            Account kitchenAccount,
            String key) {
        if (kitchenAccount == null) {
            throw BusinessException.forbidden(ErrorCode.ACCESS_DENIED, "Bạn chưa
đăng nhập");
        }
        if (key != null && key.length() > 64) {
            throw BusinessException.badRequest(
                    ErrorCode.VALIDATION_FAILED,
                    "Mã yêu cầu quá dài");
        }
```

[Đã lược bỏ các dòng 154–157 của phương thức gốc.]

```
        if (desiredQuantity == null || desiredQuantity.signum() <= 0) {
            throw BusinessException.badRequest(
                    ErrorCode.VALIDATION_FAILED,
                    "Sản lượng thành phẩm mong muốn phải lớn hơn 0");
        }

        String requestKey = (key == null || key.isBlank())
                ? UUID.randomUUID().toString()
                : key;
```

```
        accountRepository.findByIdWithLock(kitchenAccount.getId())
```

[Đã lược bỏ các dòng 169–402 của phương thức gốc.]

```
                batch.getId().toString(),
                null,
                "1",
                java.util.Map.of("action", "PREPARATION"));
        realtimeEventPublisher.publishAfterCommit(
                "/topic/admin",
                "STOCK_CHANGED",
                batch.getId().toString(),
                null,
                "1",
                java.util.Map.of("action", "PREPARATION"));
    }
```

2.3.5.4.Giải thích code

Không bịa định lượng khi source/cấu hình chưa có; một số overload là wrapper, báo cáo trích overload xử lý chính.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findByIdWithLock(); recipeRepository.findById(); recipeRepository.findByOutputMaterialIdAndActiveTrue(); batchRepository.findByIdempotencyKey(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/inventory.html; src/main/resources/static/js/admin/admin.js.

2.3.6.Nhập nguyên liệu thô vào kho tổng

2.3.6.1.Mô tả chức năng

Bảng 2.42. Đặc tả: Nhập nguyên liệu thô vào kho tổng

Tiêu chí	Nội dung
Mục đích	Ghi phiếu nhập và cập nhật đơn giá bình quân theo dữ liệu nhập.
Tác nhân	Nhân viên bếp
Điều kiện trước	Chỉ THO được nhập tại KHO; số lượng và đơn giá hợp lệ.
Đầu vào	Danh sách nguyên liệu, lượng nhập, giá nhập; idempotencyKey và người ghi.
Kết quả / điều kiện sau	Tồn và giá vốn phản ánh phiếu nhập; giữ chứng từ thay vì sửa số lượng ngầm.
Trường hợp lỗi	Nguyên liệu không tồn tại, đơn giá âm, dữ liệu vốn cũ thiếu hoặc nội dung trùng xung đột.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra key chống lặp.

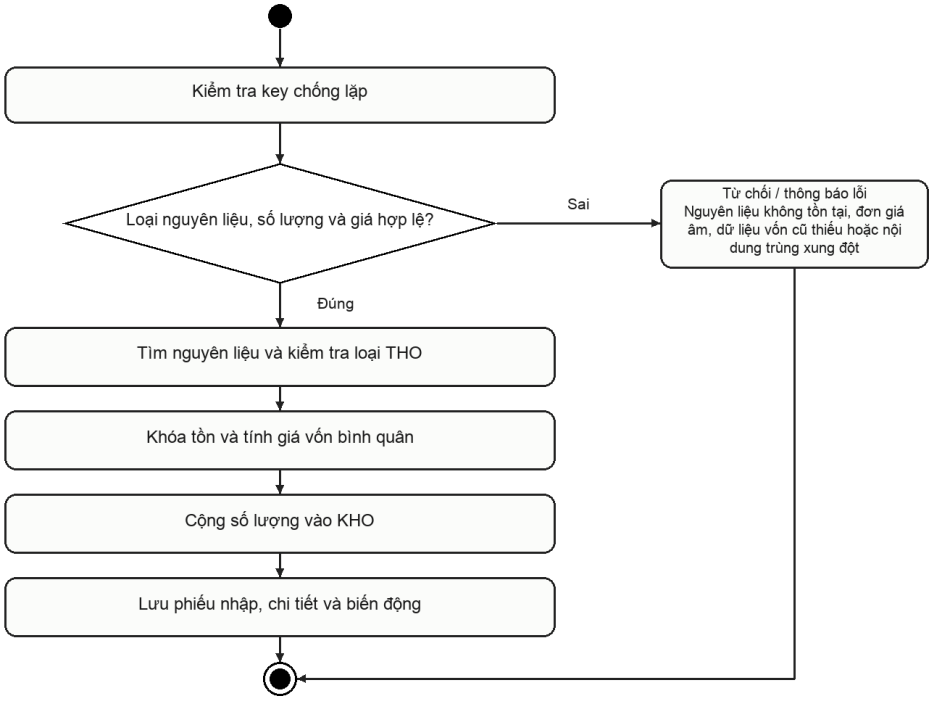
Bước 2. Tìm nguyên liệu và kiểm tra loại THO.

Bước 3. Khóa tồn và tính giá vốn bình quân.

Bước 4. Cộng số lượng vào KHO.

Bước 5. Lưu phiếu nhập, chi tiết và biến động.

2.3.6.2.Sơ đồ hoạt động



Hình 2.42. Hoạt động: Nhập nguyên liệu thô vào kho tổng

2.3.6.3.Code xử lý chức năng

Đoạn mã 2.42. InventoryService.recordImport — Nhập nguyên liệu thô vào kho tổng

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.recordImport; dòng 616–718.

```
    public InventoryDto.ImportReceiptResponse recordImport(
                InventoryDto.CreateImportReceiptRequest request,
                Account account) {
        ImportReceipt receipt = ImportReceipt.builder()
                .createdBy(account)
                .reason(request.getReason())
                .supplier(request.getSupplier())
                .status(DocumentStatus.DA_GHI_SO)
                .createdAt(Instant.now())
                .postedAt(Instant.now())
                .idempotencyKey(java.util.UUID.randomUUID().toString())
                .build();
        receipt = importReceiptRepository.save(receipt);
```

[Đã lược bỏ các dòng 629–630 của phương thức gốc.]

```
        for (var it : request.getItems()) {
            Material material = materialRepository.findById(it.getMaterialId())
                                                  .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED,  "Nguyên  liệu  không  tồn
tại: ID " + it.getMaterialId()));
```

```
            if (material.getType() != MaterialType.THO) {
                throw BusinessException.badRequest(
                        ErrorCode.VALIDATION_FAILED,
                                "Kho tổng chỉ được nhập nguyên liệu thô: " +
material.getName()
                );
            }

            BigDecimal importQty = it.getQuantity();
```

[Đã lược bỏ các dòng 642–706 của phương thức gốc.]

```
        movementRepository.saveAll(movements);
         realtimeEventPublisher.publishAfterCommit("/topic/stock", "STOCK_CHANGED",
receipt.getId().toString(), null, "1", java.util.Map.of("action", "IMPORT"));
         realtimeEventPublisher.publishAfterCommit("/topic/admin", "STOCK_CHANGED",
receipt.getId().toString(), null, "1", java.util.Map.of("action", "IMPORT"));
        return InventoryDto.ImportReceiptResponse.builder()
                .id(receipt.getId())
                .reason(receipt.getReason())
                .supplier(receipt.getSupplier())
                .status(receipt.getStatus().name())
                .createdAt(receipt.getCreatedAt())
                .createdByEmail(account != null ? account.getEmail() : null)
                .build();
    }
```

2.3.6.4.Giải thích code

Controller có kiểm tra quyền ghi nhập; không suy ra bất kỳ actor có quyền xem inventory đều được thực hiện mọi lệnh.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: importReceiptRepository.save(); materialRepository.findById(); stockRepository.findByIdWithLock(); stockRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/inventory.html; src/main/resources/static/js/admin/admin.js.

2.3.7.Ghi hao hụt và nguyên liệu loại bỏ

2.3.7.1.Mô tả chức năng

Bảng 2.43. Đặc tả: Ghi hao hụt và nguyên liệu loại bỏ

Tiêu chí	Nội dung
Mục đích	Ghi nhận nguyên liệu thất thoát/hủy bằng lượng và lý do thực tế.
Tác nhân	Nhân viên bếp
Điều kiện trước	Có tồn tại vị trí; KHO chỉ áp dụng THO; số lượng không vượt tồn.
Đầu vào	materialId, vị trí, lượng hao hụt, lý do.
Kết quả / điều kiện sau	Tồn giảm và có chứng từ giải thích hao hụt.
Trường hợp lỗi	Không có nguyên liệu/tồn, loại không phù hợp hoặc lượng vượt tồn.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm nguyên liệu và kiểm tra vị trí/loại.

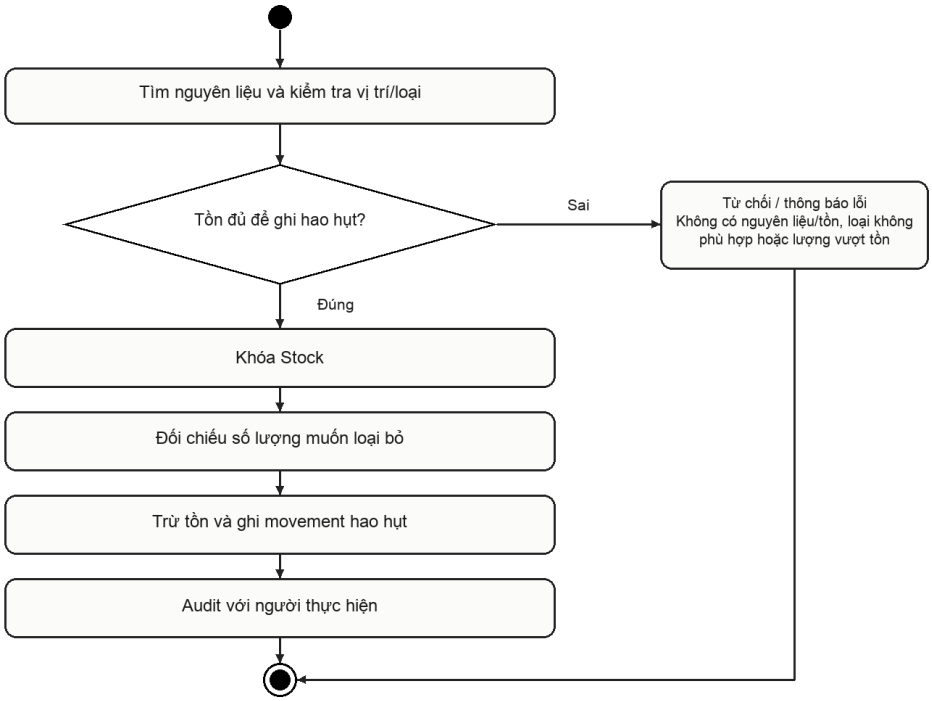
Bước 2. Khóa Stock.

Bước 3. Đối chiếu số lượng muốn loại bỏ.

Bước 4. Trừ tồn và ghi movement hao hụt.

Bước 5. Audit với người thực hiện.

2.3.7.2.Sơ đồ hoạt động



Hình 2.43. Hoạt động: Ghi hao hụt và nguyên liệu loại bỏ

2.3.7.3.Code xử lý chức năng

Đoạn mã 2.43. InventoryService.recordWaste — Ghi hao hụt và nguyên liệu loại bỏ

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.recordWaste; dòng 738–810.

```
       public void recordWaste(InventoryDto.CreateWasteRequest request, Account
account) {
        StockLocation location = StockLocation.KHO;
        List<StockMovement> movements = new ArrayList<>();
        for (var it : request.getItems()) {
            Material material = materialRepository.findById(it.getMaterialId())
                                                          .orElseThrow(()   ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED,  "Nguyên  liệu  không  tồn
tại: ID " + it.getMaterialId()));

            if (material.getType() != MaterialType.THO) {
                throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED,
                        "Hao hụt Kho tổng chỉ áp dụng cho nguyên liệu thô (THO): "
+ material.getName());
            }

            if (it.getQuantity() == null || it.getQuantity().signum() <= 0) {
```

[Đã lược bỏ các dòng 751–751 của phương thức gốc.]

```
            }

                 Stock  stock  =  stockRepository.findByIdWithLock(material.getId(),
location)
                                                          .orElseThrow(()   ->
BusinessException.notFound(ErrorCode.INVENTORY_INSUFFICIENT,
```

```
                                    "Không tìm thấy tồn kho cho nguyên liệu: " +
material.getName()));

                            BigDecimal before = stock.getQuantity() != null ?
stock.getQuantity() : BigDecimal.ZERO;
            if (before.compareTo(it.getQuantity()) < 0) {
                throw BusinessException.conflict(ErrorCode.INVENTORY_INSUFFICIENT,
                        "Tồn kho không đủ để hủy: " + material.getName() + " (Còn:
" + before + ", cần hủy: " + it.getQuantity() + ")");
            }
```

[Đã lược bỏ các dòng 763–798 của phương thức gốc.]

```
                    .build());
        }
        movementRepository.saveAll(movements);

auditRepository.save(vn.edu.ute.milktea.entity.audit.BusinessAudit.builder()
                .accountId(account != null ? account.getId() : null)
                .action("INVENTORY_WASTE_RECORDED")
                .reason(request.getReason())
                .createdAt(Instant.now())
                .build());
        realtimeEventPublisher.publishAfterCommit("/topic/stock", "STOCK_CHANGED",
location.name(), null, "1", java.util.Map.of("action", "WASTE"));
        realtimeEventPublisher.publishAfterCommit("/topic/admin", "STOCK_CHANGED",
location.name(), null, "1", java.util.Map.of("action", "WASTE"));
    }
```

2.3.7.4.Giải thích code

Không tự bù kho bằng việc sửa một đơn hàng đã hoàn tất.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: materialRepository.findById(); stockRepository.findByIdWithLock(); stockRepository.save(); movementRepository.saveAll(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/inventory.html; src/main/resources/static/js/admin/admin.js.

2.3.8.Ghi nhận và xử lý sự cố chế biến

2.3.8.1.Mô tả chức năng

Bảng 2.44. Đặc tả: Ghi nhận và xử lý sự cố chế biến

Tiêu chí	Nội dung
Mục đích	Ghi chú sự cố theo mã đơn để giữ lịch sử xử lý của bếp.
Tác nhân	Nhân viên bếp
Điều kiện trước	Đơn tồn tại; actor được API bếp cho phép.
Đầu vào	orderId, nội dung IncidentRequest, actor hiện tại.
Kết quả / điều kiện sau	Sự cố được xem lại qua getIncidents; không phát sinh một entity Incident không có trong source.
Trường hợp lỗi	Không tìm thấy đơn hoặc dữ liệu request không hợp lệ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm Order.

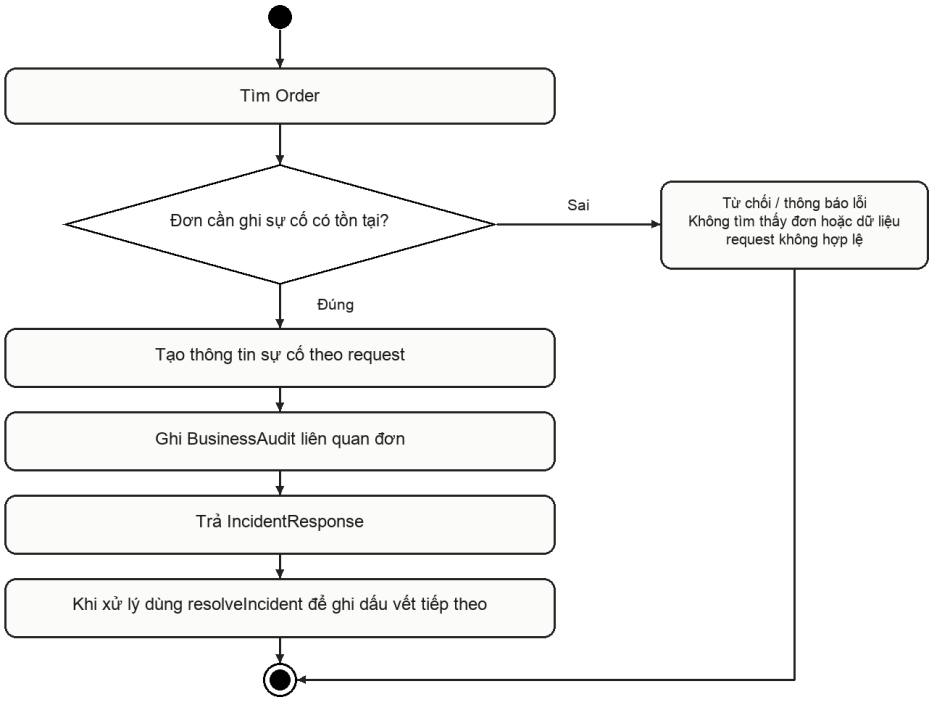
Bước 2. Tạo thông tin sự cố theo request.

Bước 3. Ghi BusinessAudit liên quan đơn.

Bước 4. Trả IncidentResponse.

Bước 5. Khi xử lý dùng resolveIncident để ghi dấu vết tiếp theo.

2.3.8.2.Sơ đồ hoạt động



Hình 2.44. Hoạt động: Ghi nhận và xử lý sự cố chế biến

2.3.8.3.Code xử lý chức năng

Đoạn mã 2.44. InventoryService.recordIncident — Ghi nhận và xử lý sự cố chế biến

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.recordIncident; dòng 950–969.

```
            public InventoryDto.IncidentResponse recordIncident(Long orderId,
InventoryDto.IncidentRequest request, Account kitchenAccount) {
        Order order = orderRepository.findById(orderId)
                                                        .orElseThrow(() ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT, "Không tìm thấy đơn
hàng #" + orderId));
                        vn.edu.ute.milktea.entity.audit.BusinessAudit audit =
vn.edu.ute.milktea.entity.audit.BusinessAudit.builder()
                .orderId(order.getId())
                .accountId(kitchenAccount != null ? kitchenAccount.getId() : null)
                .action("ORDER_INCIDENT_RECORDED")
                 .reason(request.getReason() + (request.getNotes() != null ? " (" +
request.getNotes() + ")" : ""))
                .createdAt(Instant.now())
                .build();
        audit = auditRepository.save(audit);
        return InventoryDto.IncidentResponse.builder()
                .id(audit.getId())
                .orderId(order.getId())
                .reason(audit.getReason())
                .createdAt(audit.getCreatedAt())
                 .createdByEmail(kitchenAccount != null ? kitchenAccount.getEmail()
: null)
                .resolved(false)
```

```
                .build();
    }
```

2.3.8.4.Giải thích code

Báo cáo nêu đúng lưu trữ bằng audit; không tự đặt bảng SuCoPhaChe.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findById(); auditRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/kitchen/orders.html; src/main/resources/static/js/kitchen/kitchen.js.

2.3.9.Ghi tiêu hao bổ sung khi pha bù

2.3.9.1.Mô tả chức năng

Bảng 2.45. Đặc tả: Ghi tiêu hao bổ sung khi pha bù

Tiêu chí	Nội dung
Mục đích	Tách lượng pha bù khỏi tiêu hao snapshot ban đầu của đơn.
Tác nhân	Nhân viên bếp
Điều kiện trước	Có đơn, có nguyên liệu và tồn BEP đủ.
Đầu vào	orderId, các dòng nguyên liệu và lượng dùng thêm.
Kết quả / điều kiện sau	Tồn kho có dấu vết riêng cho lượng phát sinh thực tế.
Trường hợp lỗi	Nguyên liệu/tồn không tồn tại hoặc thiếu lượng để pha bù.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm Order và dữ liệu bổ sung.

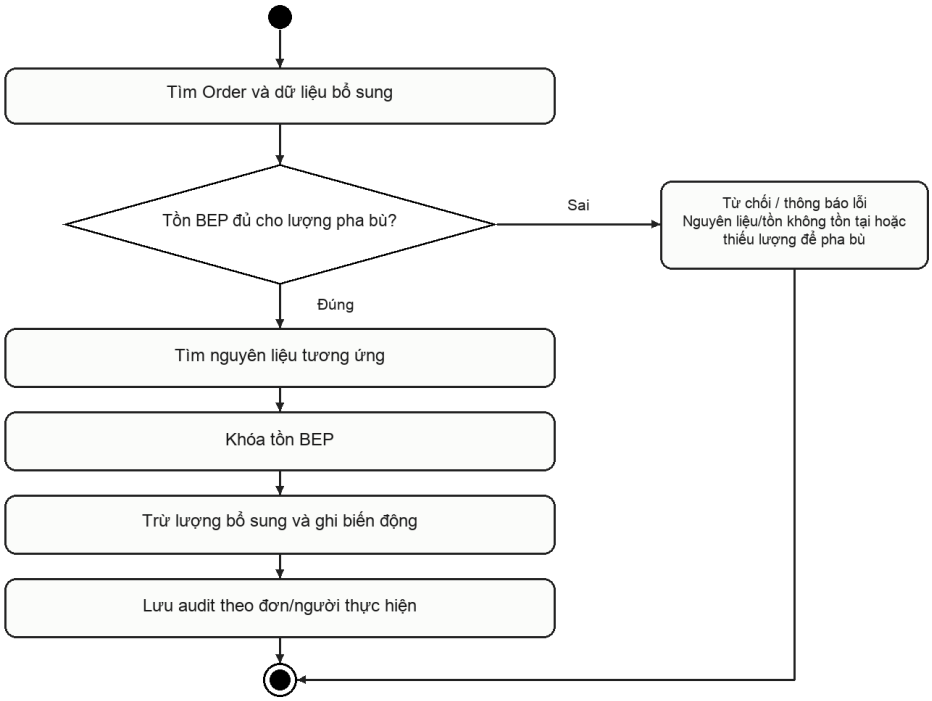
Bước 2. Tìm nguyên liệu tương ứng.

Bước 3. Khóa tồn BEP.

Bước 4. Trừ lượng bổ sung và ghi biến động.

Bước 5. Lưu audit theo đơn/người thực hiện.

2.3.9.2.Sơ đồ hoạt động



Hình 2.45. Hoạt động: Ghi tiêu hao bổ sung khi pha bù

2.3.9.3.Code xử lý chức năng

Đoạn mã 2.45. InventoryService.recordAdditionalConsumption — Ghi tiêu hao bổ sung khi pha bù

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.recordAdditionalConsumption; dòng 999–1041.

```
                public void recordAdditionalConsumption(Long orderId,
InventoryDto.AdditionalConsumptionRequest request, Account kitchenAccount) {
        Order order = orderRepository.findById(orderId)
                                                          .orElseThrow(() ->
BusinessException.notFound(ErrorCode.ORDER_STATE_CONFLICT, "Không tìm thấy đơn
hàng #" + orderId));
        List<StockMovement> movements = new ArrayList<>();
        for (var item : request.getItems()) {
            Material material = materialRepository.findById(item.getMaterialId())
                                                          .orElseThrow(() ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED, "Nguyên liệu không tồn
tại: ID " + item.getMaterialId()));
                Stock stock = stockRepository.findByIdWithLock(material.getId(),
StockLocation.BEP)
                                                          .orElseThrow(() ->
BusinessException.notFound(ErrorCode.INVENTORY_INSUFFICIENT,
                                "Không tìm thấy tồn tại Bếp cho nguyên liệu: " +
material.getName()));
            if (stock.getQuantity().compareTo(item.getQuantity()) < 0) {
                throw BusinessException.conflict(ErrorCode.INVENTORY_INSUFFICIENT,
                        "Tồn tại Bếp không đủ để pha bù: " + material.getName() +
" (Còn: " + stock.getQuantity() + ", cần: " + item.getQuantity() + ")");
```

[Đã lược bỏ các dòng 1012–1017 của phương thức gốc.]

```
                        .material(material)
                        .location(StockLocation.BEP)
                        .type(StockMovementType.TIEU_HAO)
                        .deltaQuantity(item.getQuantity().negate())
                        .beforeQuantity(before)
                        .afterQuantity(after)
                        .sourceType("REMAKE")
                        .sourceId(order.getId())
                        .sourceLine(material.getId())
                        .createdBy(kitchenAccount)
                        .createdAt(Instant.now())
```

[Đã lược bỏ các dòng 1029–1029 của phương thức gốc.]

```
                        .build());
                }
                movementRepository.saveAll(movements);

        auditRepository.save(vn.edu.ute.milktea.entity.audit.BusinessAudit.builder()
                        .orderId(order.getId())
                        .accountId(kitchenAccount != null ? kitchenAccount.getId() : null)
                        .action("ORDER_REMAKE_CONSUMED")
                        .reason(request.getReason())
                        .createdAt(Instant.now())
                        .build());
                 realtimeEventPublisher.publishAfterCommit("/topic/stock", "STOCK_CHANGED",
        order.getId().toString(), null, "1", java.util.Map.of("action", "REMAKE"));
            }
```

2.3.9.4.Giải thích code

Không thay thế hoặc sửa ngược snapshot món ban đầu để che lượng dùng thêm.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findById(); materialRepository.findById(); stockRepository.findByIdWithLock(); stockRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/kitchen/orders.html; src/main/resources/static/js/kitchen/kitchen.js.

2.3.10.Kiểm kê và kết ca kho

2.3.10.1.Mô tả chức năng

Bảng 2.46. Đặc tả: Kiểm kê và kết ca kho

Tiêu chí	Nội dung
Mục đích	Chốt tồn thực tế và hao hụt cuối ca bằng chứng từ đối chiếu sổ sách.
Tác nhân	Nhân viên bếp
Điều kiện trước	ADMIN/KITCHEN theo PreAuthorize; dữ liệu có dòng và tồn thực tế thỏa điều kiện.
Đầu vào	location, dòng kiểm kê, số lượng thực tế và lý do.
Kết quả / điều kiện sau	Biên bản ghi sổ sách, thực tế, chênh lệch và người kết ca.
Trường hợp lỗi	Danh sách rỗng, tồn không có, số lượng vượt giới hạn quy định hoặc key trùng.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Lấy preview tồn sổ sách.

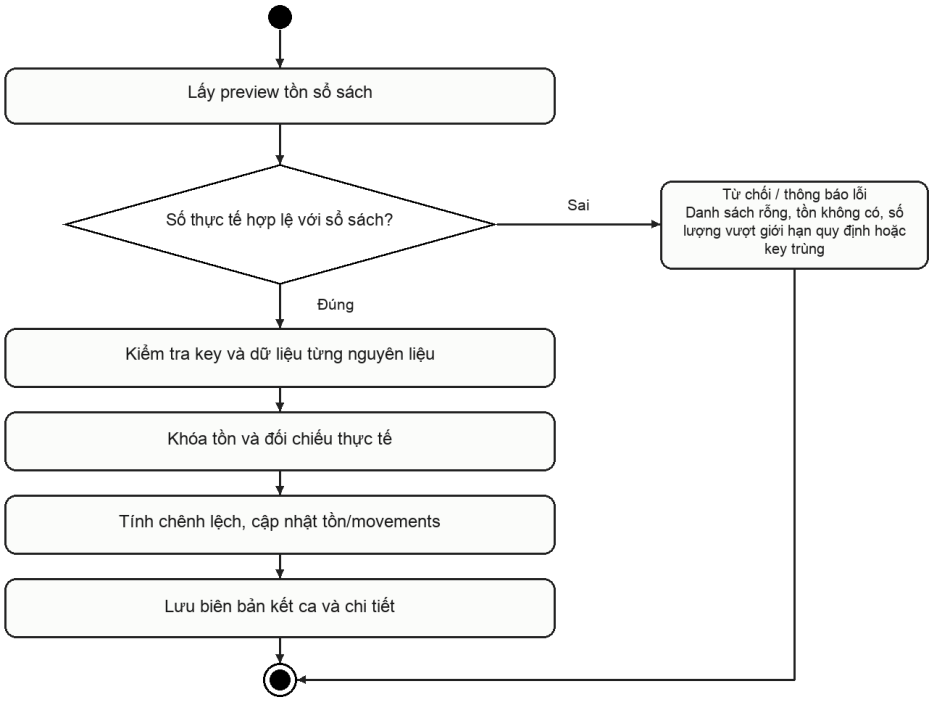
Bước 2. Kiểm tra key và dữ liệu từng nguyên liệu.

Bước 3. Khóa tồn và đối chiếu thực tế.

Bước 4. Tính chênh lệch, cập nhật tồn/movements.

Bước 5. Lưu biên bản kết ca và chi tiết.

2.3.10.2.Sơ đồ hoạt động



Hình 2.46. Hoạt động: Kiểm kê và kết ca kho

2.3.10.3.Code xử lý chức năng

Đoạn mã 2.46. InventoryService.recordShiftSettlement — Kiểm kê và kết ca kho

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.recordShiftSettlement; dòng 1140–1265.

```
    public InventoryDto.ShiftSettlementResponse recordShiftSettlement(
            InventoryDto.CreateShiftSettlementRequest request,
            Account account,
            String idempotencyKey) {
        if (account == null) {
            throw BusinessException.forbidden(ErrorCode.ACCESS_DENIED, "Bạn chưa
đăng nhập");
        }

        String reqKey = (idempotencyKey == null || idempotencyKey.isBlank())
                ? UUID.randomUUID().toString()
                : idempotencyKey;

        var existing = shiftSettlementRepository.findByIdempotencyKey(reqKey);
```

[Đã lược bỏ các dòng 1153–1153 của phương thức gốc.]

```
            return mapShiftSettlement(existing.get());
        }

        if (request.getItems() == null || request.getItems().isEmpty()) {
            throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED, "Danh
sách kiểm kê không được rỗng");
        }

        Instant now = Instant.now();
```

```
        ShiftSettlement settlement = ShiftSettlement.builder()
                .location(request.getLocation())
                .closedAt(now)
```

[Đã lược bỏ các dòng 1165–1253 của phương thức gốc.]

```
        }

        realtimeEventPublisher.publishAfterCommit(
                "/topic/stock",
                "STOCK_CHANGED",
                settlement.getId().toString(),
                null,
                "1",
                java.util.Map.of("action", "SHIFT_SETTLEMENT"));

        return mapShiftSettlement(settlement);
    }
```

2.3.10.4.Giải thích code

Các API list/get có dữ liệu lịch sử; không chỉ là một phép setStock không có chứng từ.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: shiftSettlementRepository.findByIdempotencyKey(); shiftSettlementRepository.save(); materialRepository.findById(); stockRepository.findByIdWithLock(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/inventory.html; src/main/resources/static/js/admin/admin.js.

2.3.11.Xem lịch sử nhập, sơ chế và biến động

2.3.11.1.Mô tả chức năng

Bảng 2.47. Đặc tả: Xem lịch sử nhập, sơ chế và biến động

Tiêu chí	Nội dung
Mục đích	Tra cứu nguyên nhân thay đổi tồn và các chứng từ vận hành.
Tác nhân	Nhân viên bếp
Điều kiện trước	Có quyền đọc module kho/chứng từ.
Đầu vào	materialId, location; các danh sách chứng từ có from/to/all theo controller.
Kết quả / điều kiện sau	Không sửa số lượng tồn bằng thao tác xem lịch sử.
Trường hợp lỗi	Quyền không phù hợp hoặc kết quả rỗng.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Đọc movements theo bộ lọc.

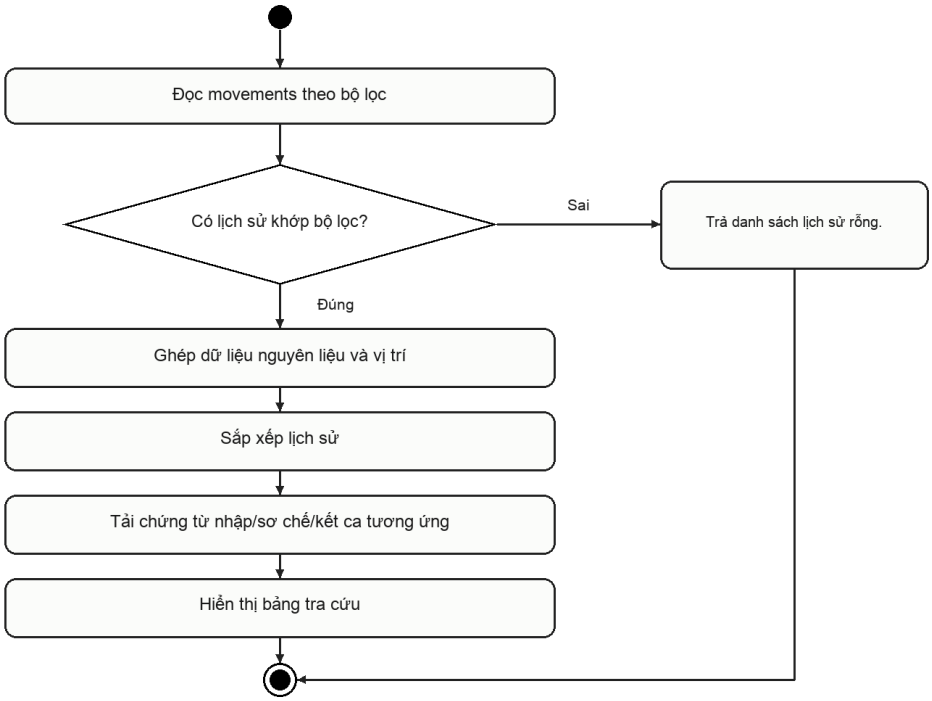
Bước 2. Ghép dữ liệu nguyên liệu và vị trí.

Bước 3. Sắp xếp lịch sử.

Bước 4. Tải chứng từ nhập/sơ chế/kết ca tương ứng.

Bước 5. Hiển thị bảng tra cứu.

2.3.11.2.Sơ đồ hoạt động



Hình 2.47. Hoạt động: Xem lịch sử nhập, sơ chế và biến động

2.3.11.3.Code xử lý chức năng

Đoạn mã 2.47. InventoryService.listMovements — Xem lịch sử nhập, sơ chế và biến động

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.listMovements; dòng 920–942.

```
    public List<InventoryDto.StockMovementResponse> listMovements(Long materialId,
StockLocation location) {
        List<StockMovement> list;
        if (materialId != null && location != null) {
                                                                    list      =
movementRepository.findByMaterialIdAndLocationOrderByCreatedAtDesc(materialId,
location);
        } else {
            list = movementRepository.findTop100ByOrderByCreatedAtDesc();
        }
        return list.stream().map(m -> InventoryDto.StockMovementResponse.builder()
                .id(m.getId())
                .materialId(m.getMaterial().getId())
                .materialName(m.getMaterial().getName())
                .location(m.getLocation())
                .type(m.getType().name())
                .deltaQuantity(m.getDeltaQuantity())
                .beforeQuantity(m.getBeforeQuantity())
                .afterQuantity(m.getAfterQuantity())
                .sourceType(m.getSourceType())
                .sourceId(m.getSourceId())
                .reason(m.getReason())
```

```
                .createdAt(m.getCreatedAt())
                .build()
        ).toList();
    }
```

2.3.11.4.Giải thích code

Giao diện dùng dữ liệu đã lưu, không suy luận mọi thay đổi từ chênh lệch giữa hai lần xem.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: movementRepository.findByMaterialIdAndLocationOrderByCreatedAtDesc(); movementRepository.findTop100ByOrderByCreatedAtDesc(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/documents.html; src/main/resources/static/js/admin/documents.js.

2.4.Quản trị viên

2.4.1.Quản lý tài khoản nhân viên

2.4.1.1.Mô tả chức năng

Bảng 2.48. Đặc tả: Quản lý tài khoản nhân viên

Tiêu chí	Nội dung
Mục đích	Tạo tài khoản nhân viên theo quyền quản trị, đồng thời quản lý hồ sơ và trạng thái.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN và dữ liệu nhân viên hợp lệ; email chưa được sử dụng.
Đầu vào	Thông tin CreateStaffRequest: email, tên, điện thoại, vai trò và mật khẩu theo DTO.
Kết quả / điều kiện sau	Nhân viên có hồ sơ được quản lý tập trung.
Trường hợp lỗi	Trùng email, vai trò không hợp lệ hoặc DTO sai.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra email và vai trò được phép.

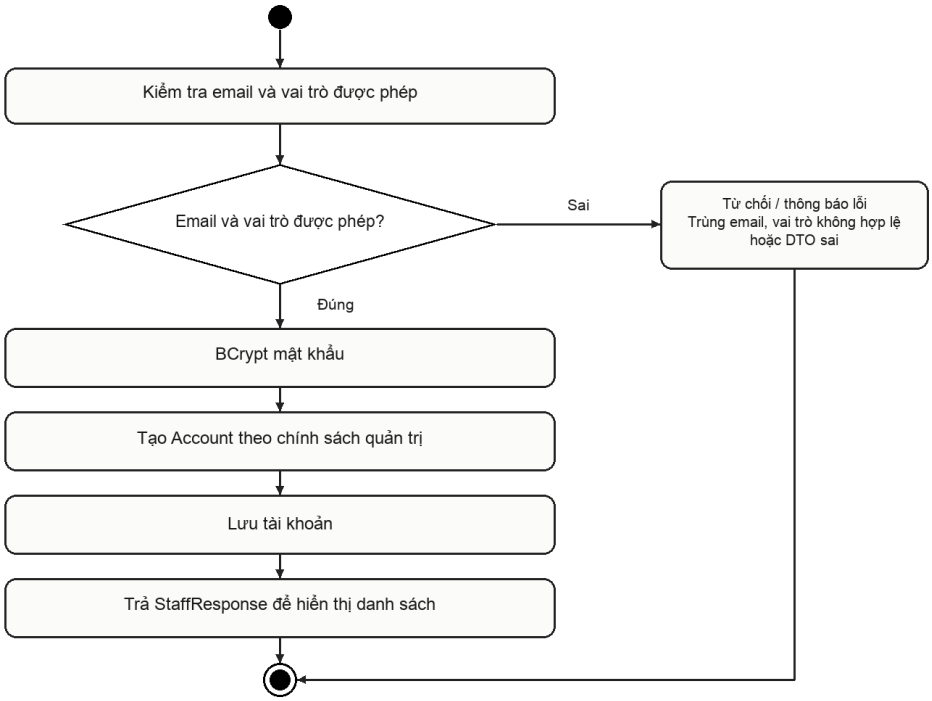
Bước 2. BCrypt mật khẩu.

Bước 3. Tạo Account theo chính sách quản trị.

Bước 4. Lưu tài khoản.

Bước 5. Trả StaffResponse để hiển thị danh sách.

2.4.1.2.Sơ đồ hoạt động



Hình 2.48. Hoạt động: Quản lý tài khoản nhân viên

2.4.1.3.Code xử lý chức năng

Đoạn mã 2.48. AdminService.createStaffAccount — Quản lý tài khoản nhân viên

Nguồn: src/main/java/vn/edu/ute/milktea/service/admin/AdminService.java; AdminService.createStaffAccount; dòng 76–99.

```
     public AdminDto.StaffResponse createStaffAccount(AdminDto.CreateStaffRequest
request) {
                                                                              if
(accountRepository.existsByEmail(request.getEmail().trim().toLowerCase())) {
             throw BusinessException.conflict(ErrorCode.VALIDATION_FAILED, "Email
đã được đăng ký trong hệ thống");
        }

        if (request.getRole() == Role.CUSTOMER) {
            throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED, "Không
thể tạo tài khoản CUSTOMER qua giao diện quản trị nhân viên");
        }

        Account account = Account.builder()
                .email(request.getEmail().trim().toLowerCase())
                .passwordHash(passwordEncoder.encode(request.getPassword()))
                .fullName(request.getFullName())
                .phone(request.getPhone())
                .role(request.getRole())
                .active(true)
                .tokenVersion(1L)
                .createdAt(Instant.now())
                .build();
```

```
        Account saved = accountRepository.save(account);
         realtimeEventPublisher.publishAfterCommit("/topic/admin", "STAFF_CHANGED",
saved.getId().toString(), null, "1", java.util.Map.of("action", "CREATE", "email",
saved.getEmail()));
        return mapStaff(saved);
    }
```

2.4.1.4.Giải thích code

Đây là quy trình quản trị, khác đăng ký CUSTOMER công khai phải nhập OTP. Không trích mật khẩu mẫu trong báo cáo.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.existsByEmail(); passwordEncoder.encode(); accountRepository.save(); realtimeEventPublisher.publishAfterCommit(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/staff.html; src/main/resources/static/js/admin/admin.js.

2.4.2.Sửa, khóa và xóa tài khoản có kiểm soát

2.4.2.1.Mô tả chức năng

Bảng 2.49. Đặc tả: Sửa, khóa và xóa tài khoản có kiểm soát

Tiêu chí	Nội dung
Mục đích	Hạn chế truy cập của tài khoản và bảo vệ tài khoản quản trị đang thao tác.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; không khóa/xóa trái quy định bảo vệ tài khoản.
Đầu vào	accountId, active và currentAdminId từ actor.
Kết quả / điều kiện sau	Filter JWT từ chối tài khoản inactive hoặc phiên bản cũ.
Trường hợp lỗi	Không tồn tại, tự tác động vào tài khoản quản trị được bảo vệ hoặc có ràng buộc nghiệp vụ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm Account.

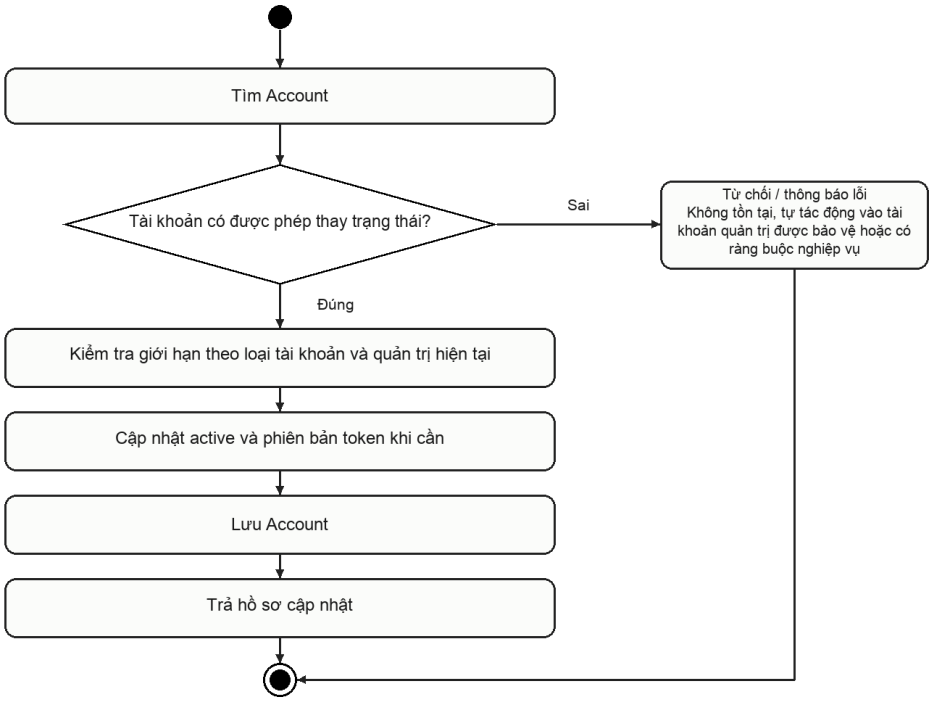
Bước 2. Kiểm tra giới hạn theo loại tài khoản và quản trị hiện tại.

Bước 3. Cập nhật active và phiên bản token khi cần.

Bước 4. Lưu Account.

Bước 5. Trả hồ sơ cập nhật.

2.4.2.2.Sơ đồ hoạt động



Hình 2.49. Hoạt động: Sửa, khóa và xóa tài khoản có kiểm soát

2.4.2.3.Code xử lý chức năng

Đoạn mã 2.49. AdminService.toggleStaffStatus — Sửa, khóa và xóa tài khoản có kiểm soát

Nguồn: src/main/java/vn/edu/ute/milktea/service/admin/AdminService.java; AdminService.toggleStaffStatus; dòng 181–218.

```
    public AdminDto.StaffResponse toggleStaffStatus(Long id, boolean active, Long
currentAdminId) {
        if (!active && id.equals(currentAdminId)) {
            throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED, "Không
thể tự khóa tài khoản của chính mình");
        }

        Account account = accountRepository.findById(id)
                                                        .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED,   "Không   tìm   thấy   tài
khoản"));

        if (!active && account.getRole() == Role.ADMIN) {
            long activeAdminCount = accountRepository.findAll().stream()
                                    .filter(a  ->  a.getRole()  ==  Role.ADMIN  &&
Boolean.TRUE.equals(a.getActive()))
                    .count();
            if (activeAdminCount <= 1) {
                   throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED,
"Không thể khóa tài khoản quản trị viên hoạt động cuối cùng");
            }
        }

        String before = "active=" + account.getActive();
```

```
        account.setActive(active);
        if (!active) {
            account.setTokenVersion(account.getTokenVersion() + 1); // Thu hồi JWT
phiên cũ lập tức
        }
        Account saved = accountRepository.save(account);
        String after = "active=" + saved.getActive();


auditRepository.save(vn.edu.ute.milktea.entity.audit.BusinessAudit.builder()
                .accountId(currentAdminId)
                                    .action(active  ?  "STAFF_ACCOUNT_UNLOCKED"  :
"STAFF_ACCOUNT_LOCKED")
                .beforeState(before)
                .afterState(after)
                 .reason((active ? "Mở khóa" : "Khóa") + " tài khoản nhân viên #" +
id)
                .createdAt(Instant.now())
                .build());

         realtimeEventPublisher.publishAfterCommit("/topic/admin", "STAFF_CHANGED",
saved.getId().toString(),   null,   "1",   java.util.Map.of("action",   active   ?
"UNLOCK" : "LOCK", "email", saved.getEmail()));

        return mapStaff(saved);
    }
```

2.4.2.4.Giải thích code

updateStaffAccount/deleteStaffAccount có kiểm tra riêng; không xem các thao tác này là xóa vật lý mọi dữ liệu liên quan.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findById(); accountRepository.findAll(); accountRepository.save(); auditRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/staff.html; src/main/resources/static/js/admin/admin.js.

2.4.3.Quản lý danh mục menu

2.4.3.1.Mô tả chức năng

Bảng 2.50. Đặc tả: Quản lý danh mục menu

Tiêu chí	Nội dung
Mục đích	Tổ chức nhóm món và quy tắc cho phép topping.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; tên danh mục đúng ràng buộc.
Đầu vào	Tên, active và allowTopping theo CategoryRequest.
Kết quả / điều kiện sau	Menu đọc từ danh mục active; việc đổi allowTopping tác động kiểm tra giỏ/đơn.
Trường hợp lỗi	Tên trùng, thiếu dữ liệu hoặc ràng buộc khi xóa.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra dữ liệu danh mục.

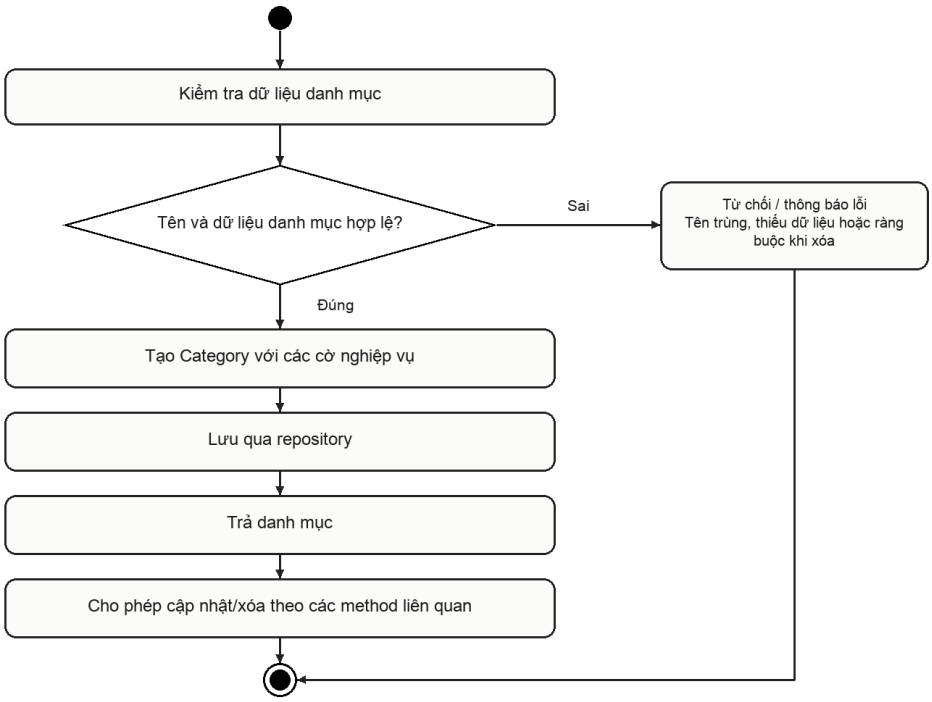
Bước 2. Tạo Category với các cờ nghiệp vụ.

Bước 3. Lưu qua repository.

Bước 4. Trả danh mục.

Bước 5. Cho phép cập nhật/xóa theo các method liên quan.

2.4.3.2.Sơ đồ hoạt động



Hình 2.50. Hoạt động: Quản lý danh mục menu

2.4.3.3.Code xử lý chức năng

Đoạn mã 2.50. AdminService.createCategory — Quản lý danh mục menu

Nguồn: src/main/java/vn/edu/ute/milktea/service/admin/AdminService.java; AdminService.createCategory; dòng 259–271.

```
        public AdminDto.CategoryResponse createCategory(AdminDto.CategoryRequest
request) {
        Category c = Category.builder()
                .name(request.getName().trim())
                .active(request.getActive() != null ? request.getActive() : true)
                                .allowTopping(request.getAllowTopping() != null ?
request.getAllowTopping() : true)
                .build();
        c = categoryRepository.save(c);
                        realtimeEventPublisher.publishAfterCommit("/topic/admin",
"CATALOG_CHANGED", c.getId().toString(), null, "1",
                    java.util.Map.of("type", "CATEGORY", "action", "CREATE", "id",
c.getId(), "name", c.getName()));
                        realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"CATALOG_CHANGED", c.getId().toString(), null, "1",
```

```
                    java.util.Map.of("type", "CATEGORY", "action", "CREATE", "id",
c.getId(), "name", c.getName()));
                                                                                return
AdminDto.CategoryResponse.builder().id(c.getId()).name(c.getName()).active(c.getAc
tive()).allowTopping(c.getAllowTopping()).build();
    }
```

2.4.3.4.Giải thích code

Không thêm module thương hiệu không có trong mô hình vì source UTE-Tea không có entity Brand tương ứng.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: categoryRepository.save(); realtimeEventPublisher.publishAfterCommit(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/catalog.html; src/main/resources/static/js/admin/admin.js.

2.4.4.Quản lý món, size và giá bán

2.4.4.1.Mô tả chức năng

Bảng 2.51. Đặc tả: Quản lý món, size và giá bán

Tiêu chí	Nội dung
Mục đích	Cập nhật danh mục, giá, mô tả và trạng thái bán của từng biến thể món.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; món và danh mục tồn tại; dữ liệu không trùng khóa menu.
Đầu vào	productId và ProductRequest gồm danh mục, tên, size, giá, mô tả, active.
Kết quả / điều kiện sau	Giá mới được dùng cho đơn mới; snapshot của đơn cũ không đổi.
Trường hợp lỗi	Không tồn tại, trùng tên/size hoặc dữ liệu vi phạm DTO.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm Product và Category.

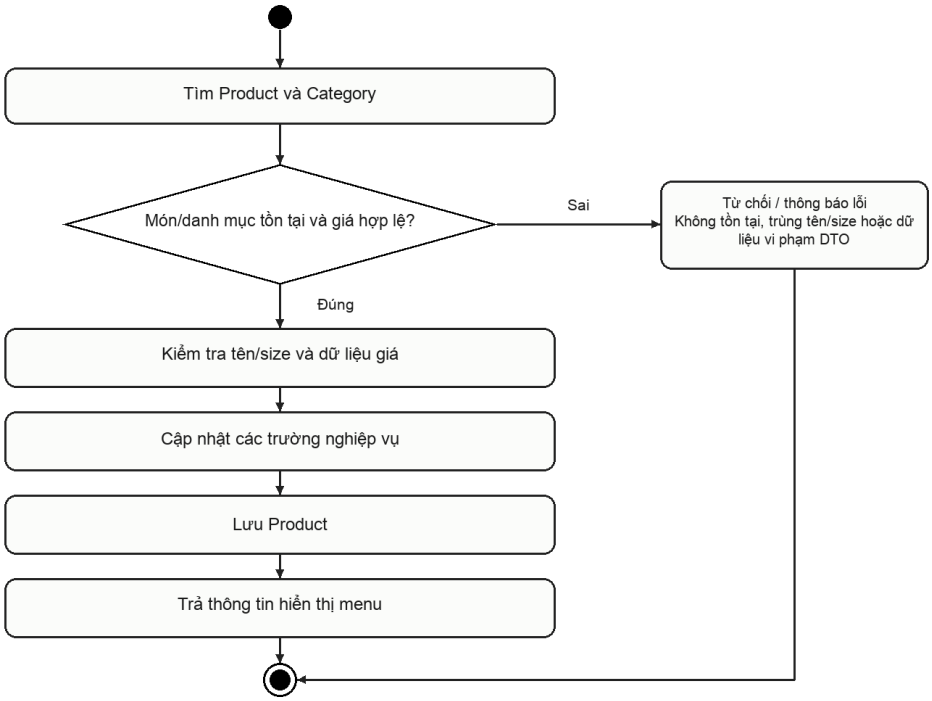
Bước 2. Kiểm tra tên/size và dữ liệu giá.

Bước 3. Cập nhật các trường nghiệp vụ.

Bước 4. Lưu Product.

Bước 5. Trả thông tin hiển thị menu.

2.4.4.2.Sơ đồ hoạt động



Hình 2.51. Hoạt động: Quản lý món, size và giá bán

2.4.4.3.Code xử lý chức năng

Đoạn mã 2.51. AdminService.updateProduct — Quản lý món, size và giá bán

Nguồn: src/main/java/vn/edu/ute/milktea/service/admin/AdminService.java; AdminService.updateProduct; dòng 381–403.

```
    public AdminDto.ProductResponse updateProduct(Long id, AdminDto.ProductRequest
request) {
        Product p = productRepository.findById(id)
                                                          .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.PRODUCT_UNAVAILABLE,  "Không  tìm  thấy  món
uống"));

        if (request.getCategoryId() != null) {
            Category cat = categoryRepository.findById(request.getCategoryId())
                                                            .orElseThrow(()   ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED,  "Không  tìm  thấy  danh
mục"));
            p.setCategory(cat);
        }
        if (request.getName() != null) p.setName(request.getName().trim());
                                        if     (request.getSize()     !=     null)
p.setSize(request.getSize().trim().toUpperCase());
                                    if     (request.getDescription()     !=     null)
p.setDescription(request.getDescription());
        if (request.getPrice() != null) p.setPrice(request.getPrice());
        if (request.getImageUrl() != null) p.setImageUrl(request.getImageUrl());
        if (request.getActive() != null) p.setActive(request.getActive());

        Product saved = productRepository.save(p);
```

```
                        realtimeEventPublisher.publishAfterCommit("/topic/admin",
"CATALOG_CHANGED", saved.getId().toString(), null, "1",
                    java.util.Map.of("type", "PRODUCT", "action", "UPDATE", "id",
saved.getId(), "name", saved.getName()));
                       realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"CATALOG_CHANGED", saved.getId().toString(), null, "1",
                    java.util.Map.of("type", "PRODUCT", "action", "UPDATE", "id",
saved.getId(), "name", saved.getName()));
        return mapProduct(saved);
    }
```

2.4.4.4.Giải thích code

Giá theo cơ sở nằm ở Product.shop; không mô tả giá frontend là nguồn quyết định hóa đơn.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: productRepository.findById(); categoryRepository.findById(); productRepository.save(); realtimeEventPublisher.publishAfterCommit(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/catalog.html; src/main/resources/static/js/admin/admin.js.

2.4.5.Quản lý ảnh sản phẩm trên Cloudinary

2.4.5.1.Mô tả chức năng

Bảng 2.52. Đặc tả: Quản lý ảnh sản phẩm trên Cloudinary

Tiêu chí	Nội dung
Mục đích	Thay ảnh món và kiểm soát file bên ngoài transaction DB.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; món tồn tại; Cloudinary hợp lệ; nội dung file được kiểm tra.
Đầu vào	productId, MultipartFile và currentAdminId.
Kết quả / điều kiện sau	Ảnh mới gắn với món khi DB commit; không bỏ mặc ảnh mồ côi theo nhánh xử lý đã triển khai.
Trường hợp lỗi	Sai loại/dung lượng, món không tồn tại hoặc lỗi upload.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra tệp thực và cấu hình dịch vụ.

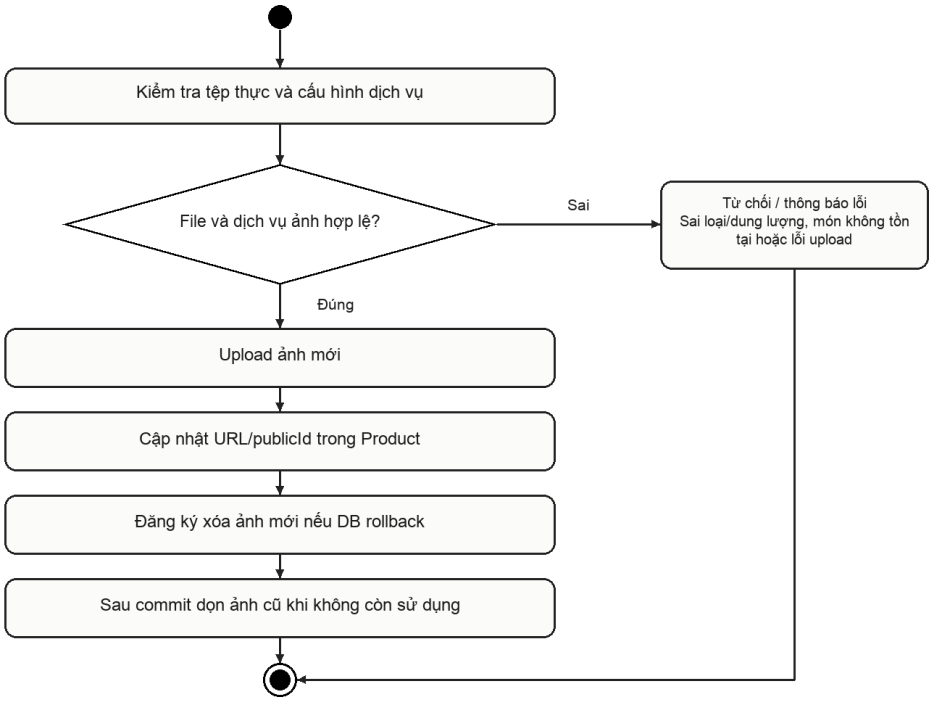
Bước 2. Upload ảnh mới.

Bước 3. Cập nhật URL/publicId trong Product.

Bước 4. Đăng ký xóa ảnh mới nếu DB rollback.

Bước 5. Sau commit dọn ảnh cũ khi không còn sử dụng.

2.4.5.2.Sơ đồ hoạt động



Hình 2.52. Hoạt động: Quản lý ảnh sản phẩm trên Cloudinary

2.4.5.3.Code xử lý chức năng

Đoạn mã 2.52. ProductImageService.uploadProductImage — Quản lý ảnh sản phẩm trên Cloudinary

Nguồn: src/main/java/vn/edu/ute/milktea/service/media/ProductImageService.java; ProductImageService.uploadProductImage; dòng 110–178.

```
          public   AdminDto.ProductImageView   uploadProductImage(Long   productId,
MultipartFile file) {
        validateImageFile(file);
        ensureCloudinaryReady();

        Product product = productRepository.findById(productId)
                .orElseThrow(
                                                                        ()   ->
BusinessException.notFound(ErrorCode.PRODUCT_UNAVAILABLE,   "Không   tìm   thấy   món
uống"));

        String oldPublicId = product.getImagePublicId();
        String secureUrl;
        String publicId;

           String customPublicId = "milktea/products/prod_" + productId + "_" +
System.currentTimeMillis() + "_"
```

[Đã lược bỏ các dòng 123–132 của phương thức gốc.]

```
            throw BusinessException.badRequest(ErrorCode.IMAGE_UPLOAD_FAILED,
                    "Không thể tải ảnh lên Cloudinary: " + e.getMessage());
        }
```

```
         // Lưu DB với cơ chế rollback ảnh mới nếu DB save thất bại (tránh ảnh mồ
côi)
        try {
            product.setImageUrl(secureUrl);
            product.setImagePublicId(publicId);
            productRepository.save(product);
        } catch (Exception dbEx) {
              log.error("Lưu DB thất bại sau khi upload Cloudinary cho món id={}.
Dọn dẹp ảnh mồ côi publicId={}",
```

[Đã lược bỏ các dòng 144–166 của phương thức gốc.]

```
          // Phát sự kiện WebSocket CATALOG_CHANGED after-commit để client tự làm
mới ảnh
        // ngay lập tức
                          realtimeEventPublisher.publishAfterCommit("/topic/admin",
"CATALOG_CHANGED", productId.toString(), null, "1",
                         Map.of("action", "IMAGE_UPDATED", "productId", productId,
"imageUrl", secureUrl));
                         realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"CATALOG_CHANGED", productId.toString(), null, "1",
                         Map.of("action", "IMAGE_UPDATED", "productId", productId,
"imageUrl", secureUrl));

        return AdminDto.ProductImageView.builder()
                .imageUrl(secureUrl)
                .imagePublicId(publicId)
                .build();
    }
```

2.4.5.4.Giải thích code

Phiên báo cáo không truyền API key thật hay tải ảnh lên dịch vụ bên ngoài.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: productRepository.findById(); productRepository.save(); productRepository.countByImagePublicId(); realtimeEventPublisher.publishAfterCommit(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/catalog.html; src/main/resources/static/js/admin/admin.js.

2.4.6.Thiết lập công thức định mức món

2.4.6.1.Mô tả chức năng

Bảng 2.53. Đặc tả: Thiết lập công thức định mức món

Tiêu chí	Nội dung
Mục đích	Khai báo lượng nguyên liệu làm cơ sở snapshot và tiêu hao khi hoàn tất đơn.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; món và từng nguyên liệu hợp lệ; định lượng dương.
Đầu vào	productId và danh sách materialId/quantity.
Kết quả / điều kiện sau	Đơn mới lấy snapshot từ công thức; đơn đã tạo giữ lượng snapshot của thời điểm đặt.
Trường hợp lỗi	Nguyên liệu không tồn tại, trùng/không hợp lệ hoặc số lượng không dương.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm sản phẩm.

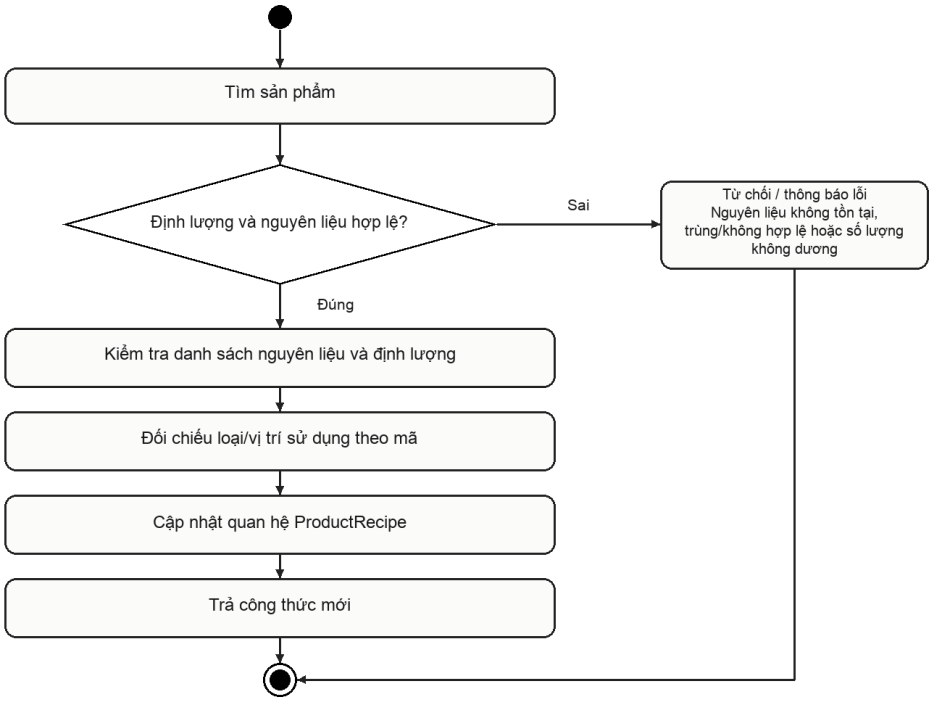
Bước 2. Kiểm tra danh sách nguyên liệu và định lượng.

Bước 3. Đối chiếu loại/vị trí sử dụng theo mã.

Bước 4. Cập nhật quan hệ ProductRecipe.

Bước 5. Trả công thức mới.

2.4.6.2.Sơ đồ hoạt động



Hình 2.53. Hoạt động: Thiết lập công thức định mức món

2.4.6.3.Code xử lý chức năng

Đoạn mã 2.53. AdminService.updateProductRecipe — Thiết lập công thức định mức món

Nguồn: src/main/java/vn/edu/ute/milktea/service/admin/AdminService.java; AdminService.updateProductRecipe; dòng 439–468.

```
       public AdminDto.ProductRecipeResponse updateProductRecipe(Long productId,
AdminDto.UpdateProductRecipeRequest request) {
        Product p = productRepository.findById(productId)
                                                        .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.PRODUCT_UNAVAILABLE,  "Không  tìm  thấy  món
uống"));

        Set<Long> materialIds = new HashSet<>();
        for (var item : request.getItems()) {
              if (item.getQuantity() == null || item.getQuantity().signum() <= 0
|| !materialIds.add(item.getMaterialId())) {
                  throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED,
"Định lượng phải lớn hơn 0 và mỗi nguyên liệu chỉ xuất hiện một lần");
            }
        }
        // Xóa công thức cũ
        productRecipeRepository.deleteByIdProductId(productId);
        productRecipeRepository.flush();

        List<ProductRecipe> newRecipes = new ArrayList<>();
        for (var item : request.getItems()) {
            Material mat = materialRepository.findById(item.getMaterialId())
```

```
                                                                .orElseThrow(() ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED, "Nguyên liệu không tồn
tại: ID " + item.getMaterialId()));

            newRecipes.add(ProductRecipe.builder()
                    .id(new ProductRecipeId(productId, mat.getId()))
                    .product(p)
                    .material(mat)
                    .quantity(item.getQuantity())
                    .build());
        }
        productRecipeRepository.saveAll(newRecipes);

        return getProductRecipe(productId);
    }
```

2.4.6.4.Giải thích code

Không tự điền công thức giả cho món chưa được cấu hình chỉ để chứng minh checkout thành công.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: productRepository.findById(); productRecipeRepository.deleteByIdProductId(); productRecipeRepository.flush(); materialRepository.findById(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/catalog.html; src/main/resources/static/js/admin/admin.js.

2.4.7.Quản lý bàn và mã QR

2.4.7.1.Mô tả chức năng

Bảng 2.54. Đặc tả: Quản lý bàn và mã QR

Tiêu chí	Nội dung
Mục đích	Tạo/điều chỉnh bàn phục vụ và mã QR khó đoán.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; tên bàn không trùng ràng buộc.
Đầu vào	Tên bàn và trạng thái hoạt động theo TableRequest.
Kết quả / điều kiện sau	Bàn có mã QR phục vụ đọc ngữ cảnh; không xem QR là mã đăng nhập quản trị.
Trường hợp lỗi	Tên trùng, ID không tồn tại hoặc xóa bàn còn quan hệ bị chặn.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra tên bàn.

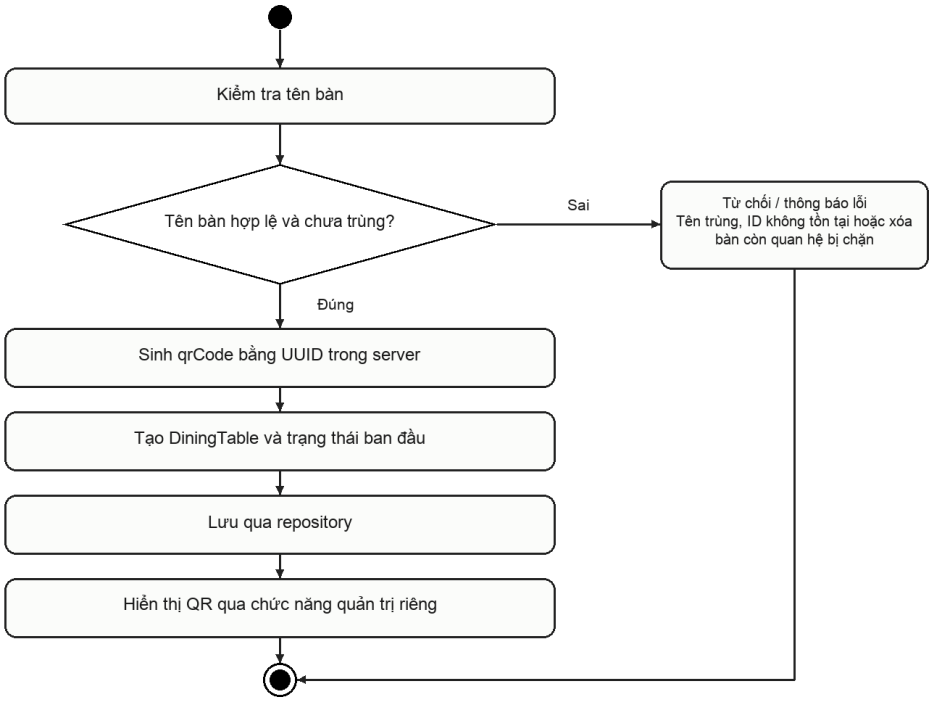
Bước 2. Sinh qrCode bằng UUID trong server.

Bước 3. Tạo DiningTable và trạng thái ban đầu.

Bước 4. Lưu qua repository.

Bước 5. Hiển thị QR qua chức năng quản trị riêng.

2.4.7.2.Sơ đồ hoạt động



Hình 2.54. Hoạt động: Quản lý bàn và mã QR

2.4.7.3.Code xử lý chức năng

Đoạn mã 2.54. AdminService.createTable — Quản lý bàn và mã QR

Nguồn: src/main/java/vn/edu/ute/milktea/service/admin/AdminService.java; AdminService.createTable; dòng 500–510.

```
    public AdminDto.TableResponse createTable(AdminDto.TableRequest request) {
          String qrCode = "TABLE_QR_" + UUID.randomUUID().toString().replace("-",
"").substring(0, 16);
        DiningTable table = DiningTable.builder()
                .name(request.getName().trim())
                .qrCode(qrCode)
                .status(TableStatus.TRONG)
                .active(request.getActive() != null ? request.getActive() : true)
                .version(0L)
                .build();
        return mapTable(tableRepository.save(table));
    }
```

2.4.7.4.Giải thích code

Ảnh báo cáo không công khai token QR thực tế, chỉ chụp màn hình danh sách không mở mã bí mật.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: tableRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/tables.html; src/main/resources/static/js/admin/table-tools.js.

2.4.8.Quản lý nguyên liệu và đơn vị

2.4.8.1.Mô tả chức năng

Bảng 2.55. Đặc tả: Quản lý nguyên liệu và đơn vị

Tiêu chí	Nội dung
Mục đích	Khai báo nguyên liệu THO/SOCHE và các thuộc tính sử dụng trong quản lý kho.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; nguyên liệu chưa trùng và các ràng buộc hợp lệ.
Đầu vào	Tên, type, unit và dữ liệu MaterialRequest.
Kết quả / điều kiện sau	Nguyên liệu có thể dùng trong định mức và chứng từ kho.
Trường hợp lỗi	Tên trùng, định dạng không hợp lệ hoặc không tồn tại khi sửa.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra thông tin nguyên liệu.

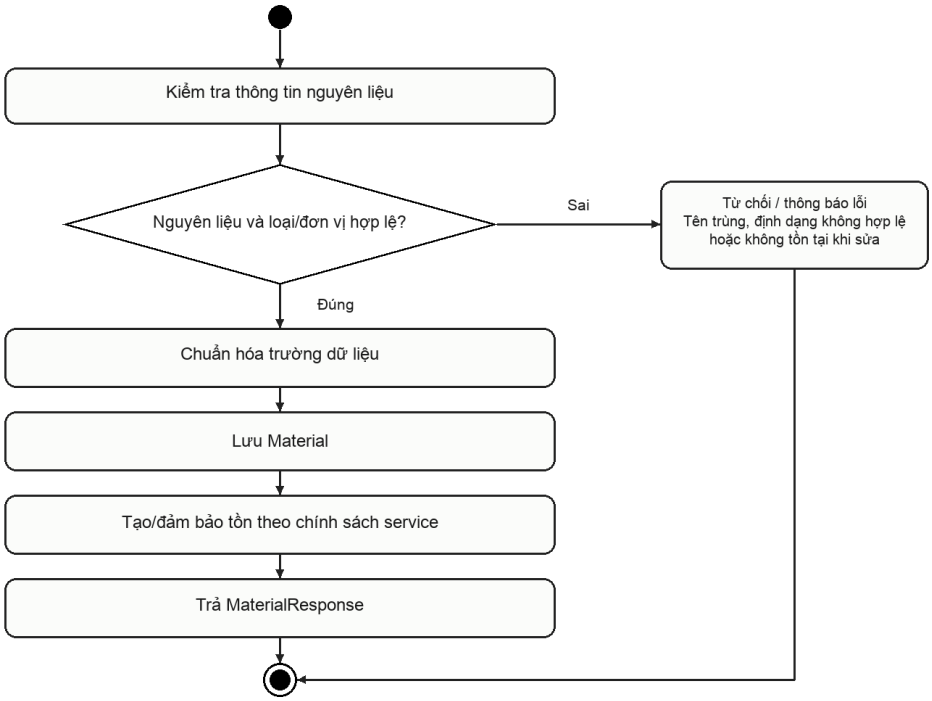
Bước 2. Chuẩn hóa trường dữ liệu.

Bước 3. Lưu Material.

Bước 4. Tạo/đảm bảo tồn theo chính sách service.

Bước 5. Trả MaterialResponse.

2.4.8.2.Sơ đồ hoạt động



Hình 2.55. Hoạt động: Quản lý nguyên liệu và đơn vị

2.4.8.3.Code xử lý chức năng

Đoạn mã 2.55. AdminService.createMaterial — Quản lý nguyên liệu và đơn vị

Nguồn: src/main/java/vn/edu/ute/milktea/service/admin/AdminService.java; AdminService.createMaterial; dòng 596–613.

```
        public AdminDto.MaterialResponse createMaterial(AdminDto.MaterialRequest
request) {
        Material m = Material.builder()
                .name(request.getName().trim())
                .type(request.getType())
                .unit(request.getUnit().trim())
                .active(request.getActive() != null ? request.getActive() : true)
                .build();
        m = materialRepository.save(m);
                        realtimeEventPublisher.publishAfterCommit("/topic/stock",
"MATERIAL_CHANGED", m.getId().toString(), null, "1", java.util.Map.of("action",
"CREATE", "name", m.getName()));
                        realtimeEventPublisher.publishAfterCommit("/topic/admin",
"MATERIAL_CHANGED", m.getId().toString(), null, "1", java.util.Map.of("action",
"CREATE", "name", m.getName()));
        return AdminDto.MaterialResponse.builder()
                .id(m.getId())
                .name(m.getName())
                .type(m.getType())
                .unit(m.getUnit())
                .active(m.getActive())
                .build();
    }
```

2.4.8.4.Giải thích code

Không tự tạo bảng biến thể ngoài mô hình hoặc các thuộc tính không tồn tại trong Entity.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: materialRepository.save(); realtimeEventPublisher.publishAfterCommit(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/inventory.html; src/main/resources/static/js/admin/admin.js.

2.4.9.Quản lý định mức sơ chế

2.4.9.1.Mô tả chức năng

Bảng 2.56. Đặc tả: Quản lý định mức sơ chế

Tiêu chí	Nội dung
Mục đích	Định nghĩa cấu trúc đầu vào/đầu ra cho mẻ sơ chế.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; thành phẩm và nguyên liệu đầu vào đúng loại; lượng dương.
Đầu vào	outputMaterialId, standardOutputQuantity, inputs và active.
Kết quả / điều kiện sau	Bếp sử dụng định mức được lưu thay vì công thức tự bịa trong giao diện.
Trường hợp lỗi	Thiếu nguyên liệu, loại không phù hợp, lặp đầu vào hoặc sản lượng không dương.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra thành phẩm sơ chế.

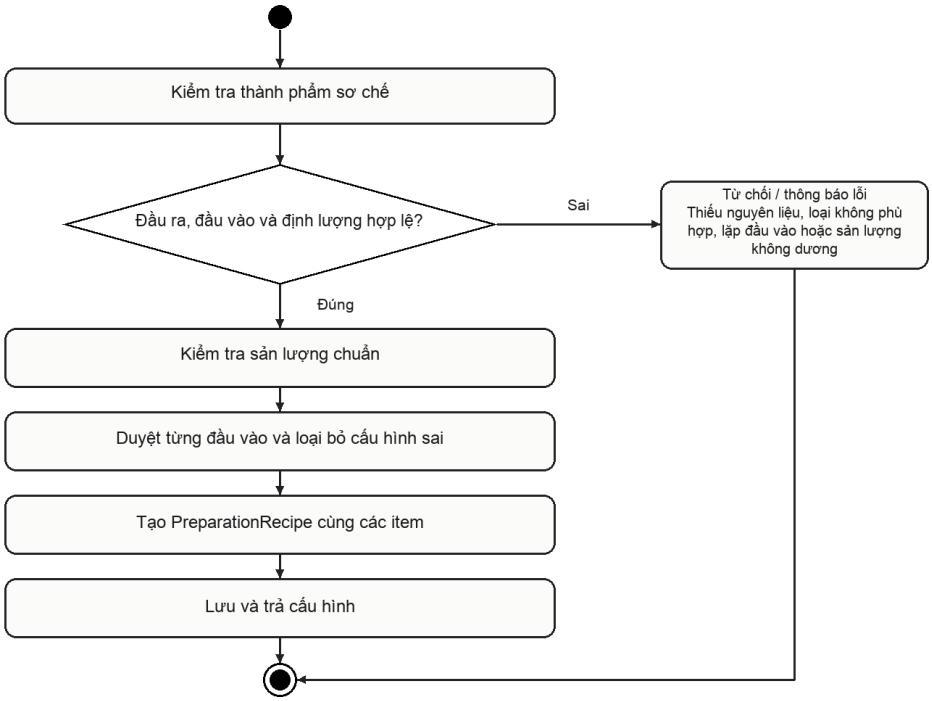
Bước 2. Kiểm tra sản lượng chuẩn.

Bước 3. Duyệt từng đầu vào và loại bỏ cấu hình sai.

Bước 4. Tạo PreparationRecipe cùng các item.

Bước 5. Lưu và trả cấu hình.

2.4.9.2.Sơ đồ hoạt động



Hình 2.56. Hoạt động: Quản lý định mức sơ chế

2.4.9.3.Code xử lý chức năng

Đoạn mã 2.56. AdminService.createPreparationRecipe — Quản lý định mức sơ chế

Nguồn: src/main/java/vn/edu/ute/milktea/service/admin/AdminService.java; AdminService.createPreparationRecipe; dòng 663–711.

```
                                    public          AdminDto.PreparationRecipeResponse
createPreparationRecipe(AdminDto.CreatePreparationRecipeRequest request) {
                                                              Material       outMat       =
materialRepository.findById(request.getOutputMaterialId())
                                                                                .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED, "Không tìm thấy bán thành
phẩm đầu ra"));

                        boolean   isActive   =   request.getActive()   ==   null   ||
Boolean.TRUE.equals(request.getActive());
                                                                  if       (isActive       &&
preparationRecipeRepository.existsByOutputMaterialIdAndActiveTrue(outMat.getId()))
{
            throw BusinessException.conflict(ErrorCode.VALIDATION_FAILED,
                    "Thành phẩm '" + outMat.getName() + "' đã có định mức đang áp
dụng. Mỗi thành phẩm chỉ được có một định mức hoạt động.");
        }

                    String   recipeName   =   (request.getName()   !=   null   &&   !
request.getName().isBlank())
                ? request.getName().trim()
                : "Định mức " + outMat.getName();
```

[Đã lược bỏ các dòng 676–687 của phương thức gốc.]

```
        var inputItems = request.getInputItems();
```

```
        if (inputItems != null && !inputItems.isEmpty()) {
            List<PreparationRecipeItem> items = new ArrayList<>();
            for (var in : inputItems) {
                                                                Material    inMat    =
materialRepository.findById(in.getInputMaterialId())
                                                                  .orElseThrow(()   ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED,  "Không  tìm  thấy  nguyên
liệu đầu vào: ID " + in.getInputMaterialId()));

                items.add(PreparationRecipeItem.builder()
                                .id(new  PreparationRecipeItemId(recipe.getId(),
inMat.getId()))
                        .recipe(recipe)
                        .inputMaterial(inMat)
```

[Đã lược bỏ các dòng 699–699 của phương thức gốc.]

```
                        .build());
            }
            preparationRecipeItemRepository.saveAll(items);
        } else {
              throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED, "Định
mức phải có ít nhất một nguyên liệu đầu vào");
        }

                                    AdminDto.PreparationRecipeResponse    resp    =
getPreparationRecipe(recipe.getId());
                         realtimeEventPublisher.publishAfterCommit("/topic/stock",
"RECIPE_CHANGED", recipe.getId().toString(), null, "1", java.util.Map.of("action",
"CREATE", "outputMaterialId", outMat.getId()));
                         realtimeEventPublisher.publishAfterCommit("/topic/admin",
"RECIPE_CHANGED", recipe.getId().toString(), null, "1", java.util.Map.of("action",
"CREATE", "outputMaterialId", outMat.getId()));
        return resp;
    }
```

2.4.9.4.Giải thích code

Sửa/xóa được kiểm tra bằng các method tương ứng; giữ dấu vết chứng từ mẻ đã tạo.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: materialRepository.findById(); preparationRecipeRepository.existsByOutputMaterialIdAndActiveTrue(); preparationRecipeRepository.save(); preparationRecipeItemRepository.saveAll(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/inventory.html; src/main/resources/static/js/admin/admin.js.

2.4.10.Điều chỉnh kho tổng có lý do

2.4.10.1.Mô tả chức năng

Bảng 2.57. Đặc tả: Điều chỉnh kho tổng có lý do

Tiêu chí	Nội dung
Mục đích	Hiệu chỉnh tồn hoặc giá vốn của nguyên liệu thô theo quyền quản trị.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; tồn KHO của nguyên liệu THO tồn tại.
Đầu vào	materialId, tồn thực tế/giá nhập, threshold và lý do điều chỉnh.
Kết quả / điều kiện sau	Điều chỉnh có thể truy vết, không thay đổi số liệu ngầm.
Trường hợp lỗi	Nguyên liệu không tồn tại, loại sai, thiếu lý do hoặc giá trị âm.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra loại THO và vị trí KHO.

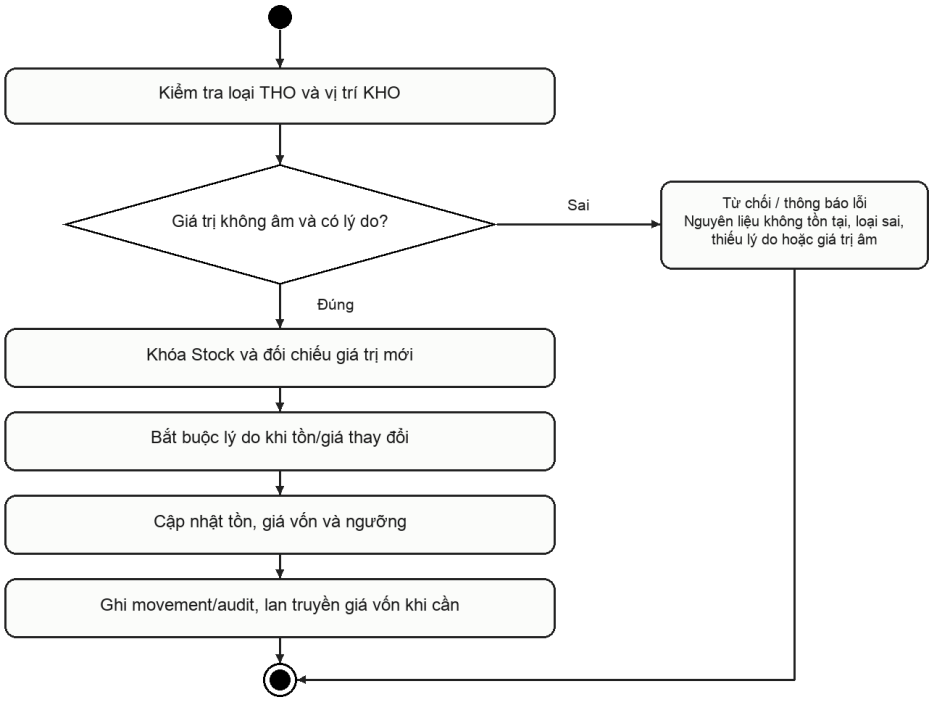
Bước 2. Khóa Stock và đối chiếu giá trị mới.

Bước 3. Bắt buộc lý do khi tồn/giá thay đổi.

Bước 4. Cập nhật tồn, giá vốn và ngưỡng.

Bước 5. Ghi movement/audit, lan truyền giá vốn khi cần.

2.4.10.2.Sơ đồ hoạt động



Hình 2.57. Hoạt động: Điều chỉnh kho tổng có lý do

2.4.10.3.Code xử lý chức năng

Đoạn mã 2.57. InventoryService.adjustRawMaterialStock — Điều chỉnh kho tổng có lý do

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.adjustRawMaterialStock; dòng 813–901.

```
      public void adjustRawMaterialStock(InventoryDto.AdjustStockRequest request,
Account adminAccount) {
        Stock khoStock = stockRepository.findByIdWithLock(request.getMaterialId(),
StockLocation.KHO)
                                                          .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED,  "Không  tìm  thấy  nguyên
liệu tại Kho tổng"));

        Material material = khoStock.getMaterial();
        if (material.getType() != MaterialType.THO) {
              throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED, "Chỉ
được điều chỉnh nguyên liệu thô tại Kho tổng");
        }

        if (request.getName() != null && !request.getName().isBlank()) {
            String newName = request.getName().trim();
            if (!newName.equalsIgnoreCase(material.getName())) {
                material.setName(newName);
```

[Đã lược bỏ các dòng 826–831 của phương thức gốc.]

```
        if (request.getThreshold() != null) {
            if (request.getThreshold().signum() < 0) {
                  throw BusinessException.badRequest(ErrorCode.VALIDATION_FAILED,
"Ngưỡng tối thiểu không được âm");
            }
```

```
            khoStock.setThreshold(request.getThreshold());
        }

                    BigDecimal oldQty = khoStock.getQuantity() != null ?
khoStock.getQuantity() : BigDecimal.ZERO;
        BigDecimal newQty = request.getQuantity();
        boolean qtyChanged = newQty != null && newQty.compareTo(oldQty) != 0;
```

[Đã lược bỏ các dòng 843–889 của phương thức gốc.]

```
                .beforeState("qty=" + oldQty + ",price=" + oldPrice)
                       .afterState("qty=" + khoStock.getQuantity() + ",price=" +
khoStock.getCostPrice())
                .createdAt(Instant.now())
                .build());

        if (priceChanged) {
            propagateRawMaterialCostChange(material.getId(), newPrice);
        }

         realtimeEventPublisher.publishAfterCommit("/topic/stock", "STOCK_CHANGED",
material.getId().toString(), null, "1", java.util.Map.of("action", "ADJUST"));
         realtimeEventPublisher.publishAfterCommit("/topic/admin", "STOCK_CHANGED",
material.getId().toString(), null, "1", java.util.Map.of("action", "ADJUST"));
    }
```

2.4.10.4.Giải thích code

Chức năng không đồng nghĩa chuyển tự do tồn của mọi chi nhánh; schema kho hiện tại theo nguyên liệu/vị trí.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: stockRepository.findByIdWithLock(); materialRepository.save(); realtimeEventPublisher.publishAfterCommit(); movementRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/inventory.html; src/main/resources/static/js/admin/admin.js.

2.4.11.Thiết lập ngưỡng cảnh báo tồn

2.4.11.1.Mô tả chức năng

Bảng 2.58. Đặc tả: Thiết lập ngưỡng cảnh báo tồn

Tiêu chí	Nội dung
Mục đích	Đặt mức tồn tối thiểu cho cảnh báo vận hành.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN theo PreAuthorize.
Đầu vào	materialId, location và threshold.
Kết quả / điều kiện sau	Thay ngưỡng không cộng/trừ số lượng đang có.
Trường hợp lỗi	Không tìm thấy dòng tồn hoặc threshold không hợp lệ theo DTO.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Tìm dòng tồn theo khóa nguyên liệu/vị trí.

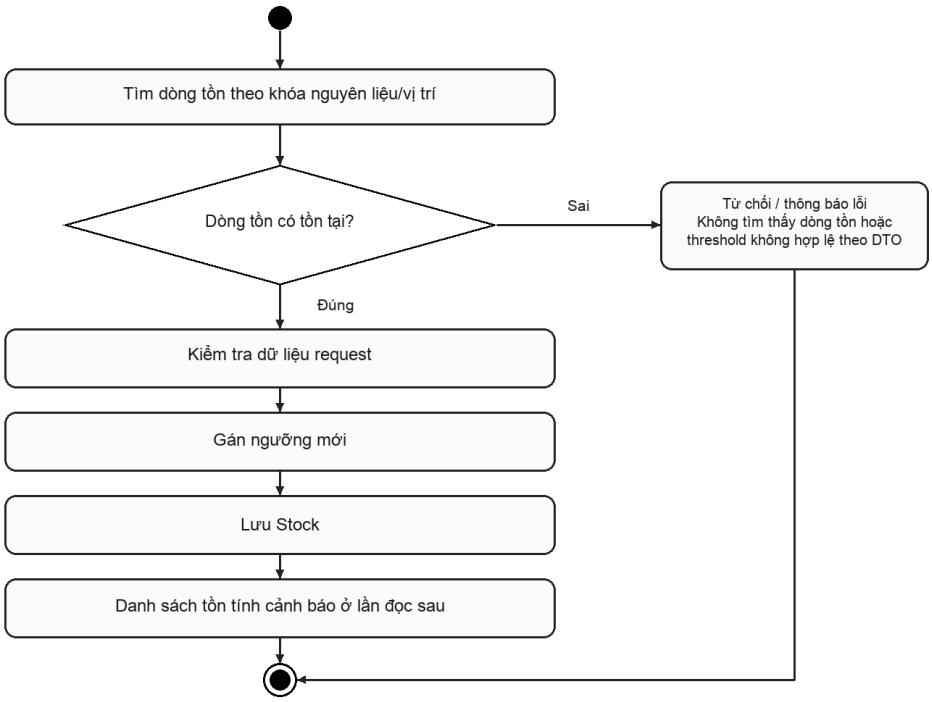
Bước 2. Kiểm tra dữ liệu request.

Bước 3. Gán ngưỡng mới.

Bước 4. Lưu Stock.

Bước 5. Danh sách tồn tính cảnh báo ở lần đọc sau.

2.4.11.2.Sơ đồ hoạt động



Hình 2.58. Hoạt động: Thiết lập ngưỡng cảnh báo tồn

2.4.11.3.Code xử lý chức năng

Đoạn mã 2.58. InventoryService.configureThreshold — Thiết lập ngưỡng cảnh báo tồn

Nguồn: src/main/java/vn/edu/ute/milktea/service/inventory/InventoryService.java; InventoryService.configureThreshold; dòng 487–492.

```
    public void configureThreshold(InventoryDto.UpdateThresholdRequest request) {
          Stock stock = stockRepository.findByIdWithLock(request.getMaterialId(),
request.getLocation())
                                                          .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED,  "Không  tìm  thấy  nguyên
liệu tại vị trí " + request.getLocation()));
        stock.setThreshold(request.getThreshold());
        stockRepository.save(stock);
    }
```

2.4.11.4.Giải thích code

Thông báo tồn thấp là tín hiệu vận hành, không tự tạo phiếu nhập hoặc mẻ sơ chế.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: stockRepository.findByIdWithLock(); stockRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/inventory.html; src/main/resources/static/js/admin/admin.js.

2.4.12.Quản lý đơn và trạng thái thanh toán

2.4.12.1.Mô tả chức năng

Bảng 2.59. Đặc tả: Quản lý đơn và trạng thái thanh toán

Tiêu chí	Nội dung
Mục đích	Tổng hợp các nhóm công việc cần xử lý cho quầy/quản trị.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN hoặc CASHIER theo endpoint; dữ liệu đọc được theo scope service.
Đầu vào	Actor và các bộ lọc danh sách liên quan.
Kết quả / điều kiện sau	Các ô tổng hợp hỗ trợ xử lý, chưa tự quyết định đơn hay tiền.
Trường hợp lỗi	Không có quyền hoặc dữ liệu không khớp; không suy ra PAID chỉ từ trạng thái hoàn thành.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Lấy đơn và lọc phạm vi.

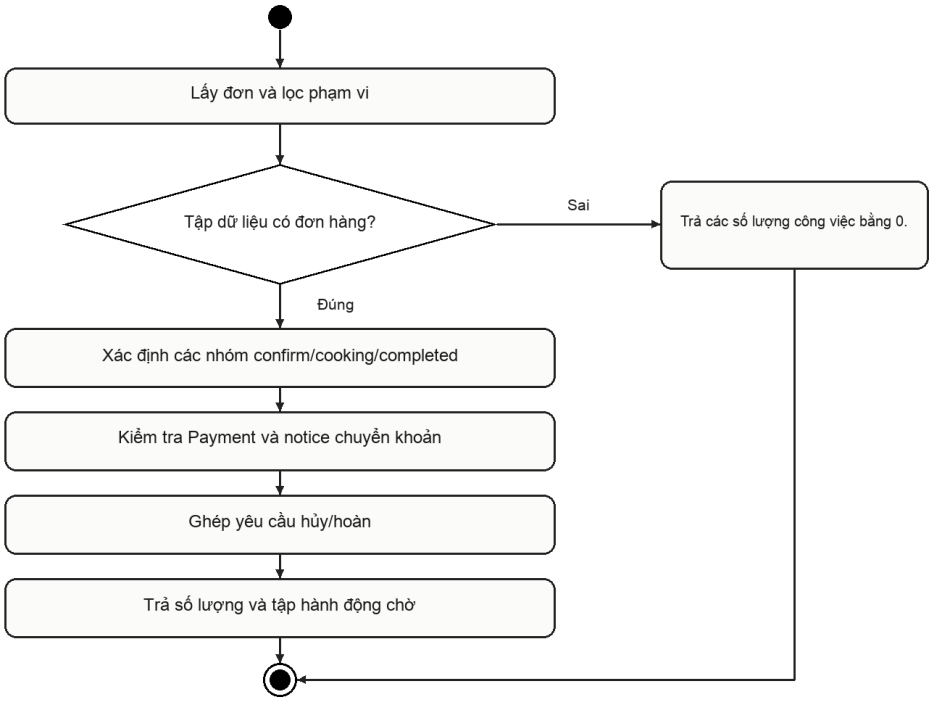
Bước 2. Xác định các nhóm confirm/cooking/completed.

Bước 3. Kiểm tra Payment và notice chuyển khoản.

Bước 4. Ghép yêu cầu hủy/hoàn.

Bước 5. Trả số lượng và tập hành động chờ.

2.4.12.2.Sơ đồ hoạt động



Hình 2.59. Hoạt động: Quản lý đơn và trạng thái thanh toán

2.4.12.3.Code xử lý chức năng

Đoạn mã 2.59. OrderService.getOrderWorkSummary — Quản lý đơn và trạng thái thanh toán

Nguồn: src/main/java/vn/edu/ute/milktea/service/order/OrderService.java; OrderService.getOrderWorkSummary; dòng 965–1035.

```
    public OrderDto.OrderWorkSummaryResponse getOrderWorkSummary() {
                                                List<Order>      allOrders      =
orderRepository.findAll().stream().filter(this::staffCanRead).toList();
        long totalCount = allOrders.size();
        long confirmCount = 0;
        long cookingCount = 0;
        long completedCount = 0;
        long transferCount = 0;
        long cancelCount = 0;
        long refundCount = 0;
        Set<Long> pendingActionOrderIds = new HashSet<>();

        var allRefunds = refundRepository.findAll();
                Map<Long,  vn.edu.ute.milktea.entity.cancellation.RefundRequest>
refundByOrderId = allRefunds.stream()
```

[Đã lược bỏ các dòng 978–979 của phương thức gốc.]

```
        for (Order o : allOrders) {
                        boolean  isStaffCreated  =  o.getAccount()  !=  null  &&
(o.getAccount().getRole()   ==   Role.CASHIER   ||   o.getAccount().getRole()   ==
Role.ADMIN);
            if (o.getStatus() == OrderStatus.CHO_XAC_NHAN && !isStaffCreated) {
                confirmCount++;
                pendingActionOrderIds.add(o.getId());
            }
```

```
                if (o.getStatus() == OrderStatus.CHO_CHE_BIEN || o.getStatus() ==
OrderStatus.DANG_CHE_BIEN) {
                cookingCount++;
            }
```

[Đã lược bỏ các dòng 991–1023 của phương thức gốc.]

```
        return OrderDto.OrderWorkSummaryResponse.builder()
                .totalCount(totalCount)
                .confirmCount(confirmCount)
                .cookingCount(cookingCount)
                .completedCount(completedCount)
                .transferCount(transferCount)
                .cancelCount(cancelCount)
                .refundCount(refundCount)
                .pendingActionCount(pendingActionOrderIds.size())
                .build();
    }
```

2.4.12.4.Giải thích code

Trang admin/orders có route và template; không tạo một dashboard chuỗi dựa trên số liệu không được truy vấn.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: orderRepository.findAll(); refundRepository.findAll(); invoiceRepository.findByOrderId(); paymentRepository.existsByInvoiceId(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/orders.html; src/main/resources/static/js/admin/admin.js.

2.4.13.Phê duyệt hoàn tiền chuyển khoản

2.4.13.1.Mô tả chức năng

Bảng 2.60. Đặc tả: Phê duyệt hoàn tiền chuyển khoản

Tiêu chí	Nội dung
Mục đích	Phê duyệt/từ chối hồ sơ hoàn tiền ngân hàng theo quy trình lưu trong service.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; yêu cầu hoàn còn trạng thái được xử lý.
Đầu vào	refundId, approve, decisionReason và adminId.
Kết quả / điều kiện sau	Chứng từ phản ánh quyết định; việc thực chi theo nhánh source tương ứng.
Trường hợp lỗi	Không tồn tại, đã quyết định hoặc trạng thái không cho thao tác.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Khóa yêu cầu hoàn.

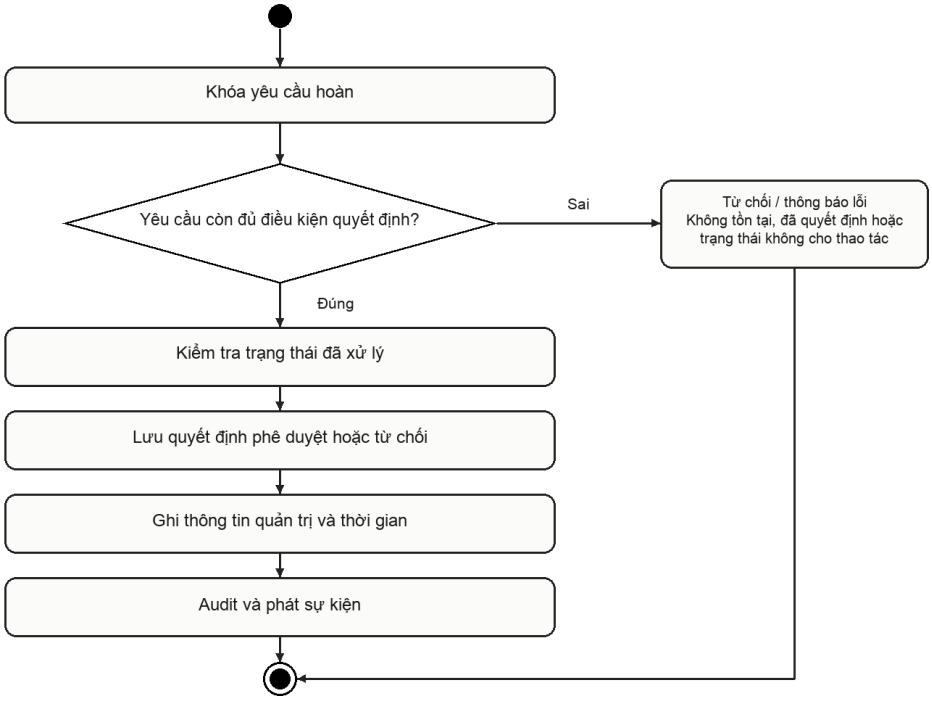
Bước 2. Kiểm tra trạng thái đã xử lý.

Bước 3. Lưu quyết định phê duyệt hoặc từ chối.

Bước 4. Ghi thông tin quản trị và thời gian.

Bước 5. Audit và phát sự kiện.

2.4.13.2.Sơ đồ hoạt động



Hình 2.60. Hoạt động: Phê duyệt hoàn tiền chuyển khoản

2.4.13.3.Code xử lý chức năng

Đoạn mã 2.60. RefundService.decideRefund — Phê duyệt hoàn tiền chuyển khoản

Nguồn: src/main/java/vn/edu/ute/milktea/service/cancellation/RefundService.java; RefundService.decideRefund; dòng 142–180.

```
    public void decideRefund(Long refundId, boolean approved, String reason, Long
adminId) {
        RefundRequest refund = refundRepository.findByIdWithLock(refundId)
                                                              .orElseThrow(()    ->
BusinessException.notFound(ErrorCode.VALIDATION_FAILED, "Không tìm thấy yêu cầu
hoàn tiền"));

        if (refund.getStatus() != RefundStatus.CHO_DUYET) {
            throw BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT, "Yêu
cầu hoàn tiền đã được xử lý trước đó");
        }

                                Account    admin    =    adminId    !=    null    ?
accountRepository.findById(adminId).orElse(null) : null;
        refund.setApprovedBy(admin);
        refund.setApprovedAt(Instant.now());
        refund.setApprovalReason(reason);
```

[Đã lược bỏ các dòng 155–155 của phương thức gốc.]

```
            refund.setStatus(RefundStatus.DA_HOAN);
            refund.setRefundedBy(admin);
            refund.setRefundedAt(refund.getApprovedAt());
            refund.setRefundReference("ADMIN_BANK_REFUND_" + refundId);
            auditRepository.save(BusinessAudit.builder()
                    .orderId(refund.getOrder().getId())
                    .accountId(adminId)
                    .action("BANK_REFUND_APPROVED")
                     .reason(reason != null && !reason.isBlank() ? reason : "Admin
 xác nhận đã hoàn chuyển khoản")
                    .createdAt(Instant.now())
                    .build());
```

[Đã lược bỏ các dòng 167–168 của phương thức gốc.]

```
            auditRepository.save(BusinessAudit.builder()
                    .orderId(refund.getOrder().getId())
                    .accountId(adminId)
                    .action("BANK_REFUND_REJECTED")
                    .reason(reason)
                    .createdAt(Instant.now())
                    .build());
        }

        refundRepository.save(refund);
        publish(refund);
    }
```

2.4.13.4.Giải thích code

Không khẳng định có API ngân hàng chuyển tiền tự động. recordRefundPayment là bước riêng cho yêu cầu đã duyệt.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: refundRepository.findByIdWithLock(); accountRepository.findById(); auditRepository.save(); refundRepository.save(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/orders.html; src/main/resources/static/js/admin/admin.js.

2.4.14.Cấu hình cửa hàng, ngân hàng và giảm giá chung

2.4.14.1.Mô tả chức năng

Bảng 2.61. Đặc tả: Cấu hình cửa hàng, ngân hàng và giảm giá chung

Tiêu chí	Nội dung
Mục đích	Quản lý tên/địa chỉ cửa hàng, tài khoản nhận tiền và kiểu giảm giá chung.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; mức giảm hợp lệ theo kiểu phần trăm hoặc số tiền.
Đầu vào	SettingsDto.UpdateSettingsRequest; actor sửa.
Kết quả / điều kiện sau	Đơn mới dùng chính sách mới; hóa đơn cũ giữ snapshot giảm giá.
Trường hợp lỗi	Giảm cố định âm hoặc dữ liệu ngoài điều kiện service.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Lấy GlobalSettings.

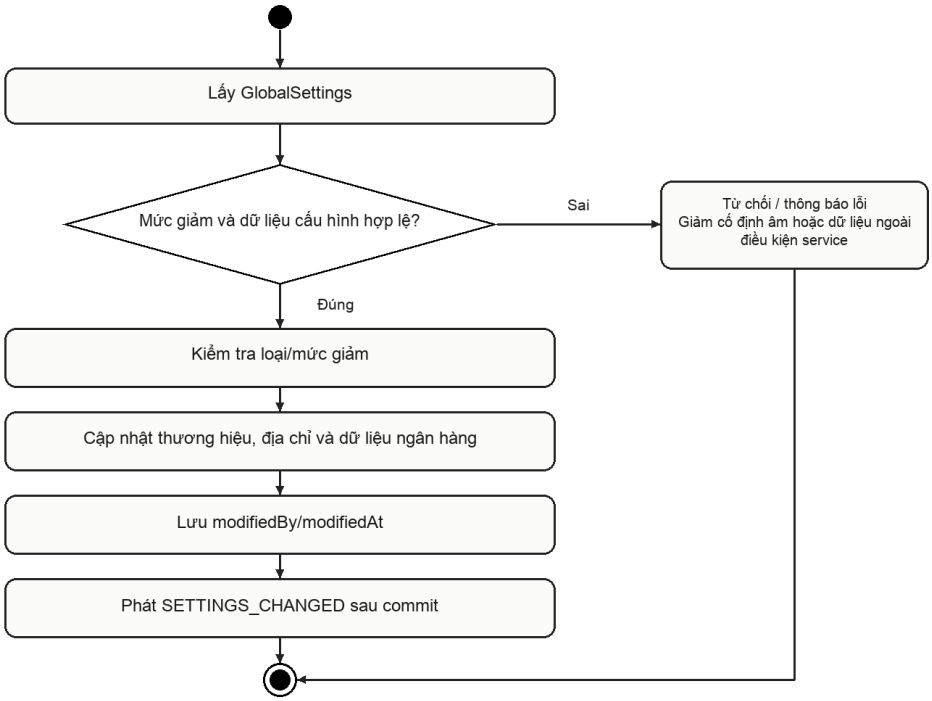
Bước 2. Kiểm tra loại/mức giảm.

Bước 3. Cập nhật thương hiệu, địa chỉ và dữ liệu ngân hàng.

Bước 4. Lưu modifiedBy/modifiedAt.

Bước 5. Phát SETTINGS_CHANGED sau commit.

2.4.14.2.Sơ đồ hoạt động



Hình 2.61. Hoạt động: Cấu hình cửa hàng, ngân hàng và giảm giá chung

2.4.14.3.Code xử lý chức năng

Đoạn mã 2.61. SettingsService.updateSettings — Cấu hình cửa hàng, ngân hàng và giảm giá chung

Nguồn: src/main/java/vn/edu/ute/milktea/service/settings/SettingsService.java; SettingsService.updateSettings; dòng 54–110.

```
      public void updateSettings(SettingsDto.UpdateSettingsRequest request, Long
adminId) {
                              Account    admin    =    adminId    !=    null    ?
accountRepository.findById(adminId).orElse(null) : null;

        GlobalSettings settings = settingsRepository.findById(1)
                .orElseGet(() -> GlobalSettings.builder()
                        .id(1)
                        .build());

        settings.setShopName(request.getShopName());
        settings.setAddress(request.getAddress());
        settings.setBankName(request.getBankName());
        settings.setBankAccountNumber(request.getBankAccountNumber());
        settings.setBankAccountHolder(request.getBankAccountHolder());
```

[Đã lược bỏ các dòng 67–68 của phương thức gốc.]

```
        if (type == null || type.isBlank()) {
            type = "PERCENT";
        }
        type = type.trim().toUpperCase();
        if (!"PERCENT".equals(type) && !"FIXED_AMOUNT".equals(type)) {
            type = "PERCENT";
        }
```

```
        BigDecimal val = request.getDiscountValue();
        if (val == null) {
                                val   =   request.getDiscountPercent()   !=   null   ?
 request.getDiscountPercent() : BigDecimal.ZERO;
```

[Đã lược bỏ các dòng 80–98 của phương thức gốc.]

```
        }

        settings.setModifiedBy(admin);
        settings.setModifiedAt(Instant.now());

        settingsRepository.save(settings);

                           realtimeEventPublisher.publishAfterCommit("/topic/admin",
 "SETTINGS_CHANGED", "1", null, "1",
                java.util.Map.of("shopName", settings.getShopName()));
                          realtimeEventPublisher.publishAfterCommit("/topic/cashier",
 "SETTINGS_CHANGED", "1", null, "1",
                java.util.Map.of("shopName", settings.getShopName()));
    }
```

2.4.14.4.Giải thích code

Báo cáo không chép số tài khoản thật, secret VNPAY hay mật khẩu SMTP vào bảng cấu hình.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: accountRepository.findById(); settingsRepository.findById(); settingsRepository.save(); realtimeEventPublisher.publishAfterCommit(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/settings.html; src/main/resources/static/js/admin/admin.js.

2.4.15.Thống kê doanh thu theo khoảng thời gian

2.4.15.1.Mô tả chức năng

Bảng 2.62. Đặc tả: Thống kê doanh thu theo khoảng thời gian

Tiêu chí	Nội dung
Mục đích	Tổng hợp số liệu doanh thu theo truy vấn đã được triển khai.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; bộ lọc ngày được chuẩn hóa trong service.
Đầu vào	fromDate/toDate theo LocalDate.
Kết quả / điều kiện sau	Giao diện báo cáo đọc được thống kê từ DB.
Trường hợp lỗi	Bộ lọc sai hoặc kết quả rỗng; không trình bày số demo như doanh thu kinh doanh thật.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Xác định khoảng thời gian.

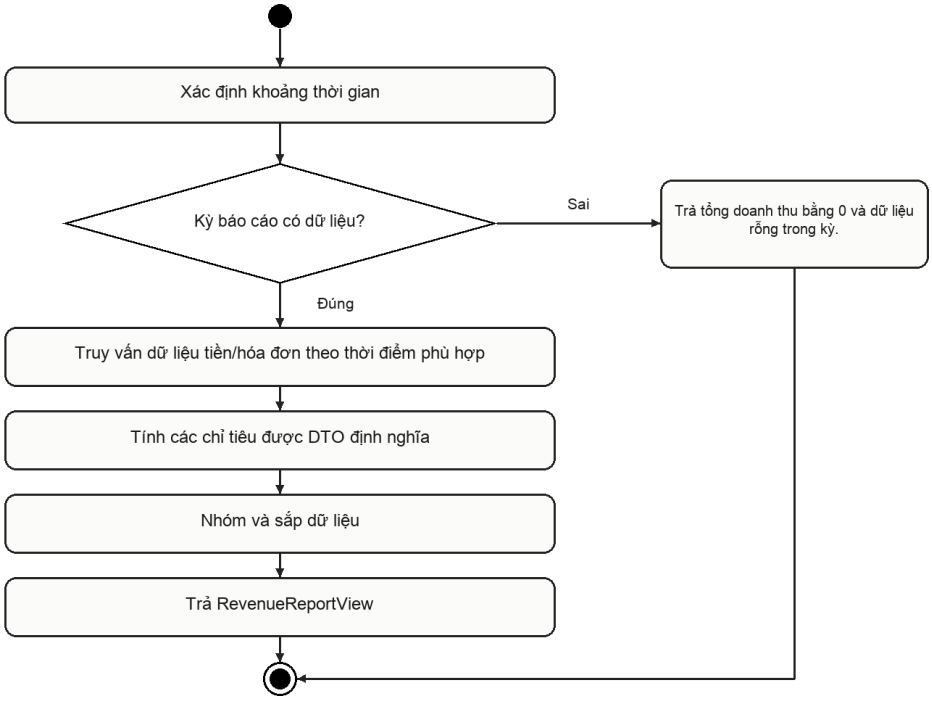
Bước 2. Truy vấn dữ liệu tiền/hóa đơn theo thời điểm phù hợp.

Bước 3. Tính các chỉ tiêu được DTO định nghĩa.

Bước 4. Nhóm và sắp dữ liệu.

Bước 5. Trả RevenueReportView.

2.4.15.2.Sơ đồ hoạt động



Hình 2.62. Hoạt động: Thống kê doanh thu theo khoảng thời gian

2.4.15.3.Code xử lý chức năng

Đoạn mã 2.62. ReportService.getRevenueReport — Thống kê doanh thu theo khoảng thời gian

Nguồn: src/main/java/vn/edu/ute/milktea/service/report/ReportService.java; ReportService.getRevenueReport; dòng 116–173.

```
        public ReportDto.RevenueReportView getRevenueReport(LocalDate fromDate,
LocalDate toDate) {
                                    if    (fromDate    ==    null)    fromDate    =
LocalDate.now(VIETNAM_ZONE).minusDays(30);
        if (toDate == null) toDate = LocalDate.now(VIETNAM_ZONE);

        Instant startInstant = fromDate.atStartOfDay(VIETNAM_ZONE).toInstant();
                                                    Instant      endInstant      =
toDate.plusDays(1).atStartOfDay(VIETNAM_ZONE).toInstant();

        // 1. Thực thu (Payments completed trong khoảng)
                                                List<Payment>      payments      =
paymentRepository.findByPaidAtBetween(startInstant, endInstant);
        BigDecimal received = payments.stream()
                .map(Payment::getAmount)
                .reduce(BigDecimal.ZERO, BigDecimal::add);
```

[Đã lược bỏ các dòng 129–140 của phương thức gốc.]

```
        for (var ord : completedOrders) {
            if (ord.getCompletedAt() != null &&
                    !ord.getCompletedAt().isBefore(startInstant) &&
                    ord.getCompletedAt().isBefore(endInstant)) {
                                                                Invoice    inv    =
invoiceRepository.findByOrderId(ord.getId()).orElse(null);
                if (inv != null && inv.getStatus() == InvoiceStatus.HIEU_LUC) {
                                                            completedOrderValue   =
completedOrderValue.add(inv.getTotalAmount());
                }
            }
        }
```

[Đã lược bỏ các dòng 152–161 của phương thức gốc.]

```
        return ReportDto.RevenueReportView.builder()
                .fromDate(fromDate)
                .toDate(toDate)
                .currency("VND")
                .received(received)
                .refunded(refunded)
                .netReceived(netReceived)
                .completedOrderValue(completedOrderValue)
                .currentOutstanding(currentOutstanding)
                .build();
    }
```

2.4.15.4.Giải thích code

Cần phân biệt ngày đặt, paidAt và hoàn tiền theo truy vấn; không đồng nhất mọi đơn tạo ra thành doanh thu đã thu.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: paymentRepository.findByPaidAtBetween(); refundRepository.findCompletedRefundsBetween(); orderRepository.findByStatusInOrderByCreatedAtAsc(); invoiceRepository.findByOrderId(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/reports.html; src/main/resources/static/js/admin/dashboard.js.

2.4.16.Dashboard đơn, sản phẩm và cảnh báo

2.4.16.1.Mô tả chức năng

Bảng 2.63. Đặc tả: Dashboard đơn, sản phẩm và cảnh báo

Tiêu chí	Nội dung
Mục đích	Tổng hợp báo cáo đơn, top sản phẩm, hao hụt và cảnh báo vận hành.
Tác nhân	Quản trị viên
Điều kiện trước	ADMIN; database có chứng từ để thống kê.
Đầu vào	fromDate/toDate và actor quản trị.
Kết quả / điều kiện sau	Không tạo dữ liệu kinh doanh giả để làm đầy biểu đồ.
Trường hợp lỗi	Không có quyền hoặc số liệu rỗng theo thời gian lọc.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Chuẩn hóa phạm vi ngày.

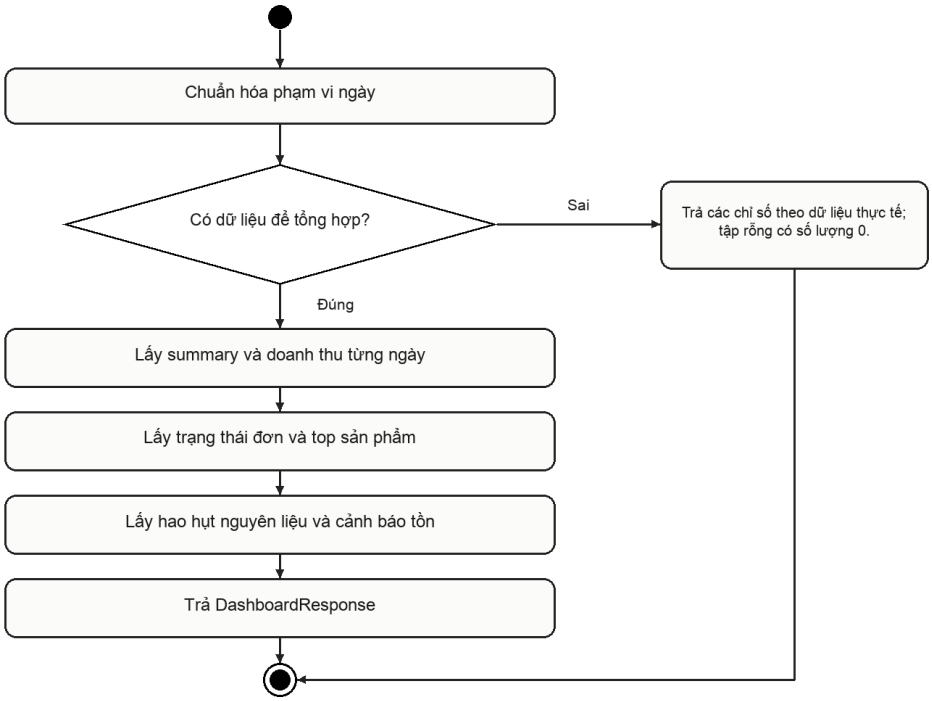
Bước 2. Lấy summary và doanh thu từng ngày.

Bước 3. Lấy trạng thái đơn và top sản phẩm.

Bước 4. Lấy hao hụt nguyên liệu và cảnh báo tồn.

Bước 5. Trả DashboardResponse.

2.4.16.2.Sơ đồ hoạt động



Hình 2.63. Hoạt động: Dashboard đơn, sản phẩm và cảnh báo

2.4.16.3.Code xử lý chức năng

Đoạn mã 2.63. ReportService.getDashboard — Dashboard đơn, sản phẩm và cảnh báo

Nguồn: src/main/java/vn/edu/ute/milktea/service/report/ReportService.java; ReportService.getDashboard; dòng 621–651.

```
public ReportDto.DashboardResponse getDashboard(
        LocalDate fromDate,
        LocalDate toDate) {

    LocalDate today = LocalDate.now(VIETNAM_ZONE);

    if (toDate == null) {
        toDate = today;
    }

    if (fromDate == null) {
        fromDate = toDate.minusDays(6);
    }

    if (fromDate.isAfter(toDate)) {
        throw new IllegalArgumentException(
                "Ngày bắt đầu không được lớn hơn ngày kết thúc"
        );
    }

    return ReportDto.DashboardResponse.builder()
            .fromDate(fromDate)
            .toDate(toDate)
            .summary(getDashboardSummary(fromDate, toDate))
```

```
                .revenueByDay(getDashboardRevenueByDay(fromDate, toDate))
                .ordersByStatus(getDashboardOrderStatuses(fromDate, toDate))
                .topProducts(getDashboardTopProducts(fromDate, toDate))
                .topWasteMaterials(getDashboardTopWasteMaterials(fromDate, toDate))
                .alerts(getDashboardAlerts())
                .build();
}
```

2.4.16.4.Giải thích code

Chụp demo phản ánh bộ dữ liệu thử nghiệm đã đánh dấu, không phải chứng minh KPI vận hành thật.

Method liên quan: ReportService.getDashboard. Controller: ReportController. Giao diện: src/main/resources/templates/admin/reports.html; src/main/resources/static/js/admin/dashboard.js.

2.4.17.Đổi logo và ảnh đại diện

2.4.17.1.Mô tả chức năng

Bảng 2.64. Đặc tả: Đổi logo và ảnh đại diện

Tiêu chí	Nội dung
Mục đích	Quản lý ảnh thương hiệu bằng pipeline kiểm tra và cleanup file ngoài hệ thống.
Tác nhân	Quản trị viên
Điều kiện trước	Có quyền tương ứng và Cloudinary đã cấu hình.
Đầu vào	MultipartFile; logo hoặc avatar có endpoint/ method riêng.
Kết quả / điều kiện sau	Ảnh mới dùng cho lần hiển thị sau; không làm lộ API key trong response.
Trường hợp lỗi	Cấu hình thiếu, tệp không phù hợp hoặc upload lỗi.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra nội dung ảnh.

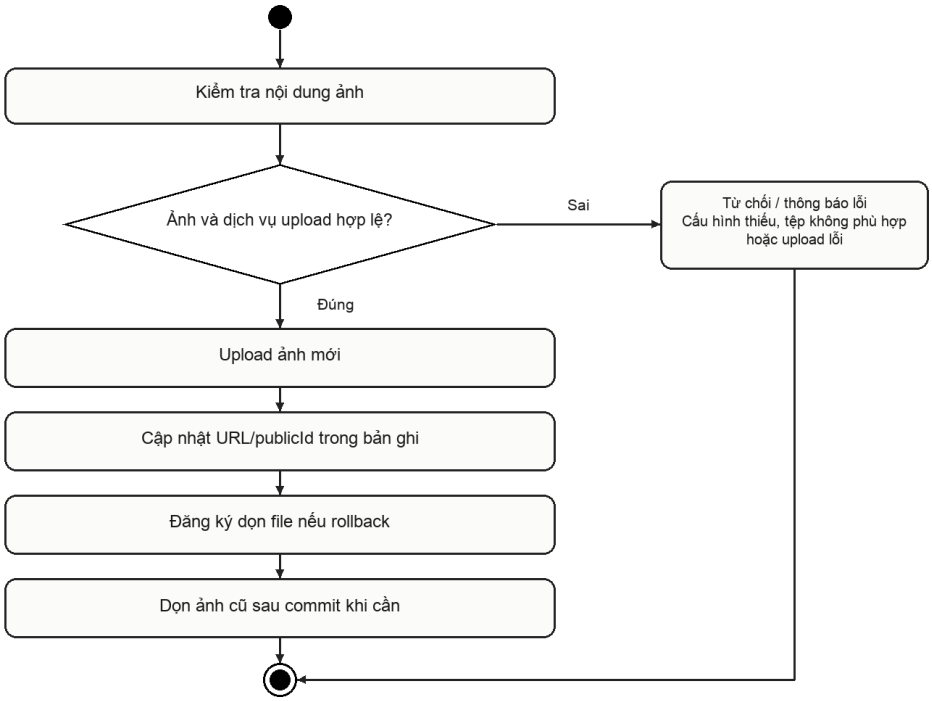
Bước 2. Upload ảnh mới.

Bước 3. Cập nhật URL/publicId trong bản ghi.

Bước 4. Đăng ký dọn file nếu rollback.

Bước 5. Dọn ảnh cũ sau commit khi cần.

2.4.17.2.Sơ đồ hoạt động



Hình 2.64. Hoạt động: Đổi logo và ảnh đại diện

2.4.17.3.Code xử lý chức năng

Đoạn mã 2.64. ProductImageService.uploadLogoImage — Đổi logo và ảnh đại diện

Nguồn: src/main/java/vn/edu/ute/milktea/service/media/ProductImageService.java; ProductImageService.uploadLogoImage; dòng 184–255.

```
       public SettingsDto.LogoImageView uploadLogoImage(MultipartFile file, Long
adminId) {
        validateImageFile(file);
        ensureCloudinaryReady();

        GlobalSettings settings = settingsRepository.findById(1)
                .orElseGet(() -> GlobalSettings.builder()
                        .id(1)
                        .shopName("MilkTea Quán 01")
                        .address("Số 1 Võ Văn Ngân, TP. Thủ Đức")
                        .modifiedAt(Instant.now())
                        .build());

        String oldPublicId = settings.getLogoPublicId();
```

[Đã lược bỏ các dòng 197–210 của phương thức gốc.]

```
            throw BusinessException.badRequest(ErrorCode.IMAGE_UPLOAD_FAILED,
                    "Không thể tải logo lên Cloudinary: " + e.getMessage());
        }

        // Lưu DB và cleanup ảnh mới nếu thất bại
        try {
            settings.setLogoUrl(secureUrl);
            settings.setLogoPublicId(publicId);
            settings.setModifiedAt(Instant.now());
```

```
            if (adminId != null) {

accountRepository.findById(adminId).ifPresent(settings::setModifiedBy);
```

[Đã lược bỏ các dòng 222–243 của phương thức gốc.]

```
         // Phát sự kiện WebSocket SETTINGS_CHANGED after-commit để header/navbar
trên
        // mọi màn hình cập nhật logo ngay lập tức
                          realtimeEventPublisher.publishAfterCommit("/topic/admin",
"SETTINGS_CHANGED", "1", null, "1",
                Map.of("action", "LOGO_UPDATED", "logoUrl", secureUrl));
                        realtimeEventPublisher.publishAfterCommit("/topic/cashier",
"SETTINGS_CHANGED", "1", null, "1",
                Map.of("action", "LOGO_UPDATED", "logoUrl", secureUrl));

        return SettingsDto.LogoImageView.builder()
                .logoUrl(secureUrl)
                .logoPublicId(publicId)
                .build();
    }
```

2.4.17.4.Giải thích code

uploadAvatarImage xác minh tài khoản; báo cáo chỉ trích mã, không thử upload bằng credentials thật.

Các điểm phối hợp thể hiện trực tiếp trong trích đoạn/phương thức: settingsRepository.findById(); accountRepository.findById(); settingsRepository.save(); realtimeEventPublisher.publishAfterCommit(). Đường dẫn giao diện liên quan: src/main/resources/templates/admin/settings.html; src/main/resources/templates/profile.html.

2.5.Vai trò cơ sở và phân hệ mở rộng

Nhóm này phản ánh VENDOR/MANAGER/SHIPPER và module marketplace/lịch sử còn trong source. Nó không thay đổi phạm vi mua mang về/tại quán của checkout hiện tại. Một API legacy còn tồn tại không được xem là bằng chứng dịch vụ giao hàng mới đang vận hành.

2.5.1.Tạo và chỉnh sửa cơ sở trong phân hệ mở rộng

2.5.1.1.Mô tả chức năng

Bảng 2.65. Đặc tả: Tạo và chỉnh sửa cơ sở trong phân hệ mở rộng

Tiêu chí	Nội dung
Mục đích	Quản lý thông tin cơ sở theo quan hệ chủ sở hữu đang có trong code.
Tác nhân	Vai trò cơ sở và phân hệ mở rộng
Điều kiện trước	Chỉ CUSTOMER/VENDOR/ADMIN được tạo theo service; cơ sở mới cần kích hoạt theo quyền quản trị.
Đầu vào	Tên, địa chỉ, area và tọa độ tùy chọn.
Kết quả / điều kiện sau	Shop giữ owner và trạng thái hoạt động theo cấu hình.
Trường hợp lỗi	Vai trò không được phép, dữ liệu địa chỉ/tọa độ không hợp lệ.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Lấy Account có khóa.

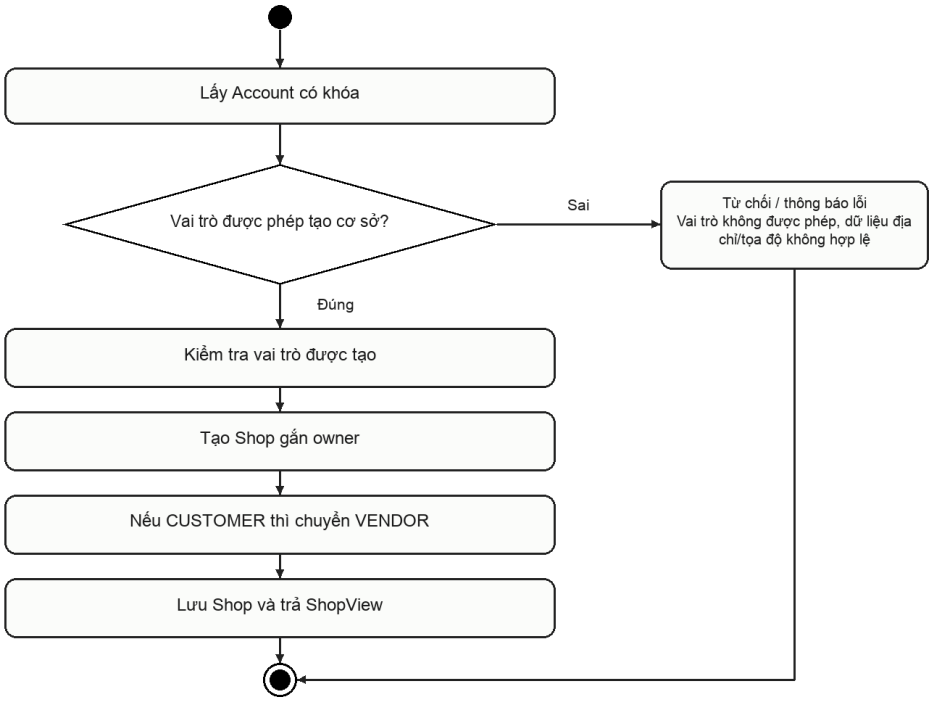
Bước 2. Kiểm tra vai trò được tạo.

Bước 3. Tạo Shop gắn owner.

Bước 4. Nếu CUSTOMER thì chuyển VENDOR.

Bước 5. Lưu Shop và trả ShopView.

2.5.1.2.Sơ đồ hoạt động



Hình 2.65. Hoạt động: Tạo và chỉnh sửa cơ sở trong phân hệ mở rộng

2.5.1.3.Code xử lý chức năng

Đoạn mã 2.65. MarketService.createShop — Tạo và chỉnh sửa cơ sở trong phân hệ mở rộng

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/MarketService.java; MarketService.createShop; dòng 51–57.

```
 public ShopView createShop(CurrentActor a,ShopInput input){
  var owner=account(a);
   if(!Set.of(Role.CUSTOMER,Role.VENDOR,Role.ADMIN).contains(owner.getRole()))throw
denied();
                                                Shop                                s=new
Shop();s.setOwner(owner);s.setName(input.name().trim());s.setAddress(input.address
().trim());location(s,input);
  if(owner.getRole()==Role.CUSTOMER)owner.setRole(Role.VENDOR);
  return shopView(shops.save(s));
 }
```

2.5.1.4.Giải thích code

Đây là module mở rộng còn hiện diện; không khẳng định chỉ có bốn role khi Role.java còn VENDOR/MANAGER/SHIPPER.

Method liên quan: MarketService.createShop. Controller: MarketController. Giao diện: src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js.

2.5.2.Phân công quản lý, thu ngân và bếp theo cơ sở

2.5.2.1.Mô tả chức năng

Bảng 2.66. Đặc tả: Phân công quản lý, thu ngân và bếp theo cơ sở

Tiêu chí	Nội dung
Mục đích	Gắn tài khoản nhân viên vào cơ sở để kiểm tra phạm vi các đơn mới.
Tác nhân	Vai trò cơ sở và phân hệ mở rộng
Điều kiện trước	ADMIN; tài khoản đích là MANAGER, CASHIER hoặc KITCHEN.
Đầu vào	shopId, accountId, enabled.
Kết quả / điều kiện sau	OrderBranchService.canAccess đọc quan hệ này khi lọc và thao tác đơn.
Trường hợp lỗi	Không có quyền, không có tài khoản hoặc role không được phân công.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra ADMIN.

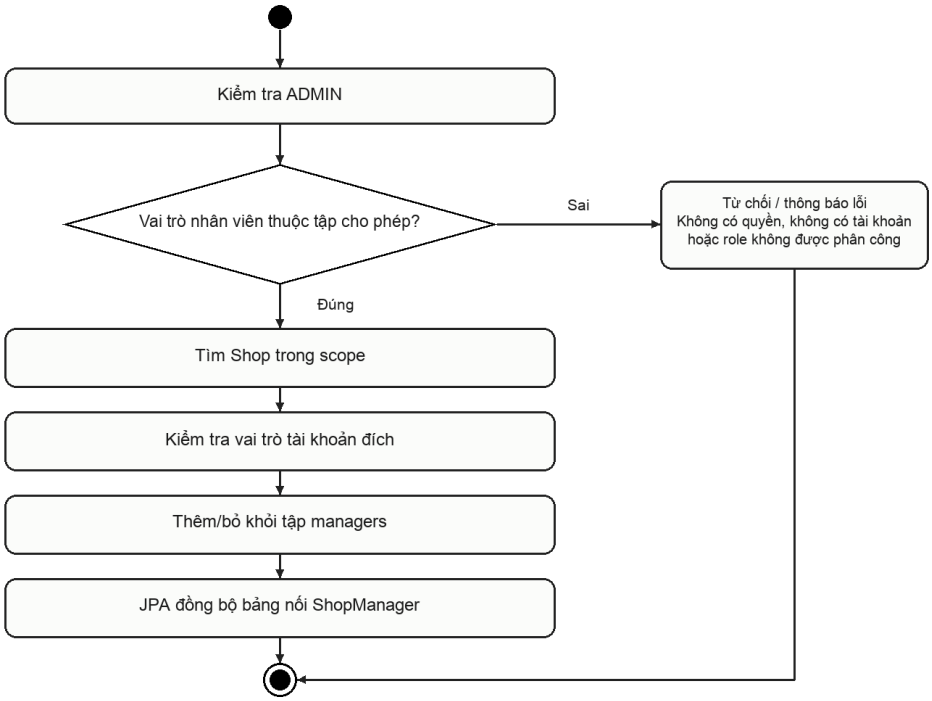
Bước 2. Tìm Shop trong scope.

Bước 3. Kiểm tra vai trò tài khoản đích.

Bước 4. Thêm/bỏ khỏi tập managers.

Bước 5. JPA đồng bộ bảng nối ShopManager.

2.5.2.2.Sơ đồ hoạt động



Hình 2.66. Hoạt động: Phân công quản lý, thu ngân và bếp theo cơ sở

2.5.2.3.Code xử lý chức năng

Đoạn mã 2.66. MarketService.assignManager — Phân công quản lý, thu ngân và bếp theo cơ sở

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/MarketService.java; MarketService.assignManager; dòng 66–70.

```
  public void assignManager(CurrentActor a,Long shopId,Long managerId,boolean
enabled){
              if(!admin(a))throw         denied();var         s=scope(a,shopId);var
m=accounts.findById(managerId).orElseThrow(this::denied);
    if(m.getRole()!=Role.MANAGER  &&  m.getRole()!=Role.CASHIER  &&  m.getRole()!
=Role.KITCHEN)throw invalid("Chỉ phân công quản lý, thu ngân hoặc bếp cho cơ sở");
          if(enabled)s.getManagers().add(m);else       s.getManagers().removeIf(x-
>x.getId().equals(managerId));
 }
```

2.5.2.4.Giải thích code

Tên bảng nối là ShopManager nhưng được tái sử dụng cho quyền quầy/bếp; không tạo bảng BranchStaff không tồn tại.

Method liên quan: MarketService.assignManager. Controller: MarketController. Giao diện: src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js.

2.5.3.Quản lý menu và tồn bán theo cơ sở

2.5.3.1.Mô tả chức năng

Bảng 2.67. Đặc tả: Quản lý menu và tồn bán theo cơ sở

Tiêu chí	Nội dung
Mục đích	Cập nhật biến thể món thuộc Shop và lượng tồn bán của module mở rộng.
Tác nhân	Vai trò cơ sở và phân hệ mở rộng
Điều kiện trước	VENDOR là owner, MANAGER được phân công hoặc ADMIN; Category thuộc đúng Shop.
Đầu vào	ProductInput gồm danh mục, tên/size, giá, menuCode, stock và version.
Kết quả / điều kiện sau	Menu có bản ghi theo Shop; checkout POS ánh xạ menuVariantIdentity để lấy đúng giá.
Trường hợp lỗi	Khác cơ sở, danh mục sai hoặc dữ liệu đã đổi ở phiên khác.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Xác minh scope.

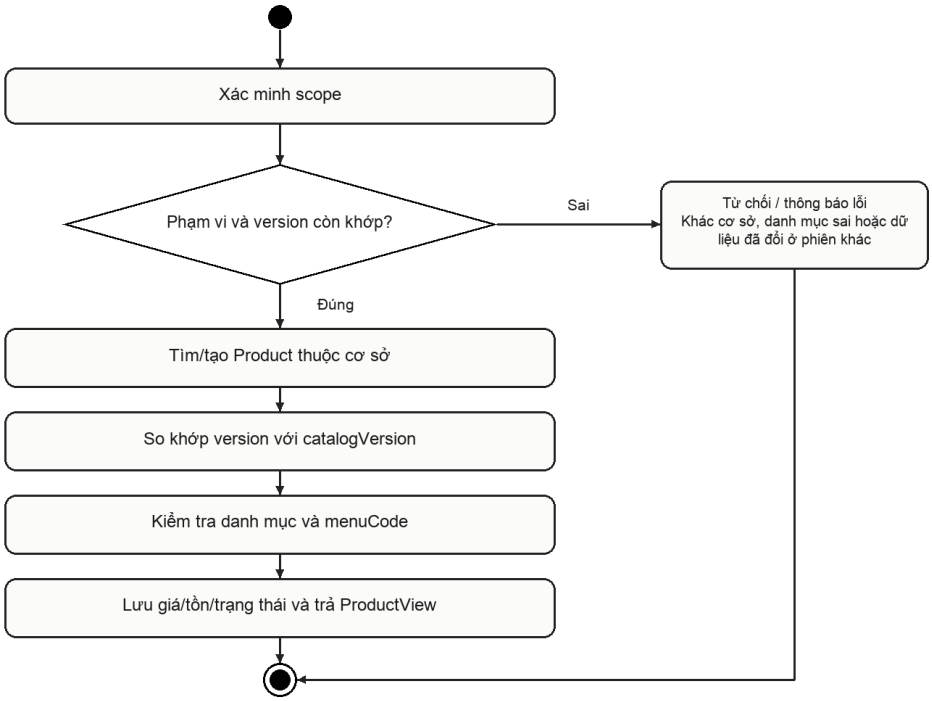
Bước 2. Tìm/tạo Product thuộc cơ sở.

Bước 3. So khớp version với catalogVersion.

Bước 4. Kiểm tra danh mục và menuCode.

Bước 5. Lưu giá/tồn/trạng thái và trả ProductView.

2.5.3.2.Sơ đồ hoạt động



Hình 2.67. Hoạt động: Quản lý menu và tồn bán theo cơ sở

2.5.3.3.Code xử lý chức năng

Đoạn mã 2.67. MarketService.saveProduct — Quản lý menu và tồn bán theo cơ sở

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/MarketService.java; MarketService.saveProduct; dòng 85–97.

```
    public      ProductView      saveProduct(CurrentActor      a,Long      shopId,Long
productId,ProductInput input){
              var         shop=scope(a,shopId);Product         p=productId==null?new
Product():products.findByIdWithLock(productId).orElseThrow(this::denied);
                                              if(productId!=null&&(p.getShop()==null||!
shopId.equals(p.getShop().getId())))throw denied();
                                                                  if(productId!=null&&!
Objects.equals(p.getCatalogVersion(),input.catalogVersion()))throw
BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT,"Món  hoặc  tồn  bán  đã
thay đổi; tải lại trước khi lưu");
    var   cat=categories.findById(input.categoryId()).orElseThrow(()->invalid("Danh
mục không tồn tại"));
  if(cat.getShop()==null||!shopId.equals(cat.getShop().getId()))throw denied();

p.setShop(shop);p.setCategory(cat);p.setName(input.name().trim());p.setSize(input.
size().trim());p.setDescription(input.description());p.setPrice(input.price());p.s
etDeliveryStock(input.stock());p.setActive(input.active());
  if(input.menuCode()!=null&&!input.menuCode().isBlank()) {
                                                                        if(!admin(a)&&!
Product.normalizeSearch(input.menuCode()).equals(p.menuIdentity())&&!
Product.normalizeSearch(input.menuCode()).equals(Product.normalizeSearch(p.getName
())+"|"+Product.normalizeSearch(p.getSize())))throw denied();
   p.setMenuCode(Product.normalizeSearch(input.menuCode()));
  }
```

```
  return productView(products.saveAndFlush(p));
 }
```

2.5.3.4.Giải thích code

Không xem deliveryStock là bằng chứng toàn bộ Stock nguyên liệu đã được phân vùng chi nhánh; hai module có mô hình riêng.

Method liên quan: MarketService.saveProduct. Controller: MarketController. Giao diện: src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js.

2.5.4.Quản lý mã khuyến mãi qua API cơ sở

2.5.4.1.Mô tả chức năng

Bảng 2.68. Đặc tả: Quản lý mã khuyến mãi qua API cơ sở

Tiêu chí	Nội dung
Mục đích	Tạo các điều kiện ưu đãi còn được backend hỗ trợ.
Tác nhân	Vai trò cơ sở và phân hệ mở rộng
Điều kiện trước	Scope quản trị/cơ sở hợp lệ.
Đầu vào	Code, số tiền giảm, đơn tối thiểu, startsAt/ endsAt, usageLimit, perAccountLimit, active.
Kết quả / điều kiện sau	Mã có dữ liệu điều kiện sử dụng trong DB.
Trường hợp lỗi	Hết hạn trước bắt đầu, code trùng hoặc thiếu quyền.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Kiểm tra phạm vi Shop.

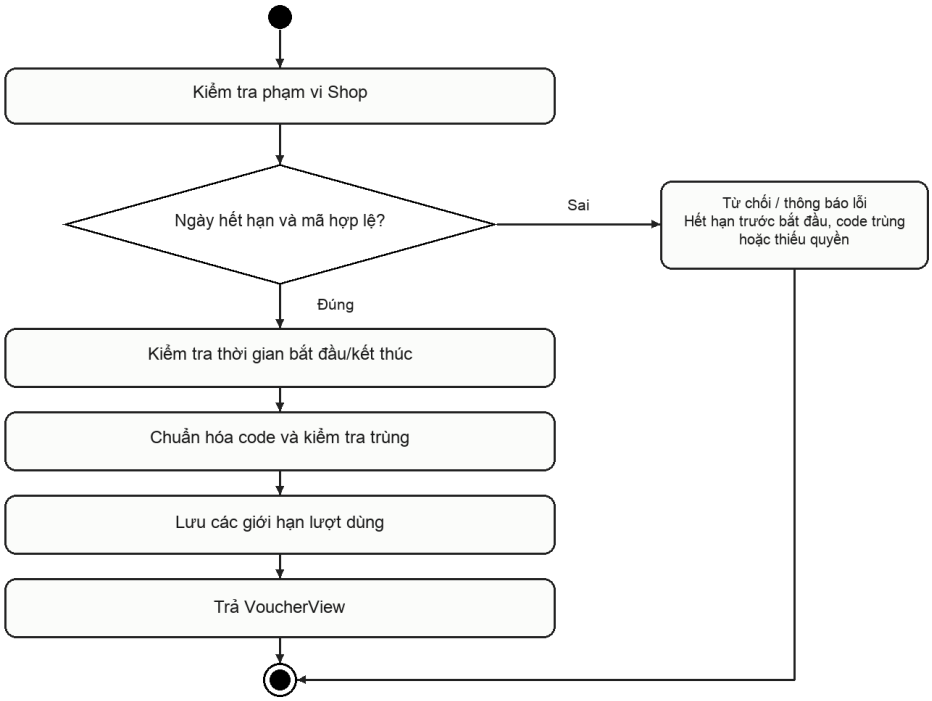
Bước 2. Kiểm tra thời gian bắt đầu/kết thúc.

Bước 3. Chuẩn hóa code và kiểm tra trùng.

Bước 4. Lưu các giới hạn lượt dùng.

Bước 5. Trả VoucherView.

2.5.4.2.Sơ đồ hoạt động



Hình 2.68. Hoạt động: Quản lý mã khuyến mãi qua API cơ sở

2.5.4.3.Code xử lý chức năng

Đoạn mã 2.68. MarketService.saveVoucher — Quản lý mã khuyến mãi qua API cơ sở

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/MarketService.java; MarketService.saveVoucher; dòng 109–115.

```
 public VoucherView saveVoucher(CurrentActor a,Long shopId,VoucherInput input){
     Shop   shop=shopId==null?null:scope(a,shopId);if(shopId==null&&!admin(a))throw
denied();
   if(!input.endsAt().isAfter(input.startsAt()))throw invalid("Ngày hết hạn phải
sau ngày bắt đầu");
  if(vouchers.findByCode(input.code()).isPresent())throw invalid("Mã đã tồn tại");
                                          Voucher                                v=new
Voucher();v.setShop(shop);v.setCode(input.code());v.setDiscountAmount(input.discou
ntAmount());v.setMinimumAmount(input.minimumAmount());v.setStartsAt(input.startsAt
());v.setEndsAt(input.endsAt());v.setUsageLimit(input.usageLimit());v.setPerAccoun
tLimit(input.perAccountLimit());v.setActive(input.active());
  return voucherView(vouchers.save(v));
 }
```

2.5.4.4.Giải thích code

Ô nhập mã đã bỏ khỏi checkout mới theo yêu cầu; có API/model không đồng nghĩa khách hiện dùng được ô nhập mã trên giao diện.

Method liên quan: MarketService.saveVoucher. Controller: MarketController. Giao diện: src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js.

2.5.5.Hạn chế khách mua tại một cơ sở

2.5.5.1.Mô tả chức năng

Bảng 2.69. Đặc tả: Hạn chế khách mua tại một cơ sở

Tiêu chí	Nội dung
Mục đích	Lưu chính sách chặn mua cục bộ thay vì khóa toàn bộ Account.
Tác nhân	Vai trò cơ sở và phân hệ mở rộng
Điều kiện trước	VENDOR/MANAGER/ADMIN có scope hợp lệ.
Đầu vào	shopId, accountId và enabled.
Kết quả / điều kiện sau	Hạn chế chỉ áp dụng tại Shop; trạng thái đăng nhập toàn hệ thống không tự đổi.
Trường hợp lỗi	Actor không thuộc scope hoặc tài khoản không tồn tại.
Trạng thái xác minh	Đã xác minh mã nguồn

Bước 1. Xác minh scope.

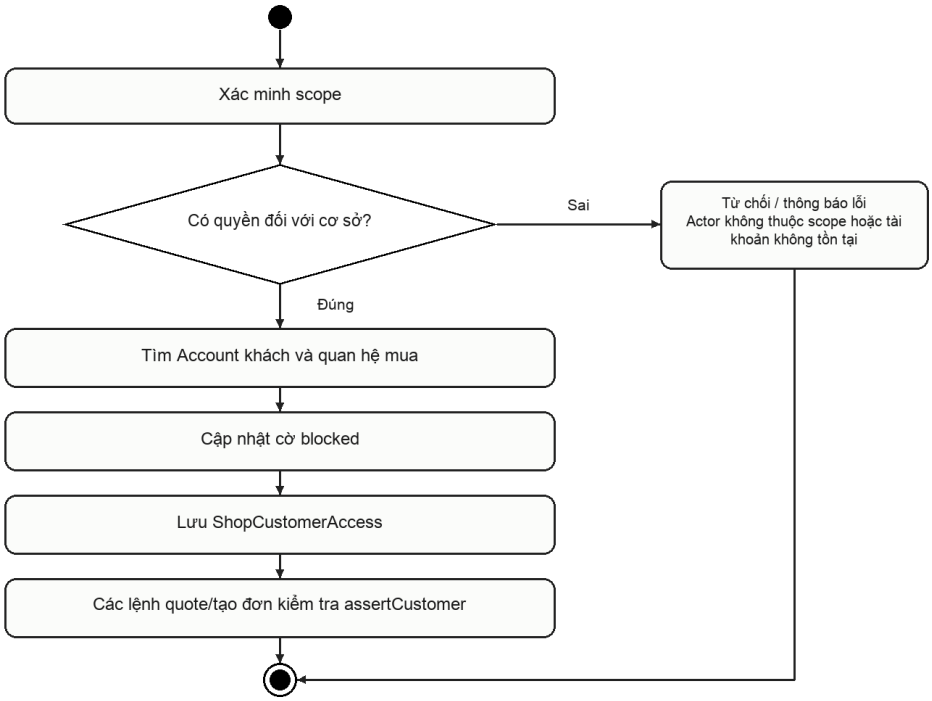
Bước 2. Tìm Account khách và quan hệ mua.

Bước 3. Cập nhật cờ blocked.

Bước 4. Lưu ShopCustomerAccess.

Bước 5. Các lệnh quote/tạo đơn kiểm tra assertCustomer.

2.5.5.2.Sơ đồ hoạt động



Hình 2.69. Hoạt động: Hạn chế khách mua tại một cơ sở

2.5.5.3.Code xử lý chức năng

Đoạn mã 2.69. MarketService.customerAccess — Hạn chế khách mua tại một cơ sở

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/MarketService.java; MarketService.customerAccess; dòng 240–243.

```
  public void customerAccess(CurrentActor a,Long shopId,Long accountId,boolean
enabled){
  scope(a,shopId);accounts.findByIdWithLock(accountId).orElseThrow(this::denied);
   var access=customers.findByShopIdAndAccountId(shopId,accountId).orElseThrow(()-
>invalid("Khách chưa có giao dịch tại shop này"));access.setBlocked(!enabled);
 }
```

2.5.5.4.Giải thích code

Khác với active=false do ADMIN khóa tài khoản; hai cơ chế không được viết lẫn trong báo cáo.

Method liên quan: MarketService.customerAccess. Controller: MarketController. Giao diện: src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js.

2.5.6.Tra cứu đơn giao hàng lịch sử theo phạm vi

2.5.6.1.Mô tả chức năng

Bảng 2.70. Đặc tả: Tra cứu đơn giao hàng lịch sử theo phạm vi

Tiêu chí	Nội dung
Mục đích	Bảo toàn truy cập dữ liệu của module giao hàng đã tồn tại.
Tác nhân	Vai trò cơ sở và phân hệ mở rộng
Điều kiện trước	Đã đăng nhập; phạm vi được tính theo owner/ manager/khách/shipper trong service.
Đầu vào	shopId tùy actor; tài khoản hiện tại.
Kết quả / điều kiện sau	Chỉ đọc các đơn cũ; không biến thành quy trình giao hàng mới của khách.
Trường hợp lỗi	Không có quyền hoặc không có lịch sử.
Trạng thái xác minh	Module lịch sử/tương thích còn mã nguồn

Bước 1. Xác định actor và scope.

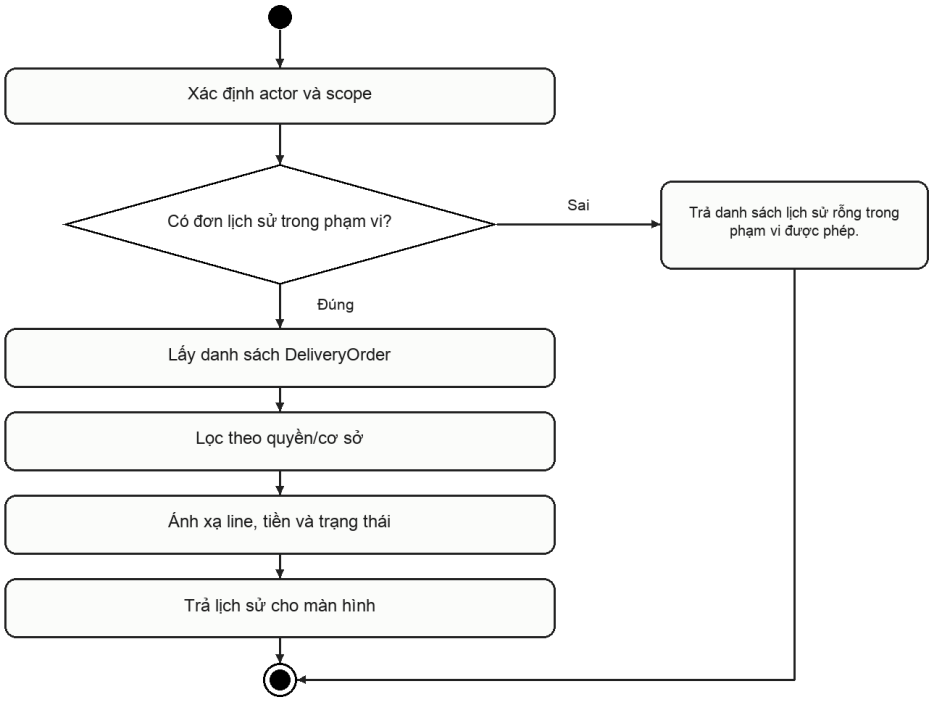
Bước 2. Lấy danh sách DeliveryOrder.

Bước 3. Lọc theo quyền/cơ sở.

Bước 4. Ánh xạ line, tiền và trạng thái.

Bước 5. Trả lịch sử cho màn hình.

2.5.6.2.Sơ đồ hoạt động



Hình 2.70. Hoạt động: Tra cứu đơn giao hàng lịch sử theo phạm vi

2.5.6.3.Code xử lý chức năng

Đoạn mã 2.70. MarketService.orders — Tra cứu đơn giao hàng lịch sử theo phạm vi

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/MarketService.java; MarketService.orders; dòng 180–184.

```
 @Transactional(readOnly=true) public List<OrderView> orders(CurrentActor a,Long
shopId){
                                    id(a);if(shopId!=null){scope(a,shopId);return
orders.findByShopIdOrderByIdDesc(shopId).stream().map(this::orderView).toList();}
               var          all=admin(a)?orders.findAll():a.hasRole(Role.SHIPPER)?
orders.findByShipperIdOrderByIdDesc(id(a)):orders.findByAccountIdOrderByIdDesc(id(
a));
                                                                          return
all.stream().sorted(Comparator.comparing(DeliveryOrder::getId).reversed()).map(thi
s::orderView).toList();
 }
```

2.5.6.4.Giải thích code

MarketController.checkout hiện chủ động từ chối tạo đơn giao hàng. Các route quote/carrier và class cũ chưa bị xóa.

Method liên quan: MarketService.orders. Controller: MarketController. Giao diện: src/main/resources/templates/customer/marketplace.html; src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js.

2.5.7.Phân công shipper cho đơn giao hàng cũ

2.5.7.1.Mô tả chức năng

Bảng 2.71. Đặc tả: Phân công shipper cho đơn giao hàng cũ

Tiêu chí	Nội dung
Mục đích	Mô tả chính xác mã nghiệp vụ còn tồn tại cho role SHIPPER.
Tác nhân	Vai trò cơ sở và phân hệ mở rộng
Điều kiện trước	Actor quản trị/cơ sở theo scope; đơn đã CONFIRMED và shipper hoạt động.
Đầu vào	orderId và shipperId.
Kết quả / điều kiện sau	Quan hệ shipper chỉ gắn với DeliveryOrder lịch sử.
Trường hợp lỗi	Sai trạng thái, sai scope hoặc shipper ngừng hoạt động.
Trạng thái xác minh	Module lịch sử/tương thích còn mã nguồn

Bước 1. Lấy DeliveryOrder có khóa.

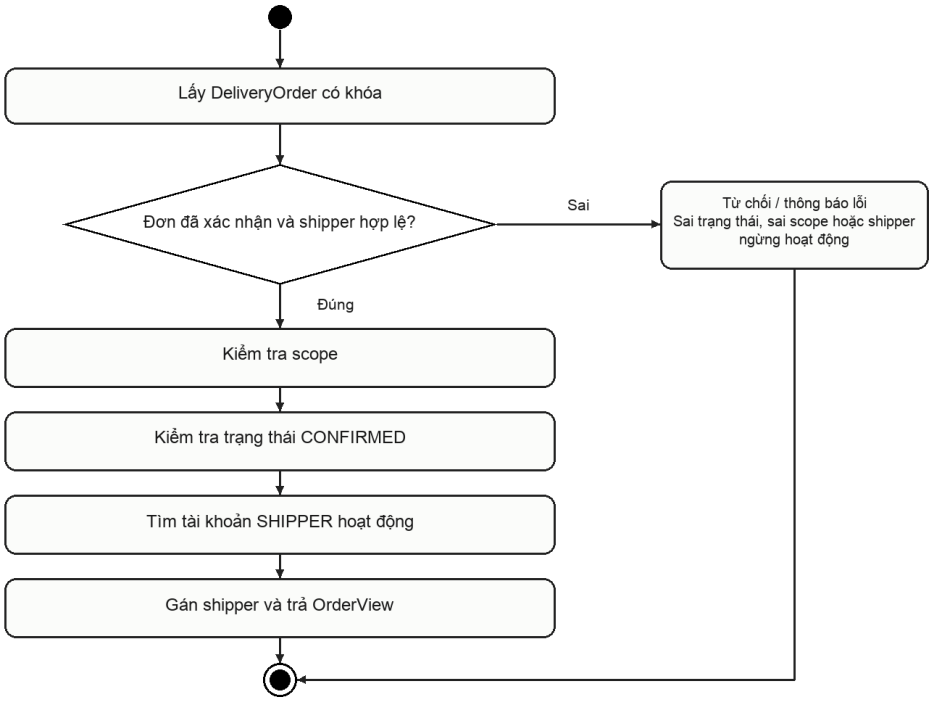
Bước 2. Kiểm tra scope.

Bước 3. Kiểm tra trạng thái CONFIRMED.

Bước 4. Tìm tài khoản SHIPPER hoạt động.

Bước 5. Gán shipper và trả OrderView.

2.5.7.2.Sơ đồ hoạt động



Hình 2.71. Hoạt động: Phân công shipper cho đơn giao hàng cũ

2.5.7.3.Code xử lý chức năng

Đoạn mã 2.71. MarketService.assignShipper — Phân công shipper cho đơn giao hàng cũ

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/MarketService.java; MarketService.assignShipper; dòng 186–190.

```
 public OrderView assignShipper(CurrentActor a,Long orderId,AssignShipper input){
                                                                                var
o=orders.lockId(orderId).orElseThrow(this::denied);scope(a,o.getShop().getId());ve
rsion(o,input.version());
   if(o.getStatus()!=DeliveryOrder.Status.CONFIRMED)throw invalid("Chỉ phân công
đơn đã xác nhận");
               var          shipper=accounts.findById(input.accountId()).filter(x-
>x.getRole()==Role.SHIPPER&&Boolean.TRUE.equals(x.getActive())).orElseThrow(()-
>invalid("Shipper không hoạt động"));o.setShipper(shipper);orders.flush();return
orderView(o);
 }
```

2.5.7.4.Giải thích code

Không tuyên bố UTE-Tea triển khai đội giao hàng cho checkout hiện tại chỉ vì role này tồn tại.

Method liên quan: MarketService.assignShipper. Controller: MarketController. Giao diện: src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js.

2.5.8.Chuyển trạng thái đơn giao hàng và trả hàng lịch sử

2.5.8.1.Mô tả chức năng

Bảng 2.72. Đặc tả: Chuyển trạng thái đơn giao hàng và trả hàng lịch sử

Tiêu chí	Nội dung
Mục đích	Lưu đồ các chuyển trạng thái còn được code hỗ trợ cho dữ liệu module cũ.
Tác nhân	Vai trò cơ sở và phân hệ mở rộng
Điều kiện trước	Actor có quyền theo vai trò và quan hệ đơn; version hiện tại.
Đầu vào	orderId, version và status đích.
Kết quả / điều kiện sau	Không sửa tùy ý status chỉ theo chuỗi client gửi.
Trường hợp lỗi	Không đủ quyền, version cũ hoặc trạng thái đích không nằm trong quy tắc.
Trạng thái xác minh	Module lịch sử/tương thích còn mã nguồn

Bước 1. Khóa DeliveryOrder và so version.

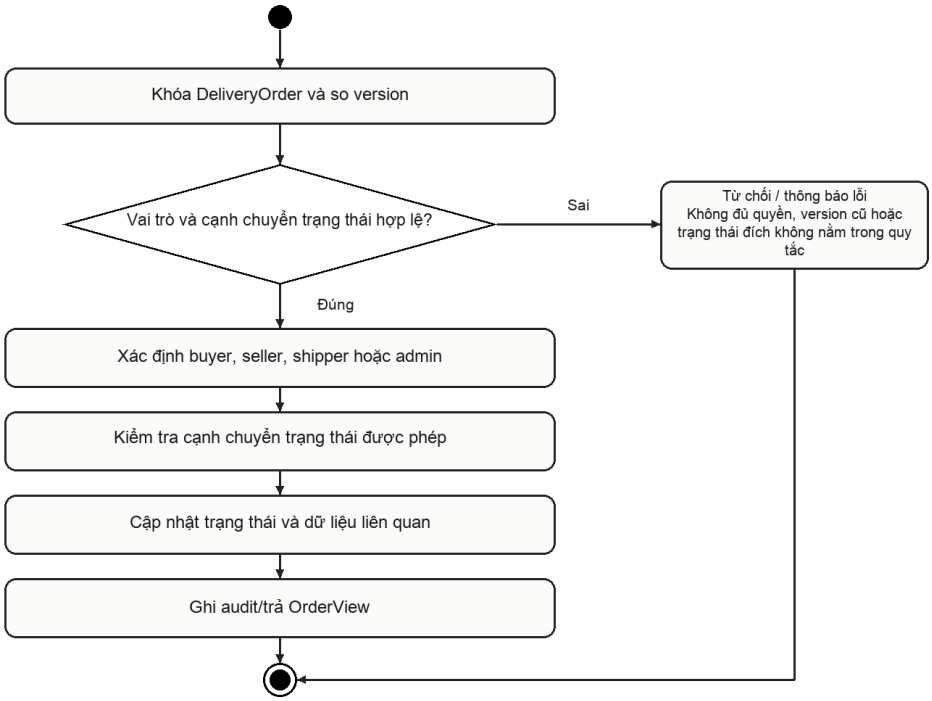
Bước 2. Xác định buyer, seller, shipper hoặc admin.

Bước 3. Kiểm tra cạnh chuyển trạng thái được phép.

Bước 4. Cập nhật trạng thái và dữ liệu liên quan.

Bước 5. Ghi audit/trả OrderView.

2.5.8.2.Sơ đồ hoạt động



Hình 2.72. Hoạt động: Chuyển trạng thái đơn giao hàng và trả hàng lịch sử

2.5.8.3.Code xử lý chức năng

Đoạn mã 2.72. MarketService.transition — Chuyển trạng thái đơn giao hàng và trả hàng lịch sử

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/MarketService.java; MarketService.transition; dòng 191–222.

```
 public OrderView transition(CurrentActor a,Long orderId,Transition input){
  var o=ownedOrder(a,orderId);version(o,input.version());
  boolean buyer=o.getAccount().getId().equals(id(a));
                boolean           shipper=a.hasRole(Role.SHIPPER)&&o.getShipper()!
=null&&o.getShipper().getId().equals(id(a));
      boolean    seller=admin(a)||shops.findAccessible(id(a)).stream().anyMatch(s-
>s.getId().equals(o.getShop().getId())&&((a.hasRole(Role.VENDOR)&&s.getOwner().get
Id().equals(id(a)))||a.hasRole(Role.MANAGER)));
  var before=o.getStatus();var next=input.status();boolean allowed=false;
  if(before==next)return orderView(o);
  switch(next){
   case CONFIRMED -> allowed=seller&&before==DeliveryOrder.Status.NEW;
                       case         SHIPPING         ->         allowed=(shipper||
admin(a))&&before==DeliveryOrder.Status.CONFIRMED&&(o.getPaymentMethod()==Delivery
Order.Method.COD||o.isPaid());
                      case         DELIVERED         ->         allowed=(shipper||
admin(a))&&before==DeliveryOrder.Status.SHIPPING;
                    case        DELIVERY_FAILED        ->        allowed=(shipper||
admin(a))&&before==DeliveryOrder.Status.SHIPPING;
       case   CANCELLED   ->   allowed=(buyer&&before==DeliveryOrder.Status.NEW)||
(seller&&(before==DeliveryOrder.Status.NEW||
before==DeliveryOrder.Status.CONFIRMED||
before==DeliveryOrder.Status.DELIVERY_FAILED));
   case RETURN_REQUESTED -> allowed=buyer&&before==DeliveryOrder.Status.DELIVERED;
```

```
   case RETURNED -> allowed=seller&&before==DeliveryOrder.Status.RETURN_REQUESTED;
   case REFUNDED -> {
            allowed=admin(a)&&o.isPaid()&&(before==DeliveryOrder.Status.RETURNED||
before==DeliveryOrder.Status.CANCELLED)&&o.getPaymentMethod()==DeliveryOrder.Metho
d.COD;
          if(allowed&&(input.reference()==null||input.reference().isBlank()))throw
invalid("Cần tham chiếu phiếu hoàn tiền thực tế");
   }
   default -> {}
  }
                                                                    if(!allowed)throw
BusinessException.conflict(ErrorCode.ORDER_STATE_CONFLICT,"Không  đủ  quyền  hoặc
chuyển trạng thái không hợp lệ");
  if(next==DeliveryOrder.Status.CANCELLED||next==DeliveryOrder.Status.RETURNED){
             for(var     line:o.getLines().stream().sorted(Comparator.comparing(l-
>l.getProduct().getId())).toList()){
                                                                          var
p=products.findByIdWithLock(line.getProduct().getId()).orElseThrow(this::denied);p
.setDeliveryStock(Math.addExact(p.getDeliveryStock(),line.getQuantity()));
   }
  }
                                              if(next==DeliveryOrder.Status.DELIVERED)
{o.setDeliveredAt(Instant.now());if(o.getPaymentMethod()==DeliveryOrder.Method.COD
)o.setPaid(true);}
  if(next==DeliveryOrder.Status.DELIVERY_FAILED)o.setDeliveryFailed(true);

if(next==DeliveryOrder.Status.REFUNDED)o.setRefundReference(input.reference().trim
());
  o.setStatus(next);orders.flush();return orderView(o);
 }
```

2.5.8.4.Giải thích code

Các trạng thái giao/hoàn ở đây không áp vào OrderStatus của POS. Giao diện mới không tạo đơn giao hàng.

Method liên quan: MarketService.transition. Controller: MarketController. Giao diện: src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js.

2.5.9.Tích hợp VNPAY của module lịch sử

2.5.9.1.Mô tả chức năng

Bảng 2.73. Đặc tả: Tích hợp VNPAY của module lịch sử

Tiêu chí	Nội dung
Mục đích	Phân tích nhánh nhận thông báo thanh toán đã có mã kiểm tra chữ ký và chống lặp.
Tác nhân	Vai trò cơ sở và phân hệ mở rộng
Điều kiện trước	Cấu hình VNPAY hợp lệ; callback qua endpoint được triển khai.
Đầu vào	Map tham số IPN từ VNPAY; mã giao dịch và chữ ký.
Kết quả / điều kiện sau	Chỉ cập nhật theo nhánh callback hợp lệ của module cũ; không đồng nghĩa mở QR POS là PAID.
Trường hợp lỗi	Chữ ký sai, đơn không khớp hoặc số tiền khác dự kiến.
Trạng thái xác minh	Có mã tích hợp; chưa xác minh giao dịch thật

Bước 1. Kiểm tra cấu hình và chữ ký.

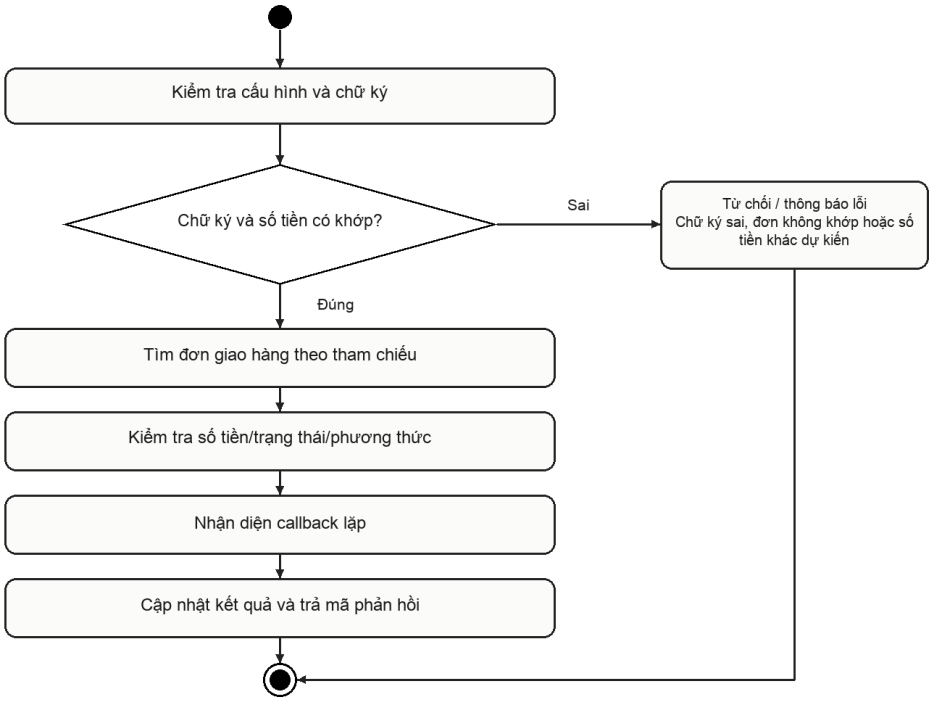
Bước 2. Tìm đơn giao hàng theo tham chiếu.

Bước 3. Kiểm tra số tiền/trạng thái/phương thức.

Bước 4. Nhận diện callback lặp.

Bước 5. Cập nhật kết quả và trả mã phản hồi.

2.5.9.2.Sơ đồ hoạt động



Hình 2.73. Hoạt động: Tích hợp VNPAY của module lịch sử

2.5.9.3.Code xử lý chức năng

Đoạn mã 2.73. VnpayService.ipn — Tích hợp VNPAY của module lịch sử

Nguồn: src/main/java/vn/edu/ute/milktea/service/market/VnpayService.java; VnpayService.ipn; dòng 42–59.

```
 @Transactional public Map<String,String> ipn(Map<String,String> input){
  if(!configured())return reply("99","Gateway not configured");
          Map<String,String>      fields=new      TreeMap<>();input.forEach((k,v)-
>{if(k.startsWith("vnp_")&&!k.equals("vnp_SecureHash")&&!
k.equals("vnp_SecureHashType"))fields.put(k,v);});
  String actual=Objects.toString(input.get("vnp_SecureHash"),"");
                                        if(!actual.matches("[0-9a-fA-F]{128}")||!
MessageDigest.isEqual(sign(fields).getBytes(StandardCharsets.US_ASCII),actual.toLo
werCase(Locale.ROOT).getBytes(StandardCharsets.US_ASCII)))return
reply("97","Invalid signature");
       if(!tmnCode.equals(fields.get("vnp_TmnCode")))return    reply("97","Invalid
merchant");
         String     ref=fields.get("vnp_TxnRef");if(ref==null||!ref.matches("[0-9]
{1,18}"))return reply("01","Order not found");
      var    optional=orders.lockId(Long.valueOf(ref));if(optional.isEmpty())return
reply("01","Order not found");var o=optional.get();
  String amount=fields.get("vnp_Amount");
                                if(amount==null||!amount.matches("[0-9]{1,20}")||!
o.getTotalAmount().movePointRight(2).toBigIntegerExact().equals(new
java.math.BigInteger(amount)))return reply("04","Invalid amount");
    if(o.getPaymentMethod()!=DeliveryOrder.Method.VNPAY)return  reply("99","Invalid
payment method");
  if(o.isPaid())return reply("02","Order already confirmed");

if(!"00".equals(fields.get("vnp_ResponseCode"))||!"00".equals(fields.get("vnp_Tran
sactionStatus")))return reply("00","Unsuccessful payment acknowledged");
```

```
      String    transaction=fields.get("vnp_TransactionNo");if(transaction==null||!
transaction.matches("[0-9]{1,15}")||
orders.existsByGatewayReference(transaction))return            reply("99","Invalid
transaction reference");
  // A verified late payment is still money received, even if the user cancelled.
  // Keep order status unchanged; expose paid cancelled orders for manual gateway
refund.
          o.setPaid(true);o.setGatewayReference(transaction);orders.flush();return
reply("00","Confirm Success");
 }
```

2.5.9.4.Giải thích code

Chưa xác minh tài khoản merchant/sandbox thực. Checkout mới chỉ CASH/ BANK_TRANSFER, không gọi paymentUrl VNPAY.

Method liên quan: VnpayService.ipn. Controller: VnpayController. Giao diện: src/main/resources/ templates/market/portal.html; src/main/resources/static/js/market/portal.js.

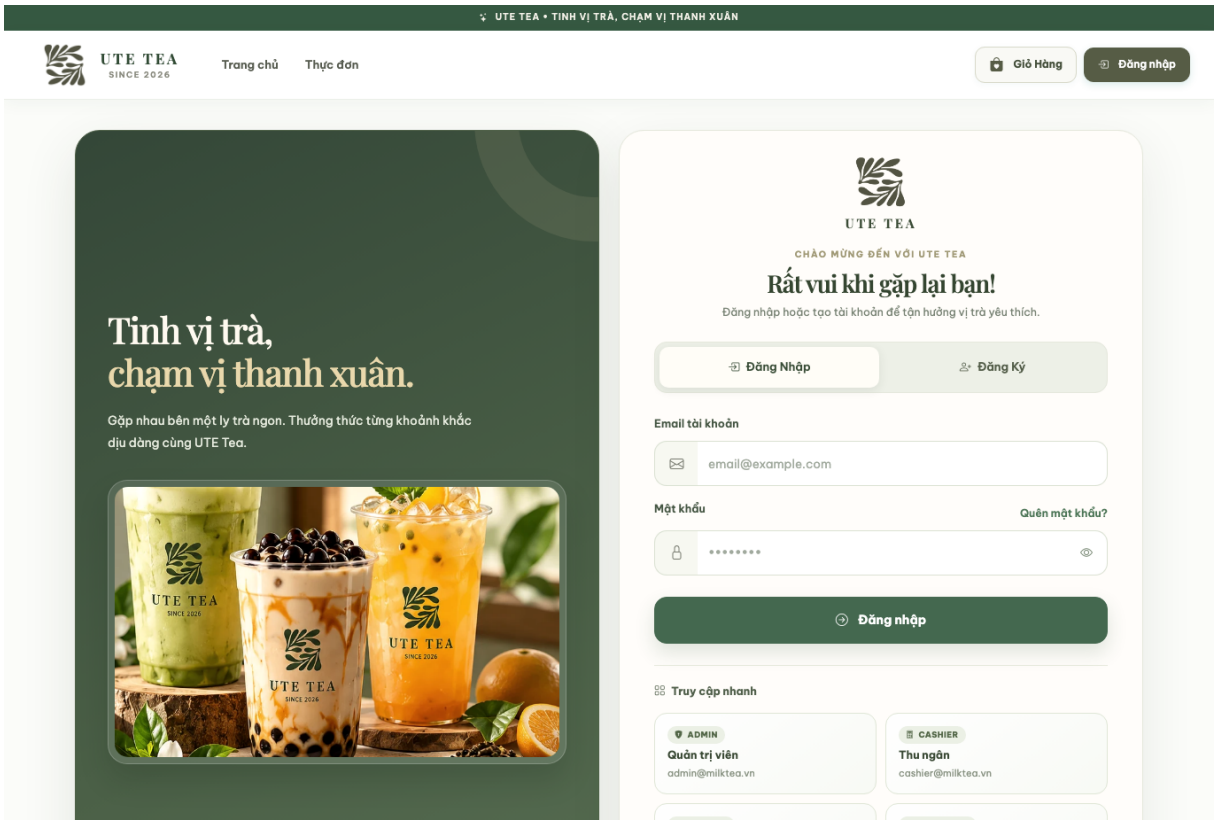
3.GIAO DIỆN HỆ THỐNG

Tất cả ảnh trong chương được chụp trực tiếp từ UTE-Tea chạy trên H2 demo tách biệt. Dữ liệu món, tài khoản và đơn trong ảnh là dữ liệu minh họa. Không sử dụng hình giao diện của dự án khác; không dùng ảnh AI dựng giao diện. Một số tác vụ dùng modal hoặc tab trong trang hiện có, không tự đặt tên một route trang độc lập.

3.1.Giao diện khách hàng

3.1.1.Đăng nhập

Xác thực tài khoản và chuyển vào chức năng phù hợp vai trò.



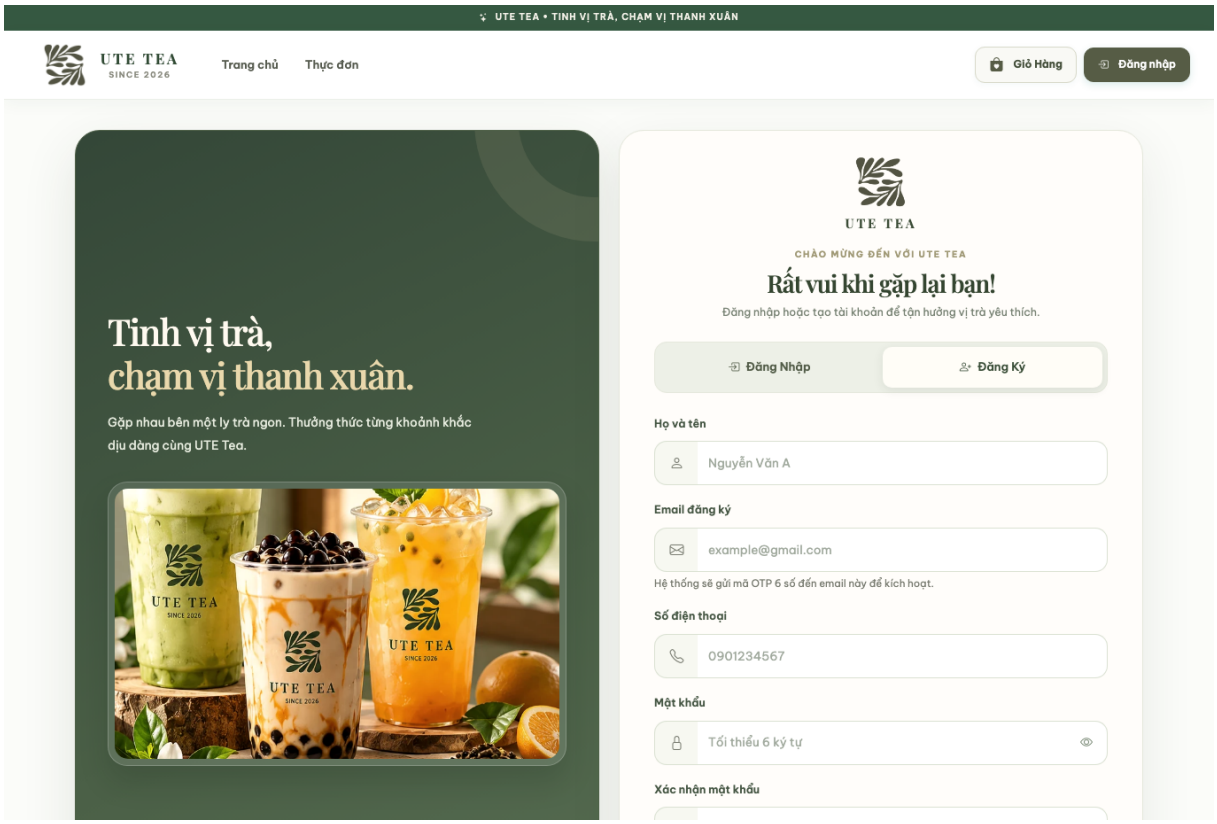
Hình 3.1. Đăng nhập — /login

Bảng 3.1. Mô tả thành phần giao diện đăng nhập

Nội dung	Mô tả
Mục đích	Xác thực tài khoản và chuyển vào chức năng phù hợp vai trò.
Thành phần chính	Tab đăng nhập/đăng ký, email, mật khẩu, quên mật khẩu.
Thao tác	Nhập thông tin và gửi đăng nhập; chuyển sang đăng ký hoặc khôi phục.
Đường dẫn	/login
File giao diện	src/main/resources/templates/auth/login.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.2.Đăng ký tài khoản

Khởi tạo tài khoản khách chờ xác thực email.



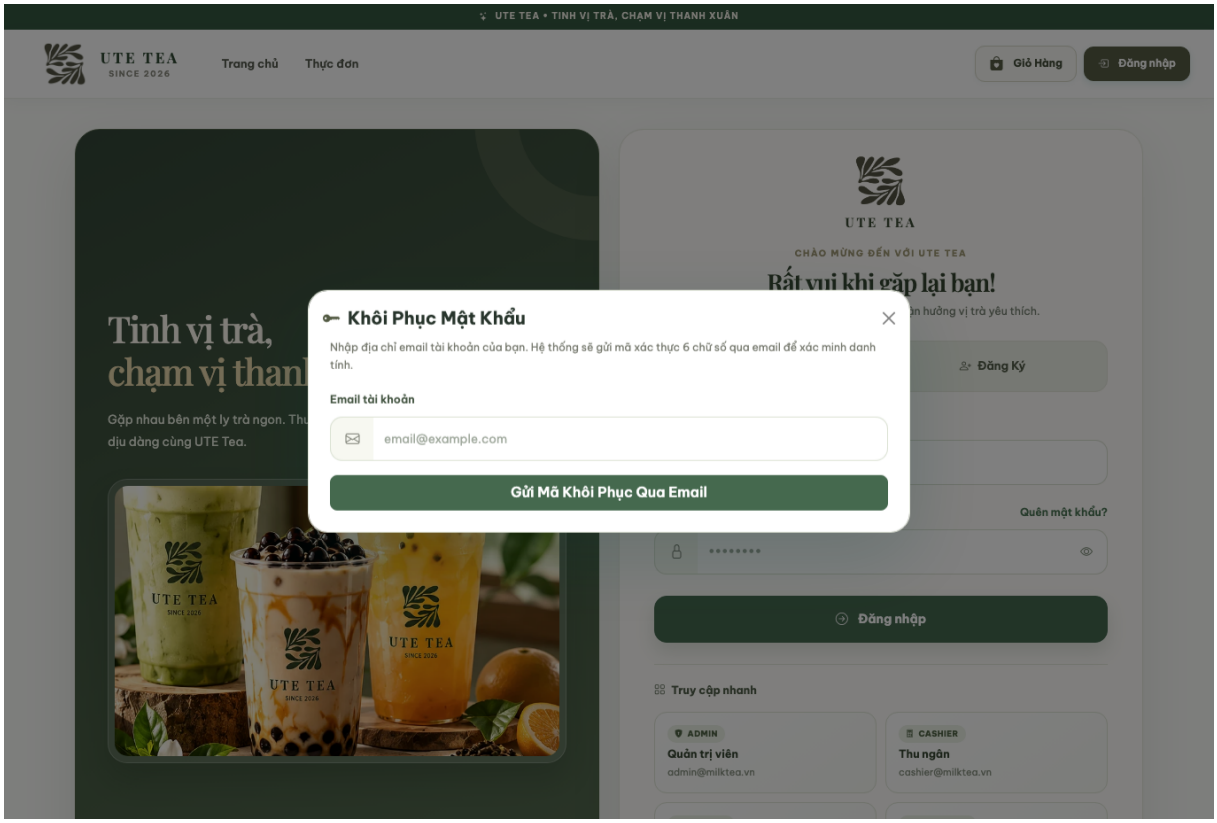
Hình 3.2. Đăng ký tài khoản — /login

Bảng 3.2. Mô tả thành phần giao diện đăng ký tài khoản

Nội dung	Mô tả
Mục đích	Khởi tạo tài khoản khách chờ xác thực email.
Thành phần chính	Họ tên, email, số điện thoại, mật khẩu và nhập lại mật khẩu.
Thao tác	Nhập dữ liệu; kiểm tra định dạng; gửi đăng ký.
Đường dẫn	/login
File giao diện	src/main/resources/templates/auth/login.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.3.Yêu cầu khôi phục mật khẩu

Yêu cầu mã xác minh khi quên mật khẩu.



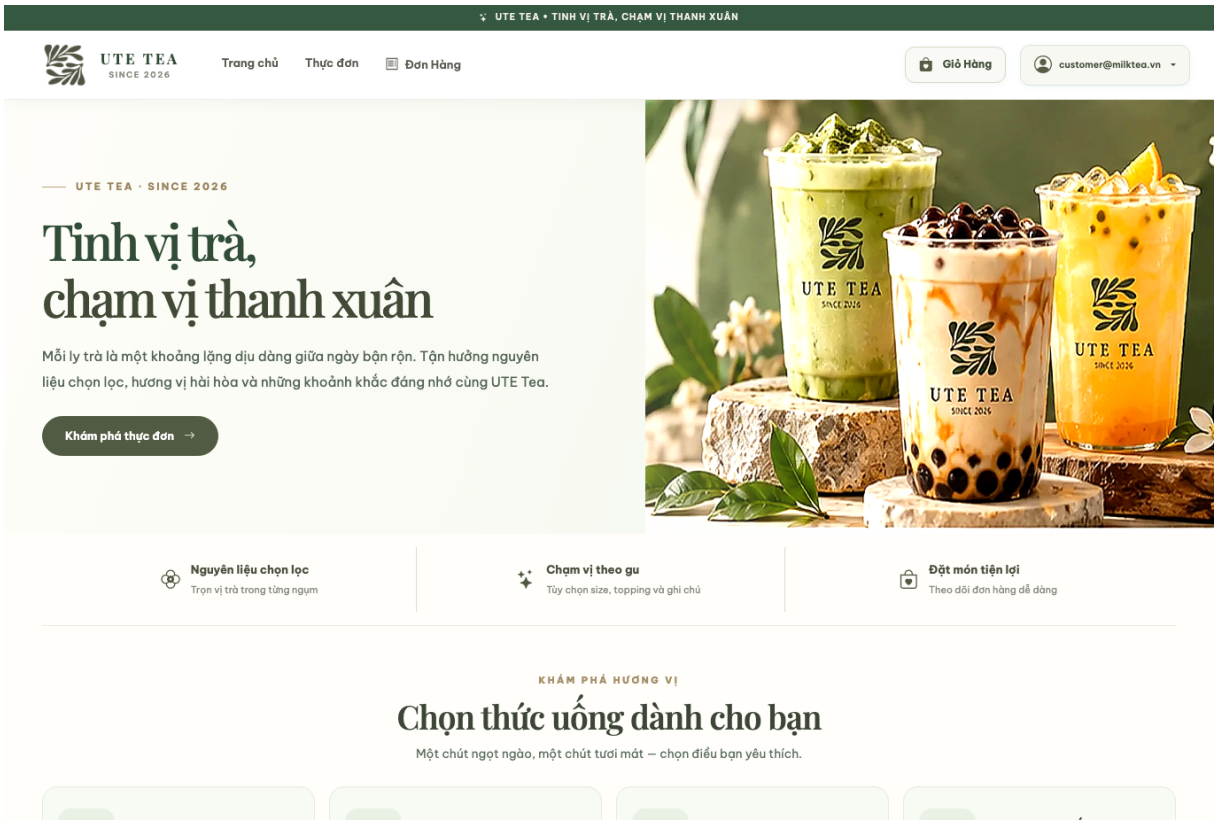
Hình 3.3. Yêu cầu khôi phục mật khẩu — /login

Bảng 3.3. Mô tả thành phần giao diện yêu cầu khôi phục mật khẩu

Nội dung	Mô tả
Mục đích	Yêu cầu mã xác minh khi quên mật khẩu.
Thành phần chính	Hộp thoại email khôi phục và nút gửi mã.
Thao tác	Nhập email, gửi yêu cầu hoặc quay lại.
Đường dẫn	/login
File giao diện	src/main/resources/templates/auth/login.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.4.Trang chủ và thực đơn

Truy cập menu chính và lựa chọn món.



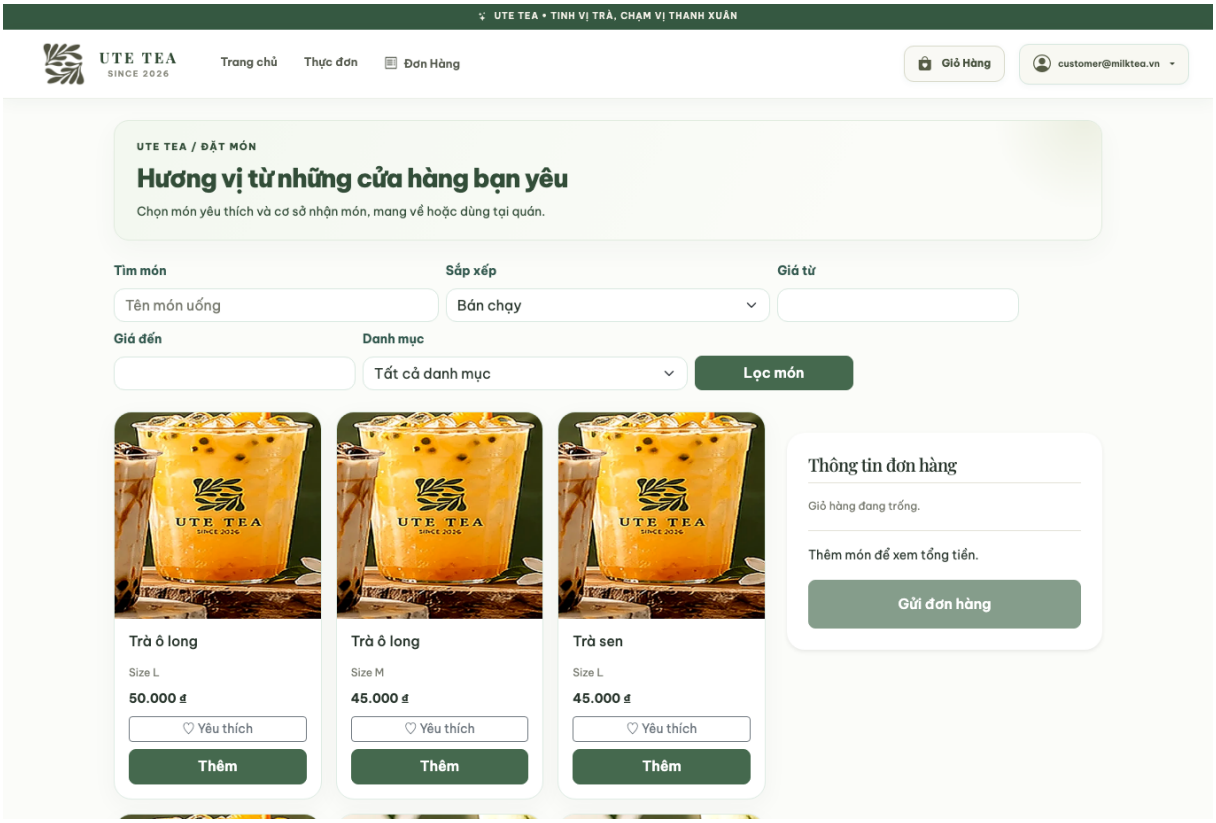
Hình 3.4. Trang chủ và thực đơn — /

Bảng 3.4. Mô tả thành phần giao diện trang chủ và thực đơn

Nội dung	Mô tả
Mục đích	Truy cập menu chính và lựa chọn món.
Thành phần chính	Banner thương hiệu, danh mục, thẻ món, giá và thanh điều hướng.
Thao tác	Lọc/tìm món, mở hộp tùy chọn và xem giỏ.
Đường dẫn	/
File giao diện	src/main/resources/templates/customer/menu.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.5.Danh sách món và bộ lọc

Tra cứu menu dùng chung và các biến thể.



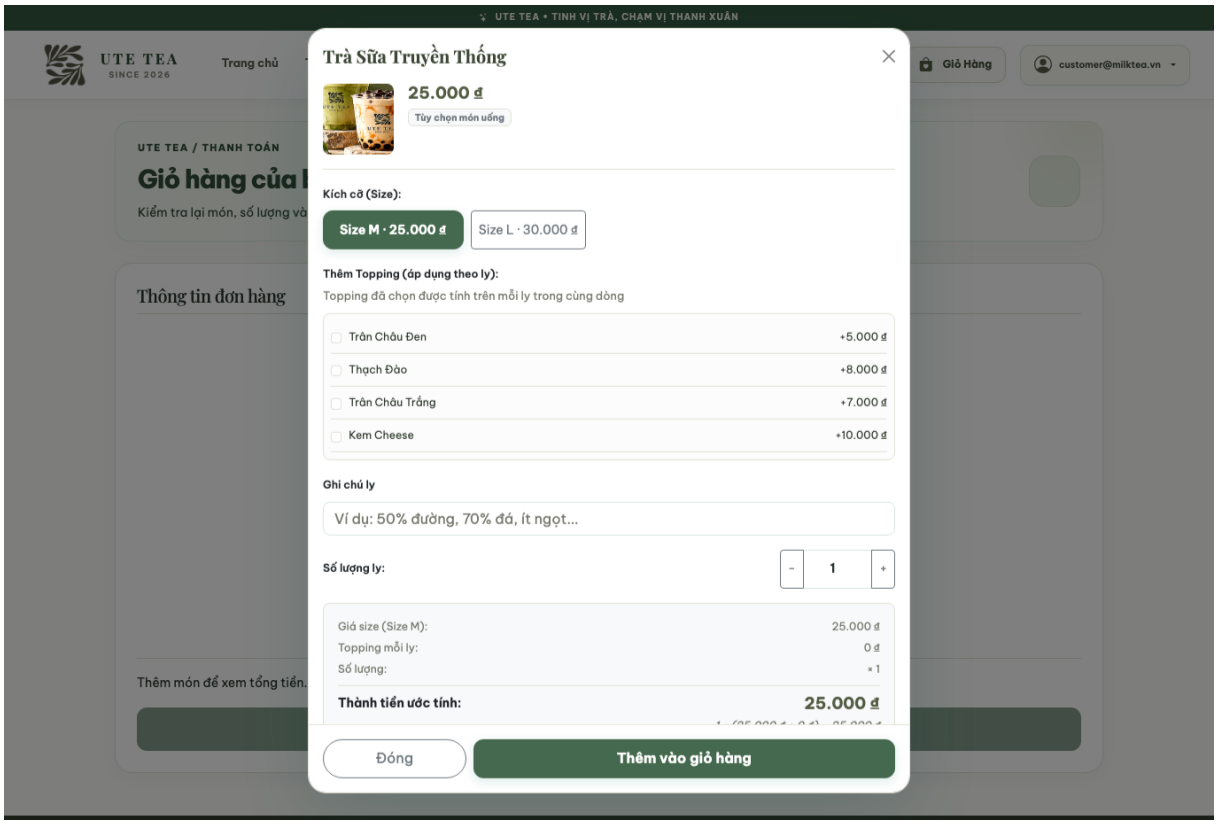
Hình 3.5. Danh sách món và bộ lọc — /marketplace

Bảng 3.5. Mô tả thành phần giao diện danh sách món và bộ lọc

Nội dung	Mô tả
Mục đích	Tra cứu menu dùng chung và các biến thể.
Thành phần chính	Ô tìm kiếm, sắp xếp, lọc giá/danh mục, grid món và phân trang.
Thao tác	Thay bộ lọc; xem đánh giá; thêm món vào giỏ.
Đường dẫn	/marketplace
File giao diện	src/main/resources/templates/customer/marketplace.html; src/main/resources/static/js/customer/marketplace.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.6.Chọn size, topping và ghi chú

Cấu hình món trước khi ghi vào giỏ.



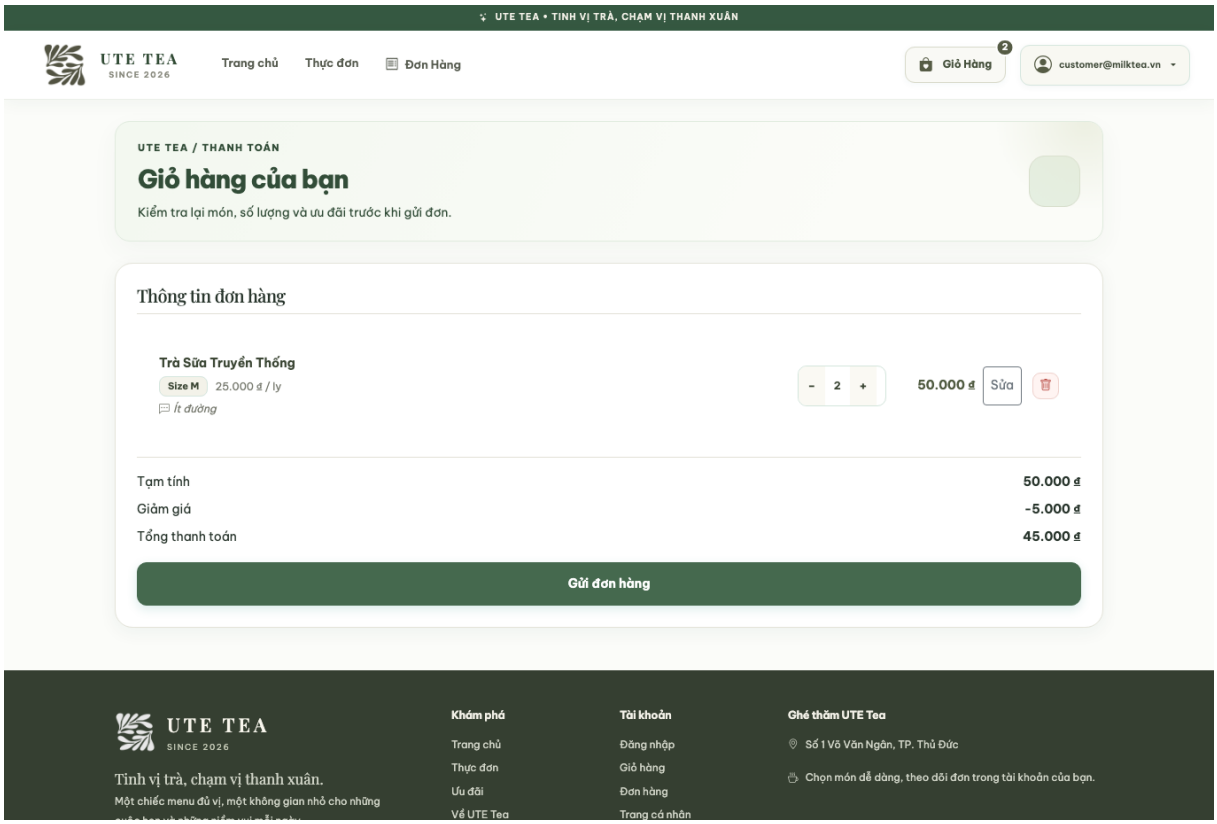
Hình 3.6. Chọn size, topping và ghi chú — /cart

Bảng 3.6. Mô tả thành phần giao diện chọn size, topping và ghi chú

Nội dung	Mô tả
Mục đích	Cấu hình món trước khi ghi vào giỏ.
Thành phần chính	Tên món, size, số lượng, topping được phép và ghi chú.
Thao tác	Chọn tùy chọn, chỉnh số lượng và thêm vào giỏ.
Đường dẫn	/cart
File giao diện	src/main/resources/templates/fragments/product-options.html; src/main/resources/static/js/customer/menu-selection.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.7.Giỏ hàng và kiểm tra tổng tiền

Kiểm tra món và tổng tiền do backend tính.



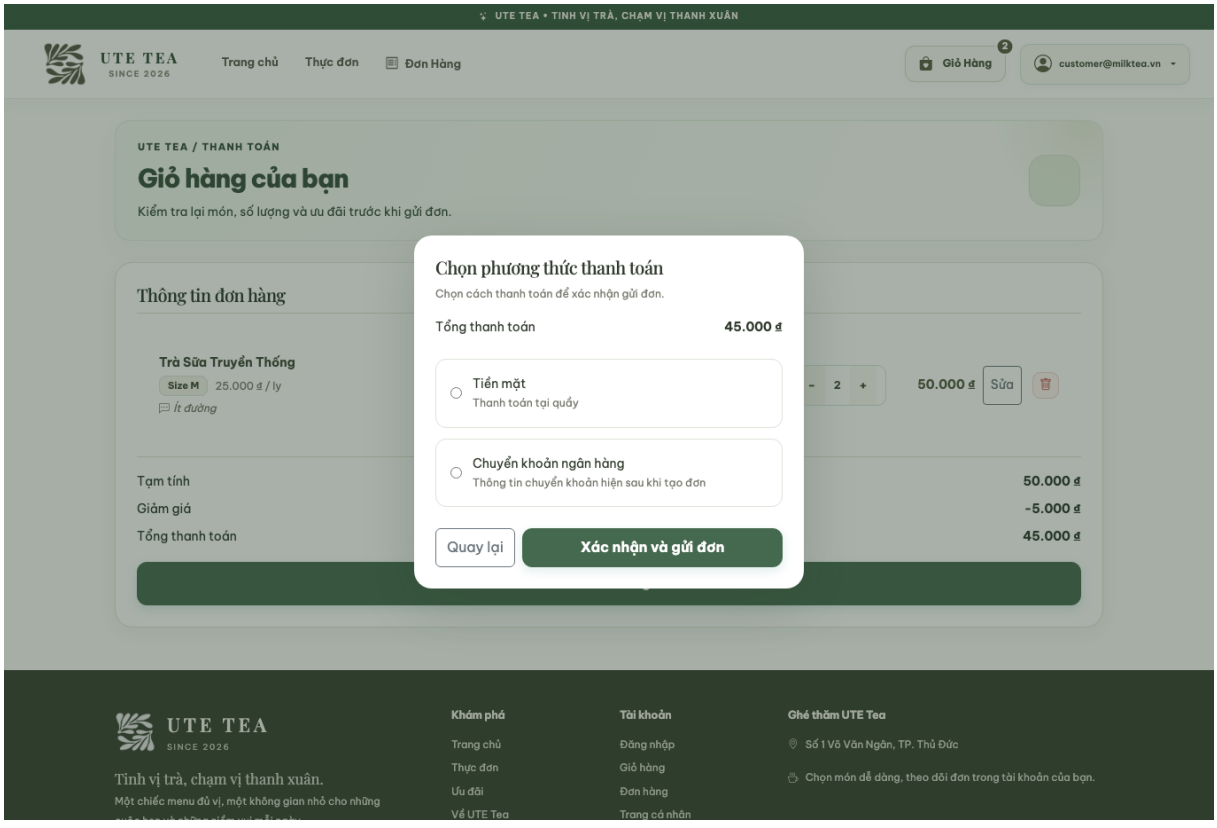
Hình 3.7. Giỏ hàng và kiểm tra tổng tiền — /cart

Bảng 3.7. Mô tả thành phần giao diện giỏ hàng và kiểm tra tổng tiền

Nội dung	Mô tả
Mục đích	Kiểm tra món và tổng tiền do backend tính.
Thành phần chính	Danh sách món, tăng/giảm/xóa/sửa, tạm tính, giảm giá chung và Gửi đơn hàng.
Thao tác	Chỉnh giỏ và bấm Gửi đơn; chưa chọn phương thức ở form chính.
Đường dẫn	/cart
File giao diện	src/main/resources/templates/customer/cart.html; src/main/resources/static/js/customer/delivery-checkout.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026. Checkout hiện tại không có ô chọn hình thức, cơ sở hoặc nhập mã giảm giá.

3.1.8.Hộp chọn phương thức thanh toán

Chọn tiền mặt hoặc chuyển khoản sau khi gửi giỏ.



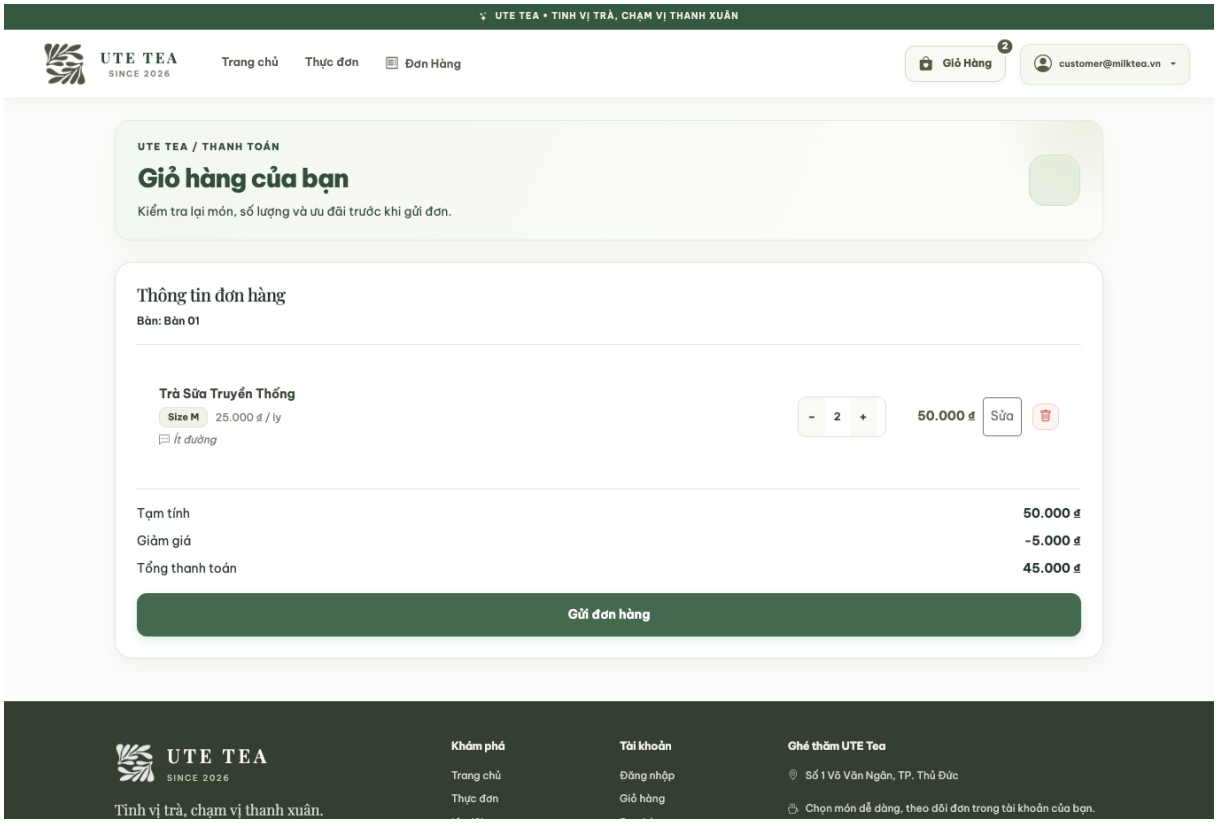
Hình 3.8. Hộp chọn phương thức thanh toán — /cart

Bảng 3.8. Mô tả thành phần giao diện hộp chọn phương thức thanh toán

Nội dung	Mô tả
Mục đích	Chọn tiền mặt hoặc chuyển khoản sau khi gửi giỏ.
Thành phần chính	Dialog, tổng thanh toán, hai phương thức, Quay lại và Xác nhận gửi đơn.
Thao tác	Chọn phương thức và xác nhận; quay lại chưa tạo đơn.
Đường dẫn	/cart
File giao diện	src/main/resources/templates/fragments/delivery-checkout.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.9.Checkout theo bàn đã quét QR

Tự xác định ngữ cảnh bàn hợp lệ.



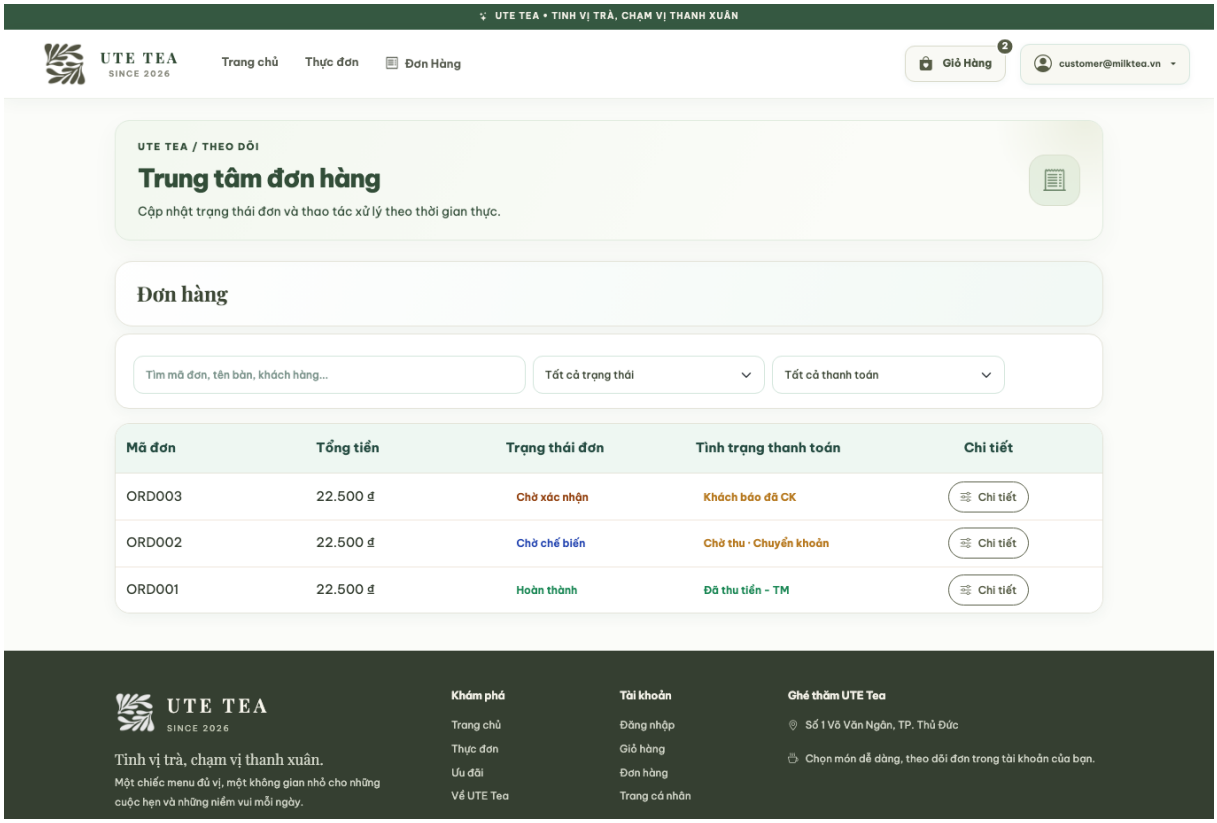
Hình 3.9. Checkout theo bàn đã quét QR — /cart

Bảng 3.9. Mô tả thành phần giao diện checkout theo bàn đã quét qr

Nội dung	Mô tả
Mục đích	Tự xác định ngữ cảnh bàn hợp lệ.
Thành phần chính	Tên bàn, giỏ món, tổng tiền và thao tác gửi đơn.
Thao tác	Sửa giỏ và gửi đơn theo đúng bàn.
Đường dẫn	/cart
File giao diện	src/main/resources/templates/customer/cart.html; src/main/resources/static/js/customer/customer.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026. Ảnh không hiển thị URL hay chuỗi QR/token.

3.1.10.Lịch sử đơn của khách

Theo dõi các đơn của tài khoản đăng nhập.



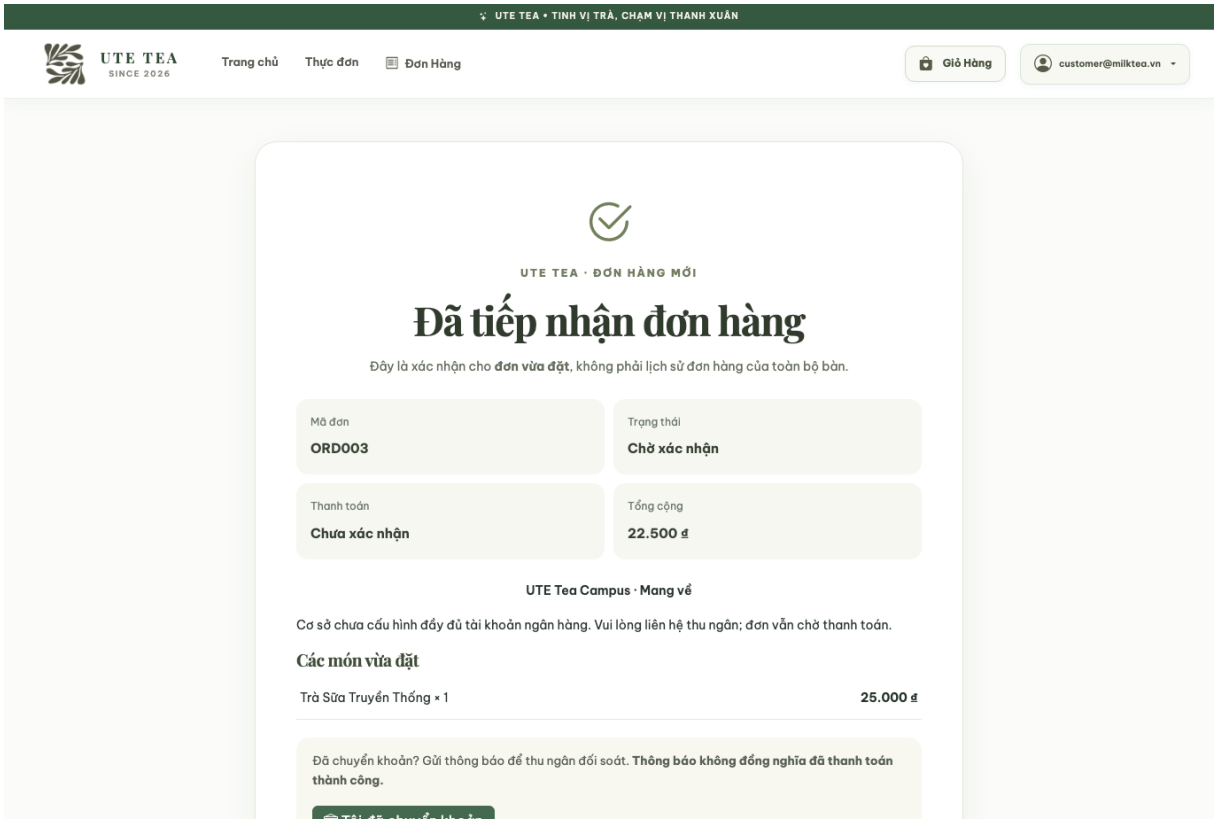
Hình 3.10. Lịch sử đơn của khách — /orders

Bảng 3.10. Mô tả thành phần giao diện lịch sử đơn của khách

Nội dung	Mô tả
Mục đích	Theo dõi các đơn của tài khoản đăng nhập.
Thành phần chính	Mã đơn, món, trạng thái, tiền, các thao tác theo điều kiện.
Thao tác	Xem chi tiết, theo dõi, yêu cầu hủy/đánh giá khi đủ điều kiện.
Đường dẫn	/orders
File giao diện	src/main/resources/templates/customer/orders.html; src/main/resources/static/js/order/orders.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.11.Xác nhận đơn và thông tin chuyển khoản

Xem chứng từ riêng của đơn đã tạo.



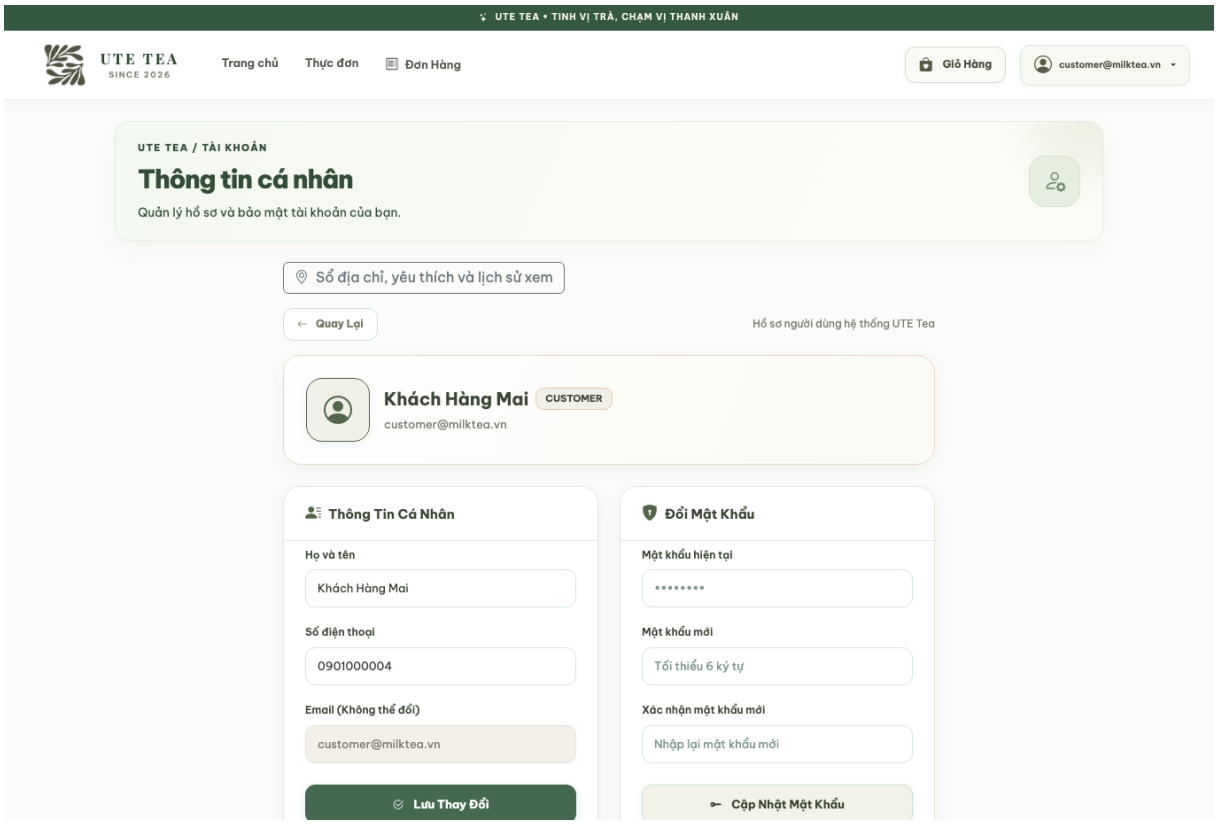
Hình 3.11. Xác nhận đơn và thông tin chuyển khoản — /order-confirmation?order=3

Bảng 3.11. Mô tả thành phần giao diện xác nhận đơn và thông tin chuyển khoản

Nội dung	Mô tả
Mục đích	Xem chứng từ riêng của đơn đã tạo.
Thành phần chính	Mã đơn, trạng thái, tổng tiền, món và thông tin thanh toán.
Thao tác	Thông báo chuyển khoản để thu ngân kiểm tra; không tự làm PAID.
Đường dẫn	/order-confirmation?order=3
File giao diện	src/main/resources/templates/customer/order-confirmation.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.12.Thông tin tài khoản

Quản lý hồ sơ cá nhân.



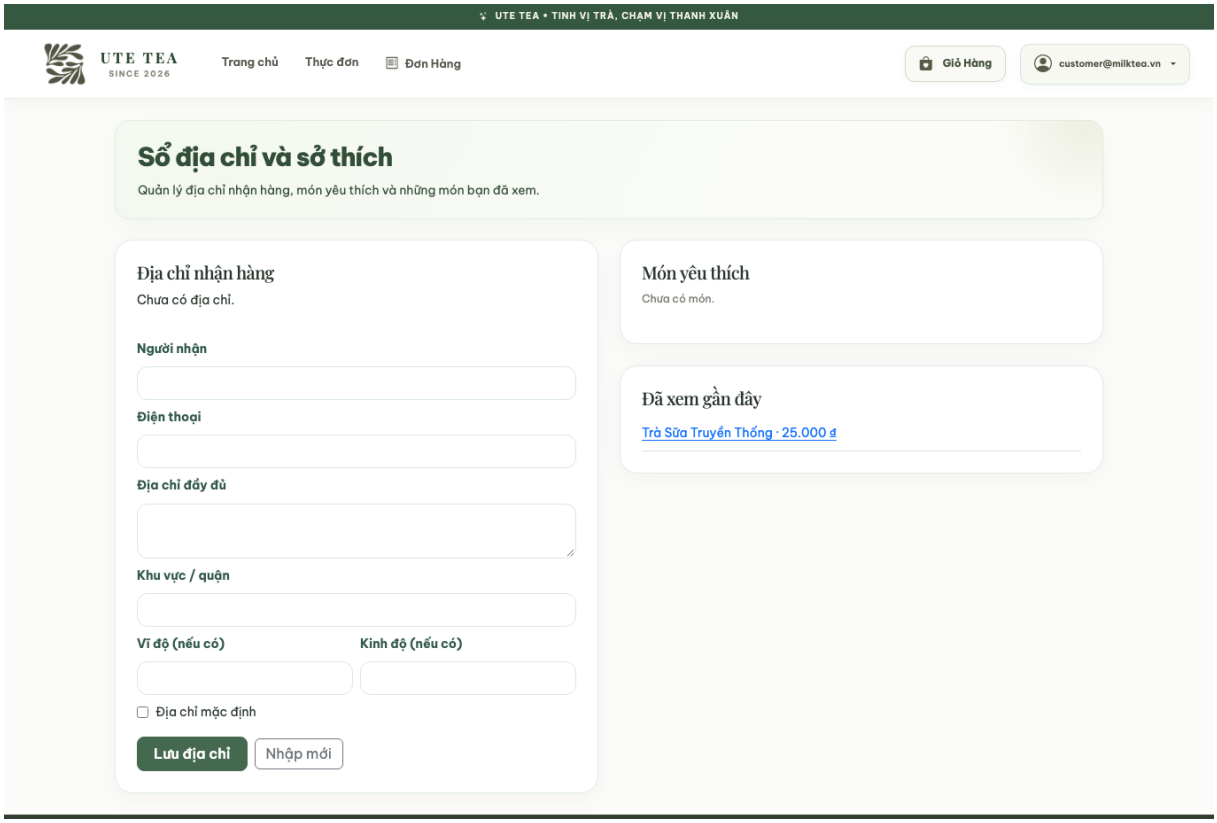
Hình 3.12. Thông tin tài khoản — /profile

Bảng 3.12. Mô tả thành phần giao diện thông tin tài khoản

Nội dung	Mô tả
Mục đích	Quản lý hồ sơ cá nhân.
Thành phần chính	Họ tên, email, điện thoại, avatar, đổi mật khẩu.
Thao tác	Sửa hồ sơ và gửi cập nhật; sử dụng chức năng mật khẩu tương ứng.
Đường dẫn	/profile
File giao diện	src/main/resources/templates/profile.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.1.13.Yêu thích, món đã xem và địa chỉ lưu

Tra cứu các dữ liệu bổ trợ theo tài khoản.



Hình 3.13. Yêu thích, món đã xem và địa chỉ lưu — /api/v1/account/preferences

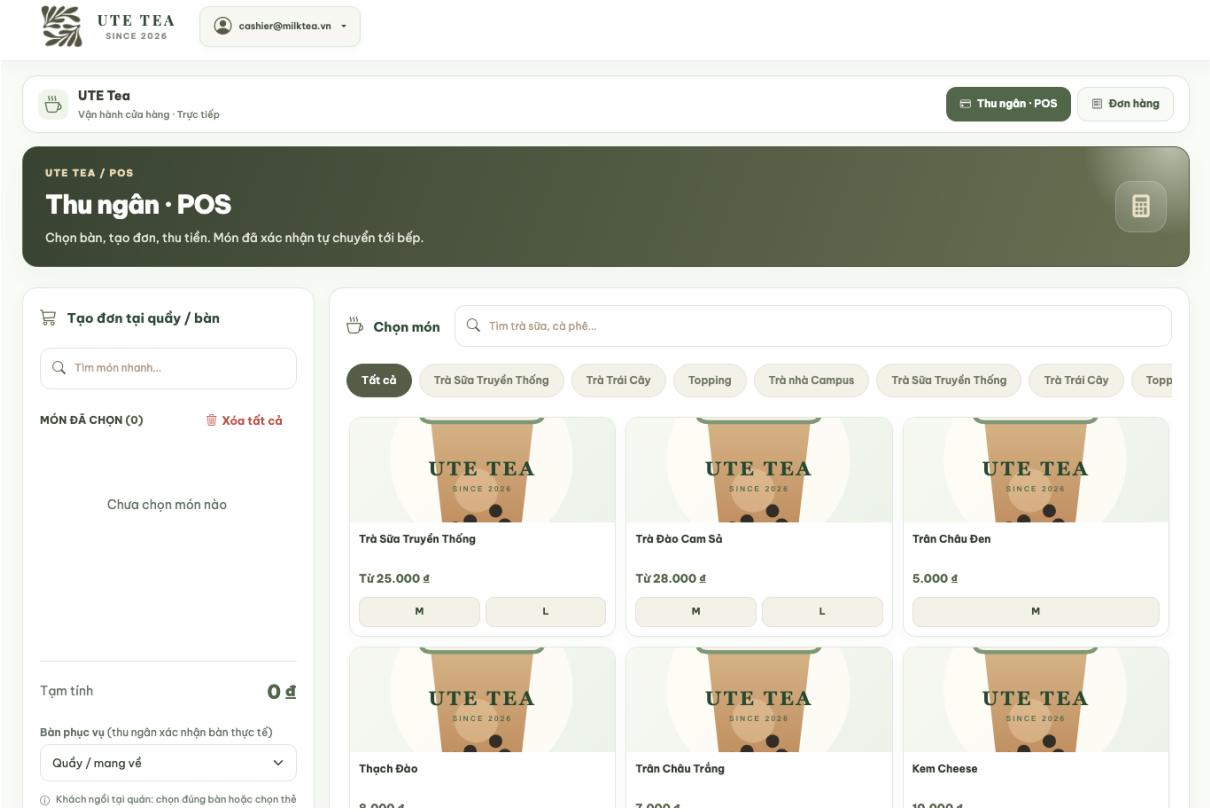
Bảng 3.13. Mô tả thành phần giao diện yêu thích, món đã xem và địa chỉ lưu

Nội dung	Mô tả
Mục đích	Tra cứu các dữ liệu bổ trợ theo tài khoản.
Thành phần chính	Các nhóm yêu thích, lịch sử xem và địa chỉ đã lưu.
Thao tác	Bật/tắt yêu thích; xem món; sửa/xóa địa chỉ.
Đường dẫn	/api/v1/account/preferences
File giao diện	src/main/resources/templates/customer/account-data.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026. Địa chỉ là dữ liệu tương thích; không bắt buộc trong checkout mới.

3.2.Giao diện thu ngân

3.2.1.Bảng điều khiển thu ngân

Tiếp nhận và xử lý đơn theo cơ sở được phân công.



Hình 3.14. Bảng điều khiển thu ngân — /cashier

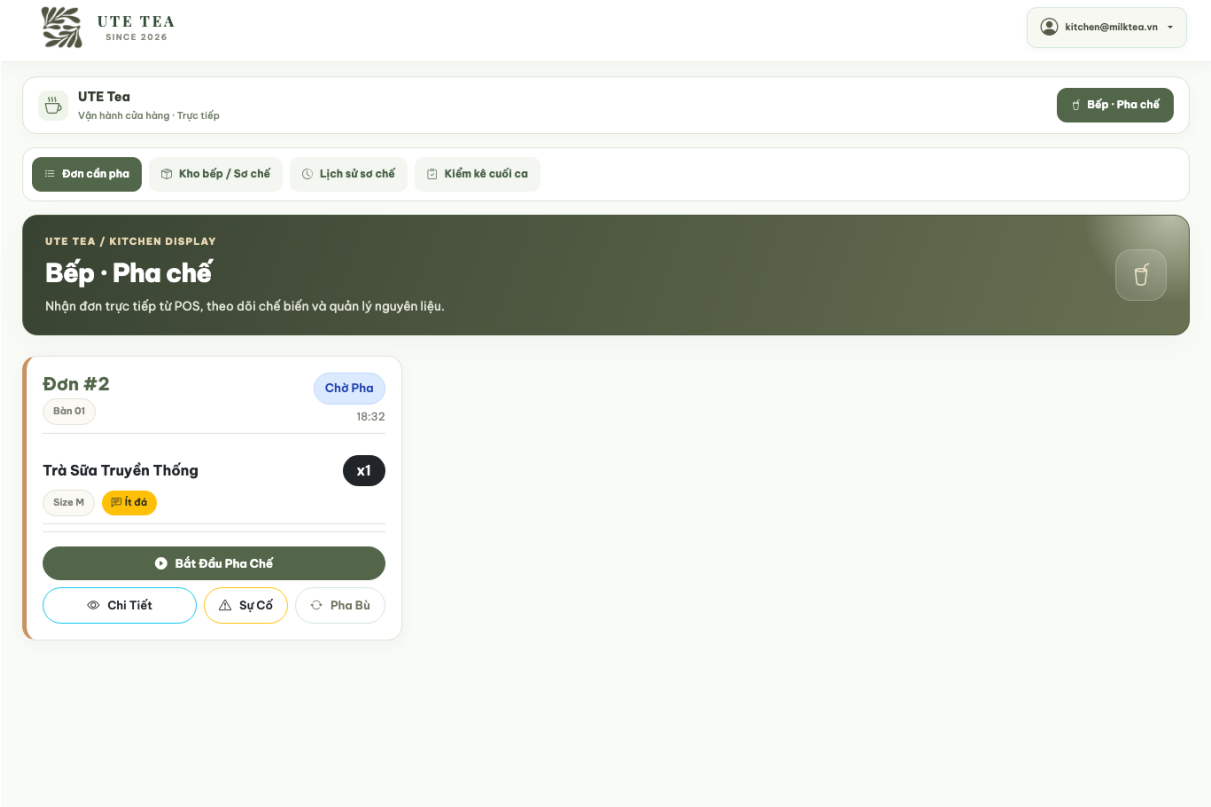
Bảng 3.14. Mô tả thành phần giao diện bảng điều khiển thu ngân

Nội dung	Mô tả
Mục đích	Tiếp nhận và xử lý đơn theo cơ sở được phân công.
Thành phần chính	Các tab công việc, mã đơn, phương thức/trạng thái tiền, bàn và hành động.
Thao tác	Xác nhận khách, ghi thu, xem chứng từ và xử lý yêu cầu.
Đường dẫn	/cashier
File giao diện	src/main/resources/templates/cashier/index.html; src/main/resources/static/js/cashier/cashier.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.3.Giao diện nhân viên bếp

3.3.1.Hàng đợi chế biến

Thực hiện các đơn được điều phối vào bếp.



Hình 3.15. Hàng đợi chế biến — /kitchen

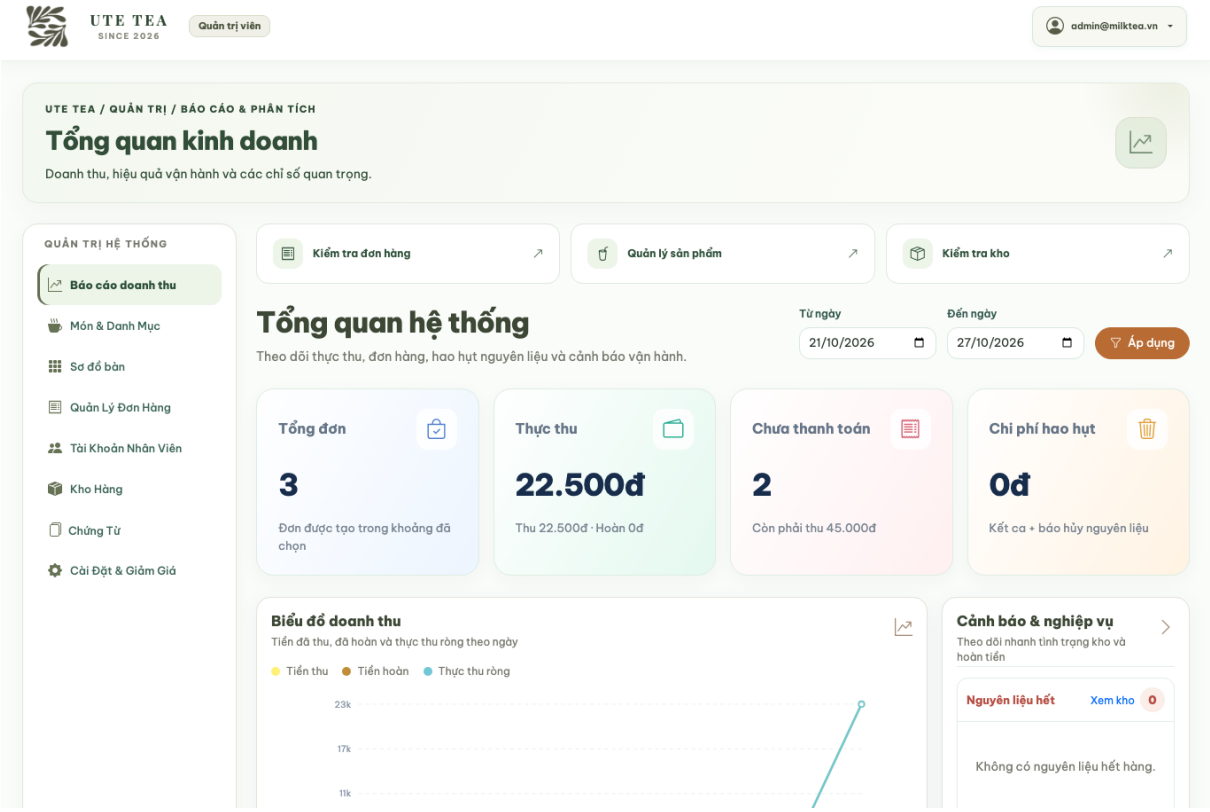
Bảng 3.15. Mô tả thành phần giao diện hàng đợi chế biến

Nội dung	Mô tả
Mục đích	Thực hiện các đơn được điều phối vào bếp.
Thành phần chính	Mã đơn, bàn, món, trạng thái và nút bắt đầu/hoàn tất.
Thao tác	Bắt đầu, hoàn tất, xem hoặc ghi sự cố theo UI.
Đường dẫn	/kitchen
File giao diện	src/main/resources/templates/kitchen/orders.html; src/main/resources/static/js/kitchen/kitchen.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.4.Giao diện quản trị viên

3.4.1.Dashboard và báo cáo

Tổng hợp dữ liệu kinh doanh thử nghiệm.



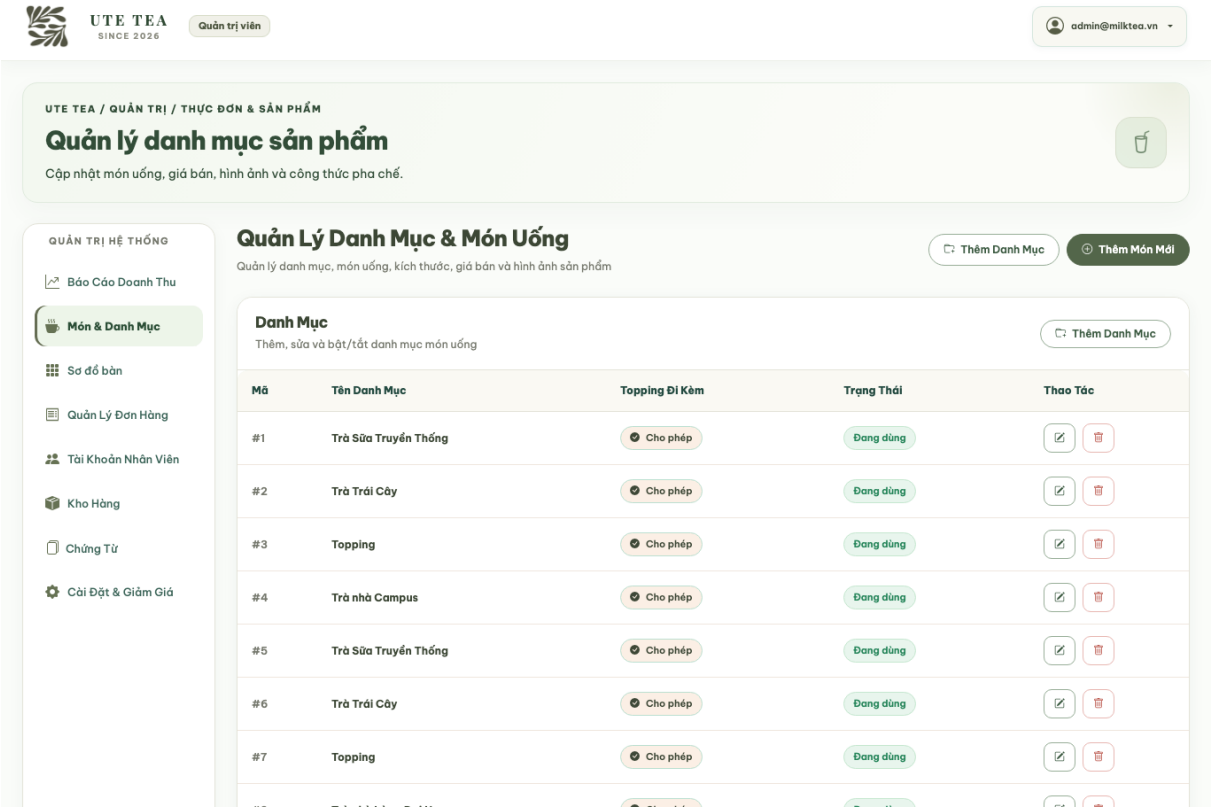
Hình 3.16. Dashboard và báo cáo — /admin/reports

Bảng 3.16. Mô tả thành phần giao diện dashboard và báo cáo

Nội dung	Mô tả
Mục đích	Tổng hợp dữ liệu kinh doanh thử nghiệm.
Thành phần chính	Bộ lọc ngày, KPI, biểu đồ, top sản phẩm và cảnh báo.
Thao tác	Lọc thời gian, xem số liệu và các nhóm thống kê.
Đường dẫn	/admin/reports
File giao diện	src/main/resources/templates/admin/reports.html; src/main/resources/static/js/admin/dashboard.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.4.2.Quản lý danh mục và món

Quản lý menu, giá và trạng thái phục vụ.



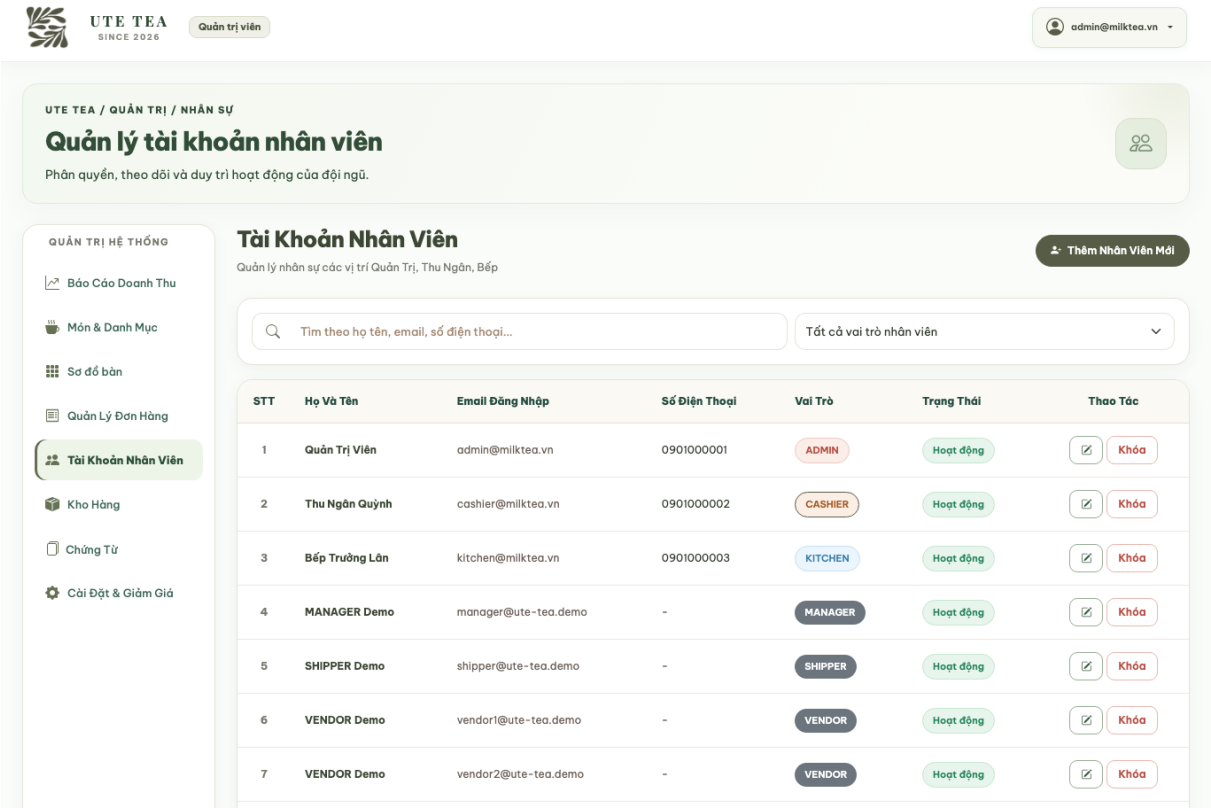
Hình 3.17. Quản lý danh mục và món — /admin/catalog

Bảng 3.17. Mô tả thành phần giao diện quản lý danh mục và món

Nội dung	Mô tả
Mục đích	Quản lý menu, giá và trạng thái phục vụ.
Thành phần chính	Bảng danh mục/món, bộ lọc, nút tạo/sửa và cấu hình công thức.
Thao tác	Tra cứu, tạo/sửa món hoặc mở cấu hình.
Đường dẫn	/admin/catalog
File giao diện	src/main/resources/templates/admin/catalog.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.4.3.Quản lý nhân viên

Quản lý tài khoản vận hành.



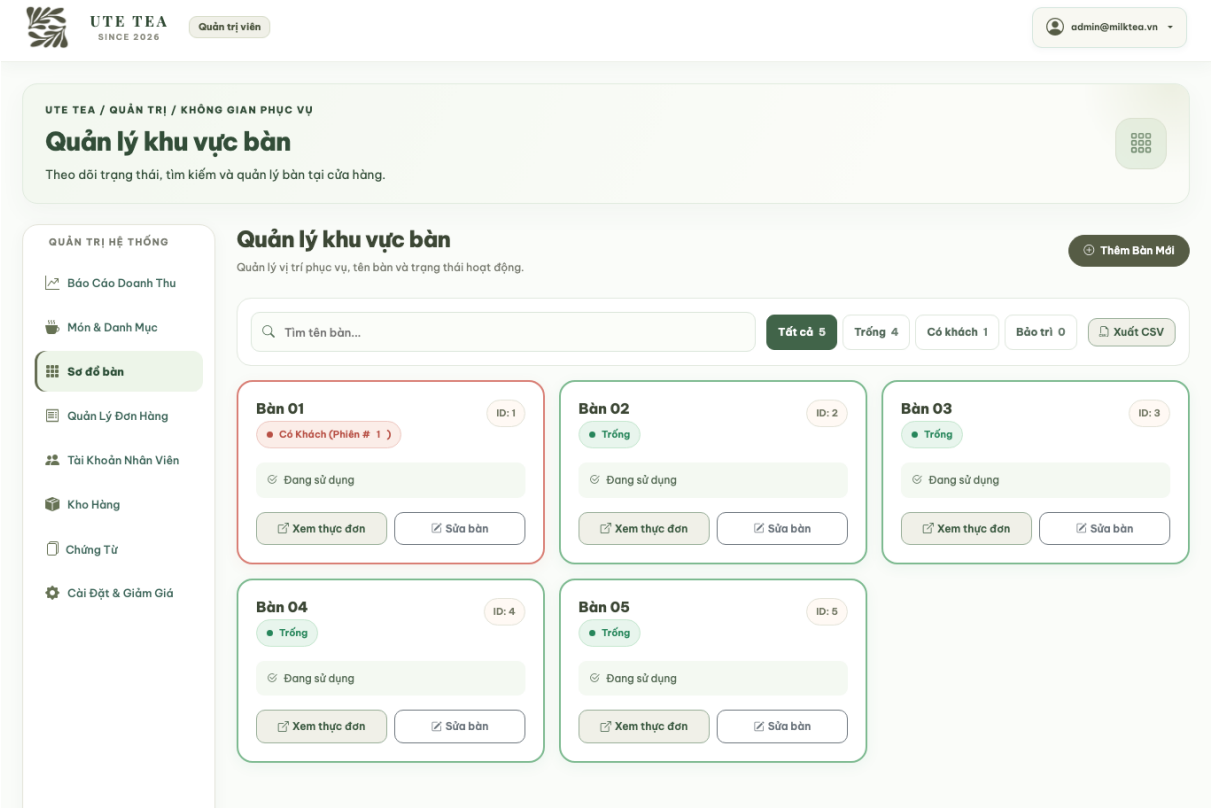
Hình 3.18. Quản lý nhân viên — /admin/staff

Bảng 3.18. Mô tả thành phần giao diện quản lý nhân viên

Nội dung	Mô tả
Mục đích	Quản lý tài khoản vận hành.
Thành phần chính	Danh sách nhân viên, thông tin vai trò và trạng thái.
Thao tác	Tạo/sửa, khóa/mở và thao tác được quyền.
Đường dẫn	/admin/staff
File giao diện	src/main/resources/templates/admin/staff.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.4.4.Quản lý bàn

Duy trì bàn và lượt phục vụ.



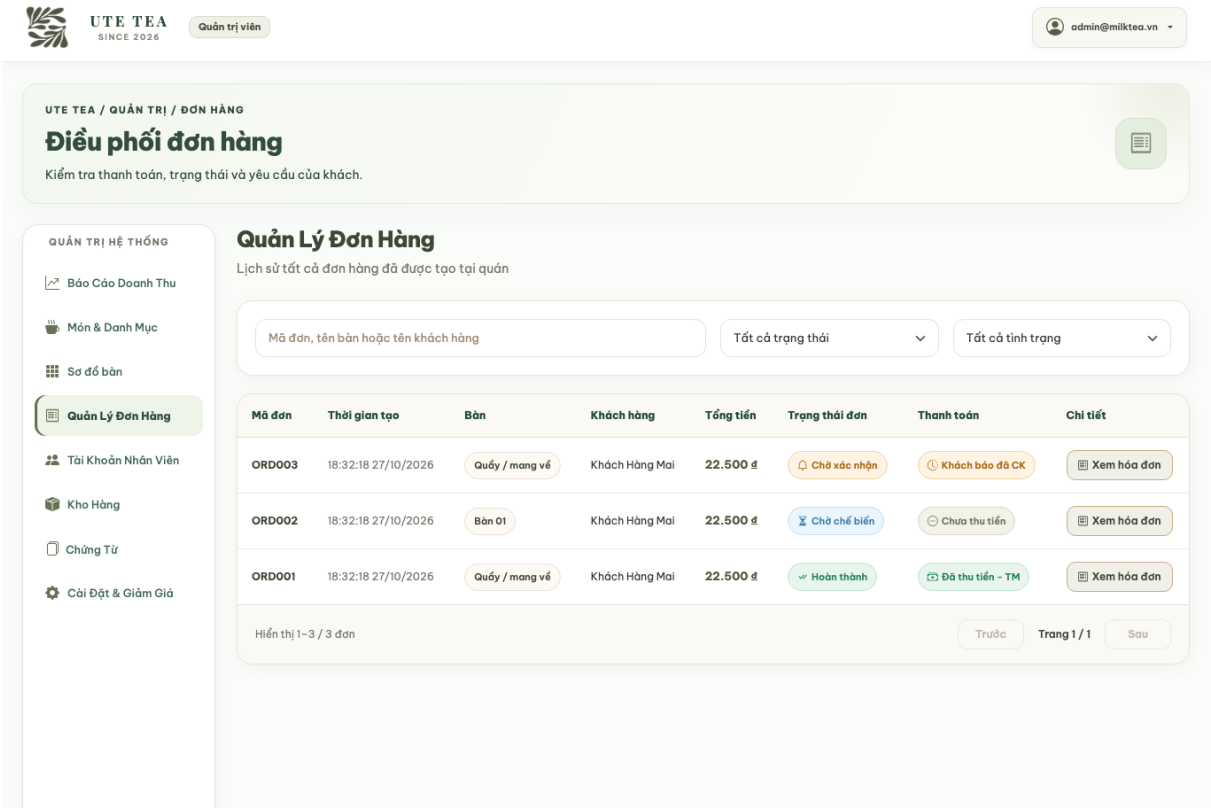
Hình 3.19. Quản lý bàn — /admin/tables

Bảng 3.19. Mô tả thành phần giao diện quản lý bàn

Nội dung	Mô tả
Mục đích	Duy trì bàn và lượt phục vụ.
Thành phần chính	Tên bàn, trạng thái, phiên hiện tại và nút thao tác.
Thao tác	Thêm/sửa bàn; các chức năng QR thuộc quản trị.
Đường dẫn	/admin/tables
File giao diện	src/main/resources/templates/admin/tables.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.4.5.Quản lý đơn và thu tiền

Theo dõi đơn toàn phạm vi quản trị.



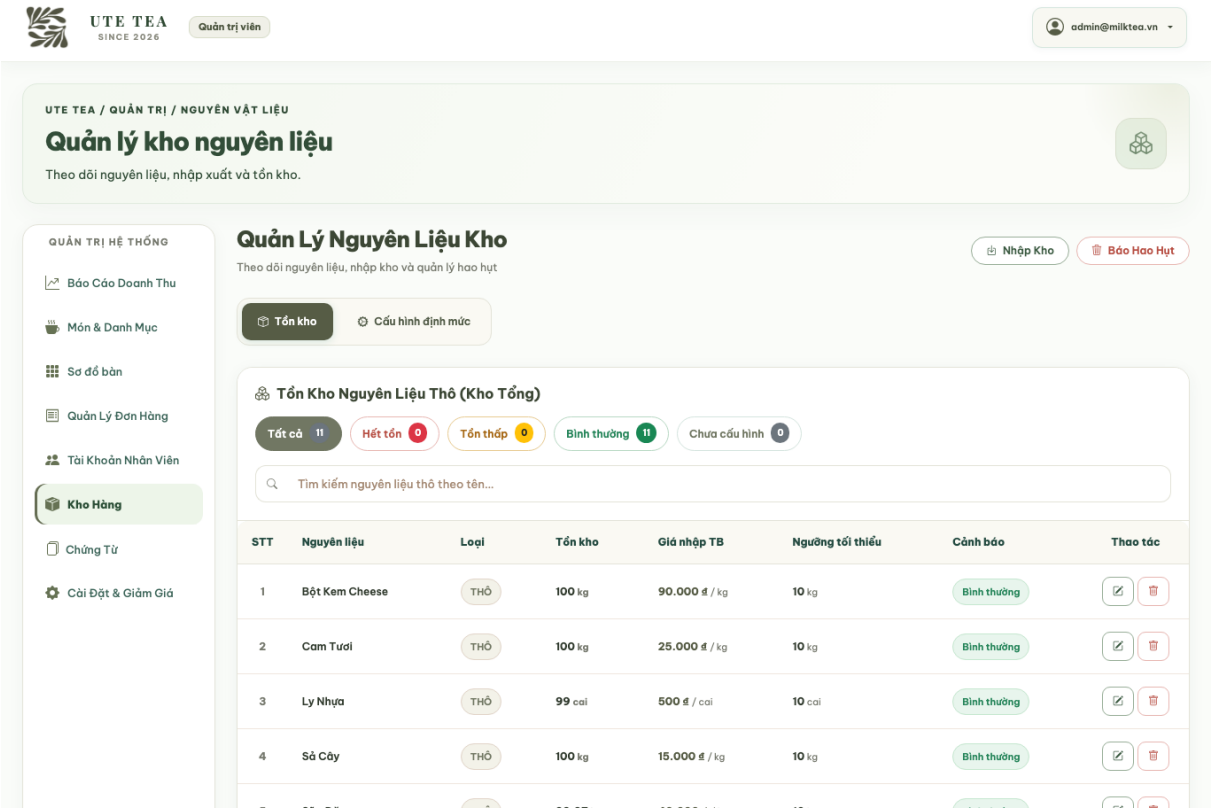
Hình 3.20. Quản lý đơn và thu tiền — /admin/orders

Bảng 3.20. Mô tả thành phần giao diện quản lý đơn và thu tiền

Nội dung	Mô tả
Mục đích	Theo dõi đơn toàn phạm vi quản trị.
Thành phần chính	Đơn, trạng thái, hóa đơn và các thao tác theo điều kiện.
Thao tác	Lọc, xem chi tiết, xử lý hủy/hoàn theo quyền.
Đường dẫn	/admin/orders
File giao diện	src/main/resources/templates/admin/orders.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.4.6.Nguyên liệu và tồn kho

Kiểm soát nguyên liệu tại KHO/BEP.



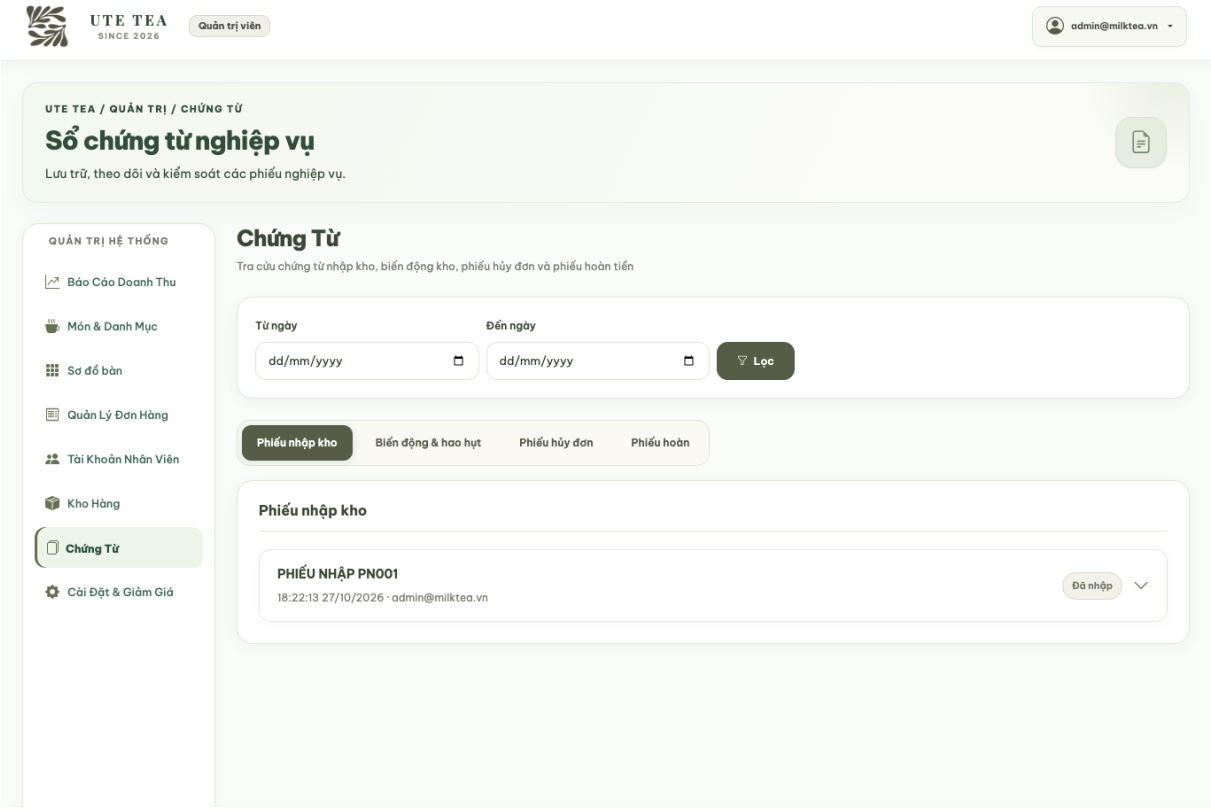
Hình 3.21. Nguyên liệu và tồn kho — /admin/inventory

Bảng 3.21. Mô tả thành phần giao diện nguyên liệu và tồn kho

Nội dung	Mô tả
Mục đích	Kiểm soát nguyên liệu tại KHO/BEP.
Thành phần chính	Danh sách tồn, ngưỡng, định mức, giá vốn và tác vụ kho.
Thao tác	Lọc kho, nhập, điều chỉnh hoặc xem công thức.
Đường dẫn	/admin/inventory
File giao diện	src/main/resources/templates/admin/inventory.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.4.7.Chứng từ vận hành kho

Tra cứu nhập, sơ chế và kiểm kê/kết ca.



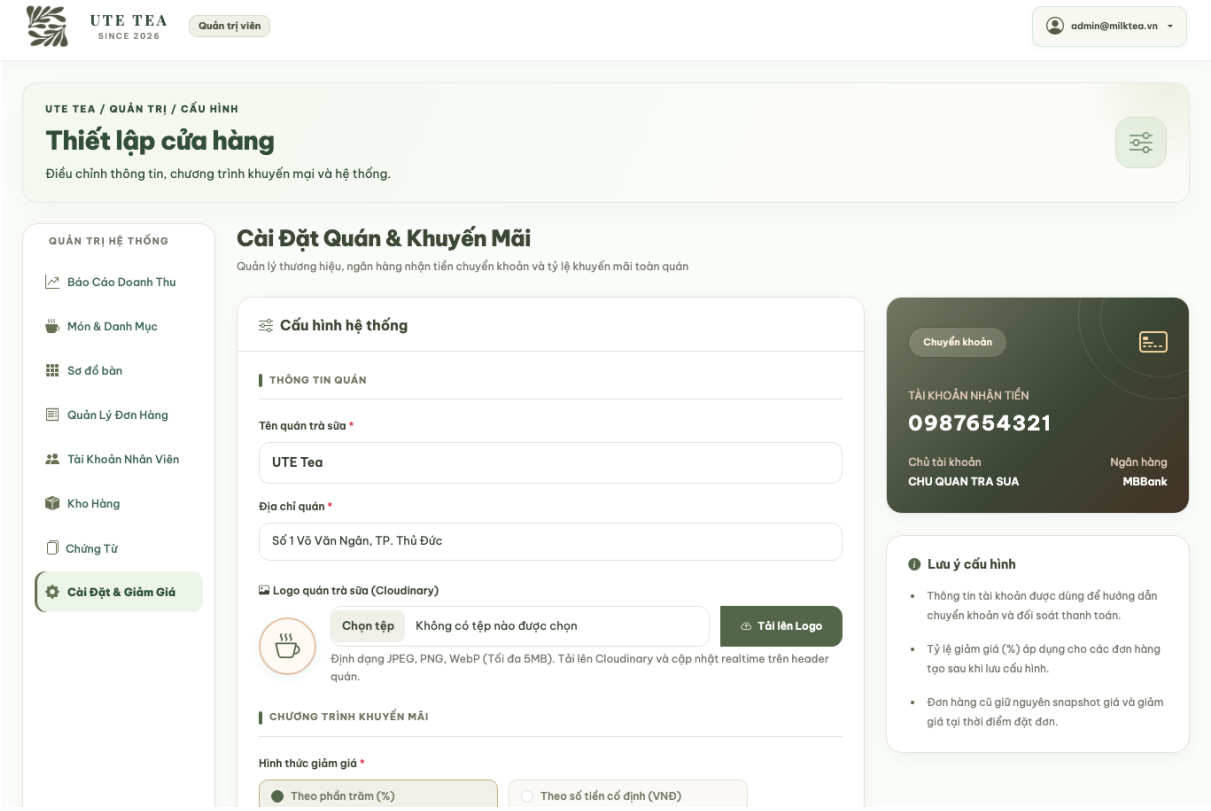
Hình 3.22. Chứng từ vận hành kho — /admin/documents

Bảng 3.22. Mô tả thành phần giao diện chứng từ vận hành kho

Nội dung	Mô tả
Mục đích	Tra cứu nhập, sơ chế và kiểm kê/kết ca.
Thành phần chính	Các nhóm chứng từ, bộ lọc ngày và dữ liệu chi tiết.
Thao tác	Đổi nhóm, lọc thời gian và xem phiếu.
Đường dẫn	/admin/documents
File giao diện	src/main/resources/templates/admin/documents.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.4.8.Cài đặt hệ thống

Quản lý thương hiệu và chính sách chung.



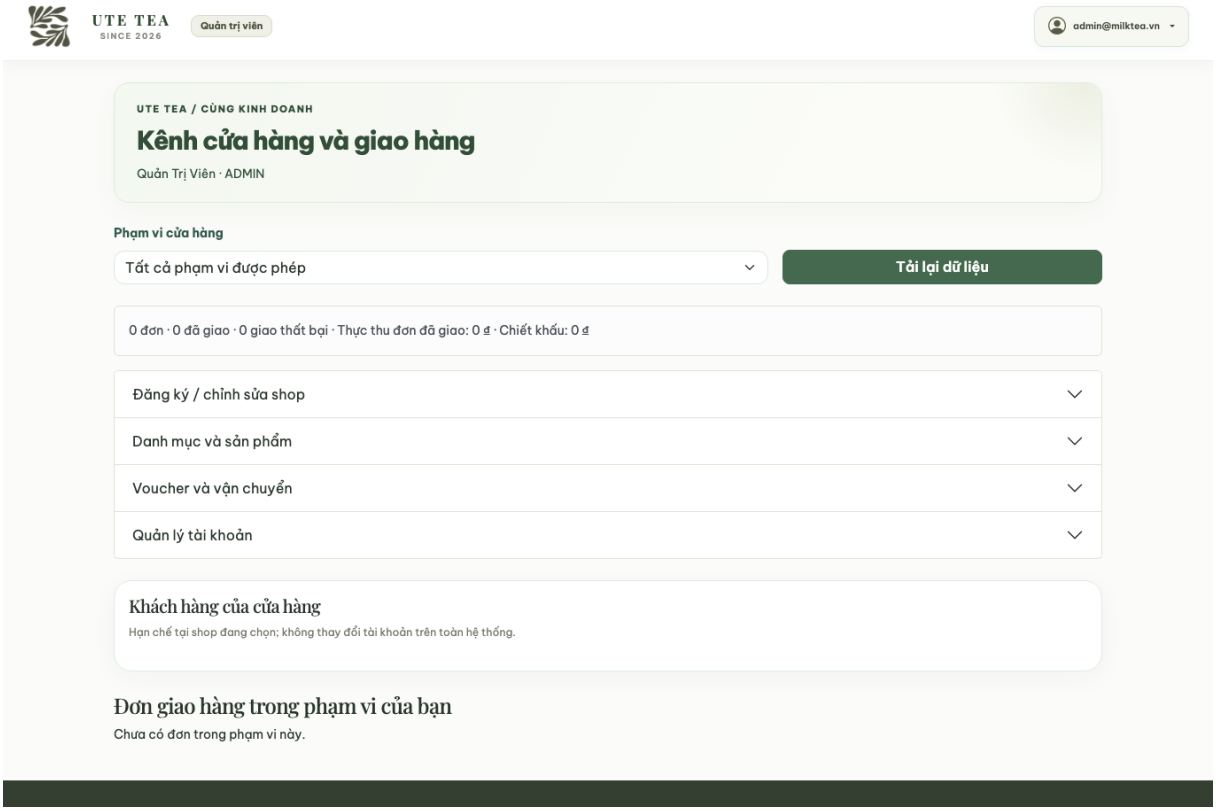
Hình 3.23. Cài đặt hệ thống — /admin/settings

Bảng 3.23. Mô tả thành phần giao diện cài đặt hệ thống

Nội dung	Mô tả
Mục đích	Quản lý thương hiệu và chính sách chung.
Thành phần chính	Tên/địa chỉ, logo, cấu hình giảm giá và tài khoản nhận tiền.
Thao tác	Sửa và lưu cấu hình; upload logo khi dịch vụ hợp lệ.
Đường dẫn	/admin/settings
File giao diện	src/main/resources/templates/admin/settings.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.4.9.Quản lý cơ sở và phân công

Quản lý module cơ sở và dữ liệu mở rộng.



Hình 3.24. Quản lý cơ sở và phân công — /merchant

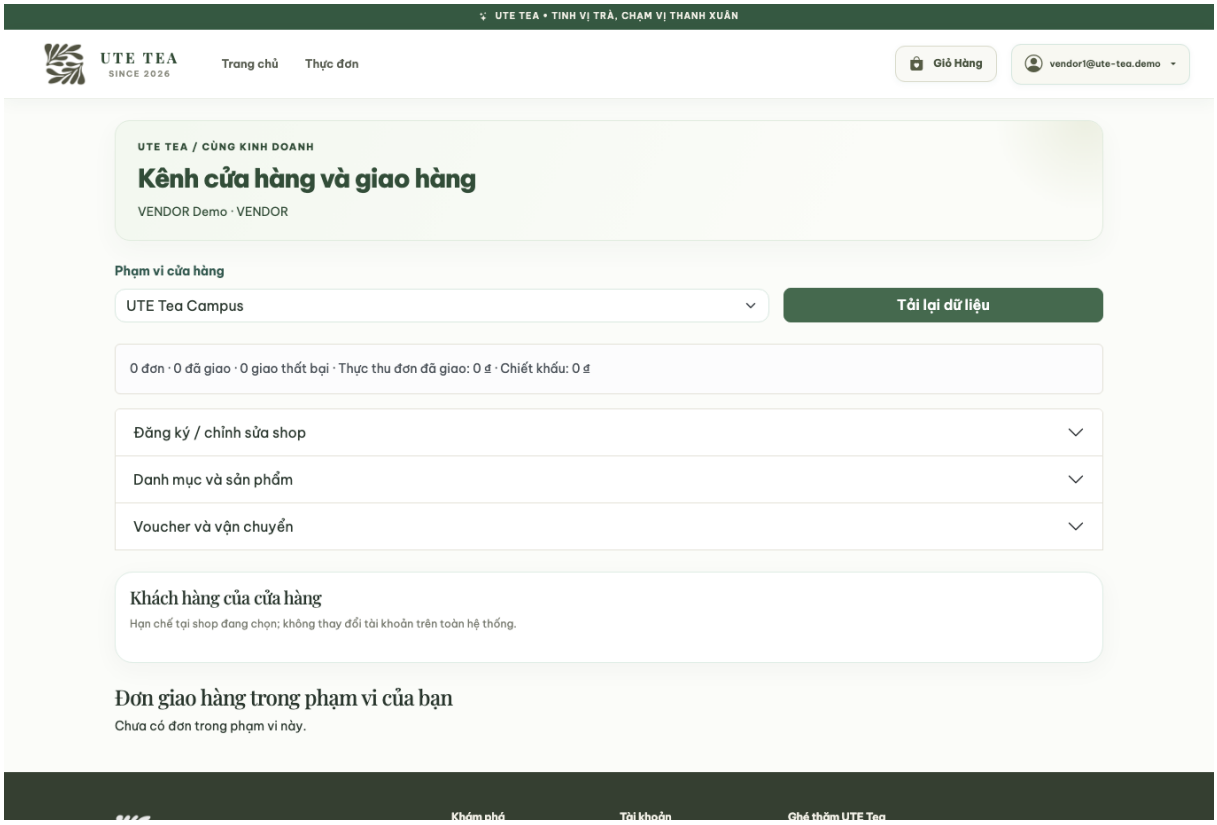
Bảng 3.24. Mô tả thành phần giao diện quản lý cơ sở và phân công

Nội dung	Mô tả
Mục đích	Quản lý module cơ sở và dữ liệu mở rộng.
Thành phần chính	Chọn Shop, cấu hình, tài khoản, phân công và nhóm nghiệp vụ cũ.
Thao tác	Tạo/sửa cơ sở, gán quản lý/thu ngân/bếp và quản lý dữ liệu trong scope.
Đường dẫn	/merchant
File giao diện	src/main/resources/templates/market/portal.html
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026.

3.5.Giao diện vai trò cơ sở và phân hệ mở rộng

3.5.1.Cổng nghiệp vụ VENDOR

Hiển thị đúng phần chức năng cho vai trò mở rộng hiện có.



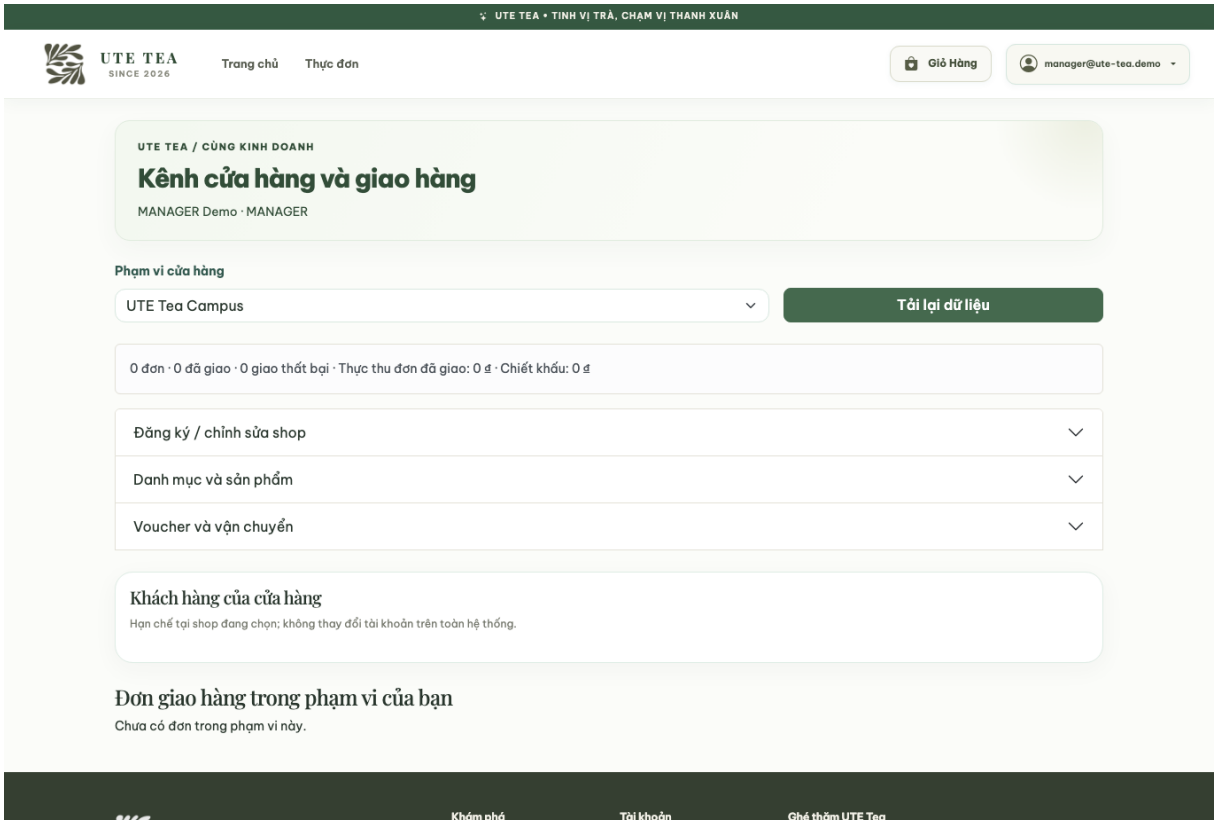
Hình 3.25. Cổng nghiệp vụ VENDOR — /merchant

Bảng 3.25. Mô tả thành phần giao diện cổng nghiệp vụ vendor

Nội dung	Mô tả
Mục đích	Hiển thị đúng phần chức năng cho vai trò mở rộng hiện có.
Thành phần chính	Các nhóm công việc có data-roles; phạm vi Shop/đơn theo service.
Thao tác	Thao tác trong scope được phân công; dữ liệu giao hàng là module cũ.
Đường dẫn	/merchant
File giao diện	src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026. Giao diện có thật; không đồng nghĩa checkout hiện tại tạo mới đơn giao hàng.

3.5.2.Cổng nghiệp vụ MANAGER

Hiển thị đúng phần chức năng cho vai trò mở rộng hiện có.



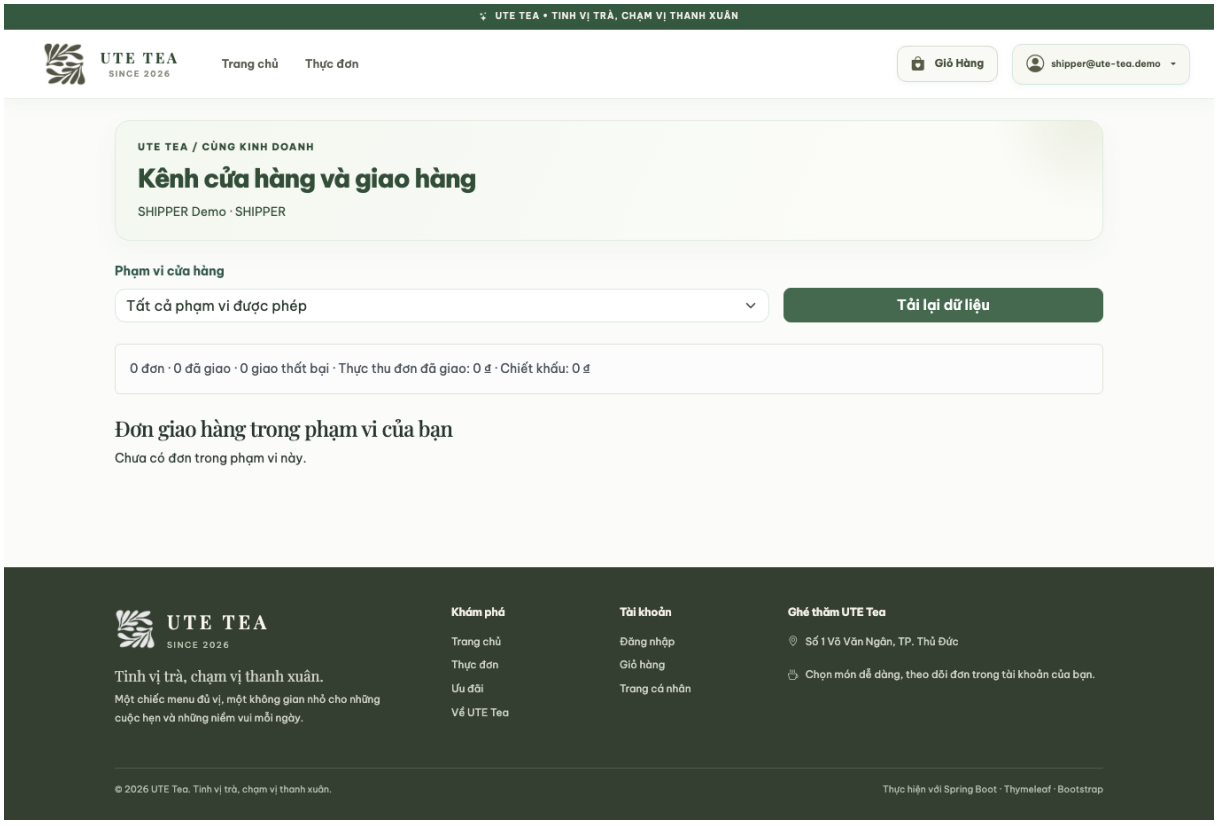
Hình 3.26. Cổng nghiệp vụ MANAGER — /merchant

Bảng 3.26. Mô tả thành phần giao diện cổng nghiệp vụ manager

Nội dung	Mô tả
Mục đích	Hiển thị đúng phần chức năng cho vai trò mở rộng hiện có.
Thành phần chính	Các nhóm công việc có data-roles; phạm vi Shop/đơn theo service.
Thao tác	Thao tác trong scope được phân công; dữ liệu giao hàng là module cũ.
Đường dẫn	/merchant
File giao diện	src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026. Giao diện có thật; không đồng nghĩa checkout hiện tại tạo mới đơn giao hàng.

3.5.3.Cổng nghiệp vụ SHIPPER

Hiển thị đúng phần chức năng cho vai trò mở rộng hiện có.



Hình 3.27. Cổng nghiệp vụ SHIPPER — /merchant

Bảng 3.27. Mô tả thành phần giao diện cổng nghiệp vụ shipper

Nội dung	Mô tả
Mục đích	Hiển thị đúng phần chức năng cho vai trò mở rộng hiện có.
Thành phần chính	Các nhóm công việc có data-roles; phạm vi Shop/đơn theo service.
Thao tác	Thao tác trong scope được phân công; dữ liệu giao hàng là module cũ.
Đường dẫn	/merchant
File giao diện	src/main/resources/templates/market/portal.html; src/main/resources/static/js/market/portal.js
Bằng chứng	Ảnh chụp Chromium từ demo H2; ngày 27/10/2026. Giao diện có thật; không đồng nghĩa checkout hiện tại tạo mới đơn giao hàng.

3.6.Đánh giá mức độ hoàn thiện và nội dung cần bổ sung

Báo cáo phản ánh các màn hình đã mở được và các chức năng được xác minh từ mã. Chức năng ảnh/video, ngân hàng, SMTP và VNPAY cần cấu hình dịch vụ ngoài; không xác nhận hoạt động thực tế chỉ từ việc có nút trên UI. Quyền cơ sở của POS được thực hiện ở một số service chính, trong khi những module cũ cần tiếp tục rà soát để đồng nhất. Các hạn chế này là nhận xét kỹ thuật, không được thay bằng kết quả hoàn thành giả định.

Bảng 3.28. Nội dung chưa đủ bằng chứng hoặc cần xác nhận

Nội dung	Trạng thái	Việc cần làm
Thông tin hành chính	[CẦN BỔ SUNG]	Xác nhận trường/khoa/logo, học phần, giảng viên, lớp, họ tên/ MSSV và địa điểm.
SQL Server thật	Chưa kiểm chứng trong phiên báo cáo	Áp dụng migration an toàn và kiểm tra ở môi trường triển khai.
SMTP/Gmail thật	Không sử dụng credentials thật trong phiên chụp ảnh	Cấu hình và thử gửi thư tại môi trường do người dùng quản lý.
Ngân hàng/VietQR	Có code cấu hình; chưa đối soát thật	Điền thông tin hợp lệ và thu ngân kiểm tra giao dịch; không tự gán PAID.
Cloudinary/VNPAY	Có code; chưa xác minh dịch vụ thật	Kiểm chứng với tài khoản dịch vụ phù hợp, không đưa secret vào báo cáo.
Các chức năng không có source	Không nhận là đã triển khai	Không thêm module Brand riêng, cổng mô phỏng, kiểm duyệt/phản hồi review hay route chi tiết sản phẩm giả.
Mục lục và caption	Có trường Word tự động	Cập nhật toàn bộ trường và kiểm tra trang trước khi in/nộp.

Danh sách đối chiếu chi tiết nằm trong reports/Bang_Doi_Chieu_Chuc_Nang.md. Tài liệu ghi rõ giới hạn kiểm tra và các mục còn thiếu tại reports/ Bao_Cao_Kiem_Tra.md. Không thay placeholder hành chính bằng dữ liệu mẫu khi chưa được xác nhận.